Finite-Difference + Feshbach-Projector Computation of $\overline{G}_n(s^2)$ on the Bounce Background

Detailed Derivation, Code Description, and Accuracy Notes

Derivation Notes

Contents

How to read this document **5**

I Concrete pipeline tour with a worked 4×4 example **6**

1 From bounce to $\overline{G}^{\mathbf{raw}}$ and $\overline{G}^{\mathbf{sub}}$, end to end, with numbers **6**

1.1 What we are computing, and the pipeline at a glance . . . 6
1.2 The actual sizes: what the code holds in memory . . . 7
1.3 Our 4×4 toy: a tiny mock-up of the real problem . . . 8
1.4 Step 1: building $\widetilde{\mathcal{M}}$ – the discretized fluctuation operator . . . 9
1.4.1 Boundary conditions: zero rows/cols, then 1 on diagonal . . . 10
1.4.2 The toy 4×4 has no BC step . . . 10
1.5 Step 2: form $A = \widetilde{\mathcal{M}} + s^2 I$ and factorize it (LU) . . . 11
1.5.1 LU of the toy A, by hand . . . 11
1.5.2 One forward/back solve, in numbers . . . 12
1.6 Step 3: estimate $\mathrm{Tr}(A^{-1})$ without forming A^{-1} – the Hutchinson trick . . . 12
1.6.1 Walked through: three probes on the toy A . . . 13
1.7 Putting it together: $\overline{G}^{\mathrm{raw}}(s^2)$ . . . 14
1.8 Why we need $\overline{G}^{\mathrm{sub}}$: the divergent eigenvalue . . . 14
1.9 Step 4: find the offending eigenvector χ – Lanczos without jargon . . . 15
1.9.1 Why we don't just diagonalize $\widetilde{\mathcal{M}}$ . . . 15
1.9.2 The simplest idea: power iteration . . . 15
1.9.3 Don't throw information away: the Krylov subspace . . . 16
1.9.4 Make the basis orthonormal: the Lanczos recurrence . . . 16
1.9.5 Diagonalize the small matrix: Ritz values . . . 16
1.9.6 Targeting the eigenvalue we actually want . . . 16
1.9.7 2 Lanczos steps on the toy, by hand . . . 17
1.9.8 The eigsh call in the production code . . . 18
1.10 Step 5: build the projector $P = I - \chi\chi^{\top}$ . . . 18
1.11 Step 6: projected Hutchinson estimator for $\overline{G}^{\mathrm{sub}}$ . . . 19
1.12 Putting it together: $\overline{G}^{\mathrm{sub}}(s^2)$ . . . 20
1.13 The full pipeline, end to end . . . 20
1.14 Sanity-check answer for the toy . . . 21
1.15 Common follow-up questions on the walkthrough . . . 22

1.16 Lanczos in greater depth: background concepts, and the two distinct `eigsh` calls . . . 32
1.17 Where to look in the rest of the document . . . 39

II Derivation from first principles 40

2 Setting up the problem 40
2.1 Euclidean theory and the bounce . . . 40
2.2 The fluctuation operator . . . 40
2.3 The object we want: $\overline{G}_n(s^2)$ . . . 40
2.4 Notation conventions used throughout this document . . . 41

3 Reducing to a 1D radial problem 41
3.1 Spherical harmonics on S^3 . . . 41
3.2 Pole structure of $\overline{G}_n$ . . . 42

4 Liouville transformation (very detailed) 42
4.1 The substitution and its derivatives . . . 42
4.2 Plug into the radial kinetic term . . . 43
4.3 Including the centrifugal and potential terms . . . 43
4.4 The measure becomes flat . . . 43
4.5 Is the Liouville transformation strictly necessary? . . . 43
4.6 Green's function in the tilde basis (derived, not assumed) . . . 45

5 Feshbach projection: removing the pole exactly 47
5.1 The general spectral identity (every step) . . . 47
5.2 Raw vs. subtracted, in code-language . . . 49
5.3 The translation zero mode for $n = 1$ . . . 49
5.4 How χ is actually computed in the code . . . 49
5.4.1 The Lanczos algorithm explained from scratch . . . 50
5.5 Why the FD subtraction has no residual spike (RK comparison) . . . 60

6 Discretization 63
6.1 The grid and the unknowns . . . 63
6.2 Second-derivative stencil . . . 63
6.3 Block matrix for the 2-field problem (and how it differs from the 1-field case) 63
6.4 Where do boundary conditions come from? . . . 65
6.5 Symmetric Dirichlet BCs (the matrix-level implementation) . . . 70
6.6 Boundary "ghost" eigenvalues . . . 81
6.7 Continuum-discrete normalization (the missing $\mathrm{d}r$) . . . 84

7 Hutchinson stochastic trace estimator 86
7.1 The problem we are trying to solve . . . 86
7.2 The mathematical identity . . . 86
7.3 The algorithm step by step . . . 86
7.4 Why this works (intuition) . . . 87
7.5 Other choices for the random vectors . . . 87
7.6 Projected variant for $\overline{G}^{\mathrm{sub}}$ . . . 88
7.7 Why the result feels surprising: three perspectives . . . 88

7.8 Putting it back into Green's-function language 89

8 End-to-end walkthrough: from problem to number 89
8.1 What we want, in plain English 90
8.2 Step A – discretize the operator 90
8.3 Step B – bake in the boundary conditions, symmetrically 90
8.4 Step C – the side-effect: 4 "ghost" eigenvalues equal to 1 90
8.5 Step D – get the special eigenvector χ 90
8.6 Step E – factor the shifted matrix once per s^2 91
8.7 Step F – estimate the trace by Hutchinson's random-probe trick 91
8.8 Step G – $\overline{G}^{\text{sub}}$ is exactly the same algorithm + 2 projections 92
8.9 Step H – repeat across the grid, save, plot 92
8.10 Recap: how every concept connects 92

9 Step-by-step computation of $\overline{G}^{\text{raw}}$ and $\overline{G}^{\text{sub}}$ (with code) 93
9.1 One-time setup (done once before the s^2 scan) 93
9.2 Per-s^2 inner loop 94
9.2.1 Step 2 in detail: what is LU decomposition and why does it help? . . 94
9.2.2 Explicit LU of *our* fluctuation operator 98
9.2.3 Derivation of the spectral trace identity $\mathrm{Tr}(A^{-1}) = \sum_L 1/(\omega_{L,n}^2 + s^2)$. 105
9.2.4 Step 3a in detail: how Hutchinson estimates $\overline{G}^{\text{raw}}$ without ever forming A^{-1} 107
9.2.5 Step 3b in detail: how the projector creates $\overline{G}^{\text{sub}}$ by killing one term in the spectral sum 110
9.2.6 Two key questions about how the code uses the eigenvectors and the linear solve 112
9.3 Putting the pieces together: the main scan 118
9.4 What the output `.npz` contains 118
9.5 Big-picture pipeline diagram 119

III What the code does, file by file 119

10 Module dependencies and choice of libraries 119

11 `fd_builder.py`: the math engine 119
11.1 `build_M_tilde_clean` 119
11.2 `find_negative_mode` 120
11.3 `build_translation_zero_mode` 120
11.4 `hutchinson_trace_raw` and `_projected` 120
11.5 `gbar_raw_fd` and `gbar_sub_fd` 120
11.6 `load_bounce` 120

12 `compute_gbar_n0_fd.py` 120

13 `compute_gbar_n1_fd.py` 121

14 `plot_gbar_n0_fd.py` and `plot_gbar_n1_fd.py` 121

15 The "v2" variants: `*_ver2.py` 121

IV Accuracy: knobs, error sources, and how to tighten 122

16 Inventory of error sources 122

17 Lever 1: increase N (better discretization) 122

18 Lever 2: 4th-order finite-difference stencil 122

19 Lever 3: increase K (Hutchinson samples) 123

20 Lever 4: spectral shift for near-pole conditioning (documented option, NOT in the current code) 123

21 Lever 5: better grid for the bounce wall 125

22 Lever 6: tighter eigensolver and BC clamping 125

23 Lever 7: ContHutch++ for next-generation variance reduction 125
23.1 What ContHutch++ does (in two sentences) . . . 125
23.2 Why it is unusually well-matched to our problem . . . 126
23.3 Concrete weaknesses of our current pipeline that ContHutch++ addresses . . 126
23.4 Specific algorithm transferred to our setting . . . 127
23.5 What it would NOT help with . . . 127
23.6 Tradeoffs and implementation cost . . . 128
23.7 When this becomes the most attractive upgrade . . . 128

24 Recommendation: priority of upgrades 128
24.1 What is sufficient *right now*? . . . 129

How to read this document

The document grew incrementally and is now long; here is a short navigation guide so you can read it in the order that fits your question.

- **If you want the math from first principles**, read Part I in order: §1–§4 build up the analytic $\overline{G}_n^{\text{raw}}$ and $\overline{G}_n^{\text{sub}}$ formulas from the Euclidean action; §5 discretizes; §6 introduces Hutchinson; §7 is a brief narrative recap (skip if pressed for time).
- **If you want to understand exactly what the FD code computes**, jump straight to §9. It walks through every line of code with the matching equations, including LU decomposition, the trace identity, and the K-probe averaging. This is the section the daily-use of the pipeline maps to.
- **If you want to know what each Python file does**, read Part II.
- **If you want to make the result more accurate or understand what controls the error bars**, read Part III. Levers 1–7 are listed in priority order, with concrete code/CLI changes.

Cross-references that appear repeatedly.

- Notation conventions ($i, j, n, L, \{m\}, \gamma$): §2.4.
- Why "ghost eigenvalues" appear and how they are excluded: §6.6.
- Why the discrete normalization absorbs the dr factor: §6.7.
- Most pedagogical Hutchinson treatment, with code: §9.2.4 and §9.2.5.
- Most pedagogical LU treatment, with worked example: §9.2.1 and §9.2.2.
- Derivation of $\mathrm{Tr}(A^{-1})$ as a spectral sum: §9.2.3.

Suggested first pass for someone new to the pipeline: §8 (the narrative walkthrough, no equations) → §9 (the same content with equations and code). After that, dive into Part I sections that resolve any conceptual gaps.

Part I
Concrete pipeline tour with a worked 4×4 example

1 From bounce to $\overline{G}^{\mathrm{raw}}$ and $\overline{G}^{\mathrm{sub}}$, end to end, with numbers

The rest of the document is dense. This new section is the *entry point*: a single self-contained walkthrough of the full pipeline that

- assumes *zero* background in numerical linear algebra (no prior knowledge of LU decomposition, Krylov subspaces, Lanczos, or Hutchinson trace estimation is needed),
- names every object as it appears in the code, with the actual sizes used in production runs,
- drops the matrix size to a tiny 4×4 toy as soon as we need to illustrate an algorithm, so that every intermediate vector and matrix can be written out on the page.

After reading this section you should be able to look at `compute_gbar_n0_fd_ver2.py` and `compute_gbar_n1_fd_ver2.py` and recognise every line as a specific manipulation of either the real 4000×4000 object or the 4×4 toy. Subsequent parts of this document then *prove* all the identities that this section uses.

1.1 What we are computing, and the pipeline at a glance

The number we want, for one fixed partial-wave index n (we run the pipeline once for $n = 0$ and once for $n = 1$) and for many values of the radial momentum s^2, is

$$\overline{G}_n(s^2) \;=\; \sum_{L \geq 0} \frac{1}{\omega_{L,n}^2 + s^2}\,, \tag{1}$$

where $\omega_{L,n}^2$ are the eigenvalues of the radial fluctuation operator $\widetilde{\mathcal{M}}_n$ in the Liouville basis. The sum is over all discrete radial modes L at fixed n.

The key identity that makes everything tractable. Equation (1) can be rewritten as a single trace

$$\boxed{\overline{G}_n(s^2) \;=\; \mathrm{Tr}\big(\,(\widetilde{\mathcal{M}}_n + s^2 I)^{-1}\,\big)} \tag{2}$$

(derived in detail in §9.2.3). This means that the entire problem reduces to: *given a matrix* $A := \widetilde{\mathcal{M}}_n + s^2 I$, *estimate* $\mathrm{Tr}(A^{-1})$.

The two flavours, raw and subtracted. For $n = 0$ the operator $\widetilde{\mathcal{M}}_0$ has *one* negative eigenvalue $\lambda_{\mathrm{neg}} < 0$ (the Coleman mode). The trace (2) therefore has a pole at $s^2 = -\lambda_{\mathrm{neg}} > 0$. For $n = 1$ the operator has a near-zero eigenvalue $\lambda_{\mathrm{zm}} \approx 0$ (the discrete translation mode), giving a pole at $s^2 \approx 0$.

We therefore compute two quantities:

$$\overline{G}_n^{\mathrm{raw}}(s^2) \;=\; \mathrm{Tr}\big(A^{-1}\big) \;=\; \sum_{\alpha} \frac{1}{\lambda_\alpha + s^2}\,, \tag{3}$$

$$\overline{G}_n^{\mathrm{sub}}(s^2) \;=\; \mathrm{Tr}\big(PA^{-1}P\big) \;=\; \sum_{\alpha \neq \alpha_\star} \frac{1}{\lambda_\alpha + s^2}\,, \tag{4}$$

where $P = I - \chi\chi^T$ is the projector that removes the offending eigenmode χ (eigenvalue $\lambda_{\alpha_\star} \in \{\lambda_{\mathrm{neg}}, \lambda_{\mathrm{zm}}\}$). $\overline{G}^{\mathrm{sub}}$ is the same sum with that one divergent term excised; it is finite at the pole.

The pipeline at a glance. Once you accept the trace identity (2), the entire production pipeline is:

1. **Build $\widetilde{\mathcal{M}}_n$** once (sparse, 4000×4000 for $N = 2000$ radial points and 2 fields).
2. **Diagonalise once** (only one eigenpair) to get χ.
3. **For each s^2 in the grid:**
   (a) form $A = \widetilde{\mathcal{M}}_n + s^2 I$,
   (b) factorise $A = LU$ once (sparse LU),
   (c) draw $K = 400$ random probe vectors and average $v_k^\top A^{-1} v_k$ to estimate $\overline{G}^{\mathrm{raw}}$,
   (d) also average $v_k^\top P A^{-1} P v_k$ to estimate $\overline{G}^{\mathrm{sub}}$,
4. **Save and plot.**

The remainder of this section explains each item, in order, with a concrete 4×4 example threaded through every step.

1.2 The actual sizes: what the code holds in memory

Before dropping to the 4×4 toy, write down the real sizes once so they are not mysterious later.

- Number of radial grid points: $N = 2000$ (default in `compute_gbar_n0_fd_ver2.py`, line: `--N 2000`).
- Number of fields: 2 (the two scalars ϕ_1, ϕ_2).
- Total degrees of freedom (rows/columns of $\widetilde{\mathcal{M}}_n$): $N_2 = 2N = 4000$. Every vector in code, including each random probe v, has length 4000; the first 2000 entries are field 1 sampled at $r_1, \ldots, r_N$, and the last 2000 entries are field 2 sampled at the same points.
- Number of s^2 samples: about 200–300 (more near the pole).
- Hutchinson probes per s^2: $K = 400$.

So in production we are repeatedly inverting (in the LU sense; see below) one 4000×4000 *sparse* symmetric matrix that has only about $\sim 10^4$ non-zero entries (the tridiagonal of each block plus the on-diagonal coupling). Naively inverting a generic 4000×4000 matrix would take about $4000^3 \approx 6.4 \times 10^{10}$ floating-point operations; the algorithms below get it done in seconds because they exploit sparsity and avoid ever forming A^{-1} explicitly.

1.3 Our 4×4 toy: a tiny mock-up of the real problem

For illustration, replace $N \to 2$, so $N_2 = 4$. Pretend we have 2 radial points and 2 fields. We will pick the four numerical entries so that the toy has the same qualitative features as the real problem:

- it is symmetric (real $\widetilde{\mathcal{M}}_n$ is symmetric);
- it is block-structured into two 2×2 field blocks ($\widetilde{\mathcal{M}}_{11}, \widetilde{\mathcal{M}}_{22}$) coupled by an off-diagonal block $\widetilde{\mathcal{M}}_{12}$;
- it has *exactly one* eigenvalue equal to zero, with a known eigenvector χ. This zero eigenvalue plays the role of $\lambda_{\rm zm} \approx 0$ in the real $n = 1$ problem (or, with a sign flip in s^2, of the negative Coleman mode in $n = 0$).

Concretely, we use

$$\widetilde{\mathcal{M}} \;=\; \begin{pmatrix} \widetilde{\mathcal{M}}_{11} & \widetilde{\mathcal{M}}_{12} \\ \widetilde{\mathcal{M}}_{12}^{\top} & \widetilde{\mathcal{M}}_{22} \end{pmatrix} \;=\; \begin{pmatrix} 2 & -1 & 1 & 0 \\ -1 & 2 & 0 & 1 \\ 1 & 0 & 2 & -1 \\ 0 & 1 & -1 & 2 \end{pmatrix}, \qquad \widetilde{\mathcal{M}}_{11} = \widetilde{\mathcal{M}}_{22} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}, \quad \widetilde{\mathcal{M}}_{12} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{5}$$

The diagonal blocks $\widetilde{\mathcal{M}}_{11} = \widetilde{\mathcal{M}}_{22} = \left(\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right)$ mimic a 2-point discrete kinetic operator; the off-diagonal block $\widetilde{\mathcal{M}}_{12} = I_2$ mimics the cross-field coupling U''_{12}.

Eigenvalues of the toy. Direct computation gives spectrum

$$\mathrm{spec}(\widetilde{\mathcal{M}}) \;=\; \{\,0,\; 2,\; 2,\; 4\,\}. \tag{6}$$

The zero eigenvalue has eigenvector

$$\chi \;=\; \tfrac{1}{2}\,(1,\; 1,\; -1,\; -1)^{\top} \quad \big(\text{check: } \widetilde{\mathcal{M}}\chi = 0\big). \tag{7}$$

This χ plays the role of the offending eigenmode below.

Pick s^2 for the worked example. We will use $s^2 = 2$ throughout. Then

$$A \;:=\; \widetilde{\mathcal{M}} + s^2 I \;=\; \begin{pmatrix} 4 & -1 & 1 & 0 \\ -1 & 4 & 0 & 1 \\ 1 & 0 & 4 & -1 \\ 0 & 1 & -1 & 4 \end{pmatrix} \quad \text{with} \quad \mathrm{spec}(A) = \{2, 4, 4, 6\}. \tag{8}$$

Ground-truth answers (for cross-checking later). From the spectrum,

$$\overline{G}^{\rm raw} \;=\; \mathrm{Tr}(A^{-1}) \;=\; \tfrac{1}{2} + \tfrac{1}{4} + \tfrac{1}{4} + \tfrac{1}{6} \;=\; \tfrac{7}{6} \;\approx\; 1.167, \tag{9}$$

$$\overline{G}^{\rm sub} \;=\; \tfrac{1}{4} + \tfrac{1}{4} + \tfrac{1}{6} \;=\; \tfrac{2}{3} \;\approx\; 0.667 \quad \text{(zero-eigenvalue term removed)}. \tag{10}$$

We will rederive both numbers below by running the actual code path (LU, Hutchinson probes, projector) by hand on this toy.

Mapping toy ↔ real:

- Real $\widetilde{\mathcal{M}}$ has size 4000×4000; toy has 4×4.
- Real χ comes from `eigsh` (Lanczos); toy χ is written down in closed form, but we will still walk through what `eigsh` does.
- Real LU is computed by `scipy.sparse.linalg.splu`; toy LU is computed by hand below in fractions.
- Real Hutchinson averages $K = 400$ probes; toy uses 3 probes to make every step fit on a page.
- Real $\overline{G}^{\text{raw}}$ is unknown ahead of time; toy $\overline{G}^{\text{raw}} = 7/6$ is known from (9) and acts as a control answer.

1.4 Step 1: building $\widetilde{\mathcal{M}}$ – the discretized fluctuation operator

The matrix $\widetilde{\mathcal{M}}_n$ encodes the discretized version of the radial differential operator

$$\widetilde{\mathcal{M}}_n \;=\; -\frac{\mathrm{d}^2}{\mathrm{d}r^2} + \frac{n(n+2)+\frac{3}{4}}{r^2}\, I_2 + U''(\phi_b(r)), \tag{11}$$

where $U''(\phi_b(r))$ is the 2×2 Hessian of the potential evaluated on the bounce profile $\phi_b(r)$. The Liouville factor $+\frac{3}{4}/r^2$ comes from absorbing the r^3 measure of the 4D Euclidean radial integral into the wave function (see §4). The job of `build_M_tilde_clean` is to discretize (11) on a uniform grid $r_j = r_{\min} + (j-1)\,\mathrm{d}r$, $j = 1, \ldots, N$.

Three pieces inside $\widetilde{\mathcal{M}}$. The discretization produces three contributions, which are then assembled into one large sparse matrix:

1. **Second-derivative stencil** (one per field, identical): $-d^2/dr^2$ becomes the tridiagonal matrix $L_2 = \frac{1}{\mathrm{d}r^2}\,\mathrm{tridiag}(1, -2, 1)$ of size $N \times N$. The minus sign in $-L_2$ is what makes $\widetilde{\mathcal{M}}$ positive-semi-definite.
2. **Centrifugal piece** (one per field, depends on n): the diagonal vector $V_{\text{rad}}(r_j) = [n(n+2)+\frac{3}{4}]/r_j^2$.
3. **Potential mass-matrix**: at each r_j, evaluate the 2×2 Hessian $U''(\phi_b(r_j))$. Its diagonal entries $U_{11}(r_j), U_{22}(r_j)$ go into the diagonal of the field-1 and field-2 blocks; the off-diagonal entry $U_{12}(r_j)$ becomes the diagonal of the cross-field block $\widetilde{\mathcal{M}}_{12}$.

These ingredients are then *block-stacked* into the full operator

$$\widetilde{\mathcal{M}}_n \;=\; \begin{pmatrix} -L_2 + \mathrm{diag}\big(V_{\text{rad}} + U_{11}\big) & \mathrm{diag}\big(U_{12}\big) \\ \mathrm{diag}\big(U_{12}\big) & -L_2 + \mathrm{diag}\big(V_{\text{rad}} + U_{22}\big) \end{pmatrix}. \tag{12}$$

Code reference. This is exactly what the following lines do (extracted from `fd_builder_ver2.py`, function `build_M_tilde_clean`):

```
e  = np.ones(N)
L2 = sp.diags([e, -2*e, e], [-1, 0, 1], shape=(N, N)) / (dr**2)

V_radial = (n*(n+2) + 0.75) / r**2
U11 = U_pp[:, 0, 0]
U22 = U_pp[:, 1, 1]
U12 = U_pp[:, 0, 1]

M11 = -L2 + sp.diags(V_radial + U11, 0)
M22 = -L2 + sp.diags(V_radial + U22, 0)
M12 = sp.diags(U12, 0)

M_tilde = sp.bmat([[M11, M12], [M12, M22]]).tocsr()
```

1.4.1 Boundary conditions: zero rows/cols, then 1 on diagonal

The continuum operator needs Dirichlet boundary conditions $\widetilde{u}(r_{\min}) = \widetilde{u}(r_{\max}) = 0$ for each field. At the matrix level this corresponds to fixing four entries of every vector to zero: indices 0, $N-1$, N, $2N-1$ (the first and last radial point of each field block).

The way to enforce this without breaking symmetry is the *symmetric trick*:

1. Build a diagonal mask matrix D that has 1 on every row *except* the four boundary rows (where it has 0).
2. Replace $\widetilde{\mathcal{M}} \to D\widetilde{\mathcal{M}}D$. Pre-multiplying by D zeroes the four boundary rows; post-multiplying by D zeroes the four boundary columns. The result is symmetric.
3. Add $I_{\mathcal{B}}$, the matrix with 1 on the four boundary diagonal entries and 0 elsewhere. This makes the matrix non-singular on those four rows.

The result $\widetilde{\mathcal{M}}_{\text{BC}} := D\widetilde{\mathcal{M}}D + I_{\mathcal{B}}$ is block-diagonal: a non-trivial *interior* block, plus a 4×4 identity *boundary* block whose four eigenvalues are all 1. These four eigenvalues at $\lambda = 1$ are called *ghost eigenvalues*; they are an unavoidable side-effect of making the BC-enforced matrix invertible (see §6.6 for the full analysis). They are excluded from the trace at probe-time by zeroing the four boundary entries of every random vector v.

The code does exactly this:

```
boundary = [0, N - 1, N, 2 * N - 1]
mask = np.ones(N2, dtype=float)
for k in boundary:
    mask[k] = 0.0
D = sp.diags(mask, 0, format='csr')
M_tilde = D @ M_tilde @ D

diag_bc = np.zeros(N2)
for k in boundary:
    diag_bc[k] = 1.0
M_tilde = M_tilde + sp.diags(diag_bc, 0, format='csr')
```

1.4.2 The toy 4×4 has no BC step

For the toy in (5) we *deliberately* chose a matrix with no boundary indices to clamp – both "radial points" in the toy are interior. This keeps the toy small enough to write every matrix on the page. In production you can imagine the four boundary rows/columns sit somewhere off this 4×4 slice; they are zeroed at every step and so contribute nothing.

1.5 Step 2: form $A = \widetilde{\mathcal{M}} + s^2 I$ and factorize it (LU)

Why we factorise. For each s^2 we want to solve many systems of the form $Ax = v$ – one per Hutchinson probe, $K = 400$ of them. Solving $Ax = v$ from scratch is expensive. But if we factorise once, $A = LU$ *with L lower-triangular and U upper-triangular*, then every subsequent solve becomes

$$Ax = v \implies \underbrace{L\,y = v}_{\text{forward solve, } O(N)} \quad \text{then} \quad \underbrace{U\,x = y}_{\text{back solve, } O(N)} . \tag{13}$$

The factorization itself costs $O(N^{1.5})$ for our sparse band-structured matrix (much less than the naive $O(N^3)$ of dense Gauss–Jordan elimination); the per-probe solve is then linear in N.

A full theoretical exposition is in §9.2.1; this sub-section does it on the toy.

1.5.1 LU of the toy A, by hand

Starting from

$$A = \begin{pmatrix} 4 & -1 & 1 & 0 \\ -1 & 4 & 0 & 1 \\ 1 & 0 & 4 & -1 \\ 0 & 1 & -1 & 4 \end{pmatrix} . \tag{14}$$

We perform Gaussian elimination, which produces L (the multipliers used to eliminate) and U (the upper-triangular result).

Eliminate column 1. The pivot is $a_{11} = 4$. Set $\ell_{21} = a_{21}/a_{11} = -1/4$, $\ell_{31} = 1/4$, $\ell_{41} = 0$. Subtract appropriate multiples of row 1 from rows $2, 3, 4$:

$$A \to \begin{pmatrix} 4 & -1 & 1 & 0 \\ 0 & \frac{15}{4} & \frac{1}{4} & 1 \\ 0 & \frac{1}{4} & \frac{15}{4} & -1 \\ 0 & 1 & -1 & 4 \end{pmatrix} . \tag{15}$$

Eliminate column 2. The pivot is $\frac{15}{4}$. Set $\ell_{32} = (\frac{1}{4})/(\frac{15}{4}) = \frac{1}{15}$, $\ell_{42} = 1/(\frac{15}{4}) = \frac{4}{15}$. Update rows $3, 4$:

$$\to \begin{pmatrix} 4 & -1 & 1 & 0 \\ 0 & \frac{15}{4} & \frac{1}{4} & 1 \\ 0 & 0 & \frac{56}{15} & -\frac{16}{15} \\ 0 & 0 & -\frac{16}{15} & \frac{56}{15} \end{pmatrix} . \tag{16}$$

Eliminate column 3. Pivot $\frac{56}{15}$. $\ell_{43} = (-\frac{16}{15})/(\frac{56}{15}) = -\frac{2}{7}$. Update row 4:

$$U = \begin{pmatrix} 4 & -1 & 1 & 0 \\ 0 & \frac{15}{4} & \frac{1}{4} & 1 \\ 0 & 0 & \frac{56}{15} & -\frac{16}{15} \\ 0 & 0 & 0 & \frac{24}{7} \end{pmatrix}, \qquad L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{4} & 1 & 0 & 0 \\ \frac{1}{4} & \frac{1}{15} & 1 & 0 \\ 0 & \frac{4}{15} & -\frac{2}{7} & 1 \end{pmatrix} . \tag{17}$$

Sanity check via determinant. $\det A = \prod(\text{pivots}) = 4 \cdot \frac{15}{4} \cdot \frac{56}{15} \cdot \frac{24}{7} = 192 = 2 \cdot 4 \cdot 4 \cdot 6$, matching $\det A = \prod_\alpha \lambda_\alpha(A)$ for the spectrum $\{2, 4, 4, 6\}$ of (8). Good.

1.5.2 One forward/back solve, in numbers

Take the probe $v_1 = (1, 1, 1, 1)^\top$. We want $x_1 = A^{-1}v_1$.

Forward solve $L\,y = v_1$ proceeds top-down:

$$y_1 = 1, \tag{18}$$
$$y_2 = 1 - (-\tfrac{1}{4}) \cdot 1 = \tfrac{5}{4}, \tag{19}$$
$$y_3 = 1 - \tfrac{1}{4} \cdot 1 - \tfrac{1}{15} \cdot \tfrac{5}{4} = \tfrac{2}{3}, \tag{20}$$
$$y_4 = 1 - 0 \cdot 1 - \tfrac{4}{15} \cdot \tfrac{5}{4} - (-\tfrac{2}{7}) \cdot \tfrac{2}{3} = \tfrac{6}{7}. \tag{21}$$

Back solve $U\,x = y$ proceeds bottom-up:

$$x_4 = y_4 / \tfrac{24}{7} = \tfrac{6}{7} \cdot \tfrac{7}{24} = \tfrac{1}{4}, \tag{22}$$
$$x_3 = \big(y_3 - (-\tfrac{16}{15})\,x_4\big) / \tfrac{56}{15} = \big(\tfrac{2}{3} + \tfrac{16}{15} \cdot \tfrac{1}{4}\big) \cdot \tfrac{15}{56} = \tfrac{1}{4}, \tag{23}$$
$$x_2 = \big(y_2 - \tfrac{1}{4}x_3 - 1 \cdot x_4\big) / \tfrac{15}{4} = \tfrac{1}{4}, \tag{24}$$
$$x_1 = \big(y_1 - (-1)\,x_2 - 1 \cdot x_3 - 0 \cdot x_4\big) / 4 = \tfrac{1}{4}. \tag{25}$$

So $A^{-1}v_1 = (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})^\top$. A direct check by multiplying by A confirms $Ax_1 = v_1$.

Code reference. The production code calls `splu` once per s^2 and then `lu.solve` once per probe:

```
A  = (M_tilde + s2 * sp.eye(N2, format='csr')).tocsc()
lu = splu(A)
# ... later, inside the Hutchinson loop:
x = lu.solve(v)   # one forward + back solve
```

`splu` computes (17) (the sparse, 4000×4000 analogue); each `lu.solve` is a forward+back solve like the one we just ran by hand.

1.6 Step 3: estimate $\mathrm{Tr}(A^{-1})$ without forming A^{-1} – the Hutchinson trick

The problem. We want $\mathrm{Tr}(A^{-1}) = \sum_i (A^{-1})_{ii}$. To compute it the naive way, we would need each diagonal entry of A^{-1}, which would require N separate solves $Ax_i = e_i$ (e_i the standard basis vectors). For $N_2 = 4000$ that is 4000 solves per s^2, which is too expensive.

The fix: random probes. Instead, use random vectors and exploit a beautiful identity. Let $v \in \mathbb{R}^{N_2}$ be a random vector with independent entries $v_i \in \{-1, +1\}$ each with probability $\frac{1}{2}$ (*Rademacher variables*). Then

$$\mathbb{E}[v_i v_j] \;=\; \delta_{ij}, \tag{26}$$

because $v_i^2 = 1$ deterministically and $\mathbb{E}[v_i v_j] = \mathbb{E}[v_i]\mathbb{E}[v_j] = 0$ for $i \neq j$ by independence. Therefore

$$\begin{aligned} \mathbb{E}\big[v^\top A^{-1} v\big] &= \mathbb{E}\left[\sum_{i,j} v_i\,(A^{-1})_{ij}\,v_j\right] = \sum_{i,j} (A^{-1})_{ij}\,\mathbb{E}[v_i v_j] \\ &= \sum_i (A^{-1})_{ii} = \mathrm{Tr}(A^{-1}). \end{aligned} \tag{27}$$

That is: *a single quadratic form $v^\top A^{-1} v$ is an unbiased random estimator of* $\mathrm{Tr}(A^{-1})$.

The algorithm.

1. Draw K independent Rademacher probes $v_1, \ldots, v_K$.
2. For each probe, compute $x_k = A^{-1}v_k$ via the LU of step 2 (forward+back solve, $O(N)$).
3. The estimator is $\hat{T} = \frac{1}{K}\sum_k v_k^\top x_k$.

The standard error scales as $\sigma/\sqrt{K}$, where σ is the standard deviation of one sample. With Rademacher probes for our problem σ is $O(1)$, so $K = 400$ gives a standard error $\sim 1/\sqrt{400} = 0.05$ relative to typical sample magnitudes, plenty for our pole-fitting needs.

1.6.1 Walked through: three probes on the toy A

We will use three probes:

$$v_1 = (1,1,1,1)^\top, \quad v_2 = (1,-1,1,-1)^\top, \quad v_3 = (1,1,-1,-1)^\top. \tag{28}$$

(Note: in production, every v_k is drawn fresh from `rng.choice([-1,1], size=N2)`; we are picking three specific ones here for pedagogy.)

Probe 1: $v_1 = (1,1,1,1)^\top$**.** We already solved $Ax_1 = v_1$ in §1.5.2: $x_1 = (\frac{1}{4},\frac{1}{4},\frac{1}{4},\frac{1}{4})^\top$.

$$\text{sample}_1 \;=\; v_1^\top x_1 \;=\; 4\cdot\tfrac{1}{4} \;=\; 1. \tag{29}$$

Probe 2: $v_2 = (1,-1,1,-1)^\top$**.** Solve $Ax_2 = v_2$ using the same LU. The arithmetic (worked out in the same forward/back-solve style as before, omitted for brevity) gives

$$x_2 = \left(\tfrac{1}{6}, -\tfrac{1}{6}, \tfrac{1}{6}, -\tfrac{1}{6}\right)^\top, \quad \text{sample}_2 \;=\; v_2^\top x_2 \;=\; 4\cdot\tfrac{1}{6} \;=\; \tfrac{2}{3}. \tag{30}$$

Probe 3: $v_3 = (1,1,-1,-1)^\top$**.** This probe is special: it is exactly 2χ, where χ is the zero-eigenvalue eigenvector (7). Since $\widetilde{\mathcal{M}}\chi = 0$, we have $A\chi = (M + s^2 I)\chi = s^2\chi = 2\chi$, so $A^{-1}\chi = \frac{1}{2}\chi$, and

$$x_3 = \tfrac{1}{2}v_3 = (\tfrac{1}{2},\tfrac{1}{2},-\tfrac{1}{2},-\tfrac{1}{2})^\top, \qquad \text{sample}_3 \;=\; v_3^\top x_3 \;=\; 4\cdot\tfrac{1}{2} \;=\; 2. \tag{31}$$

Average.

$$\hat{T}_3 \;=\; \tfrac{1}{3}(1 + \tfrac{2}{3} + 2) \;=\; \tfrac{11}{9} \;\approx\; 1.222. \tag{32}$$

The true answer is $\overline{G}^{\text{raw}} = \frac{7}{6} \approx 1.167$. With only 3 probes the estimate is off by $\sim 5\%$. Average over $K = 400$ probes (drawn at random rather than handpicked) and the discrepancy shrinks to roughly $1/\sqrt{400} \approx 5\%$ *of the single-sample standard deviation*, i.e. much smaller in absolute terms.

Code reference. The estimator code is short and matches the steps above one-for-one:

```
def hutchinson_trace_raw(lu_factor_obj, N2, K=100, rng=None,
                         boundary_indices=None):
    samples = np.empty(K)
    for k in range(K):
        v = rng.choice([-1.0, 1.0], size=N2)
        if boundary_indices is not None:
            v[boundary_indices] = 0.0     # exclude ghost rows
```

```
        x = lu_factor_obj.solve(v)          # forward+back solve
        samples[k] = v @ x                  # one Hutchinson sample
    mean = float(np.mean(samples))
    sem  = float(np.std(samples, ddof=1) / np.sqrt(K))
    return mean, sem
```

Note the boundary-clamp: setting $v[\text{boundary}] = 0$ ensures the four ghost rows (eigenvalue 1, see §6.6) contribute zero, so the estimator measures the trace of the *interior* block only.

1.7 Putting it together: $\overline{G}^{\text{raw}}(s^2)$

Steps 1-3 give us, for one fixed s^2:

$$\boxed{\overline{G}_n^{\text{raw}}(s^2) \;\approx\; \frac{1}{K}\sum_{k=1}^{K} v_k^\top \left[(\widetilde{\mathcal{M}} + s^2 I)^{-1} v_k\right]} \tag{33}$$

with the inner solve done by `splu` once and `lu.solve` K times.

Code reference.

```
def gbar_raw_fd(M_tilde, s2, dr, N2, K=100, rng=None,
                boundary_indices=None):
    A  = (M_tilde + s2 * sp.eye(N2, format='csr')).tocsc()
    lu = splu(A)
    return hutchinson_trace_raw(lu, N2, K=K, rng=rng,
                                boundary_indices=boundary_indices)
```

The driver script `compute_gbar_n0_fd_ver2.py` loops over s^2 in an adaptive grid (denser near the pole at $-\lambda_{\text{neg}}$) and calls this function once per grid point.

What we have so far: we can compute $\overline{G}_n^{\text{raw}}(s^2)$ for any s^2 that is not exactly at a pole. But *at* the pole the trace diverges. To get a finite answer there we need $\overline{G}^{\text{sub}}$, which removes the offending eigenmode analytically.

1.8 Why we need $\overline{G}^{\text{sub}}$: the divergent eigenvalue

The spectral expansion

$$\text{Tr}(A^{-1}) \;=\; \sum_{\alpha} \frac{1}{\lambda_\alpha + s^2} \tag{34}$$

makes the source of trouble obvious: any single term where $\lambda_\alpha + s^2 \to 0$ blows up. There are exactly two physical cases where this matters:

- $n = 0$, **Coleman negative mode.** $\widetilde{\mathcal{M}}_0$ has one strictly negative eigenvalue $\lambda_{\text{neg}} < 0$. $\overline{G}_0^{\text{raw}}$ has a pole at $s^2 = -\lambda_{\text{neg}} > 0$.
- $n = 1$, **translation zero mode.** $\widetilde{\mathcal{M}}_1$ has an eigenvalue $\lambda_{\text{zm}} \approx 0$ that would be exactly zero in the continuum limit (translational symmetry of the bounce). $\overline{G}_1^{\text{raw}}$ has a pole at $s^2 = -\lambda_{\text{zm}} \approx 0$.

In both cases the offending eigenmode is well-localized: the Coleman mode is one specific eigenvector, and the translation mode is one specific eigenvector. The fix is to find that eigenvector once and then compute the trace *minus* that single divergent term:

$$\overline{G}_n^{\text{sub}}(s^2) \;=\; \sum_{\alpha \neq \alpha_\star} \frac{1}{\lambda_\alpha + s^2} \;=\; \mathrm{Tr}(A^{-1}) - \frac{1}{\lambda_{\alpha_\star} + s^2}. \tag{35}$$

The key trick is that we never compute the right-hand subtraction explicitly. Instead, we project the offending mode out of the operator before tracing:

$$\overline{G}_n^{\text{sub}}(s^2) \;=\; \mathrm{Tr}\big(P\,A^{-1}\,P\big), \qquad P = I - \chi\chi^\top, \tag{36}$$

where χ is the unit eigenvector of $\widetilde{\mathcal{M}}_n$ with eigenvalue $\lambda_{\alpha_\star}$. The proof (§9.2.3) is short: P commutes with A^{-1} when χ is an eigenvector of A, so $PA^{-1}P = A^{-1}P^2 = A^{-1}P$, and $\mathrm{Tr}(A^{-1}P)$ removes exactly the $\alpha_\star$-term from the spectral sum.

So the only new ingredient we need is χ. The next two subsections explain how to find it without diagonalizing the full 4000×4000 matrix.

1.9 Step 4: find the offending eigenvector χ – Lanczos without jargon

1.9.1 Why we don't just diagonalize $\widetilde{\mathcal{M}}$

A full diagonalization of a 4000×4000 dense matrix is $O(N^3) \approx 6.4 \cdot 10^{10}$ floating-point operations. Even with sparse exploitation, full eigendecomposition is heavy. But we don't *need* the whole spectrum: we need exactly *one* eigenpair (the smallest, in some appropriate sense). For that we use an iterative method that builds up only a tiny subspace where the dominant eigenvectors live, then diagonalizes a small matrix restricted to that subspace.

The method is called **Lanczos**. It is a streamlined version of *power iteration* that uses every intermediate vector, not just the final one. The next pieces explain it from scratch.

1.9.2 The simplest idea: power iteration

Start with a random vector v_0. Multiply repeatedly by $M := \widetilde{\mathcal{M}}$:

$$v_1 = Mv_0, \quad v_2 = Mv_1 = M^2 v_0, \quad \ldots, \quad v_k = M^k v_0. \tag{37}$$

Why does this help? Expand v_0 in the eigenbasis of M: $v_0 = \sum_\alpha c_\alpha\, \xi_\alpha$ (where $M\xi_\alpha = \lambda_\alpha \xi_\alpha$). Then $v_k = \sum_\alpha c_\alpha\, \lambda_\alpha^k\, \xi_\alpha$. *The eigenvalue with the largest magnitude wins:* as $k \to \infty$, v_k aligns with ξ_{α_*} where $|\lambda_{\alpha_*}|$ is maximum.

So power iteration reliably finds the *dominant* eigenvector. But we want the *smallest-eigenvalue* eigenvector. Two fixes:

1. Apply power iteration to M^{-1} instead of M – the smallest λ of M is the largest λ^{-1} of M^{-1}. This is the *shift-invert* idea.

2. Or apply it to $-M$ – the most-negative eigenvalue of M is the largest of $-M$.

1.9.3 Don't throw information away: the Krylov subspace

Power iteration only keeps the latest v_k. But the sequence $\{v_0, Mv_0, M^2v_0, \ldots, M^{m-1}v_0\}$ contains *much* more information than the last vector alone. The span of this sequence is the **Krylov subspace**

$$\mathcal{K}_m(M, v_0) \;=\; \operatorname{span}\bigl\{v_0,\, Mv_0,\, M^2v_0,\, \ldots,\, M^{m-1}v_0\bigr\}. \tag{38}$$

For $m \ll N$, this is a tiny subspace inside $\mathbb{R}^N$ that nevertheless is biased toward containing M's extremal eigenvectors. (Intuition: each application of M pulls the iterate toward the dominant eigenvector, so the cumulative span captures a lot of that direction; but the earlier iterates also retain some information about near-dominant directions.) The plan is therefore to find the best approximate eigenvectors of M *inside this subspace*.

1.9.4 Make the basis orthonormal: the Lanczos recurrence

The vectors $v_0, Mv_0, M^2v_0, \ldots$ are not orthogonal, and they get more and more parallel as k grows (because they all lean toward ξ_{α_*}). To work with the Krylov subspace numerically, we orthogonalize as we go.

For *symmetric* M (our case), the Gram–Schmidt procedure is miraculously cheap: a brand-new Krylov vector Mq_j only needs to be orthogonalised against the *two* previous vectors q_j and q_{j-1} – not all j predecessors. This is because of the identity $\langle q_i, Mq_j\rangle = \langle Mq_i, q_j\rangle$, which forces the inner products with all but the two adjacent basis vectors to vanish.

The result is the **Lanczos three-term recurrence**:

$$\beta_{j+1}\, q_{j+1} \;=\; M\, q_j \;-\; \alpha_j\, q_j \;-\; \beta_j\, q_{j-1}, \tag{39}$$

$$\alpha_j \;=\; q_j^{\top} M q_j, \quad \beta_{j+1} \;=\; \|\, Mq_j - \alpha_j q_j - \beta_j q_{j-1}\,\|, \tag{40}$$

seeded with a normalized random q_0 and $\beta_1 = 0$. After m steps we have an orthonormal basis $Q_m = [q_0, q_1, \ldots, q_{m-1}]$ of the Krylov subspace and a small tridiagonal matrix

$$T_m \;=\; \begin{pmatrix} \alpha_0 & \beta_1 & & \\ \beta_1 & \alpha_1 & \beta_2 & \\ & \beta_2 & \alpha_2 & \ddots \\ & & \ddots & \ddots \end{pmatrix} \;=\; Q_m^{\top} M Q_m\,. \tag{41}$$

1.9.5 Diagonalize the small matrix: Ritz values

The eigenvalues of T_m (size $m \times m$, with $m \sim 20$ typically) are called the **Ritz values**: they approximate M's eigenvalues. The corresponding eigenvectors of T_m lifted back to $\mathbb{R}^N$ via $Q_m\, y$ are the **Ritz vectors**: they approximate M's eigenvectors. Crucially, the extreme Ritz values converge *first*, so we get a good handle on the smallest/largest eigenvalues of M from a small Krylov subspace.

That is exactly what we want.

1.9.6 Targeting the eigenvalue we actually want

For $n = 0$ we want the most negative eigenvalue (Coleman). The flag `which='SA'` in `scipy.sparse.linalg.eigsh` stands for "smallest algebraic" – it returns the most-negative Ritz values first, which is what we need. The code is:

```
eigvals, eigvecs = eigsh(M_tilde, k=1, which='SA',
                         tol=1e-9, maxiter=20000)
```

For $n = 1$ we want the eigenvalue *closest to zero*, not the most negative – in our discretized operator there is a tiny *positive* near-zero eigenvalue from the would-be translation mode, plus a slightly negative spurious mode that we don't care about. To target $\lambda \approx 0$ we use *shift-invert mode*: the package internally factorizes $M - \sigma I$ at $\sigma = 0$ and runs Lanczos on the inverse, so the Ritz values that converge first are those with λ *closest to* 0. The code is:

```
eigvals, eigvecs = eigsh(M_tilde, k=5, sigma=0.0, which='LM',
                         tol=1e-9, maxiter=20000)
```

(`which='LM'` means "largest magnitude", and on the inverse operator that is the same as "closest to σ" on the original.)

1.9.7 2 Lanczos steps on the toy, by hand

Take our toy $\widetilde{\mathcal{M}}$ from (5). Seed with a random unit vector $q_0 = \frac{1}{2}(1,1,1,1)^\top$ (norm 1).

Step $j = 0$ (compute α_0, β_1, q_1).

$$Mq_0 = \tfrac{1}{2}\,\widetilde{\mathcal{M}}\,(1,1,1,1)^\top = \tfrac{1}{2}(2,2,2,2)^\top = (1,1,1,1)^\top, \tag{42}$$

$$\alpha_0 = q_0^\top M q_0 = \tfrac{1}{2}\cdot 4 = 2, \tag{43}$$

$$r_0 = Mq_0 - \alpha_0 q_0 = (1,1,1,1)^\top - 2\cdot\tfrac{1}{2}(1,1,1,1)^\top = (0,0,0,0)^\top. \tag{44}$$

We hit a *lucky breakdown*: $r_0 = 0$ means the Krylov subspace has stopped growing – which means our seed was *exactly* an eigenvector. Indeed, q_0 is the eigenvector with eigenvalue 2: $\widetilde{\mathcal{M}} q_0 = 2q_0$. So with this seed the algorithm converges in one step, returning $\alpha_0 = 2$ as a Ritz value.

The lucky breakdown is artificial here – in practice seeds are random and will have nonzero overlap with all eigenvectors. To illustrate the general case, restart with a seed that has nonzero overlap with all four eigenvectors: $q_0 = \frac{1}{\sqrt{2}}(1,0,0,1)^\top$.

Restart, $j = 0$.

$$Mq_0 = \tfrac{1}{\sqrt{2}}\,\widetilde{\mathcal{M}}\,(1,0,0,1)^\top = \tfrac{1}{\sqrt{2}}(2,0,0,2)^\top = \sqrt{2}\,(1,0,0,1)^\top\cdot\tfrac{1}{\sqrt{2}}\ldots \tag{45}$$

Carefully:

$$\begin{aligned}\widetilde{\mathcal{M}}\,(1,0,0,1)^\top &= (\,2-0+0+0,\ -1+0+0+1,\ 1+0+0-1,\ 0+0+0+2\,)^\top \\ &= (2,0,0,2)^\top, \end{aligned}\tag{46}$$

$$Mq_0 = \tfrac{1}{\sqrt{2}}(2,0,0,2)^\top, \tag{47}$$

$$\alpha_0 = q_0^\top M q_0 = \tfrac{1}{\sqrt{2}}\cdot\tfrac{1}{\sqrt{2}}\,(1\cdot 2+0+0+1\cdot 2) = \tfrac{1}{2}\cdot 4 = 2. \tag{48}$$

Then

$$r_0 = Mq_0 - \alpha_0\, q_0 = \tfrac{1}{\sqrt{2}}(2,0,0,2)^\top - 2\cdot\tfrac{1}{\sqrt{2}}(1,0,0,1)^\top = 0. \tag{49}$$

Lucky breakdown again – this seed also turned out to lie in the $\lambda = 2$ eigenspace (which is two-dimensional in our toy because $\lambda = 2$ is a double eigenvalue).

A truly generic seed must have nonzero overlap with both $\lambda = 0$ and $\lambda = 4$ eigenspaces. Take $q_0 = \frac{1}{\sqrt{4}}(1,2,0,1)^\top$ (length $\sqrt{6}/2$, so let us renormalize: $q_0 = \frac{1}{\sqrt{6}}(1,2,0,1)^\top$). We will not work out the full arithmetic on the page – the takeaway is clear:

Mechanism summary (without numbers): After m Lanczos steps on a generic seed, T_m (small $m \times m$ tridiagonal) has eigenvalues that approximate the extremal eigenvalues of M. With $m = 4$ on a 4×4 matrix, T_4 is similar to M and recovers all eigenvalues exactly. With $m \ll N$ on a 4000×4000 matrix, T_m recovers only the few extremal eigenvalues, but those are precisely the ones we asked for.

1.9.8 The eigsh call in the production code

The functions in `fd_builder_ver2.py`:

```
def find_negative_mode(M_tilde, verbose=True, n_eig=3):
    """ Smallest-algebraic eigenpair (the n=0 Coleman mode). """
    eigvals, eigvecs = eigsh(M_tilde, k=n_eig, which='SA',
                             tol=1e-9, maxiter=20000)
    order = np.argsort(eigvals)
    return float(eigvals[order[0]]), eigvecs[:, order[0]]
```

```
def find_discrete_zero_mode(M_tilde, verbose=True, n_eig=5,
                            use_shift_invert=True):
    """ Eigenpair closest to zero (the n=1 translation mode). """
    eigvals, eigvecs = eigsh(M_tilde, k=n_eig, sigma=0.0,
                             which='LM', tol=1e-9, maxiter=20000)
    idx = int(np.argmin(np.abs(eigvals)))
    return float(eigvals[idx]), eigvecs[:, idx]
```

After either call, the returned eigenvector is normalised and used as χ in the projector.

Cleaning the boundary noise. The eigsolver is iterative and the four boundary entries of the returned χ may have tiny non-zero noise. We zero them explicitly to avoid contaminating the projector with ghost-mode content, then renormalise:

```
chi[boundary_indices] = 0.0
chi = chi / np.linalg.norm(chi)
```

1.10 Step 5: build the projector $P = I - \chi\chi^\top$

Definition. Given a unit vector χ, the projector

$$P \;=\; I - \chi\chi^\top \tag{50}$$

removes the χ-component of any vector: for any w,

$$Pw \;=\; w - (\chi^\top w)\,\chi. \tag{51}$$

That is, Pw is w minus the projection of w onto χ. Two basic identities:

- $P\chi = \chi - (\chi^\top\chi)\chi = \chi - \chi = 0$. (Kills χ.)
- For any $w_\perp \perp \chi$: $Pw_\perp = w_\perp$. (Identity on the orthogonal complement.)
- $P^2 = P$ (idempotent). Proof: $P^2 w = P(w - (\chi^\top w)\chi) = w - (\chi^\top w)\chi - (\chi^\top w)\,P\chi = w - (\chi^\top w)\chi = Pw$.

Toy example. For our toy $\chi = \frac{1}{2}(1, 1, -1, -1)^\top$:

$$P = I - \tfrac{1}{4}\begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \end{pmatrix}\begin{pmatrix} 1 & 1 & -1 & -1 \end{pmatrix} = I - \tfrac{1}{4}\begin{pmatrix} 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \end{pmatrix}, \tag{52}$$

$$P = \frac{1}{4}\begin{pmatrix} 3 & -1 & 1 & 1 \\ -1 & 3 & 1 & 1 \\ 1 & 1 & 3 & -1 \\ 1 & 1 & -1 & 3 \end{pmatrix}. \tag{53}$$

Sanity: $P\chi = \frac{1}{2}P(1, 1, -1, -1)^\top = 0$, as expected.

Why we never form P explicitly. The matrix P is generally *dense*, so storing it as a 4000×4000 object would defeat the sparsity advantage. We exploit the rank-1 structure: every projection $Pw = w - (\chi^\top w)\chi$ takes only $2N_2$ floating-point operations. The code never materialises P.

1.11 Step 6: projected Hutchinson estimator for $\overline{G}^{\text{sub}}$

The recipe. For each Hutchinson probe v:

1. Compute $w = Pv = v - (\chi^\top v)\chi$.
2. Solve $Ax = w$ via the LU.
3. Compute $y = Px = x - (\chi^\top x)\chi$.
4. Sample $= v^\top y$.

Average over K probes. The expectation $\mathbb{E}[v^\top y] = \mathbb{E}[v^\top P A^{-1} P v]$ is exactly $\mathrm{Tr}(PA^{-1}P)$ by the same argument as (27), and that equals $\overline{G}^{\text{sub}}$ by (36).

Walked through: the same three probes, projected. Probe 1: $v_1 = (1, 1, 1, 1)^\top$**.** $\chi^\top v_1 = \frac{1}{2}(1 + 1 - 1 - 1) = 0$, so $Pv_1 = v_1$ (the probe is already orthogonal to χ). $x_1 = A^{-1}v_1 = (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})^\top$ (from before). $\chi^\top x_1 = 0$, so $y_1 = x_1$. $\text{sample}_1^{\text{sub}} = v_1^\top y_1 = 1$.

Probe 2: $v_2 = (1, -1, 1, -1)^\top$**.** $\chi^\top v_2 = \frac{1}{2}(1 - 1 - 1 + 1) = 0$, so $Pv_2 = v_2$. $x_2 = (\frac{1}{6}, -\frac{1}{6}, \frac{1}{6}, -\frac{1}{6})^\top$ (from before). $\chi^\top x_2 = 0$, so $y_2 = x_2$. $\text{sample}_2^{\text{sub}} = v_2^\top y_2 = \frac{2}{3}$.

Probe 3: $v_3 = (1, 1, -1, -1)^\top = 2\chi$**.** $\chi^\top v_3 = \frac{1}{2} \cdot 4 = 2$, so $Pv_3 = v_3 - 2\chi = 2\chi - 2\chi = 0$. The whole sample collapses: $w_3 = 0$, $x_3 = 0$, $y_3 = 0$, $\text{sample}_3^{\text{sub}} = 0$.

This is exactly the point of the projector: the offending direction contributes *nothing* to the estimator.

Average.

$$\hat{T}_3^{\text{sub}} \;=\; \tfrac{1}{3}(1 + \tfrac{2}{3} + 0) \;=\; \tfrac{5}{9} \;\approx\; 0.556. \tag{54}$$

True $\overline{G}^{\text{sub}} = \frac{2}{3} \approx 0.667$. With only 3 specific probes the estimate is off by $\sim 17\%$ – comparable to the raw case once you remember we just discarded one of three samples. With $K = 400$ random Rademacher probes the estimate converges to the true value.

Code reference.

```
def hutchinson_trace_projected(lu_factor_obj, N2, chi, K=100, rng=None,
                               boundary_indices=None):
    samples = np.empty(K)
    for k in range(K):
        v = rng.choice([-1.0, 1.0], size=N2)
        if boundary_indices is not None:
            v[boundary_indices] = 0.0
        w = v - chi * (chi @ v)          # P v
        x = lu_factor_obj.solve(w)       # A^{-1} P v
        y = x - chi * (chi @ x)          # P A^{-1} P v
        samples[k] = v @ y               # one projected sample
    mean = float(np.mean(samples))
    sem  = float(np.std(samples, ddof=1) / np.sqrt(K))
    return mean, sem
```

1.12 Putting it together: $\overline{G}^{\mathrm{sub}}(s^2)$

$$\boxed{\overline{G}_n^{\mathrm{sub}}(s^2) \;\approx\; \frac{1}{K}\sum_{k=1}^{K} v_k^{\top}\Big[\,P\,(\widetilde{\mathcal{M}}+s^2 I)^{-1}\,P\,v_k\Big]} \tag{55}$$

with $P = I - \chi\chi^{\top}$ and the inner solve done by `lu.solve` on the same LU factor used for $\overline{G}^{\mathrm{raw}}$. The driver code is mirror-image of the raw case:

```
def gbar_sub_fd(M_tilde, s2, dr, N2, chi, K=100, rng=None,
                boundary_indices=None):
    A  = (M_tilde + s2 * sp.eye(N2, format='csr')).tocsc()
    lu = splu(A)
    return hutchinson_trace_projected(lu, N2, chi, K=K, rng=rng,
                                      boundary_indices=boundary_indices)
```

For $n = 1$ the script also multiplies the result by the SO(4) degeneracy $(n+1)^2 = 4$, since the partial-wave decomposition collects $(n+1)^2$ identical radial sectors at fixed n:

```
degeneracy = (1 + 1) ** 2  # = 4 for n=1
sub_val *= degeneracy
```

For $n = 0$ the degeneracy is $(0+1)^2 = 1$ and no multiplication is needed. (See §3.1 for the derivation of this counting.)

1.13 The full pipeline, end to end

The complete $n = 0$ run is the script `compute_gbar_n0_fd_ver2.py`; the $n = 1$ run is `compute_gbar_n1_fd_ver2.py`. In skeleton form:

```
# 1. Load the bounce profile from disk
b = load_bounce(bounce_path)
pot_lin = CTShiftedLiftedPotential(b["params"], b["false_vac"])

# 2. Build M_tilde (Step 1 of this section)
M_tilde, r, dr, U_pp = build_M_tilde_clean(
    n=N_INDEX, R_bounce=b["R"],
    X_prime_bounce=b["X_prime"], Y_prime_bounce=b["Y_prime"],
    pot_lin=pot_lin, N=2000, r_min=1e-4, fd_order=2,
)

```

```
# 3. Diagonalize once for the offending eigenvector chi (Step 4)
if N_INDEX == 0:
    lambda_alpha, chi = find_negative_mode(M_tilde)
else:
    lambda_alpha, chi = find_discrete_zero_mode(M_tilde)

# (boundary clean-up + renormalise)
chi[boundary_indices] = 0.0
chi = chi / np.linalg.norm(chi)

# 4. Build the s^2 grid (denser near the pole at -lambda_alpha)
s2_grid = build_s2_grid(...)

# 5. Loop over s^2: for each, do Steps 2, 3, and 6
for i, s2 in enumerate(s2_grid):
    raw_val, raw_err = gbar_raw_fd(M_tilde, s2, dr, N2,
                                   K=400, rng=rng,
                                   boundary_indices=boundary_indices)
    sub_val, sub_err = gbar_sub_fd(M_tilde, s2, dr, N2, chi,
                                   K=400, rng=rng,
                                   boundary_indices=boundary_indices)
    if N_INDEX == 1:
        raw_val *= 4; sub_val *= 4   # SO(4) degeneracy
    gbar_raw[i], gbar_sub[i] = raw_val, sub_val

# 6. Save
np.savez(out, s2_grid=s2_grid,
         gbar_raw=gbar_raw, gbar_sub=gbar_sub,
         chi=chi, lambda_alpha=lambda_alpha)
```

That is the entire production code in 30 lines. Every block maps directly to one of the steps walked through on the toy:

Pipeline block	**Maps to (worked toy section)**
`build_M_tilde_clean`	Step 1, §1.4: assemble $\widetilde{\mathcal{M}}$ from L_2, V_{rad}, U''; symmetric BC trick.
`find_negative_mode` / `find_discrete_zero_mode`	Step 4, §1.9: Lanczos / shift-invert to find χ.
$A = \widetilde{\mathcal{M}} + s^2 I$; `splu(A)`	Step 2, §1.5: LU factorisation, once per s^2.
`hutchinson_trace_raw`	Step 3, §1.6: random-probe trace estimator for $\overline{G}^{\text{raw}}$.
`hutchinson_trace_projected`	Step 6, §1.11: same with $PA^{-1}P$ for $\overline{G}^{\text{sub}}$.
boundary clamp $v[\text{bnd}] = 0$	exclude ghost eigenvalues (§6.6).
$\times(n+1)^2$ for $n = 1$	SO(4) partial-wave degeneracy (§3.1).

1.14 Sanity-check answer for the toy

To close the loop, plug the toy numbers into the boxed formulas (33) and (55) with the $K = 3$ specific probes used in §1.6 and §1.11:

	Probe 1 $(1,1,1,1)$	**Probe 2** $(1,-1,1,-1)$	**Probe 3** $(1,1,-1,-1)=2\chi$
$v^\top A^{-1}v$ (raw)	1	$\frac{2}{3}$	2
$v^\top PA^{-1}Pv$ (sub)	1	$\frac{2}{3}$	0

Averaging gives

$$\widehat{\overline{G}}_3^{\text{raw}} = \tfrac{1}{3}(1+\tfrac{2}{3}+2) = \tfrac{11}{9} \approx 1.222 \qquad \text{(true: } \tfrac{7}{6} \approx 1.167\text{)}, \tag{56}$$

$$\widehat{\overline{G}}_3^{\text{sub}} = \tfrac{1}{3}(1+\tfrac{2}{3}+0) = \tfrac{5}{9} \approx 0.556 \qquad \text{(true: } \tfrac{2}{3} \approx 0.667\text{)}. \tag{57}$$

The handpicked $K = 3$ estimate is correct in mechanism but noisy in absolute value – exactly the regime that production $K = 400$ random probes drives to convergence.

1.15 Common follow-up questions on the walkthrough

The six questions below are the ones a careful first reader most often asks. Each is answered with the same toy as above so you can compute along. Q0 is a quick glossary so the notation lines up between the math and the code.

Q0: code-to-math glossary – what *is* `M_tilde` in the code?

The code variable `M_tilde` returned by `build_M_tilde_clean` is exactly the discretized fluctuation operator $\widetilde{\mathcal{M}}_n$ from equation (11) – with the symmetric Dirichlet BC trick *already applied*. So every time the math in this document writes $\widetilde{\mathcal{M}}$ (or $\widetilde{\mathcal{M}}_n$), the corresponding object in the code is the matrix $\widetilde{\mathcal{M}}_{\text{BC}} = D\widetilde{\mathcal{M}}D + I_{\mathcal{B}}$, stored as `scipy.sparse.csr_matrix` of shape 4000×4000.

The Liouville transformation $\widetilde{u}(r) = r^{3/2}u(r)$ which turned the original radial operator $-\mathrm{d}_r^2 - (3/r)\mathrm{d}_r + n(n+2)/r^2 + U''$ into the cleaner $-\mathrm{d}_r^2 + [n(n+2)+\frac{3}{4}]/r^2 + U''$ (no first-derivative term, flat measure) is performed *analytically* on the page (§4) and then enters the code only as the value $+\frac{3}{4}/r^2$ that the function adds to the diagonal.

The full code-to-math glossary used in this section:

Code variable	**Math meaning**
`M_tilde`	$\widetilde{\mathcal{M}}_n$ with Dirichlet BCs baked in (size 4000×4000).
`r`, `dr`	radial grid points and spacing.
`N`, `N2`	$N = 2000$ radial points; $N_2 = 2N = 4000$ DOFs.
`U_pp`	$U''(\phi_b(r_j))$, the 2×2 Hessian at each r_j.
`boundary_indices`	$\{0, N-1, N, 2N-1\}$, the four indices where u vanishes.
`chi`	the unit eigenvector χ, used to build $P = I - \chi\chi^\top$.
`lambda_neg`, `lambda_zm`	the eigenvalues $\lambda_{\text{neg}} < 0$ (Coleman) and $\lambda_{\text{zm}} \approx 0$ (translation).
`A = M_tilde + s2*sp.eye(...)`	$A_n(s^2) = \widetilde{\mathcal{M}}_n + s^2 I$ (the matrix we factorize each s^2).
`lu = splu(A)`	sparse LU factorization $A = LU$.
`lu.solve(v)`	one forward+back solve $x = A^{-1}v$.
`v = rng.choice([-1,1], size=N2)`	one Hutchinson Rademacher probe.
`K`	number of Hutchinson probes (default $K = 400$).

So whenever the document writes "$\widetilde{\mathcal{M}}$" or "$A = \widetilde{\mathcal{M}} + s^2 I$", the corresponding code variable is `M_tilde` or `A`. Whenever a derivation says "apply A^{-1} to a vector", the corresponding code action is `lu.solve(v)`. Whenever the derivation says "project with P", the code does `w = v - chi * (chi @ v)`.

Q1: When we replace $\widetilde{\mathcal{M}} \to D\widetilde{\mathcal{M}}D + I_{\mathcal{B}}$, do we change the physics? And why is only the "inside" of $\widetilde{\mathcal{M}}$ relevant, not the zero square around it?

Let us look at this with explicit indices. Suppose $N = 3$ on *one* field (so the full matrix would be 3×3, not the toy 4×4). Indices 0 and 2 are boundary, index 1 is interior. Imagine

$$\widetilde{\mathcal{M}} \;=\; \begin{pmatrix} m_{00} & m_{01} & m_{02} \\ m_{10} & m_{11} & m_{12} \\ m_{20} & m_{21} & m_{22} \end{pmatrix}. \tag{58}$$

The mask is $D = \mathrm{diag}(0, 1, 0)$ – it kills the boundary indices.

$$D\widetilde{\mathcal{M}}D \;=\; \begin{pmatrix} 0 & 0 & 0 \\ 0 & m_{11} & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad D\widetilde{\mathcal{M}}D + I_{\mathcal{B}} \;=\; \begin{pmatrix} 1 & 0 & 0 \\ 0 & m_{11} & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{59}$$

Three facts you can read off this picture.

1. **The interior block is untouched.** After the trick, the middle entry m_{11} is exactly the same as before. If we had $N = 2000$ rows, the (1996×1996) interior block would be byte-for-byte identical to the original interior block of $\widetilde{\mathcal{M}}$. *We did not change a single matrix entry that matters for the interior physics.*

2. **The "zero square around it" is the decoupling.** The off-diagonal entries $m_{01}, m_{10}, m_{02}, m_{20}, m_1$ all become zero. Why is that fine?

   - Dirichlet BCs say $u_0 = u_{N-1} = 0$. A vector that obeys the BCs has $u_0 = u_{N-1} = 0$.
   - For such a vector, $(\widetilde{\mathcal{M}}u)_{\text{interior}} = m_{1,0}u_0 + m_{1,1}u_1 + m_{1,2}u_2 = m_{1,1}u_1$ – the boundary terms drop out anyway because $u_0 = u_2 = 0$.
   - So whether the off-diagonal entries are $m_{1,0}$ or 0 makes no difference to interior physics. Zeroing them is just making explicit what Dirichlet already guaranteed.

3. **The $+I_{\mathcal{B}}$ is purely a numerical patch.** Without it, the matrix has rows 0 and 2 equal to zero – it is singular and cannot be LU-factorized. Putting 1 on those diagonal entries makes it invertible at no physical cost: the boundary block becomes a 4×4 identity (in the full 2-field $N = 2000$ case) that is completely decoupled from the interior. Solving $(\widetilde{\mathcal{M}}_{\text{BC}} + s^2 I)x = v$ for any input vector v gives:

   - x_{interior} from the interior equations alone (untouched physics);
   - $x_{\text{boundary}} = v_{\text{boundary}}/(1 + s^2)$ from the trivial boundary equations (the unit-eigenvalue "ghosts").

Why we then probe with $v_{\text{boundary}} = 0$. The Hutchinson estimator computes $v^\top A^{-1} v = v_{\text{int}}^\top A_{\text{int}}^{-1} v_{\text{int}} + v_{\text{bnd}}^\top A_{\text{bnd}}^{-1} v_{\text{bnd}}$ because A is block-diagonal. The boundary block contributes $v_{\text{bnd}}^\top v_{\text{bnd}}/(1+s^2) = N_{\mathcal{B}}/(1+s^2)$ on average – a constant unrelated to physics. By forcing $v_{\text{bnd}} = 0$ we kill that contribution exactly and keep only the physical interior trace. *This is why the $+I_{\mathcal{B}}$ ghosts are harmless: we never let any random probe sample them.*

One-sentence summary. Zeroing the rows/columns plus adding $I_{\mathcal{B}}$ *decouples* the boundary from the interior without altering a single interior entry; the $+I_{\mathcal{B}}$ ghosts are then sterilised by clamping every probe to zero on the boundary indices.

But isn't the interior block of A uninvertible "by itself" once we split off the boundary? This is the worry behind the question. When we write $v^\top A^{-1} v = v_{\text{int}}^\top A_{\text{int}}^{-1} v_{\text{int}} + v_{\text{bnd}}^\top A_{\text{bnd}}^{-1} v_{\text{bnd}}$ we are inverting two blocks separately. The user's concern is whether deleting the off-diagonal coupling damaged the interior block's invertibility. The answer is that the interior block was never coupled to the boundary block in the first place after the trick, so we have nothing to "put back": they invert independently.

Concretely, take the same 3×3 example. After the trick and adding $s^2 I$:

$$A \;=\; \widetilde{\mathcal{M}}_{\text{BC}} + s^2 I \;=\; \begin{pmatrix} 1+s^2 & 0 & 0 \\ 0 & m_{11}+s^2 & 0 \\ 0 & 0 & 1+s^2 \end{pmatrix}. \tag{60}$$

A is block-diagonal: a 1×1 interior block $A_{\text{int}} = [m_{11} + s^2]$ and a 2×2 boundary block $A_{\text{bnd}} = (1+s^2)\, I_2$. The two blocks share no off-diagonal entries, so their inverses are computed independently:

$$A^{-1} \;=\; \begin{pmatrix} \frac{1}{1+s^2} & 0 & 0 \\ 0 & \frac{1}{m_{11}+s^2} & 0 \\ 0 & 0 & \frac{1}{1+s^2} \end{pmatrix}, \quad A_{\text{int}}^{-1} = \big[\,(m_{11}+s^2)^{-1}\,\big], \quad A_{\text{bnd}}^{-1} = \tfrac{1}{1+s^2}\, I_2. \tag{61}$$

Both are well-defined as long as $m_{11} + s^2 \neq 0$ and $1 + s^2 \neq 0$. The first is the physical condition (avoid the pole); the second is trivially $s^2 > -1$, automatic for positive s^2. *No information was lost when we "split"*, because the two blocks were never coupled after the trick. The zeros around the interior block are not "dropped" – they are correctly present in the matrix and dictate that the two block inverses don't talk to each other.

In production, A_{int} is the $(N_2 - 4) \times (N_2 - 4)$ submatrix of $\widetilde{\mathcal{M}} + s^2 I$ on interior indices: the original discretized operator restricted to the interior. It is symmetric and (away from the poles) positive definite, hence non-singular. `splu(A)` factorizes the full block-diagonal A in one shot and internally just produces independent factorizations of the two blocks. We never have to invoke the split ourselves.

What "physics unchanged" means operationally. "The physics" is, by construction, the values u_{int} that the BC-respecting wave function takes at the interior grid points, given a source f. There are two procedures one could use to compute it:

1. *Original problem with explicit BCs*: solve the linear system formed only by the interior rows of the original $\widetilde{\mathcal{M}}$, acting on the interior unknowns:

$$\widetilde{\mathcal{M}}_{\text{int,int}}\; u_{\text{int}} \;=\; f_{\text{int}} \quad \text{with implicit } u_{\text{bnd}} = 0. \tag{62}$$

   This is the textbook way of solving a Dirichlet PDE numerically.

2. *BC-tricked problem*: solve the full system on all N_2 unknowns:

$$(D\widetilde{\mathcal{M}}D + I_{\mathcal{B}})\, u \;=\; f. \tag{63}$$

   Because the matrix is block-diagonal, this decouples into two independent linear systems: $\widetilde{\mathcal{M}}_{\text{int,int}}\, u_{\text{int}} = f_{\text{int}}$ (same as above) and $u_{\text{bnd}} = f_{\text{bnd}}$ (a trivial copy of the boundary entries of the source).

The first procedure gives u_{int}. The second procedure gives the same u_{int} *plus* a meaningless boundary entry $u_{\text{bnd}} = f_{\text{bnd}}$ that we throw away. *The interior solution is bit-for-bit identical between the two procedures.*

That identity is the precise sense in which physics is unchanged: **any quantity computed from u_{int} is the same before and after the trick**. The trick adds four trivial decoupled equations whose solutions we discard, in exchange for a matrix that is non-singular and easy to factorize. No interior entry of $\widetilde{\mathcal{M}}$ is altered; no eigenvalue of the interior block changes; no solution that respects the BC changes value.

The Hutchinson estimator inherits this immediately. We only ever sample $\text{Tr}(A^{-1})$ via probes that have $v_{\text{bnd}} = 0$, so we never see u_{bnd}. The estimator measures only the interior trace $\text{Tr}(A_{\text{int}}^{-1})$, which is the same number we would have computed without the trick.

Q2: "Rademacher with probability $1/2$" – does that mean exactly half $+1$ and half -1 inside one vector, or is that just on average?

It is per-*entry*, not per-vector.

Per-entry interpretation. Each component v_i of one probe vector is decided *independently* by an unbiased coin flip:

$$v_i = \begin{cases} +1 & \text{with probability } 1/2 \\ -1 & \text{with probability } 1/2 \end{cases} \quad \text{independently of all other } v_j. \tag{64}$$

For a length-N_2 vector ($N_2 = 4000$ in production), the number of $+1$'s in any given draw is a binomial random variable $\text{Bin}(N_2, 1/2)$. Its mean is $N_2/2 = 2000$ and its standard deviation is $\sqrt{N_2/4} = \sqrt{1000} \approx 32$. So a typical draw might have, say, 1973 entries equal to $+1$ and 2027 equal to -1, or any other split within ± 100 of half-and-half. Each specific vector is *not* balanced exactly.

Why the identity $\mathbb{E}[v_i v_j] = \delta_{ij}$ still holds. The two ingredients are

- $\mathbb{E}[v_i] = (+1)\cdot\frac{1}{2} + (-1)\cdot\frac{1}{2} = 0$ (unbiased coin),
- v_i and v_j are independent for $i \neq j$.

From these,

$$\mathbb{E}[v_i v_j] \;=\; \begin{cases} \mathbb{E}[v_i^2] = \mathbb{E}[1] = 1 & i = j, \\ \mathbb{E}[v_i]\,\mathbb{E}[v_j] = 0 & i \neq j. \end{cases} \tag{65}$$

Note carefully: *this is an expectation over many draws, not a property of one draw.* A single Rademacher vector v_1 does *not* satisfy $v_1^\top v_1 = N_2$ on the off-diagonals – only the average $\langle v^\top A^{-1} v\rangle$ does the cancellation that produces $\text{Tr}(A^{-1})$.

Code reference.

```
v = rng.choice([-1.0, 1.0], size=N2)
```

`rng.choice` draws each of the $N_2 = 4000$ entries independently from $\{-1, +1\}$, exactly the per-entry recipe above.

Q3: What is Monte Carlo? How is Hutchinson a "Monte Carlo" method?

Monte Carlo in one sentence. *Monte Carlo* is the strategy "replace a number that is hard to compute exactly with an average over random samples". The random samples need to satisfy one design rule: their average must equal the number you wanted (*unbiasedness*). Then by drawing many samples and averaging, you converge to the true value at rate $1/\sqrt{K}$.

Three classical examples.

1. **Estimating π by darts.** Throw K uniformly random points $(x, y) \in [-1, 1]^2$. Count the fraction $\hat{p}$ that satisfy $x^2 + y^2 \leq 1$. Then $\hat{p} \to \pi/4$, so $\hat{\pi} = 4\hat{p}$. The true area ratio is $\pi/4$; one dart is a Bernoulli sample of that ratio; the average of K darts is an unbiased estimator with error $\sim 1/\sqrt{K}$.

2. **Estimating an integral.** For $I = \int_{[0,1]^d} f(x)\, \mathrm{d}^d x$ in high dimension d, draw K uniformly random $x_k \in [0, 1]^d$ and let $\hat{I} = \frac{1}{K} \sum_k f(x_k)$. Then $\mathbb{E}[\hat{I}] = I$ exactly, again with error $\sim 1/\sqrt{K}$. This is what makes Monte Carlo essential for high-dimensional quantum-field-theory and statistical-physics integrals.

3. **Estimating a trace (Hutchinson, our case).** For $T = \mathrm{Tr}(A^{-1})$, draw K Rademacher probes v_k and let $\hat{T} = \frac{1}{K} \sum_k v_k^\top A^{-1} v_k$. The identity (27) guarantees $\mathbb{E}[\hat{T}] = T$, and the error is again $\sim \sigma/\sqrt{K}$ where σ is the standard deviation of one sample.

What makes Hutchinson Monte Carlo, exactly. The list of features:

- the quantity we want, $\mathrm{Tr}(A^{-1})$, is a definite deterministic number;
- we cannot afford to compute it directly (would need all N_2 diagonal entries of A^{-1});
- we replace it with a *random* estimator $v^\top A^{-1} v$ that we *can* compute (one LU solve);
- the design $v \sim$ Rademacher makes the estimator unbiased;
- averaging over K independent samples reduces the variance linearly in K, hence the standard error decays as $1/\sqrt{K}$;
- this is exactly the same flavour of trade-off as "throwing darts to estimate π".

Why Monte Carlo is acceptable here. Direct computation of $\mathrm{Tr}(A^{-1})$ would cost N_2 independent solves; Hutchinson with K probes costs K solves. We pick $K = 400$ which is much less than $N_2 = 4000$, so we gain a factor of 10 in speed at the cost of a known statistical error of order $\sigma/\sqrt{K} \approx 5\%\, \sigma$. The standard error reported in the saved `*_err` arrays is exactly this Monte-Carlo error.

Q4: Power iteration – why does "the largest-magnitude eigenvalue win" as $k \to \infty$?

The clearest way to see it is with explicit numbers. Take a 4×4 symmetric matrix M with eigenvalues $\{\lambda_1, \lambda_2, \lambda_3, \lambda_4\} = \{0.5, 1, 2, 4\}$ and orthonormal eigenvectors $\xi_1, \xi_2, \xi_3, \xi_4$.

Pick a starting vector with non-zero overlap with all four:

$$v_0 \;=\; \xi_1 + \xi_2 + \xi_3 + \xi_4 \quad (\text{coefficients } c_\alpha = 1). \tag{66}$$

Apply M once. By the eigenvalue equation $M\xi_\alpha = \lambda_\alpha \xi_\alpha$,

$$v_1 \;=\; M v_0 \;=\; 0.5\,\xi_1 + 1\,\xi_2 + 2\,\xi_3 + 4\,\xi_4. \tag{67}$$

Each eigenvector has been multiplied by its own eigenvalue. Apply M again:

$$v_2 \;=\; M v_1 \;=\; 0.5^2\,\xi_1 + 1^2\,\xi_2 + 2^2\,\xi_3 + 4^2\,\xi_4 \;=\; 0.25\,\xi_1 + 1\,\xi_2 + 4\,\xi_3 + 16\,\xi_4. \tag{68}$$

Continue:

k	coefficient on ξ_1	coefficient on ξ_2	coefficient on ξ_3	coefficient on ξ_4	ratio to ξ_4-coef
0	1	1	1	1	1
1	0.5	1	2	4	0.13
5	0.031	1	32	1024	$\sim 10^{-4.5}$
10	9.8×10^{-4}	1	1024	1.05×10^{6}	$\sim 10^{-9}$
20	9.5×10^{-7}	1	$\sim 10^{6}$	$\sim 10^{12}$	$\sim 10^{-18}$

After 20 iterations, the ξ_4-coefficient is 10^{18} times bigger than the ξ_1-coefficient. The vector v_{20}, to 15 decimal digits, is parallel to ξ_4. *That is what "the largest-magnitude eigenvalue wins" means:* every other eigendirection's amplitude shrinks by the ratio $|\lambda_\alpha/\lambda_*|^k$ relative to the dominant one, and these ratios are below 1 so they vanish exponentially in k.

The general formula.

$$v_k \;=\; \sum_\alpha c_\alpha\, \lambda_\alpha^k\, \xi_\alpha \;=\; \lambda_*^k \left[c_*\,\xi_* + \sum_{\alpha \neq *} c_\alpha \underbrace{\left(\tfrac{\lambda_\alpha}{\lambda_*}\right)^k}_{\to 0} \xi_\alpha \right], \tag{69}$$

where λ_* is the eigenvalue with the largest absolute value. The bracketed sum is dominated by $c_*\xi_*$ for large k because $|\lambda_\alpha/\lambda_*| < 1$ raised to a high power vanishes.

What if you want the smallest eigenvalue? Apply power iteration to M^{-1}, whose largest eigenvalue is $1/\lambda_{\min}(M)$. Or equivalently apply it to $(M - \sigma I)^{-1}$ (*shift-invert*), whose largest eigenvalue is $1/(\lambda - \sigma)$ for the eigenvalue λ of M closest to σ. This is exactly how `eigsh` with `sigma=0` finds the eigenvalue of $\widetilde{\mathcal{M}}$ closest to zero in the $n = 1$ run.

Q5: The Lanczos recurrence – atomic walkthrough

We will rebuild Lanczos from scratch in this answer, in small enough sub-steps that every multiplication and inner product is on the page. The plan:

1. **(Goal)** explain what we want and why simple power iteration is not enough.

2. **(Idea)** explain the Krylov subspace and why we want an orthonormal basis of it.
3. **(Mechanism)** explain why, for symmetric matrices, the Gram–Schmidt step against *all previous* basis vectors collapses to a Gram–Schmidt step against *just the last two*. This is what makes Lanczos cheap.
4. **(Algorithm)** write out the recurrence explicitly with a name for every quantity.
5. **(Worked toy)** run the recurrence on $\widetilde{\mathcal{M}}$ from (5), multiplication by multiplication, until lucky breakdown.
6. **(Tridiagonal matrix)** read off the 3×3 tridiagonal T_3 and diagonalize it to get the Ritz values $\{0, 2, 4\}$.
7. **(Ritz vector)** lift the $\lambda = 0$ eigenvector of T_3 back to $\mathbb{R}^4$ and verify that it is exactly the offending mode χ from (7).
8. **(Code)** connect to the `eigsh` call.

Step (Goal). $\widetilde{\mathcal{M}}$ is a 4×4 symmetric matrix ((5)) with eigenvalues $\{0, 2, 2, 4\}$ – pretend we do not know this. We want one specific eigenpair: the one whose eigenvalue is smallest in absolute value (this is the offending mode that will become χ). *We must do this without diagonalizing the full matrix, so that the same procedure scales to the real* 4000×4000 *problem.*

Step (Idea). Power iteration would let us track the largest-magnitude eigenvector by repeatedly multiplying by $\widetilde{\mathcal{M}}$ (Q4). Lanczos generalizes this by saving *every* intermediate vector, not just the latest. The collection

$$\{q_0,\ \widetilde{\mathcal{M}}q_0,\ \widetilde{\mathcal{M}}^2 q_0,\ \widetilde{\mathcal{M}}^3 q_0, \dots\} \tag{70}$$

is called the *Krylov sequence*. Its span is the Krylov subspace $\mathcal{K}_m = \operatorname{span}\{q_0, \widetilde{\mathcal{M}}q_0, \dots, \widetilde{\mathcal{M}}^{m-1}q_0\}$. The crucial property is: $\widetilde{\mathcal{M}}$ maps $\mathcal{K}_m$ into $\mathcal{K}_{m+1}$, that is, into itself plus *one* new direction. So if we represent $\widetilde{\mathcal{M}}$ in a basis of $\mathcal{K}_m$, the matrix has a very thin shape – and for a symmetric $\widetilde{\mathcal{M}}$, that shape is exactly tridiagonal.

We want an *orthonormal* basis $q_0, q_1, \dots, q_{m-1}$ of $\mathcal{K}_m$ so that the represented matrix is symmetric in addition to tridiagonal. Lanczos is the recipe for building this basis cheaply.

Step (Mechanism: why Gram–Schmidt collapses to two projections). Suppose we have already produced $q_0, q_1, \dots, q_j$ orthonormal, and we want q_{j+1}. The naive Gram–Schmidt prescription is to take $\widetilde{\mathcal{M}}q_j$ and subtract its projection onto each of the previous basis vectors:

$$\widetilde{q}_{j+1} \;=\; \widetilde{\mathcal{M}}q_j \;-\; \sum_{i=0}^{j} \left(q_i^\top \widetilde{\mathcal{M}} q_j\right) q_i. \tag{71}$$

This costs $j+1$ inner products at step j, totalling $O(m^2)$ over all steps. We claim that for symmetric $\widetilde{\mathcal{M}}$, all but the $i = j$ and $i = j-1$ inner products are *automatically zero*, so the sum collapses to two terms.

The proof is one line. By symmetry of $\widetilde{\mathcal{M}}$,

$$q_i^\top \widetilde{\mathcal{M}} q_j \;=\; \left(\widetilde{\mathcal{M}} q_i\right)^\top q_j. \tag{72}$$

By the previous Lanczos step ($i \le j-1$), we have $\widetilde{\mathcal{M}} q_i = \alpha_i q_i + \beta_{i+1} q_{i+1} + \beta_i q_{i-1} \in \mathrm{span}\{q_{i-1}, q_i, q_{i+1}\}$. Therefore

$$\big(\widetilde{\mathcal{M}} q_i\big)^\top q_j \;=\; 0 \quad \text{whenever} \quad j \notin \{i-1, i, i+1\}, \;\; \text{i.e. } |j-i| \ge 2. \tag{73}$$

So the sum in (71) keeps only $i = j$ and $i = j-1$:

$$\widetilde{q}_{j+1} \;=\; \widetilde{\mathcal{M}} q_j - \alpha_j \, q_j - \beta_j \, q_{j-1}, \tag{74}$$

where $\alpha_j := q_j^\top \widetilde{\mathcal{M}} q_j$ and $\beta_j := q_{j-1}^\top \widetilde{\mathcal{M}} q_j$. (The naming $\beta_j := \|r_{j-1}\|$ is convention – the two formulas agree because the previous step's normalization sets the scale.)

Step (Algorithm). Initialize $\beta_0 = 0$ and $q_{-1} = 0$ (a phantom that drops out of step $j = 0$). For $j = 0, 1, 2, \ldots$:

1. Compute $w_j := \widetilde{\mathcal{M}} q_j$. *(One sparse matrix-vector multiply.)*
2. Compute the diagonal coefficient $\alpha_j := q_j^\top w_j$. *(One inner product; this i component along q_j.)*
3. Compute the residual $r_j := w_j - \alpha_j q_j - \beta_j q_{j-1}$. *(Subtract the q_j- and components.)*
4. Compute the off-diagonal coefficient $\beta_{j+1} := \|r_j\|$.
5. If $\beta_{j+1} = 0$, **stop** (lucky breakdown: invariant subspace found).
6. Otherwise normalize $q_{j+1} := r_j / \beta_{j+1}$.

After m steps, the basis is $Q_m = [q_0, q_1, \ldots, q_{m-1}]$ and the recurrence has pro $\alpha_0, \ldots, \alpha_{m-1}$ and $\beta_1, \ldots, \beta_{m-1}$. The matrix

$$T_m = \begin{pmatrix} \alpha_0 & \beta_1 & & & \\ \beta_1 & \alpha_1 & \beta_2 & & \\ & \beta_2 & \alpha_2 & \ddots & \\ & & \ddots & \ddots & \beta_{m-1} \\ & & & \beta_{m-1} & \alpha_{m-1} \end{pmatrix}$$

is exactly $Q_m^\top \widetilde{\mathcal{M}} Q_m$, i.e. the matrix representation of $\widetilde{\mathcal{M}}$ on the Krylov subspace

Step (Worked toy). Take $\widetilde{\mathcal{M}}$ from (5). Pick the simplest non-trivial seed

$$q_0 \;=\; (1, 0, 0, 0)^\top, \quad \|q_0\| = 1. \tag{76}$$

Initialize $\beta_0 = 0$, $q_{-1} = 0$.

Iteration $j = 0$.

Sub-step 0.1. Multiply: since q_0 is 1 at position 0 and 0 elsewhere, $\widetilde{\mathcal{M}} q_0$ is just the first column of $\widetilde{\mathcal{M}}$:

$$w_0 \;=\; \widetilde{\mathcal{M}}\, q_0 \;=\; \begin{pmatrix} 2 \\ -1 \\ 1 \\ 0 \end{pmatrix}. \tag{77}$$

Sub-step 0.2. Compute α_0:

$$\alpha_0 \;=\; q_0^\top w_0 \;=\; (1)(2)+(0)(-1)+(0)(1)+(0)(0) \;=\; 2. \tag{78}$$

Sub-step 0.3. Form the residual. With $\beta_0 = 0$ and $q_{-1} = 0$, the third subtraction is zero:

$$r_0 \;=\; w_0 - \alpha_0 q_0 \;=\; \begin{pmatrix} 2 \\ -1 \\ 1 \\ 0 \end{pmatrix} - 2 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \;=\; \begin{pmatrix} 0 \\ -1 \\ 1 \\ 0 \end{pmatrix}. \tag{79}$$

Sub-step 0.4. Norm: $\beta_1 = \sqrt{0+1+1+0} = \sqrt{2}$.

Sub-step 0.5. Normalize: $q_1 = r_0/\beta_1 = \frac{1}{\sqrt{2}}(0,-1,1,0)^\top$.

Iteration $j = 1$.

Sub-step 1.1. Multiply $\widetilde{\mathcal{M}}$ by $(0,-1,1,0)^\top$ component by component (read off rows of $\widetilde{\mathcal{M}}$):

$$\widetilde{\mathcal{M}} \begin{pmatrix} 0 \\ -1 \\ 1 \\ 0 \end{pmatrix} \;=\; \begin{pmatrix} 2(0)+(-1)(-1)+1(1)+0(0) \\ (-1)(0)+2(-1)+0(1)+1(0) \\ 1(0)+0(-1)+2(1)+(-1)(0) \\ 0(0)+1(-1)+(-1)(1)+2(0) \end{pmatrix} \;=\; \begin{pmatrix} 2 \\ -2 \\ 2 \\ -2 \end{pmatrix}. \tag{80}$$

Therefore $w_1 = \frac{1}{\sqrt{2}}(2,-2,2,-2)^\top$.

Sub-step 1.2. Compute α_1:

$$\alpha_1 \;=\; q_1^\top w_1 \;=\; \tfrac{1}{\sqrt{2}}(0,-1,1,0)\cdot\tfrac{1}{\sqrt{2}}(2,-2,2,-2) \;=\; \tfrac{1}{2}\,(0+2+2+0) \;=\; 2. \tag{81}$$

Sub-step 1.3. Compute the residual term by term:

$$\alpha_1 q_1 = 2\cdot\tfrac{1}{\sqrt{2}}(0,-1,1,0) = (0,-\sqrt{2},\sqrt{2},0), \tag{82}$$

$$\beta_1 q_0 = \sqrt{2}\,(1,0,0,0) = (\sqrt{2},0,0,0), \tag{83}$$

$$\begin{aligned} r_1 &= w_1 - \alpha_1 q_1 - \beta_1 q_0 \\ &= (\sqrt{2},-\sqrt{2},\sqrt{2},-\sqrt{2}) - (0,-\sqrt{2},\sqrt{2},0) - (\sqrt{2},0,0,0) \\ &= (\sqrt{2}-0-\sqrt{2},\; -\sqrt{2}+\sqrt{2}-0,\; \sqrt{2}-\sqrt{2}-0,\; -\sqrt{2}-0-0) \\ &= (0,0,0,-\sqrt{2}). \end{aligned} \tag{84}$$

Sub-step 1.4. Norm: $\beta_2 = \sqrt{0+0+0+2} = \sqrt{2}$.

Sub-step 1.5. Normalize: $q_2 = r_1/\beta_2 = (0,0,0,-1)^\top$.

Iteration $j = 2$.

Sub-step 2.1. Multiply: q_2 is -1 at position 3 and 0 elsewhere, so $\widetilde{\mathcal{M}} q_2$ is the negative of the fourth column of $\widetilde{\mathcal{M}}$:

$$w_2 \;=\; -\begin{pmatrix} 0 \\ 1 \\ -1 \\ 2 \end{pmatrix} \;=\; \begin{pmatrix} 0 \\ -1 \\ 1 \\ -2 \end{pmatrix}. \tag{85}$$

Sub-step 2.2. Compute α_2:

$$\alpha_2 \;=\; q_2^\top w_2 \;=\; (0)(0)+(0)(-1)+(0)(1)+(-1)(-2) \;=\; 2. \tag{86}$$

Sub-step 2.3. Compute the residual:

$$\alpha_2 q_2 = 2\,(0,0,0,-1) = (0,0,0,-2), \tag{87}$$

$$\beta_2 q_1 = \sqrt{2}\cdot\tfrac{1}{\sqrt{2}}(0,-1,1,0) = (0,-1,1,0), \tag{88}$$

$$r_2 = (0,-1,1,-2) - (0,0,0,-2) - (0,-1,1,0) = (0,0,0,0). \tag{89}$$

Sub-step 2.4. Norm: $\beta_3 = 0$.

Lucky breakdown: the residual vanished, so we have found an invariant 3-dimensional subspace of $\widetilde{\mathcal{M}}$ exactly. Stop with $m = 3$.

Step (Tridiagonal T_3). Collect the recurrence coefficients:

$$\alpha_0 = \alpha_1 = \alpha_2 = 2, \quad \beta_1 = \beta_2 = \sqrt{2}. \tag{90}$$

Form the tridiagonal matrix:

$$T_3 \;=\; \begin{pmatrix} 2 & \sqrt{2} & 0 \\ \sqrt{2} & 2 & \sqrt{2} \\ 0 & \sqrt{2} & 2 \end{pmatrix}. \tag{91}$$

This is by construction $Q_3^\top \widetilde{\mathcal{M}} Q_3$ – $\widetilde{\mathcal{M}}$ projected onto the Krylov subspace, written in the orthonormal basis $\{q_0, q_1, q_2\}$.

Diagonalize. Substitute $\mu = \lambda - 2$:

$$T_3 - 2I \;=\; \sqrt{2}\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}. \tag{92}$$

The matrix on the right is the adjacency matrix of a 3-vertex path (or, equivalently, twice the discrete Laplacian on three points). Its characteristic polynomial is $-\mu^3/\sqrt{2} + 2\mu/\sqrt{2} = 0$ once we factor out $\sqrt{2}$, giving roots $\mu/\sqrt{2} \in \{-\sqrt{2}, 0, \sqrt{2}\}$, i.e. $\mu \in \{-2, 0, +2\}$. So

$$\boxed{\text{eigenvalues of } T_3 \;=\; \{0,\ 2,\ 4\}.} \tag{93}$$

These are the three *Ritz values*. Compare with the full spectrum of $\widetilde{\mathcal{M}}$: $\{0, 2, 2, 4\}$. Lanczos with this seed returned the smallest, the largest, and one copy of the double eigenvalue. The second copy is missed because the seed $q_0 = (1,0,0,0)^\top$ happened to lie inside an invariant 3D subspace orthogonal to the missed eigenvector. With a generic random seed this happens with probability zero.

Step (Ritz vector for $\lambda = 0$). We want the eigenvector $\widehat{\chi} \in \mathbb{R}^4$ of $\widetilde{\mathcal{M}}$ with eigenvalue 0. The recipe: solve the small eigenproblem $T_3\, y = 0 \cdot y$ for $y \in \mathbb{R}^3$, then *lift* via $Q_3 y$ to $\mathbb{R}^4$.

The eigenvector of T_3 for $\mu = -2$ (i.e. $\lambda = 0$) satisfies $(T_3 - 0\cdot I)\, y = 0$, equivalently $\sqrt{2}\,(y_2, y_1 + y_3, y_2)^\top + 2(y_1, y_2, y_3)^\top = 0$:

$$\begin{cases} 2y_1 + \sqrt{2}\, y_2 = 0 \\ \sqrt{2}\, y_1 + 2y_2 + \sqrt{2}\, y_3 = 0 \\ \sqrt{2}\, y_2 + 2y_3 = 0 \end{cases} \tag{94}$$

First and third equations give $y_2 = -\sqrt{2}\,y_1 = -\sqrt{2}\,y_3$, hence $y_1 = y_3$. Pick $y_1 = y_3 = \frac{1}{2}$ for unit norm; then $y_2 = -\frac{\sqrt{2}}{2}$. Verify $\|y\| = \sqrt{1/4 + 1/2 + 1/4} = 1$.

Now lift to $\mathbb{R}^4$:

$$\begin{aligned}\widehat{\chi} = Q_3\, y \;&=\; y_1 q_0 + y_2 q_1 + y_3 q_2 \\ &= \tfrac{1}{2}(1,0,0,0) - \tfrac{\sqrt{2}}{2} \cdot \tfrac{1}{\sqrt{2}}(0,-1,1,0) + \tfrac{1}{2}(0,0,0,-1) \\ &= \tfrac{1}{2}(1,0,0,0) + \tfrac{1}{2}(0,1,-1,0) - \tfrac{1}{2}(0,0,0,1) \\ &= \tfrac{1}{2}(1,1,-1,-1)^\top. \end{aligned} \tag{95}$$

This is exactly the χ of (7). We have recovered the offending zero-mode eigenvector in three matrix-vector products, with no inversion of $\widetilde{\mathcal{M}}$ and no full 4×4 eigendecomposition.

Step (Code). The same recurrence runs on the production 4000×4000 $\widetilde{\mathcal{M}}$. After $m \sim 30$ steps – not 4000 – the small T_{30} has Ritz values that approximate the few smallest eigenvalues of $\widetilde{\mathcal{M}}$ to a relative tolerance of 10^{-9}. The single line

```
eigvals, eigvecs = eigsh(M_tilde, k=1, which='SA',
                         tol=1e-9, maxiter=20000)
```

runs the recurrence above on the production $\widetilde{\mathcal{M}}$, asks ARPACK for the single ($k = 1$) smallest-algebraic Ritz value with relative tolerance 10^{-9}, and caps the work at 20000 matrix-vector products. The returned `eigvecs[:,0]` is the Ritz vector approximating χ. ARPACK adds one piece of machinery on top – "implicit restarts" – that periodically discards unconverged Ritz directions to keep memory bounded; the core recurrence is exactly the one walked through above.

Lanczos in one paragraph. Start from a unit seed q_0. Iteratively multiply by $\widetilde{\mathcal{M}}$, orthogonalize against the last two basis vectors (which is enough by symmetry), and normalize. After m steps, the recurrence coefficients α_j (diagonal) and β_{j+1} (off-diagonal) form a small $m \times m$ tridiagonal matrix T_m whose eigenvalues approximate the extremal eigenvalues of $\widetilde{\mathcal{M}}$. Diagonalize T_m directly; lift the resulting eigenvectors back to $\mathbb{R}^N$ via $Q_m\, y$. We never form $\widetilde{\mathcal{M}}^{-1}$, never form the full eigendecomposition, and never use more than ~ 30 matrix-vector products.

1.16 Lanczos in greater depth: background concepts, and the two distinct `eigsh` calls

This subsection is for the reader who needs more foundation before Q5's recurrence "clicks". It does three things:

1. builds up every concept (eigenvalue, basis, orthonormality, span, subspace, projection, Krylov subspace, Ritz value/vector) with a tiny example for each, before any algorithm appears;

2. walks through the Lanczos recipe with the motivation *between* the steps, not just inside them;

3. shows in detail how the same Lanczos engine is used by *two distinct* code paths to find two different eigenvectors – the most-negative eigenvalue (Coleman, $n = 0$) and the closest-to-zero eigenvalue (translation, $n = 1$). These are the only two eigenvectors the whole pipeline ever needs.

Part A. The vocabulary, with mini-examples

Vector and matrix. A *vector* of length N is a column of N real numbers, e.g. $v = (1, -1, 1, -1)^\top \in \mathbb{R}^4$. A *matrix* of size $N \times N$ is a square array of N^2 real numbers, e.g. our toy $\widetilde{\mathcal{M}}$ from (5). "Multiplying $\widetilde{\mathcal{M}}$ by v" produces a new vector $\widetilde{\mathcal{M}}v$ whose i-th entry is $\sum_j \widetilde{\mathcal{M}}_{ij}\, v_j$.

Eigenvalue and eigenvector. A non-zero vector ξ is an *eigenvector* of $\widetilde{\mathcal{M}}$ with *eigenvalue* λ if

$$\widetilde{\mathcal{M}}\,\xi \;=\; \lambda\,\xi. \tag{96}$$

Geometrically, $\widetilde{\mathcal{M}}$ acts on ξ by stretching (and possibly flipping the sign), but *not rotating*. For the toy:

$$\widetilde{\mathcal{M}}\,\chi \;=\; 0\cdot\chi \quad \text{with} \quad \chi = \tfrac{1}{2}(1, 1, -1, -1)^\top \quad (\lambda = 0). \tag{97}$$

A symmetric $N \times N$ matrix has N orthonormal eigenvectors. Our 4×4 toy has four; the eigenvalue 0 corresponds to χ (the offending mode), the others have eigenvalues $2, 2, 4$.

Diagonalization. "Diagonalizing" $\widetilde{\mathcal{M}}$ means finding all N eigenvectors and eigenvalues. If we collect the eigenvectors as columns of a matrix $\Xi = [\xi_1, \xi_2, \ldots, \xi_N]$ and put the eigenvalues on the diagonal of $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_N)$, we get the factorization $\widetilde{\mathcal{M}} = \Xi\,\Lambda\,\Xi^\top$ (because Ξ is orthogonal). For $N = 4000$, computing the full factorization would cost $O(N^3) \approx 6 \times 10^{10}$ floating point operations – expensive, and overkill if we only want one eigenpair.

Basis and orthonormal basis. A *basis* of $\mathbb{R}^N$ is a set of N vectors $b_1, \ldots, b_N$ such that every vector in $\mathbb{R}^N$ is uniquely a linear combination of them. The standard basis is $e_1 = (1, 0, \ldots, 0)$, $e_2 = (0, 1, 0, \ldots)$, etc. A basis is *orthonormal* if each pair satisfies $b_i^\top b_j = \delta_{ij}$ (perpendicular, unit-length). Orthonormal bases are convenient because the coefficient $c_i = b_i^\top v$ is the orthogonal projection of v onto b_i – no matrix inversion needed to extract coefficients.

Span, subspace, dimension. Given m vectors $v_1, \ldots, v_m$ in $\mathbb{R}^N$, their *span* is the set of all linear combinations $\sum_i c_i v_i$. The span is a *subspace* of $\mathbb{R}^N$. If the v_i's are linearly independent, the span has dimension m. For $m < N$, the span is a strict subset of $\mathbb{R}^N$ – a flat "slice" through the origin.

Mini-example. In $\mathbb{R}^3$, the vectors $v_1 = (1, 0, 0)$ and $v_2 = (0, 1, 0)$ span the xy-plane. The vector $(2, 3, 0)$ is in this plane (write it as $2v_1 + 3v_2$); the vector $(0, 0, 1)$ is not.

Projection onto a subspace. Given an orthonormal basis $\{q_0, \ldots, q_{m-1}\}$ of a subspace $\mathcal{V} \subset \mathbb{R}^N$, the *orthogonal projection* of any vector $w \in \mathbb{R}^N$ onto $\mathcal{V}$ is

$$\mathrm{proj}_{\mathcal{V}}(w) \;=\; \sum_{i=0}^{m-1} \left(q_i^\top w\right) q_i. \tag{98}$$

The residual $w - \mathrm{proj}_{\mathcal{V}}(w)$ is orthogonal to $\mathcal{V}$. This is exactly the operation Gram–Schmidt performs.

Tridiagonal matrix. A symmetric tridiagonal matrix has non-zero entries only on the main diagonal and the two adjacent diagonals:

$$T_m = \begin{pmatrix} \alpha_0 & \beta_1 & 0 & \cdots & 0 \\ \beta_1 & \alpha_1 & \beta_2 & & \vdots \\ 0 & \beta_2 & \alpha_2 & \ddots & 0 \\ \vdots & & \ddots & \ddots & \beta_{m-1} \\ 0 & \cdots & 0 & \beta_{m-1} & \alpha_{m-1} \end{pmatrix}. \tag{99}$$

Tridiagonal matrices have a special property: their eigenvalues can be computed in $O(m^2)$ time (versus $O(m^3)$ for a general dense $m \times m$ matrix). LAPACK has dedicated routines (e.g. `dstebz`, `dstein`) for this. For $m \sim 30$, this is essentially free.

Part B. The Krylov subspace, with picture

Given a starting unit vector $q_0 \in \mathbb{R}^N$, the *Krylov sequence* of length m is

$$\{\, q_0,\, \widetilde{\mathcal{M}} q_0,\, \widetilde{\mathcal{M}}^2 q_0,\, \ldots,\, \widetilde{\mathcal{M}}^{m-1} q_0 \,\}. \tag{100}$$

Their span is the *Krylov subspace* $\mathcal{K}_m(\widetilde{\mathcal{M}}, q_0) \subset \mathbb{R}^N$, of dimension at most m. There are two complementary intuitions for what this subspace "contains":

- **Power-iteration intuition** (Q4): each application of $\widetilde{\mathcal{M}}$ stretches the components along the largest-eigenvalue direction more than the others, so the later vectors in the sequence point increasingly toward the dominant eigenvector. The Krylov subspace therefore contains a lot of the largest-eigenvalue direction.
- **Memory intuition**: the early vectors q_0, $\widetilde{\mathcal{M}} q_0$ also retain non-trivial components along the *other* eigenvectors. So the Krylov subspace also has small but nonzero overlap with eigenvectors of the smallest, second-largest, etc. eigenvalues. Diagonalizing $\widetilde{\mathcal{M}}$ within $\mathcal{K}_m$ is enough to recover *both* extremes accurately.

The Krylov subspace is the natural arena for a "one-shot diagonalization that returns the few most extreme eigenvalues quickly". Lanczos is the algorithm that builds an orthonormal basis for $\mathcal{K}_m$ and produces the representation of $\widetilde{\mathcal{M}}$ in that basis as a tridiagonal T_m.

Part C. Why we cannot use the raw Krylov vectors directly, and how Gram–Schmidt fixes it

The raw Krylov vectors $q_0, \widetilde{\mathcal{M}} q_0, \widetilde{\mathcal{M}}^2 q_0, \ldots$ have two big problems:

1. They are *not orthogonal*. Without orthogonality, every projection step requires a matrix inversion to extract coefficients – expensive and numerically unstable.
2. They become *nearly parallel* as $j \to \infty$, because they all bend toward the dominant eigenvector. Numerically, two vectors that differ only by floating-point noise are useless.

The fix is Gram–Schmidt: at each step, subtract the projection of the new vector onto every previous basis vector, producing a residual that is orthogonal to all previous ones, then normalize. The result is an orthonormal basis $q_0, q_1, \ldots$ spanning the same subspace.

For a general (non-symmetric) matrix this gives the *Arnoldi* iteration, in which each new vector requires $j+1$ inner products and subtractions at step j; total cost $O(m^2)$.

For a symmetric matrix – which our $\widetilde{\mathcal{M}}$ is – there is a miraculous simplification: the inner product $q_i^\top \widetilde{\mathcal{M}} q_j$ vanishes for $|i-j| \geq 2$. This means the naive Gram–Schmidt subtraction collapses to subtracting only the q_j and q_{j-1} components. We have to do *two* inner products per step, not $j+1$. Total cost $O(m)$. *This is the Lanczos algorithm.*

Part D. The Lanczos recurrence (with reasons between the steps)

The recipe is the same one boxed in Q5; here we annotate each step with what it accomplishes and why it is structured that way.

Setup. We are given a starting unit vector q_0 (typically random, with unit norm). We initialize $\beta_0 := 0$ and $q_{-1} := 0$ (a phantom vector that drops out at $j = 0$).

Iteration j.

(D.1) $w_j := \widetilde{\mathcal{M}}\, q_j$.

This is the *only* use of $\widetilde{\mathcal{M}}$ in the entire iteration, and it is sparse-matrix-times-vector: about $5\,N$ flops for our banded matrix, not the N^2 of a dense multiply. This is what makes Lanczos cheap on large sparse matrices.

(D.2) $\alpha_j := q_j^\top w_j$.

The number α_j measures how much of w_j already lies along q_j. This will become the j-th diagonal entry of T_m. Equivalently: $\alpha_j = q_j^\top \widetilde{\mathcal{M}} q_j$, the "Rayleigh quotient" of q_j – a scalar approximation to an eigenvalue of $\widetilde{\mathcal{M}}$ in the direction of q_j.

(D.3) $r_j := w_j - \alpha_j q_j - \beta_j q_{j-1}$.

This is the orthogonalization step. We subtract the q_j-component ($\alpha_j q_j$) and the q_{j-1}-component ($\beta_j q_{j-1}$). For a symmetric $\widetilde{\mathcal{M}}$, the components along $q_0, q_1, \ldots, q_{j-2}$ are mathematically zero already (Q5 "magic"), so we don't subtract them.

Why β_j specifically? Because of how β_{j+1} is defined in step D.4 below: $\beta_{j+1} = \|r_j\|$. Recursively, this is the same as $q_{j-1}^\top \widetilde{\mathcal{M}} q_j$ by the following argument. We have $\widetilde{\mathcal{M}} q_{j-1} = \alpha_{j-1} q_{j-1} + \beta_{j-1} q_{j-2} + \beta_j q_j$ from the previous step. Pre-multiply by $q_j^\top$: orthogonality gives $q_j^\top \widetilde{\mathcal{M}} q_{j-1} = \beta_j$. By symmetry, $q_{j-1}^\top \widetilde{\mathcal{M}} q_j = \beta_j$ as well. So the coefficient to subtract for the q_{j-1}-component is precisely β_j, the same number that appeared as the off-diagonal of T_m.

(D.4) $\beta_{j+1} := \|r_j\|$.

The norm of the residual measures how much w_j stuck out of $\mathrm{span}\{q_{j-1}, q_j\}$. This is the next off-diagonal entry of T_m. If $\beta_{j+1} = 0$, the residual is zero and w_j already lies entirely in the span of the previous basis vectors. The Krylov sequence has stopped growing – we have found an exactly $\widetilde{\mathcal{M}}$-invariant subspace, and we stop.

(D.5) $q_{j+1} := r_j / \beta_{j+1}$.

Normalize so that the new basis vector has unit length, completing the orthonormality requirement.

After m iterations we have collected the diagonal entries $\alpha_0, \alpha_1, \ldots, \alpha_{m-1}$ and the off-diagonals $\beta_1, \beta_2, \ldots, \beta_{m-1}$, defining

$$T_m \;=\; \begin{pmatrix} \alpha_0 & \beta_1 & & & \\ \beta_1 & \alpha_1 & \beta_2 & & \\ & \beta_2 & \alpha_2 & \ddots & \\ & & \ddots & \ddots & \beta_{m-1} \\ & & & \beta_{m-1} & \alpha_{m-1} \end{pmatrix}, \quad Q_m \;=\; [q_0, q_1, \ldots, q_{m-1}]. \tag{101}$$

T_m is the matrix of $\widetilde{\mathcal{M}}$ on the Krylov subspace $\mathcal{K}_m$ in the orthonormal basis Q_m. The mathematical identity is

$$T_m \;=\; Q_m^\top \, \widetilde{\mathcal{M}} \, Q_m. \tag{102}$$

Part E. From T_m to eigenvalues of $\widetilde{\mathcal{M}}$ (Ritz values and vectors)

Diagonalize T_m (cheap, since $m \sim 30$):

$$T_m \, y^{(k)} = \widetilde{\lambda}^{(k)} \, y^{(k)}, \quad k = 0, 1, \ldots, m-1. \tag{103}$$

The numbers $\widetilde{\lambda}^{(k)}$ are called *Ritz values*. They are exact eigenvalues of $\widetilde{\mathcal{M}}$ *restricted to the Krylov subspace*. Standard convergence theory (which we will not prove) shows:

- Extreme Ritz values converge *rapidly* to the extreme eigenvalues of $\widetilde{\mathcal{M}}$. After about $m \approx 20$ steps, the single smallest Ritz value differs from the true smallest eigenvalue by less than 10^{-10}.
- Interior Ritz values (those approximating non-extreme eigenvalues) converge much more slowly. Lanczos is therefore best suited for finding extreme eigenvalues, not interior ones.

The corresponding *Ritz vectors* are obtained by lifting the small eigenvectors $y^{(k)} \in \mathbb{R}^m$ back to $\mathbb{R}^N$ via the orthonormal basis Q_m:

$$\widetilde{\xi}^{(k)} \;=\; Q_m \, y^{(k)} \;=\; \sum_{j=0}^{m-1} y_j^{(k)} \, q_j \;\in\; \mathbb{R}^N. \tag{104}$$

For sufficiently large m, $\widetilde{\xi}^{(k)}$ is essentially exact as an eigenvector of $\widetilde{\mathcal{M}}$, with the same eigenvalue $\widetilde{\lambda}^{(k)} \approx \lambda_k(\widetilde{\mathcal{M}})$. *This is how a single eigenvector χ is extracted from the full $\widetilde{\mathcal{M}}$.*

Part F. The two distinct `eigsh` calls

The pipeline calls Lanczos twice – once for $n = 0$ and once for $n = 1$ – but the goals are different. We need to distinguish them clearly.

Call A: `find_negative_mode` – finding $\chi_{\mathbf{neg}}$ (Coleman, $n = 0$). Goal: the most-negative eigenvalue of $\widetilde{\mathcal{M}}_{n=0}$ ("smallest algebraic"). For the bounce problem this is unique and strictly negative, so it is also the eigenvalue of largest absolute value among the negative eigenvalues – and on a wide margin, among all eigenvalues smaller than ~ 1.

Strategy: pass `which='SA'` (smallest algebraic) to `eigsh`. ARPACK runs Lanczos and orders the Ritz values algebraically; convergence focuses on the leftmost end of the spectrum. Once converged, ARPACK returns $(\widetilde{\lambda}^{(0)}, \widetilde{\xi}^{(0)})$ with $\widetilde{\lambda}^{(0)} = \lambda_{\text{neg}} < 0$ and $\widetilde{\xi}^{(0)} = \chi_{\text{neg}}$.

Why "SA" works without shift-invert: the Coleman eigenvalue is well-isolated from the rest of the spectrum (there is a gap of $O(1)$ between $\lambda_{\text{neg}} < 0$ and the next eigenvalue $\lambda_1 \geq 0$). Lanczos converges fastest at isolated extreme eigenvalues, so plain `which='SA'` is already efficient.

Code path:

```
def find_negative_mode(M_tilde, verbose=True, n_eig=3):
    eigvals, eigvecs = eigsh(M_tilde, k=n_eig, which='SA',
                             tol=1e-9, maxiter=20000)
    order = np.argsort(eigvals)
    eigvals = eigvals[order]
    eigvecs = eigvecs[:, order]
    lambda_low = float(eigvals[0])
    chi_low = eigvecs[:, 0]
    chi_low = chi_low / np.linalg.norm(chi_low)
    return lambda_low, chi_low
```

Reading line by line:

- `k=n_eig`: ask for 3 eigenvalues, not 1. (Cheap insurance: it lets us print the gap between the smallest and the next two for diagnostics.)
- `which='SA'`: smallest algebraic.
- `tol=1e-9`: relative residual tolerance.
- `maxiter=20000`: hard cap on the number of inner matrix-vector multiplications.
- `np.argsort(eigvals)`: sort the returned 3 eigenvalues in ascending order. Take the first one.
- Normalize the eigenvector to unit norm.

Call B: `find_discrete_zero_mode` – finding $\chi_{\mathbf{zm}}$ (translation, $n = 1$). Goal: the eigenvalue *closest to zero* (in absolute value). The continuum theory predicts an exact zero eigenvalue from translation symmetry; on the discrete grid the value sits at a small $\lambda_{\text{zm}} \approx O(\mathrm{d}r^2)$ which can be positive or negative depending on discretization details.

Why we cannot just use "SA": $\widetilde{\mathcal{M}}_{n=1}$ might also have a slightly negative spurious eigenvalue (e.g. from discretization noise) that is more negative than λ_{zm}. "Smallest algebraic" would return that spurious mode – not the translation mode. We need to target *absolute value closest to zero*, not algebraically smallest.

Strategy: shift-invert mode. Set $\sigma = 0$. ARPACK internally factorizes $\widetilde{\mathcal{M}} - \sigma I$ once (LU) and then runs Lanczos on the operator $(\widetilde{\mathcal{M}} - \sigma I)^{-1}$ instead of on $\widetilde{\mathcal{M}}$ directly. The eigenvalues of $(\widetilde{\mathcal{M}} - \sigma I)^{-1}$ are $\nu_\alpha = 1/(\lambda_\alpha - \sigma)$. The *largest-magnitude* ν corresponds to the *closest-to-σ* λ. So setting $\sigma = 0$ and asking for "largest magnitude" (`which='LM'`) on the inverse operator returns the eigenpairs of $\widetilde{\mathcal{M}}$ whose eigenvalues are closest to zero.

Why we set `n_eig=5` and pick the closest in code: ARPACK returns the 5 Ritz values closest to $\sigma = 0$, in some order. We then pick the one with smallest $|\lambda|$ explicitly in Python, robust to any ordering ambiguity inside ARPACK.

Code path:

```python
def find_discrete_zero_mode(M_tilde, verbose=True, n_eig=5,
                            use_shift_invert=True):
    eigvals, eigvecs = eigsh(M_tilde, k=n_eig, sigma=0.0,
                             which='LM', tol=1e-9, maxiter=20000)
    idx_min = int(np.argmin(np.abs(eigvals)))
    lambda_zm = float(eigvals[idx_min])
    chi_zm = eigvecs[:, idx_min]
    chi_zm = chi_zm / np.linalg.norm(chi_zm)
    return lambda_zm, chi_zm
```

Reading line by line:

- `sigma=0.0`: target zero on the original spectrum.
- `which='LM'`: "largest magnitude" *on the inverse*. Combined with `sigma=0`, this means "λ closest to 0 on $\widetilde{\mathcal{M}}$".
- `k=n_eig=5`: return the 5 closest eigenpairs.
- `np.argmin(np.abs(eigvals))`: explicit pick of the closest-to-zero, regardless of sign or returned order.
- Normalize the chosen eigenvector to unit norm.

Side-by-side comparison.

Aspect	**Call A: $\chi_{\mathbf{neg}}$ ($n = 0$)**	**Call B: $\chi_{\mathbf{zm}}$ ($n = 1$)**
What we want	most-negative eigenvalue	eigenvalue closest to 0 in absolute value
ARPACK flag	`which='SA'`	`sigma=0.0`, `which='LM'` (shift-invert)
What Lanczos runs on	$\widetilde{\mathcal{M}}$ directly	$(\widetilde{\mathcal{M}} - 0 \cdot I)^{-1}$ via internal LU
Ritz values returned	algebraically smallest of $\widetilde{\mathcal{M}}$	largest-magnitude of $\widetilde{\mathcal{M}}^{-1}$ = closest-to-0 of $\widetilde{\mathcal{M}}$
Number of eigenpairs requested (`k`)	3 (with diagnostic margin)	5 (then pick closest-to-0 in Python)
Why this strategy is safe	Coleman mode is well-isolated below the rest of the spectrum	shift-invert directly targets $\lambda \approx 0$, robust to spurious negatives
Code function	`find_negative_mode`	`find_discrete_zero_mode`
Returned variable used downstream	`chi_neg` (then `chi` for projector P)	`chi_zm` (then `chi` for projector P)

What happens after either call. Both calls produce a unit eigenvector χ (and its eigenvalue, which we keep for diagnostic purposes only – it is not actually used in the projector itself). We then:

1. Clamp χ[boundary indices] $= 0$ to remove any ghost-mode noise leaked by ARPACK.
2. Renormalize: $\chi \leftarrow \chi/\|\chi\|$.
3. Use χ to build the projector $P = I - \chi\chi^\top$, which is applied to every Hutchinson probe in the inner loop (§1.11).

After this, the Lanczos machinery is no longer touched during the s^2 scan. It is a one-time cost of fractions of a second, paid once per n.

Lanczos in two paragraphs.
First paragraph (the algorithm). Lanczos builds an orthonormal basis $q_0, q_1, \ldots, q_{m-1}$ of the Krylov subspace $\mathcal{K}_m(\widetilde{\mathcal{M}}, q_0)$ via a three-term recurrence (cheap because $\widetilde{\mathcal{M}}$ is symmetric). The recurrence coefficients form a small tridiagonal matrix T_m that represents $\widetilde{\mathcal{M}}$ restricted to $\mathcal{K}_m$. The eigenvalues of T_m ("Ritz values") approximate the extremal eigenvalues of $\widetilde{\mathcal{M}}$; the corresponding eigenvectors of T_m, lifted via Q_m, approximate the extremal eigenvectors. We never form $\widetilde{\mathcal{M}}^{-1}$ on the original operator and never diagonalize the full 4000×4000 matrix.
Second paragraph (the two calls). For $n = 0$ we want the most-negative eigenvalue, so we call `eigsh` with `which='SA'` (smallest algebraic) on $\widetilde{\mathcal{M}}$ directly: Lanczos converges quickly to the leftmost end of the spectrum because the Coleman mode is well-isolated. For $n = 1$ we want the eigenvalue closest to zero in absolute value, so we call `eigsh` with `sigma=0.0` and `which='LM'` (shift-invert mode): ARPACK internally LU-factorizes $\widetilde{\mathcal{M}}$ and runs Lanczos on the inverse operator $\widetilde{\mathcal{M}}^{-1}$, whose largest-magnitude eigenvalue corresponds to the closest-to-zero eigenvalue of $\widetilde{\mathcal{M}}$. The two calls share the same Lanczos engine; only the spectral target changes.

1.17 Where to look in the rest of the document

The walkthrough above is enough to read and run the code. For the underlying derivations:

- Spectral identity $\overline{G} = \mathrm{Tr}(A^{-1}) = \sum_\alpha 1/(\lambda_\alpha + s^2)$: §9.2.3.
- Why Liouville: §4.
- Why Dirichlet, why the symmetric trick is correct, and the ghost-eigenvalue analysis: §6.6 and surrounding subsections.
- Why Hutchinson with Rademacher probes works (with three different intuitive perspectives): §9.2.4 and §9.2.5.
- LU decomposition explained from scratch with worked 4×4 examples (different from the one above): §9.2.1 and §9.2.2.
- Lanczos algorithm in detail (steps 1–8 with full motivation): §5.4.1.

For accuracy levers (how to make the result more precise): Part III, Levers 1–7. The pipeline as written here is the production "v2" configuration ($K = 400$, eigsh-derived χ, sparse LU).

Part II
Derivation from first principles

2 Setting up the problem

2.1 Euclidean theory and the bounce

We consider two real scalar fields $\phi = (\phi_1, \phi_2)$ in 4D Euclidean space with action

$$S_E[\phi] \;=\; \int \mathrm{d}^4x \left[\tfrac{1}{2}\,(\partial_\mu\phi_i)(\partial_\mu\phi_i) + U(\phi)\right], \qquad i = 1, 2, \tag{105}$$

and a potential U admitting a metastable false vacuum and a stable true vacuum. The decay rate per unit volume in the semiclassical approximation is

$$\frac{\Gamma}{V} \;=\; \mathcal{A}\, e^{-S_E[\phi_b]/\hbar}, \tag{106}$$

where $\phi_b(r)$ is the O(4)-symmetric *bounce* solution (an extremum of S_E saturating the appropriate boundary conditions) and the prefactor $\mathcal{A}$ contains the one-loop fluctuation determinant ratio

$$\mathcal{A} \;\propto\; \left|\frac{\det\{\}'\big(\mathcal{M}[\phi_b]\big)}{\det\big(\mathcal{M}[\phi_{\rm fv}]\big)}\right|^{-1/2}, \tag{107}$$

with the prime indicating that the negative mode (one) and the translation zero modes (four) are removed from the determinant.

2.2 The fluctuation operator

Expand $\phi = \phi_b + \eta$ in (105) and keep terms quadratic in η. The result is

$$S_E^{(2)}[\eta] \;=\; \tfrac{1}{2}\int \mathrm{d}^4x\, \eta_i(x)\, \mathcal{M}_{ij}(x)\, \eta_j(x), \tag{108}$$

with the fluctuation operator (a symmetric 2×2 matrix-valued differential operator)

$$\mathcal{M}_{ij}(x) \;=\; -\delta_{ij}\,\partial_\mu\partial_\mu \;+\; U''_{ij}\big(\phi_b(r)\big), \qquad U''_{ij}(\phi) = \frac{\partial^2 U}{\partial\phi_i\,\partial\phi_j}. \tag{109}$$

Because ϕ_b depends only on $r = |x|$, $\mathcal{M}$ commutes with the SO(4) generators, and we can decompose into partial waves (next subsection).

2.3 The object we want: $\overline{G}_n(s^2)$

Most one-loop methods (proper-time, dispersion-integral, GF^2, heat-kernel, etc.) produce $\log\det'\mathcal{M}$ as an integral over a spectral parameter s^2 of a certain trace. The unifying object is the diagonal of the resolvent of the shifted operator

$$G(x, y; s^2) \;=\; \big[\mathcal{M} + s^2\,\mathbb{I}\big]^{-1}(x, y) \;=\; \sum_\gamma \frac{\Psi_\gamma(x)\,\Psi_\gamma^\top(y)}{\omega_\gamma^2 + s^2}, \tag{110}$$

where $\gamma = (L, n, \{m\})$ is the full quantum-number label and $\{\Psi_\gamma, \omega_\gamma^2\}$ is a complete spectral basis of $\mathcal{M}$ on $\mathbb{R}^4$. We work with the partial-wave-resolved trace

$$\boxed{\overline{G}_n(s^2) \;=\; (n+1)^2 \int_0^\infty \mathrm{d}r\, r^3\, \mathrm{tr}_{\mathrm{field}}\big[G_n(r,r;s^2)\big].} \tag{111}$$

Here G_n is the radial Green's function in partial wave n (defined explicitly below), $\mathrm{tr}_{\mathrm{field}}$ is the 2×2 field-index trace, and $(n+1)^2$ is the dimension of the irreducible SO(4) representation of principal quantum number n.

2.4 Notation conventions used throughout this document

The full set of indices appears repeatedly; defining them all here so later sections don't have to.

- $i, j \in \{1, 2\}$ – field-component indices (we have two scalars ϕ_1, ϕ_2).
- $n = 0, 1, 2, \ldots$ – principal O(4) angular-momentum quantum number, labelling partial waves on S^3.
- L – orbital quantum number labelling discrete radial eigenvalues *within* a fixed partial-wave-n sector. (Some textbooks call this ℓ.)
- $\{m\}$ – magnetic / hyperspherical-azimuthal quantum numbers. The angular eigenfunctions on S^3 at fixed principal n form one irreducible SO(4) representation of total dimension $(n+1)^2$ (this is the only degeneracy that enters $\overline{G}_n$). One textbook decomposition writes this multiplet as a sum of SO(3) sub-multiplets of dimension $(2L+1)$, giving $\sum_L (2L+1) = (n+1)^2$; we do not need that decomposition for anything in this document, only the total $(n+1)^2$.
- $\gamma \;\equiv\; (L, n, \{m\})$ – the full quantum-number label of a 4D eigenmode.
- $\omega_{L,n}^2 \;=\; \omega_\gamma^2$ – the eigenvalues (s^2-independent).
- $\phi_{L,n,i}(r)$ – raw-basis radial mode, ith field component; $\tilde{\phi}_{L,n,i}(r) = r^{3/2}\phi_{L,n,i}(r)$ – tilde-basis version.

Compact aliases used in equations. To keep formulas readable I sometimes write

$$\chi_L \;\equiv\; \tilde{\phi}_{L,n}, \qquad \lambda_L \;\equiv\; \omega_{L,n}^2, \tag{112}$$

with L enumerating eigenmodes within fixed n. So the spectral trace $\mathrm{Tr}[(\widetilde{\mathcal{M}}_n + s^2)^{-1}] = \sum_L (\lambda_L + s^2)^{-1} = \sum_L (\omega_{L,n}^2 + s^2)^{-1}$ – the same object that appears in standard radial Sturm–Liouville treatments.

3 Reducing to a 1D radial problem

3.1 Spherical harmonics on S^3

On $S^3 \subset \mathbb{R}^4$ the Laplacian has eigenvalues $-n(n+2)/r^2$ with degeneracy $(n+1)^2$ (these are the $\mathcal{Y}^{n,k}$ harmonics, $k = 1, \ldots, (n+1)^2$). Writing the Laplacian in 4D radial coordinates,

$$\partial_\mu \partial_\mu \;=\; \partial_r^2 + \frac{3}{r}\,\partial_r + \frac{1}{r^2}\Delta_{S^3}, \tag{113}$$

and acting on $\eta_i(x) = u_i^{n,k}(r)\,\mathcal{Y}^{n,k}(\hat{x})$, the radial part of $\mathcal{M}$ becomes

$$\mathcal{M}_n^{ij}(r) \;=\; \delta^{ij}\left[-\partial_r^2 - \frac{3}{r}\,\partial_r + \frac{n(n+2)}{r^2}\right] \;+\; U''_{ij}(\phi_b(r)). \tag{114}$$

The radial Green's function $G_n(r, r'; s^2)$ is the kernel of $(\mathcal{M}_n + s^2)^{-1}$ in the inner product $\langle u, v\rangle_{r^3} = \sum_i \int_0^\infty u_i(r)\, v_i(r)\, r^3\,\mathrm{d}r$ (the natural one inherited from the 4D measure $\mathrm{d}^4x \to r^3\,\mathrm{d}r\,\mathrm{d}\Omega_3$, with summation over field index i).

3.2 Pole structure of $\overline{G}_n$

Using the spectral decomposition (110) restricted to partial wave n,

$$\overline{G}_n(s^2) \;=\; (n+1)^2 \sum_L \frac{1}{\omega_{L,n}^2 + s^2}. \tag{115}$$

Two facts will drive the rest of the derivation:

- In the $n = 0$ sector, $\mathcal{M}_0$ has *exactly one* negative eigenvalue $\lambda_{\mathrm{neg}} < 0$ (the Coleman mode). So $\overline{G}_0$ has a simple pole at $s^2 = -\lambda_{\mathrm{neg}} > 0$.
- In the $n = 1$ sector, translation invariance gives 4 zero modes ($\lambda_{\mathrm{zm}}^{(1)} = 0$), and so $\overline{G}_1$ has a simple pole at $s^2 = 0$. The 4-fold degeneracy is exactly the $(n+1)^2 = 4$ factor in (115) for $n = 1$.

Both poles are integrable (their residues are physical) but they make the s^2-integral that produces $\log\det'\mathcal{M}$ ill-defined unless removed – hence "sub" below.

4 Liouville transformation (very detailed)

The form (114) has a first-derivative term and an awkward $r^3\,\mathrm{d}r$ measure. The Liouville substitution removes both at once.

4.1 The substitution and its derivatives

Define

$$\tilde{u}(r) \;\equiv\; r^{3/2}\,u(r) \quad\Longleftrightarrow\quad u(r) \;=\; r^{-3/2}\,\tilde{u}(r). \tag{116}$$

We compute u' and u'' in terms of $\tilde{u}$. Direct differentiation gives

$$u'(r) \;=\; \frac{\mathrm{d}}{\mathrm{d}r}\big(r^{-3/2}\tilde{u}\big) \;=\; -\tfrac{3}{2}\,r^{-5/2}\,\tilde{u} + r^{-3/2}\,\tilde{u}', \tag{117}$$

$$\begin{aligned} u''(r) \;&=\; \frac{\mathrm{d}}{\mathrm{d}r}\Big[-\tfrac{3}{2}\,r^{-5/2}\tilde{u} + r^{-3/2}\tilde{u}'\Big] \\ &=\; -\tfrac{3}{2}\,(-\tfrac{5}{2})\,r^{-7/2}\tilde{u} \;-\; \tfrac{3}{2}\,r^{-5/2}\,\tilde{u}' \;-\; \tfrac{3}{2}\,r^{-5/2}\,\tilde{u}' \;+\; r^{-3/2}\,\tilde{u}'' \\ &=\; \tfrac{15}{4}\,r^{-7/2}\,\tilde{u} \;-\; 3\,r^{-5/2}\,\tilde{u}' \;+\; r^{-3/2}\,\tilde{u}''. \end{aligned} \tag{118}$$

4.2 Plug into the radial kinetic term

The radial kinetic operator is $-u'' - (3/r)u'$. Substituting (117) and (118):

$$\begin{aligned} -u'' - \tfrac{3}{r}u' \;&=\; -\Big[\tfrac{15}{4}\, r^{-7/2}\tilde{u} - 3\, r^{-5/2}\tilde{u}' + r^{-3/2}\tilde{u}''\Big] \\ &\quad - \tfrac{3}{r}\Big[-\tfrac{3}{2}\, r^{-5/2}\tilde{u} + r^{-3/2}\tilde{u}'\Big] \\ &=\; -\tfrac{15}{4}\, r^{-7/2}\tilde{u} + 3\, r^{-5/2}\tilde{u}' - r^{-3/2}\tilde{u}'' + \tfrac{9}{2}\, r^{-7/2}\tilde{u} - 3\, r^{-5/2}\tilde{u}'. \end{aligned} \tag{119}$$

The two $\tilde{u}'$ pieces cancel: $+3\, r^{-5/2}\tilde{u}' - 3\, r^{-5/2}\tilde{u}' = 0$. The two $\tilde{u}$ pieces combine: $-\frac{15}{4} + \frac{18}{4} = +\frac{3}{4}$. So

$$-u'' - \tfrac{3}{r}u' \;=\; -r^{-3/2}\,\tilde{u}'' \;+\; \tfrac{3}{4}\, r^{-7/2}\,\tilde{u} \;=\; r^{-3/2}\Big[-\tilde{u}'' + \frac{3/4}{r^2}\,\tilde{u}\Big]. \tag{120}$$

4.3 Including the centrifugal and potential terms

The remaining pieces of $\mathcal{M}_n$ acting on $u = r^{-3/2}\tilde{u}$ are multiplicative:

$$\frac{n(n+2)}{r^2}\, u + U''(\phi)\, u \;=\; r^{-3/2}\Big[\frac{n(n+2)}{r^2}\,\tilde{u} + U''(\phi)\,\tilde{u}\Big]. \tag{121}$$

Combine with (120) and pull the overall $r^{-3/2}$ factor:

$$\mathcal{M}_n\, u \;=\; r^{-3/2}\,\widetilde{\mathcal{M}}_n\,\tilde{u}, \qquad \boxed{\widetilde{\mathcal{M}}_n \;=\; -\partial_r^2 \;+\; \frac{n(n+2)+3/4}{r^2} \;+\; U''\big(\phi_b(r)\big).} \tag{122}$$

$\widetilde{\mathcal{M}}_n$ is a simple Schrödinger-type operator: no first-derivative term, and the radial measure for $\tilde{u}$ is the flat one (next subsection).

4.4 The measure becomes flat

Inner products on the u space are r^3-weighted. With $u = r^{-3/2}\tilde{u}$,

$$\begin{aligned} \langle u_1, u_2\rangle_{r^3} \;&=\; \int_0^\infty u_1(r)\, u_2(r)\, r^3\, \mathrm{d}r \\ &=\; \int_0^\infty r^{-3/2}\tilde{u}_1 \,\cdot\, r^{-3/2}\tilde{u}_2 \,\cdot\, r^3\, \mathrm{d}r \;=\; \int_0^\infty \tilde{u}_1(r)\, \tilde{u}_2(r)\, \mathrm{d}r. \end{aligned} \tag{123}$$

So $\widetilde{\mathcal{M}}_n$ is the natural symmetric operator on $L^2(0,\infty;\mathrm{d}r)$, which is what we want for finite-difference discretization.

4.5 Is the Liouville transformation strictly necessary?

Short answer: **no, mathematically it is not necessary – the trace gives the same number in either basis**. The transformation is purely a *numerical* convenience for the FD method. Let us prove this and then list why we still use it.

Step 1: equivalence at the level of the spectral sum. Within a fixed partial wave n, label the radial eigenmodes by the orbital quantum number L. The total degeneracy of the partial wave n is $(n+1)^2$ (the dimension of the SO(4) irreducible representation), and this is the only multiplicity that appears in $\overline{G}_n$. For the *raw* radial operator $\mathcal{M}_n$ in 4D, the physical eigenmodes $\phi_{L,n,i}(r)$ ($i = 1, 2$ field component) are normalized in the r^3-weighted inner product

$$\sum_i \int_0^\infty \mathrm{d}r\, r^3\, \phi_{L,n,i}(r)\, \phi_{L',n,i}(r) \;=\; \delta_{LL'}, \tag{124}$$

which is exactly the convention in your notes (page 21, $\sum_i \int dr\, r^3\, (\phi^{\pm}_{L,n,i})^* \phi^{\pm}_{L,n,i} = 1$). The resolvent's diagonal then satisfies

$$\int_0^\infty \mathrm{d}r\, r^3\, \mathrm{tr}_{\mathrm{field}}\big[G_n(r,r;s^2)\big] \;=\; \sum_L \int_0^\infty \mathrm{d}r\, r^3\, \frac{\sum_i \phi_{L,n,i}(r)^2}{\omega_{L,n}^2 + s^2} \;=\; \sum_L \frac{1}{\omega_{L,n}^2 + s^2}, \tag{125}$$

because the $r^3\mathrm{d}r$ integral collapses each $\sum_i \phi^2_{L,n,i}$ factor to 1 by (124).

In the *tilde* basis, $\tilde{\phi}_{L,n,i} = r^{3/2}\phi_{L,n,i}$, the normalization becomes flat

$$\sum_i \int_0^\infty \mathrm{d}r\, \tilde{\phi}_{L,n,i}(r)\, \tilde{\phi}_{L',n,i}(r) \;=\; \delta_{LL'}, \tag{126}$$

and the same trace reads

$$\int_0^\infty \mathrm{d}r\, \mathrm{tr}_{\mathrm{field}}\big[\widetilde{G}_n(r,r;s^2)\big] \;=\; \sum_L \frac{1}{\omega_{L,n}^2 + s^2}. \tag{127}$$

Equations (125) and (127) are *identical*. The Liouville transformation has shuffled r^3 between the measure and the kernel without changing the value of the integral.

Step 2: so why bother with $\tilde{u}$? For analytic work the choice of basis is immaterial – whichever you prefer. For *finite-difference numerics*, the tilde basis is strongly preferred for three reasons:

1. **The matrix is symmetric.** The raw radial operator $\mathcal{M}_n = -\partial_r^2 - (3/r)\partial_r + V$ contains a first-derivative term. Discretizing ∂_r with central finite differences produces an *antisymmetric* matrix: $D_{i,i+1} = +1/(2\mathrm{d}r)$, $D_{i,i-1} = -1/(2\mathrm{d}r)$. So the FD representation of $-\partial_r^2 - (3/r)\partial_r$ is not symmetric. The Liouville-transformed $\widetilde{\mathcal{M}}_n = -\partial_r^2 + V$ has no first-derivative term, and FD gives a manifestly symmetric tridiagonal matrix. Symmetry is what lets us:
   - use `eigsh` (ARPACK Lanczos) instead of the more expensive non-symmetric `eigs`;
   - rely on the spectral theorem to claim the eigenmodes are orthogonal – crucial for the projection mechanism in §4;
   - use sparse Cholesky-like factorizations (or symmetric LU) for the Hutchinson solves.
2. **The discrete inner product matches the continuum one without weighting.** In the raw basis, two FD vectors ϕ_i and ϕ'_i are "orthogonal in the physical sense" only if $\sum_i r_i^3\, \phi_i\, \phi'_i\, \mathrm{d}r = 0$. Linear-algebra routines do not know about the r^3 weight;

they would compute $\sum_i \phi_i \phi_i'$ and report the wrong inner product. We would have to manually weight every dot product, every Hutchinson sample, every projection, by $r^3\,\mathrm{d}r$ – an error-prone exercise. In the tilde basis the discrete dot product directly approximates $\int \tilde{u}_1 \tilde{u}_2\,\mathrm{d}r$, so standard linear-algebra operations *just work*.

3. **Eigenmodes are well-behaved at $r = 0$.** The raw $\phi_{L,n,i}(r) \sim r^L$ near the origin, so the small-r contribution to $\int r^3 \phi^2\,\mathrm{d}r$ comes from r^{2L+3}, which is finite but uses a singular weight. For $L = 0$ the integrand $r^3\,\phi_0^2$ goes to zero at the origin only because of the weight, not because the mode is small there. In the tilde basis $\tilde{\phi}_{L,n,i}(r) \sim r^{L+3/2}$, so the modes vanish at the origin all by themselves, and the discretization error near $r = 0$ is much better controlled.

Bottom line. The Liouville transformation does not change physics, and you could in principle do the entire FD computation with r^3-weighted inner products on the raw operator. We chose not to, because the bookkeeping overhead and the loss of matrix symmetry would dwarf any benefit. The tilde basis is the standard "Schrödinger form" that makes a Sturm-Liouville problem look like a quantum-mechanics problem on the half-line, where every standard sparse-linear-algebra tool applies without modification.

4.6 Green's function in the tilde basis (derived, not assumed)

The Liouville substitution (116) was derived for a single mode function u. For the Green's function $G_n(r, r'; s^2)$ – which is not itself a mode but the *integral kernel* of $(\widetilde{\mathcal{M}}_n + s^2)^{-1}$ – the corresponding transformation is *forced* by consistency. Let us derive it explicitly.

What u and v mean here (and a note on overloaded notation). In §3.1 – §3.4 the symbol $u(r)$ denoted a specific mode function: an eigenfunction of $\mathcal{M}_n$ that we transformed via $\tilde{u} = r^{3/2}\,u$. Below, $u(r)$ and $v(r)$ play a different role: they are *generic radial functions* (any element of the function space, not specifically modes), used as the input/output of the resolvent. The Green's function is defined as the kernel that maps an arbitrary source v to the corresponding response u; we never pick particular v's, we only need the relationship to hold for any v.

The Liouville substitution $\tilde{f} = r^{3/2}\,f$ is a *change of basis on the whole function space* – it applies equally to mode functions, sources, responses, or any other element. That is exactly why we can substitute it on both sides of the integral relation below to derive the kernel's transformation rule.

Step 1: the operator's defining equation. For arbitrary radial source $v_j(r')$ (any element of the r^3-weighted function space) and the corresponding response $u_i(r) = [(\mathcal{M}_n + s^2)^{-1}\,v]_i(r)$, the Green's function is *defined* by the "inversion" relation

$$u_i(r) \;=\; \int_0^\infty \mathrm{d}r'\; r'^3\; G_n^{ij}(r, r'; s^2)\; v_j(r'), \tag{128}$$

which says: "apply $(\mathcal{M}_n + s^2)^{-1}$ to the source v and you get the response u". The crucial point is that the integral over r' carries the r'^3 measure, because $(\mathcal{M}_n + s^2)$ acts on the r^3-weighted Hilbert space (cf. §2.1).

Step 2: change basis on both sides. Substitute $u_i = r^{-3/2}\tilde{u}_i$ on the left and $v_j = r'^{-3/2}\tilde{v}_j$ on the right of (128):

$$r^{-3/2}\,\tilde{u}_i(r) \;=\; \int_0^\infty \mathrm{d}r'\; r'^{3}\; G_n^{ij}(r,r';s^2)\; r'^{-3/2}\,\tilde{v}_j(r'). \tag{129}$$

Multiply through by $r^{3/2}$ on the left and combine the r' powers on the right ($r'^{3}\cdot r'^{-3/2} = r'^{3/2}$):

$$\tilde{u}_i(r) \;=\; \int_0^\infty \mathrm{d}r'\; \underbrace{(rr')^{3/2}\,G_n^{ij}(r,r';s^2)}_{\equiv\,\widetilde{G}_n^{ij}(r,r';s^2)}\;\tilde{v}_j(r'). \tag{130}$$

The integral over r' now carries the *flat* measure $\mathrm{d}r'$, exactly as required for $\widetilde{\mathcal{M}}_n$ to be the natural operator on $L^2(0,\infty;\mathrm{d}r)$.

Step 3: read off the kernel transformation. Comparing with the abstract relation $\tilde{u} = (\widetilde{\mathcal{M}}_n + s^2)^{-1}\,\tilde{v}$, we identify

$$\boxed{\;\widetilde{G}_n(r,r';s^2) \;=\; (rr')^{3/2}\,G_n(r,r';s^2).\;} \tag{131}$$

This is *not* an assumption that the Green's function "factorizes as a product of two modes". It is forced by the requirement that (128) continues to hold after we change basis on u and v separately. The factor r^3 is split symmetrically as $r^{3/2}$ for each argument, mirroring exactly the way each of $u(r)$ and $v(r')$ transforms.

Step 4: take the diagonal and perform the radial integral. Setting $r' = r$ in (131),

$$\widetilde{G}_n(r,r;s^2) \;=\; r^3\,G_n(r,r;s^2). \tag{132}$$

Now plug into the definition (111) of $\overline{G}_n$:

$$\overline{G}_n(s^2) \;=\; (n+1)^2\int_0^\infty \mathrm{d}r\, r^3\,\mathrm{tr}_{\text{field}}\big[G_n(r,r;s^2)\big] \tag{133}$$

$$\;=\; (n+1)^2\int_0^\infty \mathrm{d}r\,\mathrm{tr}_{\text{field}}\big[\widetilde{G}_n(r,r;s^2)\big]. \tag{134}$$

The r^3 has moved *from the measure into the kernel* via the substitution (131). Nothing has been thrown away; the same physical quantity is being computed, just rewritten in the tilde basis. So the radial integral is now performed against the flat measure $\mathrm{d}r$, but it is still a genuine $\mathrm{d}r$ integral.

Step 5: the integral is the trace. The final step uses the spectral resolution of the resolvent (eigenmodes $\tilde{\chi}_L$ of $\widetilde{\mathcal{M}}_n$, eigenvalues λ_L; continuum-normalized so $\int \tilde{\chi}_L(r)^2\,\mathrm{d}r = 1$):

$$\widetilde{G}_n(r,r;s^2) \;=\; \sum_L \frac{\tilde{\chi}_L(r)\,\tilde{\chi}_L(r)^\top}{\lambda_L + s^2} \quad\Longrightarrow\quad \int_0^\infty \mathrm{d}r\,\mathrm{tr}_{\text{field}}[\widetilde{G}_n(r,r;s^2)] \;=\; \sum_L \frac{1}{\lambda_L+s^2}. \tag{135}$$

The radial integral is performed automatically by the eigenmode normalization. For each a, $\int \tilde{\chi}_L^2\,\mathrm{d}r = 1$ collapses the numerator integral to 1, and what remains is the spectral sum. This is the operator-theoretic identity

$$\boxed{\;\overline{G}_n(s^2) \;=\; (n+1)^2\,\mathrm{Tr}\big[(\widetilde{\mathcal{M}}_n + s^2\,\mathbb{I})^{-1}\big].\;} \tag{136}$$

The $\mathrm{d}r$ integral and the operator trace are equal because the trace is *defined* as $\sum_L \langle\tilde{\chi}_L|\cdot|\tilde{\chi}_L\rangle$ with the standard L^2 inner product $\langle\cdot,\cdot\rangle = \int(\cdot)\,\mathrm{d}r$.

Step 6: the discrete analogue. On the FD grid we never write the dr explicitly because of the discrete-vs-continuum normalization mismatch derived in detail in §6.7 below. The short version: discrete eigenvectors satisfy $\sum_i \chi_{a,i}^2 = 1$ (no dr), so the matrix trace

$$\mathrm{Tr}\big[\widetilde{\mathcal{M}}^{(N)\,-1}\big] \;=\; \sum_L \frac{\sum_i \chi_{a,i}^2}{\lambda_L + s^2} \;=\; \sum_L \frac{1}{\lambda_L + s^2} \tag{137}$$

gives the *same number* as the continuum integral (135). The Riemann-sum factor dr that one would naively need to multiply the matrix trace by has been absorbed by switching from continuum normalization $\int |\tilde{\chi}_L|^2 \mathrm{d}r = 1$ to discrete normalization $\sum_i \chi_{a,i}^2 = 1$, and the two conventions differ by exactly a factor dr per squared eigenvector entry.

5 Feshbach projection: removing the pole exactly

5.1 The general spectral identity (every step)

The aim of this subsection is to show, eigenvector by eigenvector, how the projector P removes *exactly one* term from the spectral sum and leaves all others untouched. The mechanism is just orthogonality of eigenvectors of a symmetric operator – but it is worth being explicit.

Setup. Let χ be one specific normalized eigenvector of $\widetilde{\mathcal{M}}_n$ that we want to project out (the negative mode for $n = 0$ or the translation zero mode for $n = 1$):

$$\widetilde{\mathcal{M}}_n\,\chi = \lambda_\chi\,\chi, \qquad \chi^\top\chi = 1. \tag{138}$$

Define the L^2-orthogonal projector onto the complement of χ:

$$P \;=\; \mathbb{I} - \chi\chi^\top, \qquad P^2 = P, \qquad P\chi = 0. \tag{139}$$

The complete spectral resolution of $\widetilde{\mathcal{M}}_n$ uses an orthonormal eigenbasis $\{\chi_L\}_{a=1,2,\ldots}$:

$$\widetilde{\mathcal{M}}_n\,\chi_L = \lambda_L\,\chi_L, \qquad \chi_L^\top\chi_{L'} = \delta_{LL'}. \tag{140}$$

Pick a labeling so that the special eigenvector χ corresponds to some specific index $a = \bar{L}$, i.e. $\chi_{\bar{L}} = \chi$ and $\lambda_{\bar{L}} = \lambda_\chi$.

Step 1: spectral form of the resolvent. From the eigendecomposition $\widetilde{\mathcal{M}}_n = \sum_L \lambda_L\,\chi_L\chi_L^\top$ and the identity $\sum_L \chi_L\chi_L^\top = \mathbb{I}$,

$$(\widetilde{\mathcal{M}}_n + s^2\,\mathbb{I})^{-1} \;=\; \sum_L \frac{\chi_L\,\chi_L^\top}{\lambda_L + s^2}. \tag{141}$$

This is just the statement that in the eigenbasis the matrix is diagonal with entries $1/(\lambda_L + s^2)$.

Step 2: sandwich with P. Multiply (141) by P on both sides. Since P does not depend on a, it slides inside the sum, and using $P^\top = P$,

$$P\,(\widetilde{\mathcal{M}}_n + s^2\,\mathbb{I})^{-1}\,P \;=\; \sum_L \frac{(P\,\chi_L)(P\,\chi_L)^\top}{\lambda_L + s^2}. \tag{142}$$

Each term in the sum has been replaced by a rank-1 outer product of the *projected* eigenvector with itself, divided by the same spectral denominator.

Step 3: compute $P\chi_L$ for each kind of eigenvector. Apply $P = \mathbb{I} - \chi\chi^\top$ explicitly:

$$P\,\chi_L \;=\; \chi_L \;-\; \chi\,(\chi^\top\chi_L). \tag{143}$$

Two cases.

Case A: $L = \bar{L}$, i.e. $\chi_L = \chi$. Then $\chi^\top\chi_L = \chi^\top\chi = 1$ (unit norm), so

$$P\,\chi \;=\; \chi - \chi\cdot 1 \;=\; 0. \tag{144}$$

The projector annihilates the special direction completely.

Case B: $L \neq \bar{L}$, i.e. $\chi_L \neq \chi$. Here we use that $\widetilde{\mathcal{M}}_n$ is symmetric, and that χ and χ_L are eigenvectors with different eigenvalues ($\lambda_L \neq \lambda_\chi$). The spectral theorem then guarantees *orthogonality*:

$$\chi^\top\chi_L \;=\; 0 \quad \text{for every} \quad L \neq \bar{L}. \tag{145}$$

Substituting back into (143),

$$P\,\chi_L \;=\; \chi_L \;-\; \chi\cdot 0 \;=\; \chi_L. \tag{146}$$

The projector leaves *every other* eigenvector untouched.

Why orthogonality holds (sanity check). For a symmetric operator, $\chi^\top\widetilde{\mathcal{M}}_n\chi_L$ can be evaluated two ways: acting $\widetilde{\mathcal{M}}_n$ to the right gives $\lambda_L\,\chi^\top\chi_L$; acting to the left gives $\lambda_\chi\,\chi^\top\chi_L$. So $(\lambda_L - \lambda_\chi)\,\chi^\top\chi_L = 0$, and if $\lambda_L \neq \lambda_\chi$ then $\chi^\top\chi_L = 0$. (Within a degenerate subspace of dimension > 1 we can always orthogonalize the eigenvectors by hand; here our χ is non-degenerate – the negative mode is unique, the four translation zero modes for $n = 1$ share the same radial profile so we only need one χ.)

Step 4: assemble the projected resolvent. Insert Cases A and B into (142):

$$\begin{aligned} P\,(\widetilde{\mathcal{M}}_n + s^2\,\mathbb{I})^{-1}\,P \;&=\; \underbrace{\frac{(P\chi)(P\chi)^\top}{\lambda_\chi + s^2}}_{=\,0/(\lambda_\chi+s^2)\,=\,0} \;+\; \sum_{L\neq\bar{L}} \frac{(P\chi_L)(P\chi_L)^\top}{\lambda_L + s^2} \\ &=\; 0 \;+\; \sum_{L\neq\bar{L}} \frac{\chi_L\,\chi_L^\top}{\lambda_L + s^2}. \end{aligned} \tag{147}$$

The pole-bearing term $(\lambda_\chi + s^2)^{-1}$ has been multiplied by zero, which removes it analytically. All other terms pass through unchanged because P acts as the identity on them.

Step 5: take the trace. Use $\mathrm{Tr}[\chi_L\,\chi_L^\top] = \chi_L^\top\chi_L = 1$ (unit-normalized eigenvectors):

$$\boxed{\mathrm{Tr}\big[\,P\,(\widetilde{\mathcal{M}}_n + s^2\,\mathbb{I})^{-1}\,P\,\big] \;=\; \sum_{L\neq\bar{L}} \frac{1}{\lambda_L + s^2}.} \tag{148}$$

Summary of the mechanism. The projector does not "know" about poles or zero modes. It just removes the component along χ. The reason this cleanly drops exactly one term from the spectral sum is that χ *coincides with one specific eigenvector* of $\widetilde{\mathcal{M}}_n$, so by the symmetric spectral theorem it is automatically orthogonal to all the others. That orthogonality is what makes $P\chi_L = \chi_L$ for $L \neq \bar{L}$ and $P\chi = 0$. The pole at $s^2 = -\lambda_\chi$ has been removed *analytically*, not by a numerical fit.

5.2 Raw vs. subtracted, in code-language

$$\overline{G}_n^{\text{raw}}(s^2) \;=\; (n+1)^2\,\mathrm{Tr}\big[(\widetilde{\mathcal{M}}_n + s^2)^{-1}\big] \;=\; (n+1)^2 \sum_L \frac{1}{\lambda_L + s^2}, \tag{149}$$

$$\overline{G}_n^{\text{sub}}(s^2) \;=\; (n+1)^2\,\mathrm{Tr}\big[P(\widetilde{\mathcal{M}}_n + s^2)^{-1}P\big] \;=\; (n+1)^2 \sum_{L\neq\bar{L}} \frac{1}{\lambda_L + s^2}. \tag{150}$$

Near the pole, denoting $s_\star = -\lambda_\chi$,

$$\overline{G}_n^{\text{raw}}(s^2) \;\sim\; \frac{(n+1)^2}{s^2 - s_\star} \;+\; B(s^2), \qquad \overline{G}_n^{\text{sub}}(s_\star) \;=\; B(s_\star) \quad \text{(smooth)}. \tag{151}$$

$B(s_\star)$ is what the RK pole-fit method estimates by least-squares, and what our FD method computes directly.

5.3 The translation zero mode for $n = 1$

Translation invariance of S_E gives $\mathcal{M}\big(\partial_\mu\phi_b\big) = 0$. The four zero modes $\partial_\mu\phi_b$ ($\mu = 1,\ldots,4$) sit in the $n = 1$ partial wave because $\partial_\mu\phi_b(|x|) = \hat{x}_\mu\,\phi_b'(r)$, and $\hat{x}_\mu$ are exactly the four $\mathcal{Y}^{1,k}$ on S^3. The radial profile is $\phi_b'(r)$, and in the tilde basis

$$\chi_{\text{zm}}(r) \;\propto\; r^{3/2}\,\phi_b'(r), \qquad \chi_{\text{zm}}^\top\chi_{\text{zm}} = 1. \tag{152}$$

The factor $(n+1)^2 = 4$ in $\overline{G}_1$ then accounts for all four zero modes by acting only on the radial χ_{zm}.

5.4 How χ is actually computed in the code

A natural worry: to write the projector $P = \mathbb{I} - \chi\chi^\top$ we need to *know* the special eigenvector χ and (for $n = 0$) its eigenvalue λ_χ. Do we therefore need to diagonalize the whole operator $\widetilde{\mathcal{M}}_n$? **No, only one eigenpair** – the single special mode that we want to project out. Everything else takes care of itself through orthogonality (§4.1).

What the projection actually requires. Look once more at the spectral identity (148):

$$\mathrm{Tr}\big[P(\widetilde{\mathcal{M}}_n + s^2)^{-1}P\big] \;=\; \sum_{L\neq\bar{L}} \frac{1}{\lambda_L + s^2}. \tag{153}$$

The right-hand side knows about *every* interior eigenvalue, but the left-hand side only requires us to construct P. To build P we need exactly two ingredients:

- the special eigenvector χ (so we can form $\chi\chi^\top$), and
- nothing else – in particular, we do *not* need any other eigenvector χ_L for $L \neq \bar{L}$.

The other eigenvalues λ_L ($L \neq \bar{L}$) appear in the answer implicitly, because Hutchinson's stochastic estimator

$$\frac{1}{K}\sum_k v_k^\top P\,(\widetilde{\mathcal{M}} + s^2)^{-1}\,P\,v_k \;\xrightarrow{K\to\infty}\; \mathrm{Tr}\big[P(\widetilde{\mathcal{M}} + s^2)^{-1}P\big] \tag{154}$$

samples the entire interior spectrum automatically through repeated sparse-LU solves of $Ax = w$. We never have to enumerate the eigenvalues.

For $n = 0$: ARPACK Lanczos for the negative mode. The Coleman negative mode is the algebraically smallest eigenpair $(\lambda_{\text{neg}}, \chi_{\text{neg}})$ of $\widetilde{\mathcal{M}}_0$, with $\lambda_{\text{neg}} < 0$. We compute it with `scipy.sparse.linalg.eigsh`, which is a Lanczos iteration specialized for symmetric sparse matrices:

```
eigvals, eigvecs = eigsh(M_tilde, k=3, which='SA', tol=1e-9).
```
(155)

Mechanics:

- `k=3`: ask for the three lowest eigenpairs (only one is negative; the other two give a sanity check and let us see how well-separated λ_{neg} is from the rest).
- `which='SA'`: "smallest algebraic" – smallest eigenvalue including sign, which finds $\lambda_{\text{neg}} < 0$ first.
- `tol=1e-9`: convergence tolerance for the Lanczos iteration. Internally, ARPACK builds a Krylov subspace $\text{span}\{v_0, \widetilde{\mathcal{M}}v_0, \widetilde{\mathcal{M}}^2 v_0, \ldots\}$, tridiagonalizes the operator restricted to that subspace, and finds eigenpairs of the small tridiagonal matrix.

Output: λ_{neg} is a single number (≈ -0.31 for the F2/T0 bounce); χ_{neg} is a length-$2N$ vector normalized as $\chi_{\text{neg}}^\top \chi_{\text{neg}} = 1$.

The code is the function `find_negative_mode(M_tilde)` in `fd_builder.py`. It returns $(\lambda_{\text{neg}}, \chi_{\text{neg}})$, prints diagnostics on the three lowest eigenvalues so the user can verify that exactly one is negative, and clamps the eigenvector at boundary indices (eigsh noise can leak into the ghost subspace).

5.4.1 The Lanczos algorithm explained from scratch

This subsubsection builds up the Lanczos algorithm step by step, starting from the simplest possible eigenvalue-finding idea (power iteration) and arriving at what `eigsh` actually does. The goal is that by the end, you understand exactly how a few hundred matrix-vector multiplications produce the negative-mode eigenpair of our 4000×4000 operator.

The problem we are solving. Given a symmetric sparse $\widetilde{\mathcal{M}}$ of size $N \times N$ with $N = 4000$, find one eigenpair: either

- the smallest-algebraic eigenvalue $\lambda_{\text{neg}} < 0$ and its eigenvector χ_{neg} (for $n = 0$, the Coleman negative mode), or
- the eigenvalue closest to zero λ_{zm} and its eigenvector χ_{zm} (for $n = 1$, the discrete translation zero mode).

Why we don't just diagonalise $\widetilde{\mathcal{M}}$ fully. The brute-force way is `numpy.linalg.eigh(M.toarray(`
it computes *all* 4000 eigenpairs, costing $\mathcal{O}(N^3) \approx 6{\times}10^{10}$ flops and many gigabytes of memory. We would then throw away 3999 of them. Wasteful and slow. We want an algorithm whose cost grows like N per iteration and that converges in a small number of iterations to the one eigenpair we care about.

Step 1: power iteration (the simplest idea)

The most basic eigenvalue-finder is *power iteration*. Start with a random vector $v_0 \in \mathbb{R}^N$. Apply $\widetilde{\mathcal{M}}$ over and over:

$$v_0,\ \widetilde{\mathcal{M}}v_0,\ \widetilde{\mathcal{M}}^2 v_0,\ \widetilde{\mathcal{M}}^3 v_0,\ \ldots \tag{156}$$

Each application costs one sparse matvec ($\mathcal{O}(N)$ for our banded $\widetilde{\mathcal{M}}$).

Why this converges. Decompose v_0 in the eigenbasis of $\widetilde{\mathcal{M}}$:

$$v_0 \;=\; \sum_L c_L\,\chi_L, \qquad \widetilde{\mathcal{M}}\,\chi_L = \lambda_L\,\chi_L. \tag{157}$$

Then

$$\widetilde{\mathcal{M}}^k v_0 \;=\; \sum_L c_L\,\lambda_L^k\,\chi_L. \tag{158}$$

The term with the *largest* $|\lambda_L|$ grows fastest. If we label the eigenvalues so $|\lambda_1| > |\lambda_2| > \ldots$, then

$$\widetilde{\mathcal{M}}^k v_0 \;=\; \lambda_1^k\,c_1\,\chi_1 \;+\; \mathcal{O}\big(|\lambda_2/\lambda_1|^k\big)\,\chi_2 + \cdots \tag{159}$$

After enough iterations k, the first term dominates, and $\widetilde{\mathcal{M}}^k v_0/\|\widetilde{\mathcal{M}}^k v_0\|$ converges to χ_1 – the eigenvector of largest-magnitude eigenvalue. The dominant eigenvalue is then $\lambda_1 = (\chi_1^\top \widetilde{\mathcal{M}}\chi_1)$.

Why power iteration alone is not enough for us. Three issues:

1. It converges only to the *largest-magnitude* eigenvalue, not the smallest or closest-to-zero. Our negative mode and zero mode are extreme but they are at the small end, so we'd need a shift trick.

2. Convergence is slow when the spectrum has near-degeneracies ($|\lambda_2/\lambda_1|$ close to 1).

3. It throws away enormous amounts of information: at each step we have $v_0, \widetilde{\mathcal{M}}v_0, \ldots, \widetilde{\mathcal{M}}^k v_0$ in our hands but we only *use* the very last one. That's wasteful.

Step 2: the Krylov subspace – "don't throw information away"

Lanczos is the upgrade: instead of overwriting earlier vectors as we go, *keep all of them*. The collection

$$v_0,\ \widetilde{\mathcal{M}}v_0,\ \widetilde{\mathcal{M}}^2 v_0,\ \ldots,\ \widetilde{\mathcal{M}}^{m-1} v_0 \tag{160}$$

spans a subspace of $\mathbb{R}^N$ called the **Krylov subspace** of dimension m:

$$\mathcal{K}_m(\widetilde{\mathcal{M}}, v_0) \;\equiv\; \mathrm{span}\big\{v_0,\ \widetilde{\mathcal{M}}v_0,\ \ldots,\ \widetilde{\mathcal{M}}^{m-1}v_0\big\} \;\subset\; \mathbb{R}^N. \tag{161}$$

What is this subspace good for? By the same argument as power iteration, the higher powers of $\widetilde{\mathcal{M}}$ contain more and more information about the *extreme* eigenvectors of $\widetilde{\mathcal{M}}$ (largest *and* smallest, in different directions). The Krylov subspace $\mathcal{K}_m$ is therefore an m-dimensional "best subset" of $\mathbb{R}^N$: it contains the dominant eigenvectors with high accuracy and the bulk of the others approximately.

Concrete picture. Imagine $\widetilde{\mathcal{M}}$ has 4000 eigenvalues spread out over some range. Each multiplication $v \mapsto \widetilde{\mathcal{M}}v$ is like a filter: components along large-$|\lambda|$ eigenvectors get emphasised, others suppressed. After m iterations the vectors $\{v_0, \widetilde{\mathcal{M}}v_0, \ldots, \widetilde{\mathcal{M}}^{m-1}v_0\}$ collectively "see" the top m extreme eigenpairs of $\widetilde{\mathcal{M}}$ accurately and the rest poorly. So if we just need

a few extreme eigenpairs, $\mathcal{K}_m$ for $m \sim 30$–100 is enough. This is the central observation Lanczos exploits.

Cost of building $\mathcal{K}_m$. $m-1$ sparse matvecs at $\mathcal{O}(N)$ each. For $m = 30$, $N = 4000$ banded: about $30 \times 16{,}000 = 5 \times 10^5$ flops – five orders of magnitude cheaper than full diagonalisation.

Step 3: the Lanczos recurrence – get an orthonormal basis cheaply

The naive way doesn't work numerically. You might try just storing $v_0, \widetilde{\mathcal{M}} v_0, \widetilde{\mathcal{M}}^2 v_0, \ldots$ as the basis. Bad idea: in floating point they all become near-parallel to χ_1 (the dominant eigenvector) after a few iterations. The basis collapses; the matrix in this basis is ill-conditioned; nothing works.

The fix: orthogonalise as you go (Gram-Schmidt-style). Build an orthonormal basis $q_1, q_2, \ldots, q_m$ of $\mathcal{K}_m$ by:

1. $q_1 = v_0/\|v_0\|$ (the first basis vector).
2. Compute $\widetilde{\mathcal{M}} q_1$. The new direction it adds to the Krylov subspace is whatever component of $\widetilde{\mathcal{M}} q_1$ is *orthogonal* to q_1. So:

$$\begin{aligned} \alpha_1 &= q_1^\top \widetilde{\mathcal{M}} q_1, && \text{(the component of } \widetilde{\mathcal{M}} q_1 \text{ along } q_1) \\ r_1 &= \widetilde{\mathcal{M}} q_1 - \alpha_1 q_1, && \text{(remove that component)} \\ q_2 &= r_1/\|r_1\|, && \text{(normalise)} \end{aligned}$$

3. Compute $\widetilde{\mathcal{M}} q_2$. Same idea: subtract off whatever it has along q_1 and q_2, then normalise to get q_3.
4. Continue.

The miracle of symmetry. For a generic matrix, the orthogonalisation at step k would require subtracting off components along ALL previous $q_1, \ldots, q_k$ – a sum that grows in cost with k. But for symmetric $\widetilde{\mathcal{M}}$, a small calculation shows that $\widetilde{\mathcal{M}} q_k$ is automatically orthogonal to $q_1, q_2, \ldots, q_{k-2}$. We only need to subtract off components along q_k and q_{k-1}. This is the **three-term Lanczos recurrence**:

$$r_k \;=\; \widetilde{\mathcal{M}} q_k - \alpha_k q_k - \beta_{k-1} q_{k-1}, \qquad q_{k+1} = r_k/\|r_k\|, \tag{162}$$

with

$$\alpha_k = q_k^\top \widetilde{\mathcal{M}} q_k, \qquad \beta_k = \|r_k\|. \tag{163}$$

Why three-term? Because for symmetric $\widetilde{\mathcal{M}}$ and the recurrence above, q_{k+1} is automatically orthogonal to all q_j for $j \le k-2$. (Concrete proof: for $j \le k-2$, $q_j^\top r_k = q_j^\top \widetilde{\mathcal{M}} q_k - \alpha_k q_j^\top q_k - \beta_{k-1} q_j^\top q_{k-1}$. The middle and last terms are zero by orthogonality of the q's. The first term: $q_j^\top \widetilde{\mathcal{M}} q_k = (\widetilde{\mathcal{M}} q_j)^\top q_k$ by symmetry, and $\widetilde{\mathcal{M}} q_j$ lies in $\mathcal{K}_{j+1} \subset \mathcal{K}_{k-1}$, which is orthogonal to q_k. So $q_j^\top r_k = 0$ for all $j \le k-2$.) This is the "three-term recurrence" magic that makes Lanczos cheap: $\mathcal{O}(N)$ per step, not $\mathcal{O}(Nk)$.

Cost of the full Lanczos basis. m matvecs of $\mathcal{O}(N)$ + a constant number of dot products and rescalings per step. Total $\mathcal{O}(Nm)$.

Step 4: the matrix in the Lanczos basis is tridiagonal

After m Lanczos steps we have orthonormal $q_1, \dots, q_m$ spanning $\mathcal{K}_m$, plus the scalars α_k and β_k from the recurrence. Stack the q's into a tall-thin matrix $Q_m = [q_1 \; q_2 \; \cdots \; q_m] \in \mathbb{R}^{N \times m}$. Then the matrix that represents $\widetilde{\mathcal{M}}$ *in the basis* $\{q_1, \dots, q_m\}$ is

$$T_m \;=\; Q_m^{\top}\, \widetilde{\mathcal{M}}\, Q_m \;=\; \begin{pmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & \\ & \beta_2 & \alpha_3 & \ddots & \\ & & \ddots & \ddots & \beta_{m-1} \\ & & & \beta_{m-1} & \alpha_m \end{pmatrix}. \tag{164}$$

T_m is a small $m \times m$ symmetric tridiagonal matrix – typically $m \sim 30$. We have just *compressed* the relevant spectral information from the 4000×4000 matrix $\widetilde{\mathcal{M}}$ into a 30×30 matrix T_m.

Why this is amazing. A 30×30 tridiagonal symmetric matrix can be diagonalised in milliseconds using textbook methods (e.g. tridiagonal QR), giving us all of its eigenvalues and eigenvectors. So in $\mathcal{O}(Nm + m^2)$ flops total, we get a small problem whose eigenstructure approximates the eigenstructure of $\widetilde{\mathcal{M}}$.

Step 5: Ritz values approximate eigenvalues of $\widetilde{\mathcal{M}}$

Diagonalise the small T_m:

$$T_m\, y_i \;=\; \theta_i\, y_i, \qquad i = 1, 2, \dots, m, \tag{165}$$

giving m eigenvalues $\theta_1 \le \theta_2 \le \cdots \le \theta_m$ (the **Ritz values**) and corresponding eigenvectors $y_i \in \mathbb{R}^m$ (small). The full-space approximations to the eigenvectors of $\widetilde{\mathcal{M}}$ are

$$\chi_i^{\mathrm{Ritz}} \;=\; Q_m\, y_i \;\in\; \mathbb{R}^N, \tag{166}$$

the so-called **Ritz vectors**.

What's the relation between θ_i and the eigenvalues of $\widetilde{\mathcal{M}}$? Two key results:

1. The Ritz values are guaranteed to lie in the convex hull of the true eigenvalues: $\lambda_{\min} \le \theta_i \le \lambda_{\max}$ for all i.
2. The *extreme* Ritz values (θ_1 and θ_m) converge to the extreme eigenvalues of $\widetilde{\mathcal{M}}$ (the smallest and the largest) very quickly – typically a few iterations after their target's eigengap is resolved.

The "interior" Ritz values (the middle ones) converge more slowly. So Lanczos is naturally an *extreme*-eigenvalue finder. For our negative mode (the smallest-algebraic eigenvalue), this is exactly what we need.

When we declare convergence. The error of the Ritz value θ_i as an approximation to a true eigenvalue λ of $\widetilde{\mathcal{M}}$ is bounded by the *residual norm*

$$\|\widetilde{\mathcal{M}}\, \chi_i^{\mathrm{Ritz}} - \theta_i\, \chi_i^{\mathrm{Ritz}}\|. \tag{167}$$

This residual is computable on the fly: in fact, it equals $|\beta_m| \cdot |y_{i,m}|$ – the product of the last off-diagonal of T_m and the last component of y_i. ARPACK uses this to declare a Ritz pair "converged" once the residual is below a user-specified tolerance `tol` (default $\sim 10^{-9}$).

Step 6: targeting the smallest, not the largest – spectral transformations

Plain Lanczos converges fastest to the eigenvalues of LARGEST *absolute value* of $\widetilde{\mathcal{M}}$. For our negative mode (the smallest-algebraic eigenvalue $\lambda_{\text{neg}} \approx -0.31$), this happens to also be the largest in absolute value among the low-lying spectrum, so plain Lanczos works – but ARPACK uses a more reliable approach via spectral transformations:

(a) For `which='SA'` (smallest algebraic): run Lanczos on $-\widetilde{\mathcal{M}}$ instead of $\widetilde{\mathcal{M}}$. The largest eigenvalues of $-\widetilde{\mathcal{M}}$ correspond to the most-negative eigenvalues of $\widetilde{\mathcal{M}}$. So λ_{neg} becomes the *largest*-magnitude (and largest-algebraic) eigenvalue of $-\widetilde{\mathcal{M}}$, where Lanczos converges fastest. ARPACK negates the result internally so the user sees the original eigenvalue.

(b) For shift-invert (`sigma=0, which='LM'`): run Lanczos on $\widetilde{\mathcal{M}}^{-1}$ (or, equivalently, do the recurrence with the action $w = \widetilde{\mathcal{M}}^{-1}v$ instead of $w = \widetilde{\mathcal{M}}v$). The largest eigenvalues of $\widetilde{\mathcal{M}}^{-1}$ are the *reciprocals* of the smallest-magnitude eigenvalues of $\widetilde{\mathcal{M}}$. So eigenvalues near zero of $\widetilde{\mathcal{M}}$ become the largest eigenvalues of $\widetilde{\mathcal{M}}^{-1}$ – exactly where Lanczos converges fastest. This is how `find_discrete_zero_mode` finds the eigenvalue closest to zero, regardless of sign. The matvec $w = \widetilde{\mathcal{M}}^{-1}v$ is computed by solving $\widetilde{\mathcal{M}}w = v$ via sparse LU at the start of the Lanczos run – a one-time cost that makes shift-invert tractable.

For general σ (any target value): `sigma=`σ`, which='LM'` runs Lanczos on $(\widetilde{\mathcal{M}} - \sigma\mathbb{I})^{-1}$, finding eigenvalues of $\widetilde{\mathcal{M}}$ closest to σ. Useful if you know roughly where in the spectrum the eigenpair you want sits.

Step 7: the "implicit restart" refinement that ARPACK adds

In practice, the basic Lanczos algorithm has two issues:

1. *Loss of orthogonality.* In floating point, the q_k slowly drift away from being mutually orthogonal as k grows. After a few hundred steps the basis is contaminated.

2. *Memory.* Storing Q_m requires $N \cdot m$ floats. For $m = 1000$, $N = 4000$, that's 32 MB – not catastrophic, but wasteful when we only want a few eigenvalues.

ARPACK's *Implicitly Restarted Lanczos Method* (Sorensen 1992) addresses both: every ~ 50 iterations, it restarts the Lanczos basis with a cleverly chosen new v_0 that filters out the spectral components we don't want and keeps the components of the already-converged eigenpairs. The result is that the algorithm runs in fixed memory (only stores $\mathcal{O}(k)$ basis vectors, where k is the number of wanted eigenpairs) and recovers full orthogonality through the restarts. The "IR" in IRLM is what `eigsh` relies on for production-grade eigenvalue computation.

Step 8: putting it all together for our two modes

Finding χ_{neg} for $n = 0$ (Coleman negative mode).

```
eigvals, eigvecs = eigsh(M_tilde, k=3, which='SA', tol=1e-9, maxiter
    =20000)
```

What ARPACK does behind the scenes:

1. Pick a random starting vector v_0.

2. Run the Lanczos recurrence on $-\widetilde{\mathcal{M}}$ for ~ 7 steps (initial Krylov dimension, set by the `ncv` parameter, default $2k + 1$).

3. Form the small tridiagonal T_m and diagonalise it.

4. Check the residuals of the lowest 3 Ritz values. If all below `tol`, return them. Otherwise:

5. Implicit restart: filter out unwanted components, restart with a new v_0 that retains the converged Ritz pairs.

6. Repeat steps 2-5 until convergence (max `maxiter` iterations).

For our F2/T0 bounce, this typically converges in $\sim$ 30–60 matvecs, completing in well under a second. Returns $(\lambda_{\text{neg}}, \chi_{\text{neg}})$ with $\lambda_{\text{neg}} \approx -0.31$ and a length-$2N$ eigenvector normalised to unit length.

Finding χ_{zm} for $n = 1$ (translation zero mode).

```
eigvals, eigvecs = eigsh(M_tilde, k=5, sigma=0.0, which='LM', tol=1e-9)
```

The shift-invert mode (`sigma=0`). Behind the scenes:

1. Solve $\widetilde{\mathcal{M}}\, X = I$ via sparse LU once – this gives `splu(M_tilde)` which can rapidly evaluate $w = \widetilde{\mathcal{M}}^{-1} v$ for any v.

2. Run Lanczos on $\widetilde{\mathcal{M}}^{-1}$, with each matvec replaced by a sparse-LU solve.

3. The largest eigenvalues of $\widetilde{\mathcal{M}}^{-1}$ correspond to the eigenvalues of $\widetilde{\mathcal{M}}$ closest to zero.

4. ARPACK inverts the Ritz values internally, so the user sees the original eigenvalues of $\widetilde{\mathcal{M}}$.

5. Among the 5 returned eigenvalues, our code picks the one with smallest absolute value: `idx_min = np.argmin(np.abs(eigvals))`.

Returns $(\lambda_{\text{zm}}, \chi_{\text{zm}})$ with $\lambda_{\text{zm}} \approx -8.5 \times 10^{-3}$ for our bounce.

Why we never form $\widetilde{\mathcal{M}}^{-1}$ explicitly in the χ_{neg} case. Plain Lanczos (used for `which='SA'`) only requires the action of $\widetilde{\mathcal{M}}$ on a vector ($w = \widetilde{\mathcal{M}}\, q$), not $\widetilde{\mathcal{M}}^{-1}$. So the negative-mode eigsh call costs only matvecs and is matrix-free in the strictest sense. The shift-invert mode for the zero mode *does* require sparse LU of $\widetilde{\mathcal{M}}$ (computed once at the start), but each subsequent "inverse matvec" is just two cheap triangular solves.

Sanity checks the code performs after eigsh.

1. Print the lowest few eigenvalues. The user verifies that the expected pattern holds: for $n = 0$, exactly one is negative ($\lambda_{\text{neg}} \approx -0.31$); for $n = 1$, exactly one is near zero ($\lambda_{\text{zm}} \approx -8.5 \times 10^{-3}$).

2. Clamp the returned eigenvector at boundary indices and renormalise. ARPACK's tolerance is 10^{-9}, so the eigenvector has tiny noise components in the ghost subspace; clamping makes the projector $P = \mathbb{I} - \chi\chi^{\top}$ act exactly on the interior subspace.

3. For $n = 1$: also overlap-check the returned eigenvector against the analytic translation mode $r^{3/2}\phi_b'$ – the overlap should be > 0.95 for a well-converged bounce; lower indicates the picked eigenvector is not the translation mode.

Connection to Hutchinson and the rest of the pipeline. Notice the parallel between Lanczos and Hutchinson: both algorithms treat $\widetilde{\mathcal{M}}$ as "a thing you can apply to a vector," never as a stored matrix. Both are matrix-free in spirit. Lanczos uses N matvecs (~ 30 of them) to extract one eigenpair; Hutchinson uses K solves (~ 100 of them) to extract the trace. The two algorithms are the workhorses of large-scale sparse linear algebra, and our pipeline uses both:

- `find_negative_mode` / `find_discrete_zero_mode`: *Lanczos* on $-\widetilde{\mathcal{M}}$ or $\widetilde{\mathcal{M}}^{-1}$, returns eigenpair.
- `gbar_raw_fd` / `gbar_sub_fd`: *Hutchinson* with random probes plus sparse LU, returns trace.

Both are fundamental to making the FD method computationally tractable.

Key idea: Krylov subspaces. Pick any starting vector v_0. The *Krylov subspace* of order m is

$$\mathcal{K}_m(\widetilde{\mathcal{M}}, v_0) \;\equiv\; \mathrm{span}\big\{v_0,\; \widetilde{\mathcal{M}}v_0,\; \widetilde{\mathcal{M}}^2 v_0,\; \ldots,\; \widetilde{\mathcal{M}}^{m-1}v_0\big\} \;\subset\; \mathbb{R}^N. \tag{168}$$

Why is this useful? Because powers of $\widetilde{\mathcal{M}}$ amplify the extreme eigenvalues. If v_0 has any non-zero overlap with the eigenvector χ_{neg} (corresponding to the most negative λ_{neg}), then $\widetilde{\mathcal{M}}^m v_0$ becomes more and more parallel to χ_{neg} as m grows, because $|\lambda_{\mathrm{neg}}|$ is the largest magnitude eigenvalue (it dominates the matrix-vector products). [1] The Krylov subspace contains v_0 contaminated with ever-stronger components along the extreme eigenvectors.

Cost of building the Krylov subspace: only matrix-vector products $w = \widetilde{\mathcal{M}}\, v$. For our sparse $\widetilde{\mathcal{M}}$ (bandwidth ~ 4), each matvec is $\mathcal{O}(N) \approx 16{,}000$ flops – one per Krylov iteration. Building a Krylov space of dimension $m \sim 30$ costs $\sim 30 \times 16{,}000 = 5 \times 10^5$ flops – five orders of magnitude cheaper than full diagonalization.

The 3-term Lanczos recurrence. Building the Krylov vectors directly as $\widetilde{\mathcal{M}}^k v_0$ is numerically catastrophic – they all become near-parallel to the top eigenvector after a few iterations, losing the information about other eigenvectors. *Lanczos* avoids this by orthogonalizing as we go, producing an orthonormal basis $q_1, q_2, \ldots, q_m$ of $\mathcal{K}_m(\widetilde{\mathcal{M}}, v_0)$ via a short three-term recurrence:

$$\begin{aligned} q_1 &= v_0/\|v_0\|, \\ \alpha_1 &= q_1^\top \widetilde{\mathcal{M}}\, q_1, \\ r_1 &= \widetilde{\mathcal{M}}\, q_1 - \alpha_1\, q_1, \\ \beta_1 &= \|r_1\|, \\ q_2 &= r_1/\beta_1. \end{aligned} \tag{169}$$

For $k = 2, 3, \ldots, m$:

$$\begin{aligned} \alpha_k &= q_k^\top \widetilde{\mathcal{M}}\, q_k, \\ r_k &= \widetilde{\mathcal{M}}\, q_k - \alpha_k\, q_k - \beta_{k-1}\, q_{k-1}, \\ \beta_k &= \|r_k\|, \\ q_{k+1} &= r_k/\beta_k. \end{aligned} \tag{170}$$

[1] For "smallest-algebraic" rather than "largest-magnitude," we work with $-\widetilde{\mathcal{M}}$ or use shift-invert; same idea.

Why three terms suffice. For symmetric $\widetilde{\mathcal{M}}$, the projection $\widetilde{\mathcal{M}}\, q_k$ already has zero overlap with all q_j for $j < k-1$ (this is the magic of symmetry). So the orthogonalization only has to subtract off the components along q_k (to give α_k) and q_{k-1} (to give β_{k-1}); all earlier q_j are automatically orthogonal. That's the 3-term recurrence – $\mathcal{O}(N)$ work per step, no growing cost as k increases.

What we get out: a small tridiagonal matrix. After m Lanczos steps, collect the orthonormal vectors into a tall-thin matrix $Q_m = [q_1, q_2, \ldots, q_m] \in \mathbb{R}^{N\times m}$. The recurrence (170) is equivalent to

$$Q_m^\top \widetilde{\mathcal{M}}\, Q_m \;=\; T_m, \qquad T_m \;=\; \begin{pmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & \\ & \beta_2 & \alpha_3 & \ddots & \\ & & \ddots & \ddots & \beta_{m-1} \\ & & & \beta_{m-1} & \alpha_m \end{pmatrix} \;\in\; \mathbb{R}^{m\times m}. \tag{171}$$

T_m is a $m\times m$ *tridiagonal symmetric* matrix – much smaller than $\widetilde{\mathcal{M}}$ ($m \sim 30$ vs. $N = 4000$). We can diagonalize T_m cheaply ($\mathcal{O}(m^2)$ for tridiagonal QR, well under a millisecond).

Ritz values and Ritz vectors. The eigenvalues $\theta_1 \le \theta_2 \le \ldots \le \theta_m$ of T_m are called *Ritz values*; they approximate eigenvalues of $\widetilde{\mathcal{M}}$. If $T_m\, y_i = \theta_i\, y_i$ with eigenvector $y_i \in \mathbb{R}^m$, then the corresponding *Ritz vector* in $\mathbb{R}^N$ is $\chi_i^{(m)} = Q_m\, y_i$, approximating an eigenvector of $\widetilde{\mathcal{M}}$.

Convergence: extreme eigenvalues come first. The Ritz values converge to the *extreme* eigenvalues of $\widetilde{\mathcal{M}}$ much faster than to the interior ones. Specifically, the smallest and largest eigenvalues of $\widetilde{\mathcal{M}}$ are approximated to machine precision after $\sim \log(1/\mathrm{tol})$ Lanczos iterations *per well-separated extreme eigenvalue*. For the Coleman negative mode (which is well-separated from the rest of the spectrum – $\lambda_{\mathrm{neg}} \approx -0.31$, gap to next eigenvalue ~ 1.3), this is typically ~ 30 iterations to reach $\mathrm{tol} = 10^{-9}$.

The convergence rate is governed by the "gap ratio" between the target eigenvalue and the rest of the spectrum: a well-isolated eigenvalue converges fast, a near-degenerate one converges slowly. The scipy `tol` parameter controls how small the residual $\|\widetilde{\mathcal{M}}\chi - \theta\chi\|$ must be before declaring convergence.

Implicit restart (the "IRLM" part). Plain Lanczos has two practical issues:

1. *Loss of orthogonality.* In floating point, the q_k slowly drift away from being mutually orthogonal because of roundoff. After a few hundred iterations they're a complete mess.

2. *Memory.* Storing Q_m requires $N \cdot m$ floats; if m grows to ~ 1000, that's ~ 30 MB per stored basis – manageable but wasteful.

Implicit restart (Sorensen, 1992) periodically restarts the Lanczos iteration with a cleverly chosen new starting vector that filters out unwanted spectral components. The information from the already-converged eigenpairs is preserved, and the unwanted components are squelched. ARPACK does this automatically.

Targeting the smallest-algebraic eigenvalue: `which='SA'`. Plain Lanczos converges fastest to the largest-magnitude eigenvalues. To find the smallest-algebraic instead, ARPACK uses *spectral transformations*:

- `which='SA'` (smallest algebraic): runs Lanczos on $-\widetilde{\mathcal{M}}$. The largest eigenvalues of $-\widetilde{\mathcal{M}}$ correspond to the most-negative eigenvalues of $\widetilde{\mathcal{M}}$, so λ_{neg} emerges as a Ritz value of $-\widetilde{\mathcal{M}}$. ARPACK negates the result internally so the user sees the original eigenvalue.
- `which='LM'` (largest magnitude): plain Lanczos on $\widetilde{\mathcal{M}}$, used by default for the largest eigenvalues.
- `sigma=`σ`, which='LM'` (shift-invert): runs Lanczos on $(\widetilde{\mathcal{M}} - \sigma\mathbb{I})^{-1}$. The largest eigenvalues of the inverse correspond to the eigenvalues of $\widetilde{\mathcal{M}}$ closest to σ in absolute value. This is used in `find_discrete_zero_mode` (§9.2.2) to find eigenvalues nearest zero, with $\sigma = 0$.

Putting it together for χ_{neg}. The complete `eigsh` call we use is:

```
eigvals, eigvecs = eigsh(M_tilde, k=3, which='SA', tol=1e-9, maxiter
    =20000)
```

What ARPACK does internally:

1. Pick a random starting vector v_0 (or use a user-supplied one).
2. Run the Lanczos recurrence on $-\widetilde{\mathcal{M}}$ for some initial number of steps (default $\sim 2k + 1 = 7$ – "ncv" parameter).
3. Diagonalize the small T_m, get Ritz values θ_i.
4. Check whether the lowest 3 Ritz values have converged (residual $\|\widetilde{\mathcal{M}}\chi - \theta\chi\| < \text{tol}\cdot|\theta|$). If yes, return them. If no, perform an implicit restart: filter out unwanted components, restart Lanczos, and continue.
5. Iterate until convergence (maxiter steps maximum).

For our problem this typically converges in $\sim$ 30–60 matrix-vector products, completing in well under a second.

Why we never form $\widetilde{\mathcal{M}}^{-1}$ in this step. Lanczos only requires the action of $\widetilde{\mathcal{M}}$ on a vector ($w = \widetilde{\mathcal{M}}\, q$), which is one sparse matvec – $\mathcal{O}(N)$ work, no inversion needed. So the eigsh call that finds the negative mode is conceptually quite different from the Hutchinson trace step, which *does* need the action of $\widetilde{\mathcal{M}}^{-1}$ via sparse LU. Both algorithms share the "matrix-free, only matvecs" philosophy that makes them efficient for large sparse problems, but eigsh applies $\widetilde{\mathcal{M}}$ while Hutchinson applies A^{-1}.

Sanity checks the code does after eigsh.

1. Print the 3 lowest eigenvalues. The user verifies exactly *one* is negative; the others should be of order $\mathcal{O}(1)$ or larger. If two are negative, the bounce profile is off (cosmoTransitions hasn't converged) or the BC is wrong.

2. Clamp the returned eigenvector at boundary indices and renormalize. ARPACK's tolerance is 10^{-9}, so the eigenvector has tiny noise components in the ghost subspace ($\sim 10^{-9}$); clamping them to zero and renormalizing makes the projector $P = \mathbb{I} - \chi\chi^\top$ act exactly on the interior subspace without leaking into the boundary subspace.

3. For $n = 1$: also overlap-check the returned eigenvector against the analytic translation mode $r^{3/2}\phi_b'$ (see §9.2.4). The overlap should be > 0.95 or so for a well-converged bounce; lower indicates that the picked eigenvector is not the translation mode.

For $n = 1$: two options, both implemented. Translation invariance of the action gives an analytic continuum zero-mode profile, $\chi_{\rm zm}(r) \propto r^{3/2}\,\phi_b'(r)$. The codebase exposes two ways to obtain $\chi_{\rm zm}$ as a length-$2N$ array:

Option (a) – analytic continuum mode (v1, fd_builder.py). Implemented as `build_translation_zero_mode(r, ...)`. No eigenvalue solver involved:

1. Spline $\phi_b(r)$ from the bounce file: `x_spline = CubicSpline(R_bounce, X_prime_bounce)`. Cubic splines provide analytic first derivatives via `x_spline(r, 1)`.

2. Evaluate $\phi_b'(r_i)$ at every FD grid point r_i.

3. Multiply by $r_i^{3/2}$ for each field component: $\chi_x = r^{3/2}\,\phi_{b,1}'$, $\chi_y = r^{3/2}\,\phi_{b,2}'$.

4. Concatenate into a single length-$2N$ vector and normalize: $\chi_{\rm zm} = \chi/\|\chi\|$.

5. Clamp at the four boundary indices (Dirichlet BC sanity).

The corresponding continuum eigenvalue is $\lambda_{\rm zm} = 0$. Cost: a few spline evaluations – effectively free.

Option (b) – discrete eigenvector (v2, fd_builder_ver2.py, recommended). Implemented as `find_discrete_zero_mode(M_tilde)` which calls `eigsh` in shift-invert mode (`sigma=0, which='LM'`) to find the eigenvector of the discrete $\widetilde{\mathcal{M}}^{(N)}$ whose eigenvalue is closest to zero. For our F2/T0 bounce this returns $\lambda_{\rm zm} \approx -8.5 \times 10^{-3}$ (slightly negative due to the bounce profile not satisfying its EOM exactly – see §6.4, Q3 of §9.2.6). The pole of $\overline{G}^{\rm sub}_{n=1}$ is then at $s^2 = -\lambda_{\rm zm} \approx +0.0085$, not at exactly 0.

Why option (b) is the cleaner choice. The analytic $\chi_{\rm zm}$ from option (a) is the continuum zero mode of an exact bounce; on the discrete grid it is only approximately an eigenvector of $\widetilde{\mathcal{M}}^{(N)}$ (its overlap with the true discrete near-zero eigenvector is ~ 0.9999 for our F2/T0 bounce). Option (b) returns the actual discrete eigenvector, so the projector $P = \mathbb{I} - \chi_{\rm zm}\chi_{\rm zm}^\top$ removes the discrete near-zero mode *exactly* from the spectral sum, with no residual "hairpin" near $s^2 \approx 0.0085$. The v2 `compute_gbar_n1_fd_ver2.py` script uses option (b) by default and additionally cross-checks the returned eigenvector against the analytic formula (overlap warning if below 0.95, $|\lambda_{\rm zm}|$ warning if above 0.1, both promoted to hard errors with `--strict-zm`).

Why we get away with one mode and not the full spectrum. The trace identity is

$$\mathrm{Tr}\big[P(\widetilde{\mathcal{M}}_n + s^2)^{-1}P\big] \;=\; \sum_{L\neq\bar{L}} \frac{1}{\lambda_L + s^2}. \tag{172}$$

Algebraically the right-hand side has been pole-subtracted by virtue of P, but it still contains *every other* eigenvalue λ_L in the spectrum. We never enumerate those: the random-vector

trace estimator (Hutchinson, §7) samples them implicitly through repeated sparse-LU solves $Ax = v$. Each solve costs $\mathcal{O}(N)$ for our banded A, and $K \sim 100$ samples give $\sim 1\%$ statistical noise. That is many orders of magnitude cheaper than computing the ~ 4000 interior eigenpairs explicitly.

Summary of the workflow.

$n = 0$ **(negative mode)**	$n = 1$ **(translation zero mode)**
1. `eigsh(M_tilde, k=3, which='SA')` returns $(\lambda_{\text{neg}}, \chi_{\text{neg}})$	1. `build_translation_zero_mode(r, ...)` returns $\chi_{\text{zm}} \propto r^{3/2}\phi_b'(r)$
2. clamp χ_{neg} at boundary indices, renormalize	2. clamp χ_{zm} at boundary indices, renormalize
3. pole at $s_\star^2 = -\lambda_{\text{neg}}$	3. pole at $s_\star^2 = 0$
4. for each s^2 in adaptive grid: raw = Hutchinson$\big[(\widetilde{\mathcal{M}} + s^2 I)^{-1}\big]$ sub = Hutchinson$\big[P(\widetilde{\mathcal{M}} + s^2 I)^{-1} P\big]$ with $P = \mathbb{I} - \chi_{\text{neg}}\chi_{\text{neg}}^\top$	4. for each s^2 in log+linear grid: raw = Hutchinson$\big[(\widetilde{\mathcal{M}} + s^2 I)^{-1}\big]$ sub = Hutchinson$\big[P(\widetilde{\mathcal{M}} + s^2 I)^{-1} P\big]$ with $P = \mathbb{I} - \chi_{\text{zm}}\chi_{\text{zm}}^\top$
5. save `gbar_n0_fd_F{F}_T{T}.npz`	5. multiply by $(n+1)^2 = 4$, save `gbar_n1_fd_F{F}_T{T}.npz`

5.5 Why the FD subtraction has no residual spike (RK comparison)

A practical advantage of the FD/Feshbach method over the RK/Wronskian method is that $\overline{G}^{\text{sub}}$ comes out *cleanly* – smooth and finite through the pole region, with no residual spike. The RK implementation in `compute_gbar_n0.py` and `compute_gbar_n1.py`, by contrast, leaves a visible leftover spike near the pole even after subtraction. The reason is that the two methods do mathematically different things.

What RK does: numerical pole fit. RK computes the raw $\overline{G}(s^2)$ via Wronskian construction, then fits the leading singular form across a window near the pole:

$$\overline{G}(s^2) \;\approx\; \frac{A}{s^2 - s_\star} \;+\; B \qquad \text{(fit ansatz)}. \tag{173}$$

The fit returns numerical estimates $\hat{A}$, $\hat{s}_\star$, $\hat{B}$ from a least-squares solve. The subtracted curve is built point by point as

$$\overline{G}^{\text{sub}}_{\text{RK}}(s^2) \;=\; \overline{G}(s^2) \;-\; \frac{\hat{A}}{s^2 - \hat{s}_\star}. \tag{174}$$

This is the line in `compute_gbar_n0.py`:

```
gbar_sub[idx] = gbar_arr[idx] - A_fit / (s2 - s_star)
```

Why the RK subtraction leaves a spike. Equation (173) is only the *leading* singular behavior. The exact $\overline{G}$ has higher-order corrections,

$$\overline{G}(s^2) \;=\; \frac{A_{\text{true}}}{s^2 - s_\star^{\text{true}}} \;+\; B(s^2) \;+\; \mathcal{O}(s^2 - s_\star^{\text{true}}), \tag{175}$$

and the fit captures only the first two. Three sources of residual spike emerge:

1. **Coefficient error.** The fit returns $\hat{A} \neq A_{\text{true}}$; the residual is $(\hat{A} - A_{\text{true}})/(s^2 - s_\star)$, still divergent at $s^2 = s_\star$.

2. **Pole-location error.** The fit returns $\hat{s}_\star \neq s_\star^{\text{true}}$; even with the exact A, $A/(s^2 - \hat{s}_\star) \neq A/(s^2 - s_\star^{\text{true}})$ near the pole, so a near-pole residual survives.

3. **Catastrophic cancellation.** For s^2 very close to $s_\star$, $\overline{G}$ and $\hat{A}/(s^2 - \hat{s}_\star)$ are both huge ($\sim 10^4$), and (174) subtracts them to extract a small finite piece. Floating-point cancellation amplifies the relative error by orders of magnitude.

The visible RK spike is a combination of all three, plus the fact that $A_{\text{true}}/(s^2 - s_\star^{\text{true}})$ *itself* is not the entire singular structure – there can be subleading singular pieces that the simple ansatz misses.

What FD does: analytic projection in the basis of eigenmodes. The FD method does *not* subtract a fitted form. It removes the entire problematic eigenmode from the operator's spectral sum *exactly*, by working in a subspace orthogonal to that mode. Reading off equations (149) and (150):

$$\overline{G}^{\text{raw}}(s^2) \;=\; (n+1)^2 \sum_{\text{all } L} \frac{1}{\lambda_L + s^2} \quad \to \quad \text{has pole at } s^2 = -\lambda_\chi, \tag{176}$$

$$\overline{G}^{\text{sub}}(s^2) \;=\; (n+1)^2 \sum_{L \neq \bar{L}} \frac{1}{\lambda_L + s^2} \quad \to \quad \text{the pole-bearing term is just absent.} \tag{177}$$

There is no fit, no estimate, no near-cancellation. The term that would have produced the pole simply does not appear in the sum, because $P\chi = 0$ (Case A in §4.1) zeroes its outer product contribution to the trace.

Why the FD residual is essentially zero. The only thing FD needs in order to do this is *the eigenvector* χ. The eigenvalue λ_χ does not enter the projector $P = \mathbb{I} - \chi\chi^\top$ at all. So even if our numerical estimate of λ_χ is slightly off, the projection still kills exactly one term – the term it would have killed using the exact eigenvector.

In the FD code, χ_{neg} comes from `eigsh` at tolerance 10^{-9} and is then clamped at boundary indices and renormalized; the residual error $\|\widetilde{\mathcal{M}}\,\chi - \lambda\,\chi\|$ is at the level of 10^{-9}. The projector built from this χ removes the pole-bearing eigenmode to the same accuracy. Whatever leftover spike exists is therefore $\sim 10^{-9}$ times the original spike height – completely invisible on any plot.

Numerical comparison side by side.

	RK / Wronskian	**FD / Feshbach**
What is subtracted	$\hat{A}/(s^2-\hat{s}_\star)$ – a *fitted* singular form	$P\,(\cdot)\,P$ – an exact *projection* that drops one term from the spectral sum
What you need to know about the mode	amplitude A and pole location $s_\star$ (both fitted from data)	the eigenvector χ (computed by `eigsh`); eigenvalue not needed for the projector
Source of residual spike	$(\hat{A}-A_{\text{true}})/(s^2-s_\star)+\hat{A}\,(s_\star-\hat{s}_\star)/[(s^2-\hat{s}_\star)(s^2-s_\star)]+$ floating-point cancellation in (174)	none, because the pole-bearing term is removed analytically by $P\chi=0$; only an $\mathcal{O}$(eigsh tol) residual that is invisible on any plot
Code line that does the subtraction	`gbar - A_fit/(s2 - s_star)` in `compute_gbar_n0.py`	`gbar_sub_fd(M_tilde, s2, dr, N2, chi)`, internally `Hutchinson(P A_inv P)`
Number of fit / numerical-tuning parameters	3 $(A, s_\star, B)$ plus a fit window choice	0
Behavior near the pole	two large numbers subtracted; floating-point error visible as a "residual spike"	no large numbers subtracted; the projected solve is itself well-defined away from $\pm$skip_radius

Connection back to the code. In `compute_gbar_n0_fd.py` the relevant line is

```
sub_val, sub_err = gbar_sub_fd(
    M_tilde, s2, dr, N2, chi_neg, K=args.K, rng=rng,
    boundary_indices=boundary_indices,
)
```

Inside `gbar_sub_fd` (`fd_builder.py`), the Hutchinson estimator with the projected variant is called – which performs the $PA^{-1}P$ trace via random probes. The `chi_neg` argument is the only ingredient that "knows" about the negative mode; once χ_{neg} is plugged in, the pole-bearing term is gone from the trace. There is no "$/(s^2-s_\star)$" anywhere in the FD code path, and consequently no source of catastrophic cancellation.

For the $n=1$ zero mode the story is identical, with χ_{zm} (built analytically from $r^{3/2}\phi_b'$) playing the role of χ_{neg}:

```
sub_val, sub_err = gbar_sub_fd(
    M_tilde, s2, dr, N2, chi_zm, K=args.K, rng=rng,
    boundary_indices=boundary_indices,
)
```

The four-fold degeneracy of the translation modes is folded in afterwards by multiplying the sub trace by $(n+1)^2=4$.

Bottom line. The FD method's $\overline{G}^{\text{sub}}$ is clean because it does not subtract two large numbers – it never builds a large pole-shaped piece in the first place. The projector excises the bad eigenmode *from the operator*, before the trace is ever evaluated. The RK subtraction

operates after the fact on a function that already contains the divergence, and is therefore at the mercy of catastrophic cancellation and pole-fit accuracy.

6 Discretization

6.1 The grid and the unknowns

Choose $r_{\min} > 0$ small (the radial coordinate is singular at $r = 0$, so we keep a non-zero cutoff) and $r_{\max} > r_b$ (well past the bounce wall). Use a uniform grid

$$r_i = r_{\min} + i\,\mathrm{d}r, \qquad i = 0, 1, \ldots, N-1, \qquad \mathrm{d}r = \frac{r_{\max} - r_{\min}}{N-1}. \tag{178}$$

For two fields, the unknown vector is

$$\mathbf{u} \;=\; \big(\,\tilde{u}_1^{(0)}, \tilde{u}_1^{(1)}, \ldots, \tilde{u}_1^{(N-1)},\; \tilde{u}_2^{(0)}, \ldots, \tilde{u}_2^{(N-1)}\,\big)^{\top} \;\in\; \mathbb{R}^{2N}. \tag{179}$$

6.2 Second-derivative stencil

Taylor-expand $\tilde{u}(r_i \pm \mathrm{d}r)$:

$$\tilde{u}_{i\pm1} \;=\; \tilde{u}_i \pm \mathrm{d}r\,\tilde{u}_i' + \tfrac{\mathrm{d}r^2}{2}\,\tilde{u}_i'' \pm \tfrac{\mathrm{d}r^3}{6}\,\tilde{u}_i''' + \tfrac{\mathrm{d}r^4}{24}\,\tilde{u}_i^{(4)} + \mathcal{O}(\mathrm{d}r^5). \tag{180}$$

Adding,

$$\tilde{u}_{i+1} + \tilde{u}_{i-1} \;=\; 2\tilde{u}_i + \mathrm{d}r^2\,\tilde{u}_i'' + \tfrac{\mathrm{d}r^4}{12}\,\tilde{u}_i^{(4)} + \mathcal{O}(\mathrm{d}r^6), \tag{181}$$

which gives the standard 2nd-order central stencil

$$\tilde{u}_i'' \;\approx\; \frac{\tilde{u}_{i+1} - 2\tilde{u}_i + \tilde{u}_{i-1}}{\mathrm{d}r^2} \;+\; \mathcal{O}(\mathrm{d}r^2). \tag{182}$$

Encoding $-\partial_r^2$ in matrix form L_2 produces a tridiagonal $N \times N$ matrix.

6.3 Block matrix for the 2-field problem (and how it differs from the 1-field case)

Our problem has $\phi = (\phi_1, \phi_2)$ – two real scalar fields – so at each grid point r_i we have *two* independent unknowns, one per field. Most of this section walks through what that "2" adds compared with the textbook one-field ($\phi = \phi_1$, no second component) version.

What the 1-field problem looks like

For a single scalar ϕ_1, the Hessian at each point is just one number $U''(\phi_b(r_i))$, and the radial fluctuation operator is the $N \times N$ symmetric tridiagonal

$$\widetilde{\mathcal{M}}_n^{(N),\,\text{1-field}} \;=\; -L_2 \;+\; \mathrm{diag}\big(V_n(r_i) + U''(\phi_b(r_i))\big), \tag{183}$$

where $V_n(r_i) = (n(n+2) + 3/4)/r_i^2$ is the centrifugal-plus-Liouville diagonal and $-L_2$ is the second-derivative tridiagonal matrix (274) from §9.2.2. The unknown vector is one length-N array $\tilde{u}_1 = (\tilde{u}_1^{(0)}, \ldots, \tilde{u}_1^{(N-1)})$.

What the 2-field problem adds

With *two* fields, the Hessian at each grid point is now a 2×2 **symmetric matrix**:

$$U''\big(\phi_b(r_i)\big) \;=\; \begin{pmatrix} U''_{11,i} & U''_{12,i} \\ U''_{12,i} & U''_{22,i} \end{pmatrix}, \qquad U''_{ab,i} \;=\; \left.\frac{\partial^2 U}{\partial\phi_a\,\partial\phi_b}\right|_{\phi_b(r_i)}. \tag{184}$$

The diagonal entries $U''_{11,i}, U''_{22,i}$ are the on-site masses-squared for fields 1 and 2; the off-diagonal $U''_{12,i}$ is the **field-coupling**, which mixes fluctuations of ϕ_1 with those of ϕ_2 at every r_i. *If $U''_{12} \equiv 0$ everywhere, the two fields decouple and the 2-field problem reduces to two independent 1-field problems.*

Concretely, the unknown vector now has $2N$ components,

$$\mathbf{u} \;=\; \big(\underbrace{\tilde{u}_1^{(0)}, \ldots, \tilde{u}_1^{(N-1)}}_{\text{field 1, length } N},\; \underbrace{\tilde{u}_2^{(0)}, \ldots, \tilde{u}_2^{(N-1)}}_{\text{field 2, length } N}\big)^{\top} \;\in\; \mathbb{R}^{2N}, \tag{185}$$

field 1 first then field 2 (this is the convention in the code: `u[0:N]` is field 1, `u[N:2*N]` is field 2). The discretized $\widetilde{\mathcal{M}}_n$ is the $2N \times 2N$ symmetric block matrix

$$\widetilde{\mathcal{M}}_n^{(N)} \;=\; \begin{pmatrix} \underbrace{-L_2 + \mathrm{diag}(V_n + U''_{11})}_{\text{field 1 self-block}} & \underbrace{\mathrm{diag}(U''_{12})}_{1\leftrightarrow 2\text{ coupling}} \\ \underbrace{\mathrm{diag}(U''_{12})}_{2\leftrightarrow 1\text{ coupling}} & \underbrace{-L_2 + \mathrm{diag}(V_n + U''_{22})}_{\text{field 2 self-block}} \end{pmatrix}. \tag{186}$$

What is new compared with the 1-field case.

1. **Vector size doubles:** $N \to 2N$. For our $N = 2000$ that means a length-4000 unknown vector instead of length-2000.

2. **Two diagonal self-blocks instead of one.** Each is structurally the same as the 1-field $\widetilde{\mathcal{M}}_n^{(N)}$ (183), but with its own on-site mass (U''_{11} for the field-1 block, U''_{22} for the field-2 block).

3. **Two new off-diagonal coupling blocks** $\mathrm{diag}(U''_{12})$, placed at rows $0 \ldots N{-}1$, columns $N \ldots 2N{-}1$ (and its transpose). These are diagonal $N \times N$ matrices, NOT tridiagonal – they couple field 1 entry i with field 2 entry i *at the same grid point*, never at different grid points. (No spatial derivative appears in the coupling because the coupling comes from the algebraic Hessian $\partial^2 U/\partial\phi_1\partial\phi_2$, not from any derivative term.)

4. **Doubled boundary index set.** For the 1-field problem, Dirichlet BCs touch indices $\{0, N{-}1\}$. For 2 fields, the boundary set is $\mathcal{B} = \{0,\ N{-}1,\ N,\ 2N{-}1\}$ – the start/end of each field block. Same symmetric trick $\widetilde{\mathcal{M}} \to D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$ applies, just with 4 indices instead of 2. The boundary ghost subspace is 4-dimensional instead of 2.

5. **Eigenmodes have two field components.** Each eigenvector χ_L of $\widetilde{\mathcal{M}}_n$ is a length-$2N$ vector that we can split into two length-N pieces, $\chi_L = (\chi_{L,1}, \chi_{L,2})$, one per field. The same applies to the special projector mode χ_{neg} (n=0) or χ_{zm} (n=1). The Hutchinson probes v are also length-$2N$, with random ± 1 entries spread across both fields.

6. **Trace sums over both fields.** The trace $\mathrm{Tr}[(\widetilde{\mathcal{M}}_n + s^2\mathbb{I})^{-1}]$ is the sum over all $2N$ diagonal entries of the inverse, picking up contributions from both field components automatically.

7. **LU has more fill-in.** For a pure tridiagonal 1-field matrix, the LU factors are bidiagonal (just sub- or super-diagonal). For our 2-field block-tridiagonal matrix, the LU acquires a few additional non-zero entries at the rows where the two fields couple via U''_{12}. SuperLU (`scipy.sparse.linalg.splu`) tracks all of these automatically – see the discussion in §9.2.2 step (vi).

Sparsity count. The whole $2N \times 2N$ matrix is sparse with $\sim 8N$ non-zero entries: each of the two $N \times N$ self-blocks contributes $\sim 3N$ (tridiagonal), and each of the two $N \times N$ coupling blocks contributes N (diagonal). For $N = 2000$ that is about 16 000 non-zeros stored in the sparse representation, instead of $(2N)^2 = 16\,000\,000$ entries that the dense version would need.

When does this all collapse to two independent 1-field problems? If the bounce keeps ϕ_2 identically zero everywhere (or the potential is separable, $U(\phi_1, \phi_2) = U_1(\phi_1) + U_2(\phi_2)$), then $U''_{12,i} = 0$ at every grid point, the two off-diagonal blocks vanish, and (186) becomes block-diagonal:

$$\widetilde{\mathcal{M}}_n^{(N)}\big|_{U''_{12}=0} \;=\; \begin{pmatrix} \widetilde{\mathcal{M}}_n^{(N),\,\text{1-field 1}} & 0 \\ 0 & \widetilde{\mathcal{M}}_n^{(N),\,\text{1-field 2}} \end{pmatrix}. \tag{187}$$

In that limit the trace splits into two independent 1-field traces (no fill-in in LU between the blocks; each can be solved separately). For our actual bounce $U''_{12} \neq 0$ at the wall, so the coupling matters and the full 2-field machinery is needed throughout.

6.4 Where do boundary conditions come from?

The continuum problem lives on the half-line $r \in [0, \infty)$. To make the spectrum discrete (as opposed to a continuum of scattering states) and to make the eigenmodes normalizable, we need to impose *boundary conditions* on $\tilde{u}(r)$ at both ends. These conditions come from physics, not from numerical convenience.

Origin behavior ($r \to 0$). Near $r = 0$ the centrifugal term in $\widetilde{\mathcal{M}}_n$ dominates, $\widetilde{\mathcal{M}}_n \sim -\partial_r^2 + (n(n+2) + 3/4)/r^2$. The two linearly independent solutions of $\widetilde{\mathcal{M}}_n \tilde{u} = E\tilde{u}$ behave as $\tilde{u} \sim r^{n+3/2}$ (regular) and $\tilde{u} \sim r^{-n-1/2}$ (singular). The singular branch is non-normalizable ($\int |\tilde{u}|^2 dr$ diverges at $r = 0$ for any $n \geq 0$), so we must keep only the regular branch:

$$\tilde{u}(r) \xrightarrow[r\to 0]{} 0 \quad \text{(Dirichlet at the origin)}. \tag{188}$$

Concretely, regular modes satisfy $\tilde{u}(0) = 0$.

Asymptotic behavior ($r \to \infty$): bound states vs. continuum. Far from the bounce wall, the potential approaches its false-vacuum value: $U''(\phi_b(r)) \to U''(\phi_{\text{fv}}) = m^2$. The *eigenvalue equation* for $\widetilde{\mathcal{M}}_n$ in the asymptotic region is $(-\partial_r^2 + m^2)\,\tilde{\phi}_L = \lambda_L\,\tilde{\phi}_L$, whose solutions are

$$\tilde{\phi}_L(r) \;\sim\; \begin{cases} e^{-\kappa_L r}, \;\; \kappa_L = \sqrt{m^2 - \lambda_L} & \text{if } \lambda_L < m^2 \quad (\textit{bound mode}), \\ \sin(k_L r + \delta), \;\; k_L = \sqrt{\lambda_L - m^2} & \text{if } \lambda_L > m^2 \quad (\textit{continuum/scattering mode}). \end{cases} \tag{189}$$

So the spectrum splits into two pieces:

- **Bound modes ($\lambda_L < m^2$):** a finite, discrete set of localized states that decay exponentially at infinity. The negative mode ($\lambda_{\rm neg} < 0 < m^2$) and the translation zero modes ($\lambda_{\rm zm} = 0 < m^2$) are both bound modes. Higher partial waves typically have a few low bound modes too.
- **Continuum modes ($\lambda_L > m^2$):** an unbounded continuous spectrum of oscillatory "scattering" states.

Why we still impose $\tilde{u}(r_{\rm max}) = 0$ everywhere ("putting the system in a box"). On the infinite half-line, only bound modes are normalizable; the continuum modes are delta-function-normalized. For numerical work we truncate the half-line to $[r_{\rm min}, r_{\rm max}]$ and impose Dirichlet BCs at *both* ends – on *all* modes, bound and continuum alike. Two consequences:

1. **Bound modes are essentially exact.** They decay as $e^{-\kappa_L r}$ with $\kappa_L \sim m$, so by $r_{\rm max} \gg 1/m$ they are already astronomically small. Imposing $\tilde{u}(r_{\rm max}) = 0$ on them is a vanishingly small perturbation.
2. **Continuum modes get discretized.** The Dirichlet box replaces the continuous spectrum with a fine discrete one – essentially particles in a 1D infinite-well of width $r_{\rm max} - r_{\rm min}$. The number of these "boxed" modes scales with N, and they fill out the high end of the spectrum. Their eigenvalues λ_L are large, so they contribute little to the trace $\sum_L 1/(\lambda_L + s^2)$ and even less to anything we care about (their full contribution is what UV renormalization removes).

So the BC is the *same* for all modes ($\tilde{u} = 0$ at both ends); the bound modes are barely affected, and the continuum modes become a discrete approximation of themselves. The argument is not "only bound modes need this BC" – the BC is imposed uniformly to turn an infinite-dimensional operator into a finite matrix, and the quality of the approximation depends on the mode.

Why "Dirichlet" specifically – and why no other choice gives the right physics

This is the question to spend a moment on, because it is easy to ask "why not Neumann or Robin?" and not see the answer until you look at the physics at each end carefully. The short version: Dirichlet $\tilde{u}(r_{\rm min}) = \tilde{u}(r_{\rm max}) = 0$ is the *only* boundary condition that simultaneously

- picks the regular (normalizable) solution at $r = 0$,
- matches the asymptotic decay of every physical bound mode at large r, and
- makes $\widetilde{\mathcal{M}}_n$ self-adjoint on $L^2(0, \infty; {\rm d}r)$, so the spectral theorem applies and the eigenvectors are orthogonal.

Below is the argument at each end and why each alternative fails.

(a) At the origin $r = 0$: only Dirichlet is normalizable. Near $r = 0$ the operator $\widetilde{\mathcal{M}}_n \approx -\partial_r^2 + (n(n+2) + 3/4)/r^2$ has two linearly independent local solutions,

$$\tilde{u}(r) \;\sim\; r^{\,n+3/2} \quad (\text{"regular"}), \qquad \tilde{u}(r) \;\sim\; r^{\,-n-1/2} \quad (\text{"singular"}). \tag{190}$$

A physical mode must be in L^2, i.e. $\int_0^\infty |\tilde{u}|^2 \, \mathrm{d}r < \infty$. Plug each branch into the integrand:

$$|r^{\,n+3/2}|^2 = r^{2n+3} \qquad \int_0^\epsilon r^{2n+3} \, \mathrm{d}r \;=\; \tfrac{\epsilon^{2n+4}}{2n+4} \;<\; \infty \quad \text{(converges)} \tag{191}$$

$$|r^{\,-n-1/2}|^2 = r^{-2n-1} \qquad \int_0^\epsilon r^{-2n-1} \, \mathrm{d}r \;=\; \begin{cases} \infty & n \geq 0 \text{ (\textit{always diverges})}. \end{cases} \tag{192}$$

The singular branch is non-normalizable, so a physical mode must contain only the regular branch, i.e. $\tilde{u}(0^+) = 0$ – this is exactly Dirichlet at the origin.

- Neumann would impose $\tilde{u}'(0) = 0$. The regular branch $r^{n+3/2}$ has derivative $(n + \frac{3}{2})\, r^{n+1/2}$, which vanishes at $r = 0$ for $n \geq 0$ – so Neumann is consistent with the regular branch *only by accident* for $n \geq 1$, but it does *not* exclude the singular branch (which has infinite derivative at $r = 0$ but is not constrained by a finite Neumann condition). The result is a different self-adjoint extension of the operator with extra spurious modes.
- Robin (mixed: $\alpha\, \tilde{u}(0) + \beta\, \tilde{u}'(0) = 0$) reduces to Dirichlet only if you set $\beta = 0$; any other choice picks a linear combination of the two branches that is still non-normalizable.

So at the origin, Dirichlet is the unique BC consistent with L^2.

(b) At $r \to \infty$: the BC argument is different for bound vs. continuum modes. Important caveat up front. The asymptotic-decay argument "modes vanish at $r \to \infty$" applies *only to bound modes*. On the infinite half-line, continuum modes ($\lambda_L > m^2$) are NOT in L^2; they oscillate as $\sin(k_L r + \delta)$ forever and never decay. They are delta-function-normalized scattering states, not square-integrable wavefunctions. So no BC at $r = \infty$ is "physically required" for continuum modes – they don't satisfy any BC there in the continuum.

What we are actually doing. We truncate to $[r_{\min}, r_{\max}]$ and impose a BC at $r_{\max}$ because the finite-matrix problem requires one. This is an unavoidable mathematical step, not a statement about physics at $r_{\max}$. For each mode type:

- **Bound modes** ($\lambda_L < m^2$, $\tilde{u} \sim e^{-\kappa_L r}$): Dirichlet $\tilde{u}(r_{\max}) = 0$ is essentially *exact*. The mode is already $\sim e^{-\kappa_L r_{\max}}$ at the right edge, machine-epsilon for $r_{\max} \gg 1/m$. Forcing the function to be zero there changes nothing measurable. (Neumann at $r_{\max}$ would also be essentially exact for bound modes alone – the derivative is tiny too. For bound modes *in isolation*, Dirichlet vs. Neumann is a non-issue.)
- **Continuum modes** ($\lambda_L > m^2$, $\tilde{u} \sim \sin(k_L r + \delta)$): the BC is *not* an approximation to something physical – it is a *deliberate boxing* of the continuum. Different BCs select different discrete subsets of the continuum:
  - Dirichlet: k_L's such that $\sin(k_L r_{\max} + \delta) = 0$ (sine-wave standing waves).
  - Neumann: k_L's such that $\cos(k_L r_{\max} + \delta) = 0$ (cosine-wave standing waves).

  These two sets are different (shifted by half a wavelength in k_L) but *both converge to the same continuum density of states as* $r_{\max} \to \infty$. They differ by an $\mathcal{O}(1/r_{\max})$ boundary contribution to the trace. *Neither is the "physical" choice* for continuum modes – in the continuum, neither sin nor cos is preferred.

Why we still pick Dirichlet at $r_{\max}$. Not because continuum modes "decay" (they don't), but for two purely mathematical reasons that link to (a):

1. Origin BC is forced to be Dirichlet (from (a) – this argument is universal, applies to bound and continuum modes alike).

2. To keep the symmetric trick $D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$ symmetric, the BCs at both ends must coincide. Dirichlet at both ends is the only choice that does this. (Mixed Dirichlet at $r = 0$ + Neumann at $r_{\max}$ is still a self-adjoint operator in the abstract sense, but the corresponding matrix is non-symmetric, breaking ARPACK and the Feshbach machinery.)

The fact that bound modes *also* happen to decay at $r_{\max}$ is a happy coincidence that makes the Dirichlet-at-$r_{\max}$ error on bound modes vanishingly small. The same Dirichlet, applied to continuum modes, is a controlled approximation whose error scales as $1/r_{\max}$ and which is removed in the renormalization step anyway.

(c) Why both ends use *the same* BC. The key spectral-theory fact is that the operator $\widetilde{\mathcal{M}}_n$ is self-adjoint on $L^2(0,\infty;\mathrm{d}r)$ *only* for specific choices of boundary conditions. The deficiency-index analysis shows that $\widetilde{\mathcal{M}}_n$ at the origin is in the "limit-point" case for $n \geq 1$ (no boundary condition is needed – regularity already does it) and in the "limit-circle" case for $n = 0$ (a single boundary condition is needed; Dirichlet is the standard physical choice). Either way, Dirichlet at the origin is consistent.

At the truncated $r_{\max}$ we are introducing an artificial boundary that doesn't exist in the continuum; we want it to be as inert as possible. Dirichlet kills the wavefunction there, which is a small perturbation if $r_{\max}$ is far enough out. Choosing *the same* BC at both ends keeps the discrete operator *symmetric*, which is what makes its eigenvalues real and its eigenvectors orthogonal – the prerequisite for the spectral theorem (§9.2.3) and the projection mechanism (§4) to work. Mixing Dirichlet at one end with Neumann at the other gives a non-symmetric matrix, breaking the very properties we relied on.

(d) What happens numerically with the wrong BC. If you wrongly impose Neumann at $r = 0$:

- ARPACK still returns eigenvalues, but they are eigenvalues of a *different* self-adjoint extension of the operator. The negative-mode eigenvalue λ_{neg} moves to a slightly different value; the discrete near-zero mode of $n = 1$ moves too. Pole locations in $\overline{G}$ shift.

- The radial profile of χ_{neg} changes near $r = 0$ – it no longer goes to zero as $r^{n+3/2}$, so it does not approximate the continuum negative mode.

- Subsequent quantities – $\overline{G}^{\text{sub}}$, the integrated log-determinant, the decay rate – come out at the wrong numerical values.

So Dirichlet is not just one of several reasonable choices; it is *the* choice required by (1) regularity / normalizability at $r = 0$ and (2) self-adjointness of the truncated operator. Any deviation gives systematically wrong physics, not just a different discretization error.

(e) Higher partial waves $n \geq 2$, where most or all modes are continuum-like. A natural follow-up: arguments (a)–(c) above lean on bound modes existing in the spectrum. What about higher n where the centrifugal barrier $n(n+2)/r^2$ is so large that there are very few or zero bound modes, and the spectrum is dominated by continuum ($\lambda_L > m^2$) modes? Does Dirichlet still uniquely apply?

Yes – the BC argument does not depend on whether the modes are bound or continuum. The reason is that the two pillars of the argument both still hold:

- **(a) at $r = 0$ becomes *stronger* for higher n.** The centrifugal term grows as $n(n+2)/r^2$, so the regular branch $\tilde{u} \sim r^{n+3/2}$ vanishes at the origin even more strongly, and the singular branch $\tilde{u} \sim r^{-n-1/2}$ diverges even more strongly. The L^2-normalizability check still kills the singular branch – in fact for any $n \geq 0$ the integral $\int_0^\epsilon r^{-2n-1}\,\mathrm{d}r$ diverges. So at the origin the Dirichlet condition $\tilde{u}(0^+) = 0$ remains the unique choice consistent with the function being in L^2. This argument has *nothing to do with whether the mode is bound or continuum*; it is about the local behaviour of the radial ODE near its regular singular point at $r = 0$.

- **(b) at $r_{\max}$ for continuum modes is the "putting the system in a box" part.** On the continuum half-line all modes with $\lambda_L > m^2$ are oscillatory and δ-function normalized, not in L^2 – they would need a careful rigged-Hilbert-space treatment. On a finite box $[r_{\min}, r_{\max}]$, choosing *any* self-adjoint BC at $r_{\max}$ replaces the continuous spectrum with a fine discrete approximation:

  – Dirichlet $\tilde{u}(r_{\max}) = 0$ picks the "sine-quantized" k_L values, $\sin(k_L\, r_{\max} + \delta) = 0$.

  – Neumann $\tilde{u}'(r_{\max}) = 0$ picks the "cosine-quantized" values, $\cos(k_L\, r_{\max} + \delta) = 0$.

  Both produce a fine discrete approximation of the same continuum. The two discretizations differ by an $\mathcal{O}(1/r_{\max})$ boundary contribution to the trace – specifically, by half a wavelength shift in the discrete k_L spacing. As $r_{\max} \to \infty$ the two BC choices converge to the same continuum density of states and the same trace.

- **Self-adjointness still selects Dirichlet at both ends.** The continuum-mode discretization is not unique on its own, but the *combined choice at both ends* is constrained by symmetry of the matrix and consistency at $r = 0$. Because we MUST use Dirichlet at the origin (from (a)), and we want a symmetric matrix (so the spectral theorem and Feshbach projection work), Dirichlet at $r_{\max}$ is the only choice that gives the same BC at both ends – which is what symmetry of the $D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$ sandwich requires. Mixing Dirichlet at $r = 0$ with Neumann at $r_{\max}$ would still be self-adjoint, but the matrix would not be symmetric, ARPACK's `eigsh` would refuse it, and the projector mechanism would lose its orthogonality guarantee.

Why the continuum-mode discretization choice doesn't matter much for $\overline{G}$ in practice. For higher n the modes are predominantly continuum-like, but their individual contributions to the trace

$$\frac{1}{\omega_{L,n}^2 + s^2} \tag{193}$$

are *small*, because $\omega_{L,n}^2$ is large (above m^2 plus the centrifugal $n(n+2)/r^2$ effective shift). Their *sum* over L is dominated by the bulk density of states, which is BC-independent in the $r_{\max} \to \infty$ limit. The contribution that does depend on the BC choice is the $\sim \mathcal{O}(1/r_{\max})$ boundary piece, which is what UV renormalization removes anyway (Lever 7 /

ContHutch++ in Part III discusses this; the standard Baacke-Kevlishvili reference treats it explicitly). So for higher n:

- Dirichlet remains the unique correct choice from origin regularity and matrix symmetry.
- The continuum modes that dominate are accurately represented *in bulk* by the Dirichlet box, with a small (and renormalization-removable) boundary artifact.
- The pole structure that we care about – the negative mode in $n = 0$ and the zero mode in $n = 1$ – only exists for low n, so for higher n there is no χ to project out and the "raw = sub" trace is itself the renormalized quantity.

In short: even when there are no bound modes at all, Dirichlet is still required by self-adjointness and origin-regularity, and the discretization of the continuum is consistent and converges to the right physics as $r_{\max} \to \infty$.

What goes wrong if we don't enforce BCs? A finite-difference matrix built without modifying the boundary rows treats r_0 and r_{N-1} as "regular interior points". The discrete operator then has stencils that reach *outside* the grid (positions r_{-1} and r_N that don't exist). The standard fix is to assume the function values there are zero – which is exactly the Dirichlet condition, but applied implicitly by construction. So strictly speaking our operator already "has Dirichlet BCs" once we use a stencil that treats off-grid values as zero; what the next subsection does is just to make this explicit and bake it into the matrix in a way that *preserves symmetry*, so that ARPACK and the projection mechanism work properly.

Two-field case. Each of the two fields has its own pair of boundary points, so the boundary index set in the $2N$-dimensional problem is

$$\mathcal{B} = \{0,\ N-1,\ N,\ 2N-1\} \tag{194}$$

($N-1$ at the right end of field 1, then N at the left end of field 2, etc.).

6.5 Symmetric Dirichlet BCs (the matrix-level implementation)

Plain-language story (no algebra yet)

Before any matrices, here is the human story.

Step 1 – the discrete problem. We have a length-N vector $\mathbf{u} = (u_0, u_1, \ldots, u_{N-1})^\top$, where u_i is the value of $\tilde{u}$ at grid point r_i. The matrix $\widetilde{\mathcal{M}}$ is built from finite-difference rules; *row i of $\widetilde{\mathcal{M}}$ is the equation enforced at grid point r_i.*

Step 2 – the BC says "the function is zero at the edges." On the grid that means $u_0 = 0$ and $u_{N-1} = 0$ – the *first* and *last* entries of the unknown vector are constrained to be zero. We want to bake that constraint into the matrix.

Step 3 – "the equation at point r_0 is: u_0 equals zero." We need one matrix row that says exactly this. The simplest way to write the equation $u_0 = 0$ as a matrix-vector

product is:

$$\underbrace{(1,0,0,\ldots,0)}_{=e_0^\top} \cdot \begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-1} \end{pmatrix} \;=\; 0. \tag{195}$$

The row vector on the left is the *unit basis vector* $e_0^\top = (1,0,0,\ldots,0)$. Dotting it with $\mathbf{u}$ literally "picks out" the first entry u_0. So if we put $e_0^\top$ as row 0 of our matrix, and 0 as the first entry of the right-hand side, the matrix equation at row 0 becomes $1\cdot u_0 + 0\cdot u_1 + \cdots = 0$, i.e. $u_0 = 0$. Same idea at the other boundary: row $N-1$ becomes $e_{N-1}^\top$ to enforce $u_{N-1} = 0$.

Why unit vectors at all? Because they are the simplest row that says "one specific entry of $\mathbf{u}$ equals (whatever the RHS says)." We use them at boundary positions to convert that matrix row into a pure constraint on a single unknown.

Step 4 – this naive trick destroys the symmetry of the matrix. If we replace boundary *rows* by $e_k^\top$, the boundary *columns* of the matrix are unchanged. Symmetric matrices have $M_{ij} = M_{ji}$; after the replacement, $M_{0,1} = 0$ (we set the entire row 0 to $e_0^\top$) but $M_{1,0}$ is still some non-zero number from the original $\widetilde{\mathcal{M}}$. So $M_{0,1} \neq M_{1,0}$ – the matrix has lost its symmetry. ARPACK's `eigsh` will refuse it; the spectral theorem (orthogonal eigenvectors, real eigenvalues) no longer applies; the Feshbach projection of §4 cannot be used.

Step 5 – the symmetric fix is to also delete the boundary columns. We zero *both* the boundary rows and the boundary columns. The row deletion enforces the BC; the column deletion restores symmetry. Together: the boundary subspace is decoupled from the interior subspace.

Step 6 – but a matrix with zero rows is singular. If row 0 is all zeros, no information is available to determine u_0 at all (the equation is $0 = 0$). To get a solvable system, we put a 1 **on the diagonal** at each boundary index. Now row 0 is $(1,0,0,\ldots,0) = e_0^\top$ – exactly the unit-vector row that the naive approach used in Step 3. So the unit vectors come back, but this time the corresponding columns are also zero (because of the symmetric column deletion in Step 5), so the matrix *stays symmetric*. The boundary block becomes a literal identity sub-matrix; the interior block is the original physics.

Step 7 – the cost of this trick is the "ghost eigenvalues." Because the boundary subspace is now a $|\mathcal{B}|$- dimensional identity block, each boundary basis vector e_k (the unit vector) is automatically an eigenvector of the modified matrix with eigenvalue 1. These are not approximations to physical modes; they exist purely because we put 1's on the boundary diagonal. §6.6 explains how to surgically exclude them from the trace – by zeroing out the random Hutchinson probe at the same boundary positions, so the probe has no overlap with the ghost subspace.

That is the entire story. The next subsubsections do it in algebra.

What problem are we solving here?

We have built the discrete operator $\widetilde{\mathcal{M}}_n^{(N)}$ from the FD stencil and the bounce Hessian. So far it knows nothing about the boundary conditions $\tilde{u}(r_{\min}) = \tilde{u}(r_{\max}) = 0$ that we just argued must be Dirichlet. We need to *bake those BCs into the matrix* so that:

1. Any solve $\widetilde{\mathcal{M}}\,\mathbf{x} = \mathbf{b}$ with Dirichlet RHS ($b_k = 0$ at boundary indices) returns $x_k = 0$ at boundary indices automatically;

2. The matrix *remains symmetric* after the modification – otherwise we lose ARPACK's `eigsh`, the spectral theorem, orthogonality of eigenvectors, and the Feshbach projection mechanism.

Goal (1) is easy in isolation; goal (2) is what makes this subsection non-trivial.

Why the naive approach fails

The simplest way to enforce $x_k = 0$ at index k in a linear solve is to *replace row k of the matrix with a unit basis vector $e_k^\top$ on the LHS* (and zero on the RHS). Then row k literally reads "$x_k = 0$."

Problem: this changes a row but not the corresponding column. The matrix is no longer symmetric. Schematically, for $N = 4$ and boundary index $k = 0$:

$$\widetilde{\mathcal{M}} \;\to\; \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ e & d_1 & e & 0 \\ 0 & e & d_2 & e \\ 0 & 0 & e & d_3 \end{pmatrix}}_{\text{not symmetric: row 0 is } e_0^\top \text{ but column 0 still has } e\text{'s.}} \tag{196}$$

ARPACK's `eigsh` requires a symmetric (or Hermitian) matrix and will refuse this one. Generic eigensolvers (`eigs`) work on non-symmetric matrices but return potentially complex eigenvalues and non-orthogonal eigenvectors. Crucially, the Feshbach projection of §4 derived its key identity $\chi^\top \chi_L = 0$ from the symmetry of $\widetilde{\mathcal{M}}$ (spectral theorem); without symmetry that identity fails and the analytic pole-removal machinery breaks.

The trick: zero rows AND columns symmetrically, then restore the boundary identity

The fix is to *symmetrize the row deletion by also deleting the column*, then add a small correction to keep the matrix invertible. Algebraically, build two simple diagonal matrices:

$$D_{ii} = \begin{cases} 0 & i \in \mathcal{B} \\ 1 & \text{otherwise} \end{cases}, \qquad (I_{\mathcal{B}})_{ii} = \begin{cases} 1 & i \in \mathcal{B} \\ 0 & \text{otherwise} \end{cases}, \tag{197}$$

which satisfy $D + I_{\mathcal{B}} = \mathbb{I}$. The replacement is

$$\boxed{\widetilde{\mathcal{M}}_n^{(N)} \;\longrightarrow\; \widetilde{\mathcal{M}}_{\mathrm{BC}} \;\equiv\; D\,\widetilde{\mathcal{M}}_n^{(N)}\,D \;+\; I_{\mathcal{B}}.} \tag{198}$$

The two pieces $D\,\widetilde{\mathcal{M}}\,D$ and $I_{\mathcal{B}}$ each play a specific role; let's unpack them.

What $D\,\widetilde{\mathcal{M}}\,D$ does. D acts as a diagonal mask: multiplying any matrix on the right by D zeros every entry whose *column* index is in $\mathcal{B}$; multiplying on the left zeros every entry whose *row* index is in $\mathcal{B}$. So $D\,\widetilde{\mathcal{M}}\,D$ has *zero entries everywhere* a boundary index appears in either row or column. In particular, the boundary rows are all zeros, and so are the boundary columns. The matrix is symmetric ($(DMD)^\top = D^\top M^\top D^\top = DMD$ because D is diagonal and $\widetilde{\mathcal{M}}$ is symmetric), and the boundary subspace is fully decoupled from the interior block. But: $D\,\widetilde{\mathcal{M}}\,D$ **is singular** – its boundary rows are all zero, so it has nullity $|\mathcal{B}|$. We can't solve linear systems with it as-is.

What adding $I_{\mathcal{B}}$ does. $I_{\mathcal{B}}$ is the diagonal matrix that is 1 at boundary indices and 0 elsewhere. Adding it to $D\,\widetilde{\mathcal{M}}\,D$ puts a 1 on the diagonal at each boundary index, while leaving every other entry of the matrix untouched. The resulting $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ has:

- the original interior $\widetilde{\mathcal{M}}$ entries unchanged in the interior $\times$ interior submatrix;
- zeros in every other position involving a boundary index;
- a single 1 on the diagonal at each boundary index.

This is invertible (the boundary subspace is now an identity block, not a zero block), still symmetric (we added a diagonal, which preserves symmetry), and decouples the boundary subspace from the interior at the matrix-algebra level.

Worked example ($N = 4$, single field, $\mathcal{B} = \{0, 3\}$). Suppose the original symmetric tridiagonal $\widetilde{\mathcal{M}}$ is

$$\widetilde{\mathcal{M}} = \begin{pmatrix} d_0 & e & 0 & 0 \\ e & d_1 & e & 0 \\ 0 & e & d_2 & e \\ 0 & 0 & e & d_3 \end{pmatrix}. \tag{199}$$

With $D = \mathrm{diag}(0, 1, 1, 0)$:

$$D\,\widetilde{\mathcal{M}}\,D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & d_1 & e & 0 \\ 0 & e & d_2 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad \text{(symmetric, but singular: rows 0 and 3 are zero).} \tag{200}$$

Add $I_{\mathcal{B}} = \mathrm{diag}(1, 0, 0, 1)$:

$$\widetilde{\mathcal{M}}_{\mathrm{BC}} \;=\; D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}} = \begin{pmatrix} \boxed{1} & 0 & 0 & 0 \\ 0 & d_1 & e & 0 \\ 0 & e & d_2 & 0 \\ 0 & 0 & 0 & \boxed{1} \end{pmatrix}. \tag{201}$$

The boxed 1's on the boundary diagonal are the only modifications beyond zeroing rows/columns. The matrix is now block-diagonal:

- A trivial 1×1 block at index 0 (the boxed 1).
- The 2×2 interior block on indices $\{1, 2\}$, identical to the original $\widetilde{\mathcal{M}}$ restricted to those rows/columns.
- A trivial 1×1 block at index 3.

Verifying the BC is enforced. Solve $\widetilde{\mathcal{M}}_{\mathrm{BC}}\,\mathbf{x} = \mathbf{b}$ with $\mathbf{b} = (0, b_1, b_2, 0)^\top$ (Dirichlet RHS):

$$\begin{aligned} \text{row 0:} \quad & 1 \cdot x_0 = 0 \implies x_0 = 0 \;\checkmark, \\ \text{rows 1, 2:} \quad & \text{the standard } 2 \times 2 \text{ interior solve}, \\ \text{row 3:} \quad & 1 \cdot x_3 = 0 \implies x_3 = 0 \;\checkmark. \end{aligned} \tag{202}$$

The Dirichlet condition is enforced *exactly*, with the interior solve unchanged, and the matrix is symmetric. We have achieved goals (1) and (2).

Side-by-side: "why isn't symmetric-trick already the same as naive?"

A natural follow-up: "the symmetric-trick matrix $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ has 1's on the boundary diagonal – isn't that already symmetric? What's actually different from the naive approach?" Both end up with 1's on the boundary diagonal; the two approaches differ in *whether the boundary columns are also zeroed*. Lining them up for $N = 4$, boundary $\mathcal{B} = \{0, 3\}$:

Strategy A – naive: replace boundary ROWS only by $e_k^\top$.

$$\widetilde{\mathcal{M}}_{\mathrm{naive}} = \begin{pmatrix} \boxed{1} & 0 & 0 & 0 \\ \boxed{e} & d_1 & e & 0 \\ 0 & e & d_2 & \boxed{e} \\ 0 & 0 & \boxed{0} & \boxed{1} \end{pmatrix} \tag{203}$$

NOT symmetric. Compare the boxed entries: $M_{\mathrm{naive}}[1,0] = e$ but $M_{\mathrm{naive}}[0,1] = 0$, and $M_{\mathrm{naive}}[2,3] = e$ but $M_{\mathrm{naive}}[3,2] = 0$. Replacing row 0 by $e_0^\top$ wiped the row, but the boundary *column* 0 was left untouched, so the original e at $(1,0)$ from the FD stencil is still there. Transposing $\widetilde{\mathcal{M}}_{\mathrm{naive}}$ gives a different matrix.

Strategy B – symmetric trick: $D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$ (zeros boundary ROWS *and* COLUMNS).

$$\widetilde{\mathcal{M}}_{\mathrm{BC}} = \begin{pmatrix} \boxed{1} & 0 & 0 & 0 \\ \boxed{0} & d_1 & e & 0 \\ 0 & e & d_2 & \boxed{0} \\ 0 & 0 & \boxed{0} & \boxed{1} \end{pmatrix} \tag{204}$$

Symmetric. Both $M_{\mathrm{BC}}[0,1] = 0$ AND $M_{\mathrm{BC}}[1,0] = 0$, both $M_{\mathrm{BC}}[2,3] = 0$ AND $M_{\mathrm{BC}}[3,2] = 0$. The boxed positions agree with their transposes. $(M_{\mathrm{BC}})^\top = M_{\mathrm{BC}}$.

Where the asymmetry would come from. The asymmetry isn't in the 1's on the diagonal – those are symmetric (a 1 at position $(0,0)$ equals the 1 at position $(0,0)$ when transposed). The asymmetry is in the boundary *column*: in Strategy A, the off-diagonal entries of column 0 (specifically $M[1,0] = e$) survive intact, while the corresponding row 0 entries (specifically $M[0,1]$) have been zeroed by the row replacement. The trick fixes this by zeroing the boundary columns too – which is exactly what the right-multiplication by D does. Three things, not just one:

	Strategy A (naive)	**Strategy B (trick)**
1 on boundary diagonal	yes	yes
boundary row zeroed off-diag	yes (by $e_k^\top$)	yes (by left-D)
boundary column zeroed off-diag	**no** – left untouched	yes (by right-D)
matrix symmetric	**NO**	yes
$M^\top = M$	fails	holds

So your reading of $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ as "obviously symmetric" is correct – the symmetric-trick matrix *is* symmetric. The non-symmetric version is the naive replace-rows-only matrix $\widetilde{\mathcal{M}}_{\mathrm{naive}}$, which we never actually use; the trick exists precisely to avoid it. The 1's on the boundary diagonal are a feature shared by both; the column zeroing is the feature that makes Strategy B symmetric.

Are we changing the fluctuation operator? Three matrices, one physics

A natural worry: the symmetric trick zeroes the boundary-column entries (the $e = -1/\mathrm{d}r^2$ FD-coupling terms between boundary points and their interior neighbours) that the original

$\widetilde{\mathcal{M}}$ *did* have. So technically the matrix entries are different. Are we secretly modifying the physics?

Three matrices coexist in this discussion:

Matrix	**Entry** $M[1,0]$	**Symmetric?**
Original $\widetilde{\mathcal{M}}$ (FD stencil only, no BC explicitly enforced)	$-1/\mathrm{d}r^2$	yes
$\widetilde{\mathcal{M}}_{\text{naive}}$ (row 0 replaced by $e_0^\top$, column intact)	$-1/\mathrm{d}r^2$	no
$\widetilde{\mathcal{M}}_{\text{BC}} = D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$ (trick: both row & column zeroed)	0	yes

Yes, $\widetilde{\mathcal{M}}_{\text{BC}}$ has *different entries* from the original $\widetilde{\mathcal{M}}$. But the entries that differ all multiply boundary values χ_0 in any matrix-vector product. **And the BC says** $\chi_0 = 0$**.** So for any vector that satisfies the BC, the lost terms contribute zero anyway:

$$\text{Row 1 of } \widetilde{\mathcal{M}}\,\chi \;=\; e\,\chi_0 + d_1\chi_1 + e\,\chi_2, \tag{205}$$

$$\text{Row 1 of } \widetilde{\mathcal{M}}_{\text{BC}}\,\chi \;=\; 0\cdot\chi_0 + d_1\chi_1 + e\,\chi_2, \tag{206}$$

$$\text{difference} \;=\; e\,\chi_0 \;=\; e\cdot 0 \;=\; 0 \qquad (\text{when } \chi_0 = 0). \tag{207}$$

So $\widetilde{\mathcal{M}}$ and $\widetilde{\mathcal{M}}_{\text{BC}}$ act identically on any vector that satisfies the Dirichlet BC.

Physical equivalence on the BC-respecting subspace. Restrict attention to the $(2N - |\mathcal{B}|)$-dimensional subspace of vectors that satisfy the BC ($\chi_k = 0$ for $k \in \mathcal{B}$). On this subspace the three matrices $\widetilde{\mathcal{M}}$, $\widetilde{\mathcal{M}}_{\text{naive}}$, $\widetilde{\mathcal{M}}_{\text{BC}}$ all agree. They only differ in what they do to vectors *that don't satisfy the BC* – and we never use those.

So why prefer $\widetilde{\mathcal{M}}_{\text{BC}}$ **over the original** $\widetilde{\mathcal{M}}$**?** The original $\widetilde{\mathcal{M}}$ doesn't *enforce* the BC – it just discretises the PDE, and assumes off-grid values are zero (the stencil truncation). Eigenvectors of the original $\widetilde{\mathcal{M}}$ have boundary entries that aren't strictly zero (small finite-volume artifacts). The trick produces eigenvectors that are *exactly* zero on the boundary by construction, which is what the projection-mechanism arguments in §4 require (clean orthogonality between the χ that we project against and everything else).

Three statements coexist, no contradiction.

1. Yes, $\widetilde{\mathcal{M}}_{\text{BC}}$ has different matrix entries from $\widetilde{\mathcal{M}}$. Strict "did the matrix change?" answer: *yes.*

2. Yes, $\widetilde{\mathcal{M}}_{\text{BC}}$ acts the same as $\widetilde{\mathcal{M}}$ on any vector satisfying the BC. Strict "did the physics change?" answer: *no.*

3. The benefit of the change is that the BC is *enforced exactly*, not just approximated, and the resulting matrix is *symmetric and BC-decoupled*, so `eigsh`, the spectral theorem, and the Feshbach projection mechanism all work cleanly.

"Physics unchanged": what exactly is invariant under the trick

The claim "physics unchanged" deserves a careful unpacking, because at the matrix level we did change entries. Let us go observable by observable.

Picture: three different operators. We are dealing with three different objects:

- the *continuum* radial operator $\widetilde{\mathcal{M}}_n$ on $L^2(0,\infty;\mathrm{d}r)$ with Dirichlet BCs at $r = 0$ and $r = \infty$ – this is what the physics actually wants;

- the discrete *stencil-truncation* approximation $\widetilde{\mathcal{M}}$, which incorporates Dirichlet implicitly by setting off-grid values to zero in the FD stencil;

- the discrete *symmetric-trick* approximation $\widetilde{\mathcal{M}}_{\mathrm{BC}}$, which enforces $u_{\mathrm{boundary}} = 0$ explicitly by zeroing rows/columns and adding $I_{\mathcal{B}}$.

$\widetilde{\mathcal{M}}$ and $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ are two different finite-matrix approximations to the *same* continuum operator. They are slightly different matrices at the boundary, but *both* converge to the same continuum spectrum as $\mathrm{d}r \to 0$. The difference between them is itself $\mathcal{O}(\mathrm{d}r^2)$ – of the same order as the FD discretization error vs. the continuum, well within the noise floor that we already accept.

(a) Eigenvectors – where the only real difference is. For $\widetilde{\mathcal{M}}$, the eigenvectors are vectors in $\mathbb{R}^{2N}$ that *approximately* vanish at boundary indices (small finite-volume artifacts of order $\mathrm{d}r^2$, because the FD equation at the boundary treats r_0, r_{N-1} as "regular interior points" coupled to off-grid zeros). For $\widetilde{\mathcal{M}}_{\mathrm{BC}}$, the boundary entries of each interior eigenvector are *exactly* zero by construction.

Both behaviours are valid finite-volume approximations to the continuum Dirichlet eigenmode that vanishes at $r = 0, r = \infty$. They differ only in whether the BC is approximate at the grid edge or exact. The *interior shape* of the eigenmode is the same in both; only the boundary entries differ.

(b) Eigenvalues – the same spectrum to leading order. The interior eigenvalues of $\widetilde{\mathcal{M}}$ and $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ are related as follows: $\widetilde{\mathcal{M}}_{\mathrm{BC}}$'s interior block *is* the original $\widetilde{\mathcal{M}}$ restricted to interior indices. $\widetilde{\mathcal{M}}$ has those same interior entries plus extra coupling to boundary entries (the FD-stencil $-1/\mathrm{d}r^2$ between r_0 and r_1, etc.). So:

- The interior eigenvalues of $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ are exactly the eigenvalues of the interior block (the $(2N - |\mathcal{B}|) \times (2N - |\mathcal{B}|)$ "interior-only" operator one would have written down with no boundary indices at all).

- The eigenvalues of the original $\widetilde{\mathcal{M}}$ differ from these by an $\mathcal{O}(\mathrm{d}r^2)$ amount that comes from the boundary coupling. The shift goes to zero as $\mathrm{d}r \to 0$.

For our $N = 2000$, both spectra agree to $\sim 10^{-5}$ on the relevant low-lying modes. Neither matters for the final $\overline{G}^{\mathrm{sub}}$ values because:

- The boundary-localized differences are at the 10^{-5} level, much smaller than our Hutchinson SEM ($\sim 10^{-2}$).

- The high-eigenvalue tail (where the boundary discretization matters more) gets removed by UV renormalization (which subtracts the false-vacuum result).

(c) Resolvent / Green's function – agree on the BC-respecting subspace. The trace $\mathrm{Tr}\big[(\widetilde{\mathcal{M}}_{\mathrm{BC}}+s^2)^{-1}\big]$ that we estimate via Hutchinson is restricted to the interior subspace (probe-zeroed at boundary). On that subspace, $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ *is* the interior-only operator. Its trace is therefore the discrete approximation to $\sum_L 1/(\omega_{L,n}^2 + s^2)$ – the very thing the continuum formula asks for. No boundary-coupling correction enters because those entries multiply zero on the BC-respecting subspace.

(d) Linear solves – exact equivalence on Dirichlet RHS. For any RHS $\mathbf{b}$ with $b_k = 0$ for $k \in \mathcal{B}$ (homogeneous Dirichlet RHS, which is the only kind we use), the solutions

satisfy

$$\widetilde{\mathcal{M}}\,\mathbf{x} = \mathbf{b}, \quad x_{\text{boundary}} \text{ free} \quad \to \quad x_{\text{interior}} = (\text{interior block})^{-1}\,\mathbf{b}_{\text{interior}}$$
$$\text{(after using the BC to zero boundary entries)},$$
$$\widetilde{\mathcal{M}}_{\text{BC}}\,\mathbf{x} = \mathbf{b} \quad \to \quad x_{\text{boundary}} = b_{\text{boundary}} = 0,$$
$$x_{\text{interior}} = (\text{interior block})^{-1}\,\mathbf{b}_{\text{interior}}.$$

Same interior solution. The Hutchinson estimator (which uses only Dirichlet RHS via the boundary-probe-zero) cannot distinguish the two operators.

(e) The pole structure that matters for $\overline{G}$. $\overline{G}_n^{\text{raw}}(s^2) = (n+1)^2 \sum_L 1/(\omega_{L,n}^2 + s^2)$ has poles at $s^2 = -\omega_{L,n}^2$ for the discrete eigenvalues. These poles are at the *interior* eigenvalues – the ghost eigenvalues at 1 would give a pole at $s^2 = -1$, which is outside our scan range ($s^2 > 0$) and is excluded by the boundary-probe-zero anyway. So the physical pole structure of $\overline{G}$ is identical for $\widetilde{\mathcal{M}}$ and $\widetilde{\mathcal{M}}_{\text{BC}}$.

(f) Renormalized determinant ratio. The final physical observable is the renormalized $\log \det' \mathcal{M}[\phi_b]/\det \mathcal{M}[\phi_{\text{fv}}]$. Even at the $\mathcal{O}(\mathrm{d}r^2)$ level where $\widetilde{\mathcal{M}}$ and $\widetilde{\mathcal{M}}_{\text{BC}}$ genuinely differ, that difference is the *same* boundary-discretization artifact in the bounce *and* the false-vacuum operators, so it cancels in the ratio. UV renormalization removes whatever remains. The renormalized quantity is invariant under the trick.

Summary: "physics unchanged" means three things.

1. *On the BC-respecting subspace*, $\widetilde{\mathcal{M}}$ and $\widetilde{\mathcal{M}}_{\text{BC}}$ act identically. Eigenvalue equations and linear systems give the same answers.

2. *In the continuum limit* ($\mathrm{d}r \to 0$), both discrete operators converge to the same continuum operator. The difference between them is $\mathcal{O}(\mathrm{d}r^2)$, of the same order as the FD discretization error vs. the continuum.

3. *For the renormalized observable* (the determinant ratio that enters the decay rate), the small boundary discretization differences cancel between bounce and false-vacuum operators in the ratio, and UV renormalization removes whatever residue is left.

Where "physics changed" would actually be problematic. What would genuinely break the physics is:

- using a non-Dirichlet BC at the origin (e.g. Neumann), which selects a different self-adjoint extension of the continuum operator and gives a different spectrum – not a discretization error, a *different physical problem.*

- failing to enforce the BC at all (the eigenvalues then drift toward those of an unbounded, non-self-adjoint operator).

- using inhomogeneous Dirichlet ($u_{\text{boundary}} = c \neq 0$) without compensating the RHS – this would solve a *different* linear system.

None of these is what the symmetric trick does. The trick is the discrete analogue of taking the original continuum operator and restricting it to its physical domain (L^2 functions vanishing at $r = 0, r = \infty$). It does not change the operator; it picks the right *subspace* of the discretized space on which to evaluate that operator, and it does so in a way that keeps all the matrix-algebra machinery (symmetry, spectral theorem, Feshbach, Hutchinson) working without modification.

When the trick fails to apply. The trick assumes *homogeneous Dirichlet* (BC value $= 0$). For inhomogeneous Dirichlet ($x_k = c \neq 0$ at boundary), you would also modify the right-hand side to compensate: solve $\widetilde{\mathcal{M}}_{\mathrm{BC}}\,\mathbf{x} = \mathbf{b}$ with $b_k = c$, and recall that the original cross-coupling entries $e\,x_k$ that contributed to the interior rows are now missing – so the interior RHS would need to be shifted by $-e\,c$. For *periodic* BCs (where the boundary points are identified with each other) the trick doesn't apply at all – you would need a different construction (e.g., a circulant matrix). For our problem – always homogeneous Dirichlet on a symmetric operator – the trick is exact, harmless, and applies universally.

Why $+I_{\mathcal{B}}$ is necessary and why it produces the ghosts

After $D\,\widetilde{\mathcal{M}}\,D$ alone, the boundary subspace is a $|\mathcal{B}|\times|\mathcal{B}|$ *zero block*: zero rows, zero columns, zero diagonal. Three consequences:

- the matrix has rank $2N - |\mathcal{B}|$ (only the interior contributes);
- its kernel is spanned by the boundary basis vectors $\{e_k : k \in \mathcal{B}\}$;
- it is **singular** – `splu` either fails or returns garbage, and we cannot estimate $\mathrm{Tr}(\widetilde{\mathcal{M}}_{\mathrm{BC}}^{-1})$ at all.

We need an invertible matrix. The minimal repair is to lift the zero rows out of the kernel by putting *something non-zero* on the boundary diagonal. The simplest "something" is 1, which is what $I_{\mathcal{B}}$ provides.

Could we use any other constant $c \neq 0$? Yes – exactly the same way. Replace $I_{\mathcal{B}}$ with $c\,I_{\mathcal{B}}$ and the boundary block becomes $c\,I$. The matrix is invertible, the interior physics is untouched, and the ghost eigenvalues are now at c:

Choice	Boundary block	Ghost eigvals of $\widetilde{\mathcal{M}}_{\mathrm{BC}}$	Ghost eigvals of $A = \widetilde{\mathcal{M}}_{\mathrm{BC}} + s^2\mathbb{I}$
$c = 1$ (our choice)	$\mathbb{I}$	1	$1 + s^2$
$c = 1000$	$1000\,\mathbb{I}$	1000	$1000 + s^2$
$c = m^2$ (false-vac mass^2)	$m^2\,\mathbb{I}$	m^2	$m^2 + s^2$

(208)

The boundary-zeroing of the Hutchinson probe (§6.6) removes the ghost contribution regardless of which c we picked. $c = 1$ is just convention – a different value would only relabel where the ghost eigenvalues sit, not change anything operationally.

The unavoidable trade-off.

- Without $+c\,I_{\mathcal{B}}$ for any c: matrix singular, pipeline broken.
- With $+c\,I_{\mathcal{B}}$ for any $c \neq 0$: matrix invertible, but $|\mathcal{B}|$ ghost eigenvalues appear at c (or $c + s^2$ after the spectral shift), contaminating $\mathrm{Tr}(A^{-1})$ by $|\mathcal{B}|/(c + s^2)$.

There is no way to have both invertibility *and* no ghost contribution simultaneously, because the only way to lift the zero rows out of the kernel is to give the boundary subspace some non-trivial spectrum, and any non-trivial spectrum shows up additively in the trace. The resolution is not to avoid the ghosts but to *exclude them from the Hutchinson estimator* – which is exactly what zeroing the probe at boundary indices accomplishes (see §6.6).

In code. The code uses $c = 1$:

```
diag_bc = np.zeros(N2)
for k in boundary:
    diag_bc[k] = 1.0          # this line picks c = 1
M_tilde = M_tilde + sp.diags(diag_bc, 0, format='csr')
```

You could change `1.0` to any non-zero number; the trace results would be identical because the boundary-probe-zeroing fix removes the ghost contribution either way. We use 1 as the simplest and most readable option.

Concrete walkthrough: how the inversion actually runs, and exactly where the ghosts enter

This subsubsection ties the whole story together with a single *concrete* 4×4 example, showing every step of the pipeline: the matrix, what we invert, what the inverse looks like, and why the ghost contribution exists but is automatically excluded. After this you should have a single, clear mental picture.

Setup. Take $N = 4$ for one field, with boundary indices $\mathcal{B} = \{0, 3\}$. Suppose the original FD matrix is the toy

$$\widetilde{\mathcal{M}} \;=\; \begin{pmatrix} 3 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 \\ 0 & -1 & 3 & -1 \\ 0 & 0 & -1 & 3 \end{pmatrix} \tag{209}$$

(the same toy as §9.2.2; the diagonal 3 plays the role of $2/\mathrm{d}r^2 + V_n + U''$, and -1 plays the role of the FD off-diagonal).

Step 1: apply the symmetric trick. $D = \mathrm{diag}(0, 1, 1, 0)$, $I_{\mathcal{B}} = \mathrm{diag}(1, 0, 0, 1)$:

$$\widetilde{\mathcal{M}}_{\mathrm{BC}} \;=\; D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}} \;=\; \begin{pmatrix} \boxed{1} & 0 & 0 & 0 \\ 0 & 3 & -1 & 0 \\ 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & \boxed{1} \end{pmatrix}. \tag{210}$$

Note the block structure: indices $\{0, 3\}$ are decoupled from $\{1, 2\}$, with the boundary diagonal $= 1$.

Step 2: form $A = \widetilde{\mathcal{M}}_{\mathbf{BC}} + s^2\mathbb{I}$. For illustration take $s^2 = 0.1$:

$$A \;=\; \widetilde{\mathcal{M}}_{\mathrm{BC}} + 0.1\,\mathbb{I} \;=\; \begin{pmatrix} \boxed{1.1} & 0 & 0 & 0 \\ 0 & 3.1 & -1 & 0 \\ 0 & -1 & 3.1 & 0 \\ 0 & 0 & 0 & \boxed{1.1} \end{pmatrix}. \tag{211}$$

The boundary block is $(1 + s^2)\,\mathbb{I}_2 = 1.1\,\mathbb{I}_2$; the interior block is the original interior plus s^2 on the diagonal. The matrix is still block-diagonal w.r.t. $\{\mathcal{B}, \mathcal{I}\}$.

Step 3: invert A via `splu` − the whole thing, ghost block included. The code does `lu = splu(A)`; the LU factorization runs over the entire matrix. But because A is block-diagonal, the LU also splits into block-diagonal pieces, and the inverse is

$$A^{-1} \;=\; \begin{pmatrix} \frac{1}{1.1} & 0 & 0 & 0 \\ 0 & \frac{3.1}{(3.1)^2-1} & \frac{1}{(3.1)^2-1} & 0 \\ 0 & \frac{1}{(3.1)^2-1} & \frac{3.1}{(3.1)^2-1} & 0 \\ 0 & 0 & 0 & \frac{1}{1.1} \end{pmatrix}. \tag{212}$$

Plug in: $(3.1)^2 - 1 = 8.61$, so the interior block of A^{-1} is $\begin{pmatrix} 0.360 & 0.116 \\ 0.116 & 0.360 \end{pmatrix}$ and the ghost block of A^{-1} is $\mathrm{diag}(1/1.1, 1/1.1) = \mathrm{diag}(0.909, 0.909)$.

Step 4: the trace of A^{-1} has two pieces.

$$\mathrm{Tr}(A^{-1}) \;=\; \underbrace{\tfrac{1}{1.1}+\tfrac{1}{1.1}}_{\text{ghost contribution}=2/(1+s^2)\approx 1.818} \;+\; \underbrace{\tfrac{3.1}{8.61}+\tfrac{3.1}{8.61}}_{\text{physical interior trace}\approx 0.720} \;\approx\; 2.538. \tag{213}$$

The full-matrix trace is ≈ 2.538. Of that, 1.818 is the ghost contribution and 0.720 is the physical part. **This is what would happen if we used Hutchinson with no probe-zeroing.**

Step 5: probe-zeroing on the boundary. The Hutchinson loop draws a Rademacher $v \in \{\pm 1\}^4$, then sets $v[0] = v[3] = 0$. So v now has only entries at indices $\{1, 2\}$, e.g. $v = (0, +1, -1, 0)$. Computing $A^{-1}v$:

$$A^{-1}\,v \;=\; \begin{pmatrix} \frac{1}{1.1} & 0 & 0 & 0 \\ 0 & 0.360 & 0.116 & 0 \\ 0 & 0.116 & 0.360 & 0 \\ 0 & 0 & 0 & \frac{1}{1.1} \end{pmatrix} \begin{pmatrix} 0 \\ +1 \\ -1 \\ 0 \end{pmatrix} \;=\; \begin{pmatrix} 0 \\ 0.244 \\ -0.244 \\ 0 \end{pmatrix}. \tag{214}$$

The output has zero at boundary indices because the input does – the ghost block had nothing to act on! Now compute the dot product:

$$v \cdot (A^{-1}v) \;=\; 0 \cdot 0 + 1 \cdot 0.244 + (-1) \cdot (-0.244) + 0 \cdot 0 \;=\; 0.488. \tag{215}$$

This is one Hutchinson sample. It contains *no contribution from the ghost block* – the boundary entries multiplied zero both on input and output. Averaging many such samples converges to

$$\mathbb{E}[v \cdot A^{-1}v]_{\text{boundary-zero probes}} \;=\; \mathrm{Tr}(A^{-1}\big|_{\text{interior block}}) \;=\; 0.720, \tag{216}$$

exactly the physical interior trace, with the ghost contribution 1.818 surgically excluded.

Step 6: tying back to the original question.

- Yes, we invert $A = \widetilde{\mathcal{M}}_{\mathrm{BC}} + s^2\mathbb{I}$ as a whole (including the $I_{\mathcal{B}}$ part). `splu` factors the entire matrix.
- The resulting A^{-1} has block-diagonal structure with a ghost block ($\mathrm{diag}(1/(1+s^2))$) and a physical interior block.
- Probe-zeroing the Hutchinson vector v at boundary indices makes the ghost block invisible to the estimator – the boundary entries of A^{-1} never get sampled.
- So the trace estimate that we report is just the interior trace – the physically meaningful quantity, identical (up to $\mathcal{O}(\mathrm{d}r^2)$) to what an interior-only operator would give.

Why this is exactly equivalent to "inverting the interior block on its own." You could imagine bypassing the trick entirely: just take the 2×2 interior block (rows/cols $\{1, 2\}$ of the original $\widetilde{\mathcal{M}}$) directly, add $s^2\mathbb{I}_2$, and invert that smaller matrix. You would get the same interior A^{-1} block we just computed. The trick achieves the same thing while keeping the matrix at full $N \times N$ size, which simplifies code (no slicing, no reordering) and makes the boundary structure visible in a diagnostic way (ghost eigenvalues at 1 are easy to spot in `eigsh` output if anything ever leaks).

The big-picture summary. The claim "the physics is unchanged" means: the trick produces a matrix whose *inverse splits into a ghost piece + a physical piece*, and the Hutchinson with boundary-zeroed probes accesses *only the physical piece*. The ghost piece is a

numerical artifact created by the BC trick (the price for keeping the matrix $N \times N$ and invertible) but it is automatically excluded from the trace estimate. So the trace we report – and every downstream quantity built from it – is the same number you would get from inverting an interior-only operator that never had ghosts to begin with. That is the precise sense in which "physics is unchanged."

Connection to the ghost eigenvalues (forward to §6.6)

The boxed 1's on the boundary diagonal of $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ are exactly what produces the four "ghost" eigenvalues that §6.6 addresses. To see this directly from (201): the boundary subspace is a $|\mathcal{B}| \times |\mathcal{B}|$ identity block, so each boundary basis vector e_k ($k \in \mathcal{B}$) is an eigenvector of $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ with eigenvalue 1. In the example above, $\widetilde{\mathcal{M}}_{\mathrm{BC}}\, e_0 = e_0$ and $\widetilde{\mathcal{M}}_{\mathrm{BC}}\, e_3 = e_3$.

These "ghost" eigenvalues

$$\lambda_{\mathrm{ghost},k} \;=\; 1, \qquad k \in \mathcal{B} \tag{217}$$

are not approximations to physical modes – they are unit spikes at single boundary points. They exist purely as a side-effect of using 1's on the boundary diagonal to make the matrix invertible. We could have used any number c instead of 1; the boundary block would then be $c\,I$ and the ghost eigenvalues would be c. Whatever value c we picked, the ghost eigenvalues would still be there.

The next subsection (§6.6) explains why these ghost eigenvalues contaminate the trace $\mathrm{Tr}\big[(\widetilde{\mathcal{M}}_{\mathrm{BC}}+s^2)^{-1}\big]$ by an amount $|\mathcal{B}|/(c+s^2)$ – and how the boundary-zeroing of the Hutchinson probe v surgically excludes them at zero cost. **The sequence is therefore:**

- §6.5 (this section): introduces the symmetric trick to bake in BCs while keeping the matrix symmetric; unavoidable side-effect is the boundary-identity block.
- §6.6 (next): explains the resulting four "ghost eigenvalues" equal to 1, why they pollute the trace, and how zeroing the Hutchinson probe at boundary indices removes their contribution.

The "trick" and the "ghosts" are two sides of the same coin: the trick keeps the matrix usable, the ghosts are its price, and the probe-zeroing fix in §6.6 pays the price for free.

For the full 2-field problem. Same construction with $\mathcal{B} = \{0, N{-}1, N, 2N{-}1\}$ – the start/end of field 1 at 0 and $N{-}1$, and the start/end of field 2 at N and $2N{-}1$. The boundary subspace becomes a 4×4 identity block, generating four ghost eigenvalues instead of two.

6.6 Boundary "ghost" eigenvalues

The trick of the previous subsection is mathematically clean but introduces 4 *spurious* eigenvalues. The story has three pieces: "what causes them", "why they spoil the trace", and "how to remove them".

Step 1: where the ghost eigenvalues come from. After the symmetric BC trick (198), the modified matrix $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ has block structure:

$$\widetilde{\mathcal{M}}_{\mathrm{BC}} \;=\; \underbrace{\begin{pmatrix} \mathbb{I}_4 & 0 \\ 0 & \widetilde{\mathcal{M}}^{\mathrm{int}} \end{pmatrix}}_{\text{block-diagonalized}} \quad \text{(after reordering rows/cols so boundary indices come first).} \tag{218}$$

The boundary block is the 4×4 identity; the interior block is the original physics. There is *no coupling* between the two blocks (this was guaranteed by the $D\widetilde{\mathcal{M}}D$ part, which zeroed every entry connecting boundary to interior).

Why (218) **is block-diagonal – and "can I always do this?"** This is worth a beat, because it ties together the BC trick, the ghost story, and a general fact about matrix structure.

(i) What "block-diagonal" means. A matrix A is *block-diagonal with respect to the partition* $\{\mathcal{B}, \mathcal{I}\}$ of its index set if $A_{ij} = 0$ whenever i and j lie in different blocks of the partition. Equivalently: the boundary-subspace vectors and the interior-subspace vectors are mapped only to themselves by A, never mixed.

(ii) Why our $\widetilde{\mathcal{M}}_{\mathbf{BC}}$ is block-diagonal. The trick (198) was *designed* to make the two subspaces decouple:

- $D\,\widetilde{\mathcal{M}}\,D$ *zeros every entry that touches a boundary index in row OR column.* So for any $i \in \mathcal{B}$ and any $j \in \mathcal{I}$ (or vice versa), $(D\widetilde{\mathcal{M}}D)_{ij} = 0$. This kills all the cross-coupling in one shot.

- $I_{\mathcal{B}}$ *only fills the boundary diagonal.* It puts 1's at (k, k) for $k \in \mathcal{B}$ and zero everywhere else. So it doesn't reintroduce any cross-coupling.

Adding the two: the boundary-row/boundary-col entries are zero *except* on the diagonal where they are 1. That is exactly the structure "boundary block = identity, interior block = $\widetilde{\mathcal{M}}$ restricted to interior, no coupling" shown in (218). The block-diagonal form is therefore *a direct algebraic consequence of the trick, not a separate operation.*

(iii) Can you always block-diagonalize a matrix? Three different senses, three different answers:

- *By permutation alone (rearranging rows/columns):* **Yes if and only if** there exists a partition of the index set such that no off-diagonal entry of the matrix crosses that partition. For our $\widetilde{\mathcal{M}}_{\mathrm{BC}}$ this is true with partition $\{\mathcal{B}, \mathcal{I}\}$ *because the trick created the zeros.* For a generic matrix with non-trivial cross-couplings, no permutation alone gets you there.

- *By symmetry (working in an irrep basis):* **Yes whenever the matrix commutes with a symmetry group's representation.** For our pre-discretized 4D operator $\mathcal{M}$, SO(4) invariance of the bounce gives the partial-wave decomposition $\mathcal{M} = \bigoplus_n \mathcal{M}_n \otimes \mathbb{I}_{(n+1)^2}$, which is block-diagonal in n. We use this implicitly: each partial-wave n is treated by its own separate matrix, never mixing with other n's.

- *By eigendecomposition (working in the eigenbasis):* **Yes for any symmetric / Hermitian matrix** – the spectral theorem gives an orthogonal basis of eigenvectors in which the matrix is diagonal (the limit of "block-diagonal" with all 1×1 blocks). The cost is solving the full eigenvalue problem, which is exactly what we want to *avoid* (Hutchinson's whole point).

So "always" has caveats. Our case is the easy one: *by construction*, the BC trick produced the cross-coupling zeros, and *a permutation* (just renaming rows/columns) reveals the block structure.

(iv) Did we actually permute the matrix in code? No. The code does `D @ M_tilde @ D + diag(diag_bc)` exactly as written – the cross-coupling entries become zero

in place, and the boundary-diagonal entries become 1 in place, but the rows and columns are not physically reordered. The matrix in memory is still indexed in the natural FD order $i = 0, 1, \ldots, 2N-1$; the "boundary indices come first" framing of (218) is a *conceptual* reordering for purposes of seeing the block structure, not something the code performs. SuperLU's sparse LU factorization is permutation-aware internally (it does its own pivoting/reordering for fill-in optimization), but our application code never sees a permuted matrix.

(v) Why we don't bother to physically reorder. For our downstream operations – sparse LU plus Hutchinson solves plus dot products – the natural FD order is just as good as any permutation. The matrix product $\widetilde{\mathcal{M}}_{\rm BC}\, v$ produces the same result regardless of how we labeled rows and columns; the boundary subspace is decoupled algebraically, not by virtue of being adjacent in memory. The only place where we explicitly use the boundary index set is in `boundary_indices = np.array([0, N-1, N, 2*N-1])`, which we pass to the Hutchinson estimator so it can zero the random probe at those positions and exclude the ghost subspace (§6.6 below). That's the entire visible footprint of the block structure in code.

A 4×4 identity matrix has 4 eigenvectors – the unit basis vectors $e_0, e_{N-1}, e_N, e_{2N-1}$ – all with eigenvalue 1. Apply $\widetilde{\mathcal{M}}_{\rm BC}$ to one of them:

$$\widetilde{\mathcal{M}}_{\rm BC}\, e_k = e_k, \qquad k \in \mathcal{B}. \tag{219}$$

So we have 4 **exact eigenvectors with exact eigenvalue** 1. These are not approximations to physical modes – they are *unit spikes at single boundary points*, not smooth functions on the grid. They exist in the matrix because of the BC trick, not because the physics put them there. We call them "ghosts".

Step 2: why ghosts contaminate the trace. The trace of a matrix's inverse is a sum over its eigenvalues:

$$\mathrm{Tr}\big[(\widetilde{\mathcal{M}}_{\rm BC} + s^2)^{-1}\big] \;=\; \sum_{\text{all eigenvalues}} \frac{1}{\lambda + s^2}. \tag{220}$$

Splitting "all" into the two blocks:

$$\mathrm{Tr}\big[(\widetilde{\mathcal{M}}_{\rm BC} + s^2)^{-1}\big] \;=\; \underbrace{\sum_{L\,\text{interior}} \frac{1}{\lambda_L + s^2}}_{\text{physical, want}} \;+\; \underbrace{4 \cdot \frac{1}{1+s^2}}_{\text{ghost contamination}}. \tag{221}$$

The ghost contribution is a deterministic bias of $4/(1+s^2)$. It has nothing to do with the Green's function we are trying to compute – it's an artifact of the BC enforcement that we built in by hand.

At the negative-mode pole $s^2 = 0.31$ (n=0 sector):

$$\frac{4}{1+0.31} \;=\; \frac{4}{1.31} \;\approx\; 3.05. \tag{222}$$

This is exactly the wrong offset that broke our $\overline{G}_0^{\rm sub}$ benchmark before we excluded the ghosts – ~ 5.6 instead of the RK answer $B \approx 2.58$, off by ~ 3.

Step 3: how to surgically exclude the ghosts in Hutchinson. The Hutchinson estimator is

$$\mathrm{Tr}(A^{-1}) \;\approx\; \frac{1}{K} \sum_k v_k^\top A^{-1} v_k, \tag{223}$$

and its identity $\mathbb{E}[v\,v^\top] = \mathbb{I}$ gives equal weight to every diagonal entry of A^{-1}, including those at the four boundary indices. We want to weight the interior entries by 1 and the boundary entries by 0 – that is, replace $\mathbb{E}[v\,v^\top] = \mathbb{I}$ with $\mathbb{E}[v\,v^\top] = D$ where D is the "zero on boundary, one on interior" diagonal we already used for the BC trick.

The way to make a probe satisfy $\mathbb{E}[v\,v^\top] = D$ is simply: **set the boundary entries of every probe to zero**.

$$v \in \{\pm 1\}^{2N}, \qquad v_k = 0 \;\text{ for } k \in \mathcal{B} = \{0, N{-}1, N, 2N{-}1\}. \tag{224}$$

With this rule:

$$\mathbb{E}[v_k\, v_l] \;=\; \begin{cases} \delta_{kl} & \text{if both } k, l \notin \mathcal{B}, \\ 0 & \text{if } k \in \mathcal{B} \text{ or } l \in \mathcal{B}, \end{cases} \qquad \Longleftrightarrow \qquad \mathbb{E}[v\,v^\top] = D. \tag{225}$$

Sandwiching with any matrix B now selects only the interior diagonal entries:

$$\mathbb{E}\big[v^\top B\, v\big] \;=\; \sum_{kl} B_{kl}\, \mathbb{E}[v_k v_l] \;=\; \sum_{k \notin \mathcal{B}} B_{kk} \;=\; \mathrm{Tr}\big[D\,B\big] \;=\; \mathrm{Tr}\big[B\big]_{\text{interior block only}}. \tag{226}$$

Equivalently: the probe v has *zero overlap* with each of the 4 ghost eigenvectors e_k (since $v_k = 0$ at those positions, $v \cdot e_k = 0$). So when the spectral expansion of $A^{-1}v$ runs over all eigenmodes, the 4 ghost contributions are killed term-by-term.

Result. The estimator

$$\widehat{\mathrm{Tr}}^{\text{interior}}(A^{-1}) \;=\; \frac{1}{K}\sum_k v_k^\top A^{-1} v_k \;\xrightarrow{K\to\infty}\; \sum_{L\,\text{interior}} \frac{1}{\lambda_L + s^2} \tag{227}$$

recovers exactly the physical trace – the 4 ghost eigenvalues are surgically excluded, at zero algorithmic cost (just set 4 entries to zero on each random draw).

Where this lives in the code. The `boundary_indices` argument in `hutchinson_trace_raw` and `hutchinson_trace_projected` (in `fd_builder.py`) is exactly this fix. Inside the loop over Hutchinson probes:

```
v = rng.choice([-1.0, 1.0], size=N2)
if boundary_indices is not None:
    v[boundary_indices] = 0.0    # <-- excludes the 4 ghosts
x = lu_factor.solve(v)
samples[k] = v @ x
```

Without that one line, the trace estimate would carry the spurious $+4/(1+s^2)$ offset on every s^2 value – enough to throw off the benchmark by a factor of ~ 2 near the pole.

6.7 Continuum-discrete normalization (the missing $\mathrm{d}r$)

This was the most subtle bug to track down. The question: does the matrix trace need to be multiplied by $\mathrm{d}r$ to match the continuum trace? Naively yes – a Riemann sum carries a $\mathrm{d}r$. Actually no – the discrete eigenvector convention absorbs it. Let us derive this explicitly.

The continuum normalization. For the tilde basis the inner product is the flat L^2 one, so eigenmodes are normalized as

$$\int_0^\infty \tilde{u}_L(r)\, \tilde{u}_L(r)\, \mathrm{d}r \;=\; 1 \qquad \text{(continuum convention)}. \tag{228}$$

(Suppressing the field index i for clarity; the full expression is $\sum_i \int |\tilde{u}_{L,i}|^2 \mathrm{d}r = 1$.)

The discrete normalization (eigsh default). `scipy.sparse.linalg.eigsh` returns eigenvectors normalized in plain Euclidean sense:

$$\sum_i \chi_{L,i}^2 \;=\; 1 \qquad \text{(discrete convention)}, \tag{229}$$

where now i enumerates the matrix entries (grid points $\times$ field components, $i = 1, \ldots, 2N$).

The two are not the same: a $\sqrt{\mathrm{d}r}$ factor. If $\chi_{L,i}$ approximates the continuum function value $\tilde{u}_L(r_i)$ on the grid, then by Riemann-summing (228) we get

$$\int |\tilde{u}_L|^2 \,\mathrm{d}r \;\approx\; \sum_i |\tilde{u}_L(r_i)|^2 \,\mathrm{d}r \;=\; 1 \quad\Longrightarrow\quad \sum_i |\tilde{u}_L(r_i)|^2 \;=\; \frac{1}{\mathrm{d}r}. \tag{230}$$

But the discrete vector is normalized to $\sum_i |\chi_{L,i}|^2 = 1$, not $1/\mathrm{d}r$. The two conventions differ by a factor $\sqrt{\mathrm{d}r}$ per entry:

$$\boxed{\chi_{L,i} \;=\; \sqrt{\mathrm{d}r}\,\tilde{u}_L(r_i).} \tag{231}$$

This relation is the key to everything that follows.

Trace computed two ways must be equal. Continuum side. The spectral form of the resolvent is

$$\widetilde{G}(r, r; s^2) = \sum_L \frac{\tilde{u}_L(r)\,\tilde{u}_L(r)}{\lambda_L + s^2}, \tag{232}$$

and integrating with the flat measure (using (228)),

$$\int_0^\infty \mathrm{d}r\, \widetilde{G}(r, r; s^2) \;=\; \sum_L \frac{\int |\tilde{u}_L|^2 \,\mathrm{d}r}{\lambda_L + s^2} \;=\; \sum_L \frac{1}{\lambda_L + s^2}. \tag{233}$$

Discrete side. The matrix-trace formula (using (229)),

$$\mathrm{Tr}\big[\widetilde{\mathcal{M}}_{\mathrm{BC}}^{-1}\big] \;=\; \sum_L \frac{\sum_i \chi_{L,i}^2}{\lambda_L} \;=\; \sum_L \frac{1}{\lambda_L}. \tag{234}$$

They are equal – both give the spectral sum $\sum_L 1/(\lambda_L + s^2)$ (or $1/\lambda_L$ at $s^2 = 0$). No $\mathrm{d}r$ appears in either, because in each case the eigenmode normalization collapses the squared-norm to 1, killing any potential $\mathrm{d}r$ factor.

Sanity check via (231)**.** What if we had foolishly used $\tilde{u}_L(r_i)$ directly as a discrete vector (i.e., not the eigsh-normalized $\chi_{L,i}$)? Then the matrix trace would be

$$\sum_L \frac{\sum_i |\tilde{u}_L(r_i)|^2}{\lambda_L} \;=\; \sum_L \frac{1/\mathrm{d}r}{\lambda_L} \;=\; \frac{1}{\mathrm{d}r} \sum_L \frac{1}{\lambda_L}, \tag{235}$$

and we would have to multiply by $\mathrm{d}r$ explicitly to get the continuum answer. With the eigsh-normalized $\chi_{L,i}$, this multiplication is built into the convention. *This is why the FD code does not multiply anything by* $\mathrm{d}r$.

7 Hutchinson stochastic trace estimator

7.1 The problem we are trying to solve

We need

$$\mathrm{Tr}\big[(\widetilde{\mathcal{M}} + s^2\mathbb{I})^{-1}\big] \;=\; \sum_{i=1}^{2N} \big[(\widetilde{\mathcal{M}} + s^2\mathbb{I})^{-1}\big]_{ii}, \tag{236}$$

i.e. the trace (sum of diagonal entries) of the inverse of a $2N \times 2N \approx 4000 \times 4000$ sparse matrix, evaluated at hundreds of s^2 values. Two naive routes both fail in practice:

- **Form the inverse explicitly.** The matrix $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ is sparse (about $8N$ nonzeros). But A^{-1} is *dense* – typically every entry is nonzero. Storing it (16 000 000 floats) and computing it (cubic-cost LU + back-solve) would burn $\sim$ 30s per s^2 point and gigabytes of memory. We don't actually need the off-diagonal entries.
- **Compute the diagonal directly.** The i-th diagonal entry is $(A^{-1})_{ii} = e_i^\top A^{-1} e_i$, which we can get by solving $Ax = e_i$ and reading x_i. Doing this $2N$ times gives the trace exactly. With sparse LU each solve is $\mathcal{O}(N)$, so the cost is $\mathcal{O}(N^2)$ per s^2. Doable, but too expensive when scanning ~ 300 values of s^2.

The Hutchinson trick estimates the trace using only $K \ll N$ *random* probes. With $K = 100$ we get $\sim 1\%$ accuracy at $\sim 40\times$ the speed of direct diagonal extraction.

7.2 The mathematical identity

Take a random vector $v \in \mathbb{R}^{2N}$ whose components are **i.i.d. Rademacher**: each v_i is $+1$ or -1 with equal probability $1/2$. The two key statistical properties:

$$\mathbb{E}[v_i] = 0, \qquad v_i^2 = 1 \;\text{ (deterministically)}, \qquad \mathbb{E}[v_i\, v_j] = \delta_{ij} \quad \text{for } i \neq j. \tag{237}$$

The off-diagonal expectation $\mathbb{E}[v_i v_j] = 0$ comes from the four equally likely sign combinations cancelling in pairs. Compactly,

$$\mathbb{E}\big[\, v\, v^\top \big] \;=\; \mathbb{I}. \tag{238}$$

Now sandwich any matrix B between v and $v^\top$:

$$\mathbb{E}\big[\, v^\top B\, v \,\big] \;=\; \mathbb{E}\Big[\sum_{i,j} v_i\, B_{ij}\, v_j\Big] \;=\; \sum_{i,j} B_{ij}\, \mathbb{E}[v_i v_j] \tag{239}$$

$$= \sum_{i,j} B_{ij}\, \delta_{ij} \;=\; \sum_i B_{ii} \;=\; \mathrm{Tr}(B). \tag{240}$$

So a single random sample $v^\top B v$ has expected value $\mathrm{Tr}(B)$. This is true for any matrix B, including the one we care about, $B = A^{-1} = (\widetilde{\mathcal{M}} + s^2\mathbb{I})^{-1}$. We never need to form A^{-1} explicitly – we only need to be able to multiply A^{-1} against a vector.

7.3 The algorithm step by step

To estimate $\mathrm{Tr}(A^{-1})$ for our matrix $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$:

1. **Factor A once.** Compute the sparse LU decomposition $A = LU$ via `scipy.sparse.linalg.splu`. L and U are sparse triangular matrices. For our banded A this costs $\mathcal{O}(Nb^2)$ where the bandwidth b is small. Done once per s^2.

2. **Loop over K random probes.** For each $k = 1, \ldots, K$:

   (a) Draw a Rademacher vector $v_k \in \{-1, +1\}^{2N}$.

   (b) Zero v_k at the four boundary indices (§6.6) to exclude the boundary ghost subspace from the trace estimate.

   (c) Solve $A\, x_k = v_k$ using the LU factors. This is two triangular solves – cheap.

   (d) Compute the sample value $s_k = v_k^\top x_k = \sum_i v_{k,i}\, x_{k,i}$.

3. **Aggregate.** The estimator is the sample mean

$$\widehat{\mathrm{Tr}}(A^{-1}) \;=\; \frac{1}{K}\sum_{k=1}^{K} s_k, \tag{241}$$

   with statistical uncertainty (standard error of the mean)

$$\mathrm{SEM} \;=\; \frac{\sigma}{\sqrt{K}}, \qquad \sigma^2 = \frac{1}{K-1}\sum_k (s_k - \bar{s})^2. \tag{242}$$

The total cost per s^2 value is one sparse LU plus K sparse triangular solves – vastly cheaper than $2N$ solves for explicit diagonal extraction.

7.4 Why this works (intuition)

Decompose the per-sample value:

$$v^\top B\, v \;=\; \underbrace{\sum_i v_i^2\, B_{ii}}_{\text{diagonal piece}} \;+\; \underbrace{\sum_{i\neq j} v_i\, v_j\, B_{ij}}_{\text{off-diagonal piece}}\,. \tag{243}$$

For Rademacher probes, $v_i^2 = 1$ deterministically, so the diagonal piece is exactly $\sum_i B_{ii} = \mathrm{Tr}(B)$ – *no randomness*. All the noise comes from the off-diagonal piece, where each $v_i v_j$ is ± 1 at random and the cross-terms partially cancel. Per-sample variance:

$$\mathrm{Var}\big[v^\top B\, v\big] \;=\; 2\sum_{i\neq j} B_{ij}^2 \;=\; 2\left(\|B\|_F^2 - \|\mathrm{diag}(B)\|^2\right), \tag{244}$$

i.e. proportional to the off-diagonal mass of $B = A^{-1}$. For our Green's function $A^{-1} = (\widetilde{\mathcal{M}} + s^2)^{-1}$, this off-diagonal mass is $\sum_L 1/(\lambda_L + s^2)^2$, which grows when we approach the pole $s^2 = -\lambda_\chi$. That is the underlying reason why the projected estimator can still be noisy near $s_\star$ even after we analytically remove the pole.

7.5 Other choices for the random vectors

The Hutchinson identity (238) works for any distribution with $\mathbb{E}[v_i] = 0$ and $\mathbb{E}[v_i^2] = 1$. Common choices:

- **Rademacher** ($v_i = \pm 1$): smallest variance for structured matrices because the diagonal is sampled deterministically. **This is what we use.**

- **Gaussian** ($v_i \sim \mathcal{N}(0,1)$): also unbiased, but the diagonal contribution is now random too, and the variance is strictly larger than Rademacher. Useful when you want fancier variance-reduction tricks (Hutch++, etc.).
- **Sparse / unit basis**: setting $v_k = e_k$ for $k = 1, \ldots, N$ recovers exact diagonal extraction. Hutchinson with random v is what you do when you can afford fewer than N solves.

7.6 Projected variant for $\overline{G}^{\mathbf{sub}}$

For the subtracted trace we want $\mathrm{Tr}\big[P\,A^{-1}\,P\big]$ with $P = \mathbb{I} - \chi\chi^\top$. Using $P^\top = P$ (symmetric) and the expectation (238),

$$\begin{aligned}\mathbb{E}\big[\,v^\top P\,A^{-1}\,P\,v\,\big] &= \mathbb{E}\big[(P\,v)^\top\,A^{-1}\,(P\,v)\big] \\ &= P_{ij}\,(A^{-1})_{jk}\,P_{kl}\,\underbrace{\mathbb{E}[v_i\,v_l]}_{\delta_{il}} \;=\; P_{ij}(A^{-1})_{jk}P_{ki} \\ &= \mathrm{Tr}\big[PA^{-1}P\big]. \end{aligned} \tag{245}$$

So we just need to apply P to the probe before the solve and to the result after. Per-sample algorithm:

1. draw Rademacher v, zero at boundary indices;
2. project the probe: $w = Pv = v - \chi\,(\chi^\top v)$ (one dot product, one rescale, one subtraction – cheap);
3. solve $A\,x = w$ via the factored LU;
4. project the result: $y = P\,x = x - \chi\,(\chi^\top x)$ (re-projection: protects against numerical drift along χ that the LU may introduce);
5. sample value: $v^\top y$ (note: $v^\top y$, not $w^\top y$ – one of the two P's was already absorbed by the expectation).

Aggregate as before to get $(\widehat{\overline{G}}^{\mathrm{sub}}, \mathrm{SEM})$. The extra cost over the unprojected variant is just the four cheap dot products / projections per sample.

7.7 Why the result feels surprising: three perspectives

It really is striking that a few hundred random vectors plus a sparse LU give us the trace of an inverse we never compute. The previous subsections gave the algebraic identity and the algorithm; here are three short complementary perspectives that may help intuition.

The detailed algebraic + spectral derivation, with explicit LU and a 4×4 worked example, is in §9.2.2–§9.2.3. That section shows line-by-line how the bounce + FD operator funnels through L, U, what one Hutchinson sample looks like end-to-end, and how the spectral identity $\mathrm{Tr}(A^{-1}) = \sum_L 1/(\omega_{L,n}^2 + s^2)$ is derived in five sub-steps. If you have not read it yet, do so now – it is the core of how the FD code works.

Geometric intuition: "isotropic averaging." The condition $\mathbb{E}[v\,v^\top] = \mathbb{I}$ says that although every individual random probe v points in some specific direction, the *ensemble* of probes is rotationally symmetric – has equal expected weight in every direction in $\mathbb{R}^n$. Sandwiching a matrix with such an ensemble averages it over all directions equally, which is exactly what the trace is: a basis-independent quantity equal to $\sum_i u_i^\top B u_i$ for any orthonormal basis $\{u_i\}$. Hutchinson replaces the deterministic basis sum with a stochastic average that produces the same answer in expectation.

Connection to Monte Carlo integration. Hutchinson is essentially Monte Carlo evaluation of a high-dimensional sum:

$$\mathrm{Tr}(B) \;=\; \sum_i B_{ii} \;=\; \mathbb{E}_{v\sim\mathrm{Rademacher}}[v^\top B\,v]. \tag{246}$$

Just as Monte Carlo approximates an integral $\int f$ by $(1/K)\sum_k f(x_k)$ for random x_k, Hutchinson approximates a trace by averaging a quadratic form over random vectors. Both have the universal $1/\sqrt{K}$ convergence and both pay a cost proportional to the integrand's variance.

Beyond Hutchinson: Hutch++ / ContHutch++. For matrices whose dominant eigenvalues are well-separated from the rest, one can do strictly better. *Hutch++* (Meyer et al., 2021) combines Hutchinson with a low-rank approximation: sketch a few large eigenpairs of A^{-1} via randomized SVD, compute their contribution to the trace exactly, then Hutchinson the residual (whose tiny eigenvalues give tiny variance). The variance scales as $\mathcal{O}(1/K^2)$ instead of $\mathcal{O}(1/K)$. Near the pole of our Green's function, where one eigenvalue dominates the variance, this would be the natural next-step improvement – see Lever 7 in Part III (§23) for the continuous-operator version (ContHutch++).

Summary in one line. Hutchinson estimates the trace of an inverse using only matrix-vector solves, by the trick that random vectors with $\mathbb{E}[v\,v^\top] = \mathbb{I}$ sample the diagonal of *any* matrix in expectation – including matrices we never actually compute, like A^{-1}.

7.8 Putting it back into Green's-function language

For our problem, $A = \widetilde{\mathcal{M}}_n + s^2\mathbb{I}$ and the trace is $\overline{G}_n^{\mathrm{raw}}/(n+1)^2$ (or $\overline{G}_n^{\mathrm{sub}}/(n+1)^2$ for the projected variant). One Hutchinson call per s^2 point gives one estimate of $\overline{G}_n(s^2)$, accompanied by an error bar SEM. Repeating across the s^2 grid produces the curves the plot scripts visualize, with error bars displayed as $\pm 1\sigma$.

8 End-to-end walkthrough: from problem to number

This section is a narrative walk through the entire computation, with no equations beyond the bare minimum, designed to connect every concept introduced so far – discretization, BCs, ghosts, Hutchinson, projection – into one coherent story. The next section (§9) gives the same content with full equations and code excerpts.

Orientation: this section vs. §9. §8 (this section) is the abstract storyline – no matrix entries, no code, just the conceptual sequence of moves. If you prefer the code-anchored version (with worked 4×4 matrices, explicit forward/back solves, and Python snippets), skip ahead to §9; you can return here later if a high-level picture would be helpful.

8.1 What we want, in plain English

We want a single real number for each value of s^2:

$$\overline{G}_n^{\text{raw}}(s^2) \;=\; (n+1)^2 \cdot \mathrm{Tr}\big[(\widetilde{\mathcal{M}}_n + s^2\,\mathbb{I})^{-1}\big], \tag{247}$$

or the pole-free version $\overline{G}_n^{\text{sub}}(s^2)$ that projects out the "bad" eigenmode. Both are traces (sums of diagonal entries) of an inverse of a sparse symmetric matrix.

The challenge: the matrix is 4000×4000 and its inverse is *dense* (16 million entries), so we can't form the inverse and read off the diagonal. We need a smarter approach.

8.2 Step A – discretize the operator

Before any of this can happen, we need a finite matrix to work with. We pick a grid of $N = 2000$ radial points from $r_{\min}$ to $r_{\max}$ and write out the Liouville-transformed radial operator $\widetilde{\mathcal{M}}_n = -\partial_r^2 + V_n(r) + U''(\phi_b(r))$ at each grid point. With central finite differences for ∂_r^2 and the two field components stacked, we get a sparse symmetric matrix of size $2N \times 2N$ – the discrete representation of $\widetilde{\mathcal{M}}_n$.

8.3 Step B – bake in the boundary conditions, symmetrically

The continuum modes of $\widetilde{\mathcal{M}}_n$ vanish at both ends of the radial domain (regularity at $r = 0$, decay at $r = \infty$ for bound modes, "put it in a box" for continuum modes – see §6.4). On the grid this means the discrete eigenvectors should be zero at the four boundary indices $\mathcal{B} = \{0, N-1, N, 2N-1\}$ (the ends of each field).

Naively zeroing only the boundary rows of $\widetilde{\mathcal{M}}$ would break symmetry, which would (i) prevent us from using `eigsh`, (ii) break orthogonality of eigenvectors, which the projection mechanism in §4 needs. The fix is the symmetric sandwich $\widetilde{\mathcal{M}} \to D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$ – it zeros boundary rows *and* columns and adds 1's on the boundary diagonal. The matrix is now symmetric, invertible, and the boundary subspace is fully decoupled from the interior.

8.4 Step C – the side-effect: 4 "ghost" eigenvalues equal to 1

The price of step B is that the modified matrix has 4 fake eigenvalues equal to 1, with eigenvectors that are unit spikes at the boundary positions. They are not physical modes – they exist purely because we put 1's on the boundary diagonal. We have to be careful that they don't sneak into our trace computation. The fix will come at step F.

8.5 Step D – get the special eigenvector χ

For $\overline{G}^{\text{sub}}$ we need to project out the pole-bearing mode. Two cases:

- $n = 0$**:** the pole comes from the Coleman negative mode. Use `eigsh` (sparse symmetric eigensolver) on $\widetilde{\mathcal{M}}_0$ to find the algebraically smallest eigenpair $(\lambda_{\text{neg}}, \chi_{\text{neg}})$. Cost: a few seconds.

- $n = 1$**:** the pole comes from the four translation zero modes. The radial profile is known analytically as $\chi_{\rm zm}(r) \propto r^{3/2}\,\phi_b'(r)$. Cost: essentially free.

After finding χ, clamp it at the 4 boundary indices and renormalize. (Eigensolver noise can leak into the boundary subspace; this cleans it up.)

For $\overline{G}^{\rm raw}$ we don't need χ – but for $\overline{G}^{\rm sub}$, this is the only piece of mode information we need. We do *not* need to know any other eigenpair of $\widetilde{\mathcal{M}}$.

8.6 Step E – factor the shifted matrix once per s^2

For each s^2 in the grid, form $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ (cheap; just adds s^2 to each diagonal entry of the sparse matrix) and compute its sparse LU decomposition $A = LU$. This is the expensive step – maybe ~ 0.1 seconds for our matrix. Once done, we can solve $Ax = b$ for any b very cheaply (two triangular solves).

8.7 Step F – estimate the trace by Hutchinson's random-probe trick

Now the heart of the computation. We want $\mathrm{Tr}(A^{-1})$, but we never form A^{-1}.

The trick: pick a random vector v with each component ± 1 with equal probability (Rademacher). For *this particular probe*, compute the dot product

$$v \cdot (A^{-1}v) \;=\; v^\top A^{-1} v. \tag{248}$$

We don't need A^{-1} itself; we just solve $Ax = v$ with the LU factors and dot v with x. One sample.

The mathematical identity is:

$$\mathbb{E}\big[v^\top A^{-1} v\big] \;=\; \mathrm{Tr}(A^{-1}). \tag{249}$$

The reason: for Rademacher entries, $v_i^2 = 1$ always, so the diagonal contribution to the sum $\sum_{ij} v_i (A^{-1})_{ij} v_j$ is exactly $\sum_i (A^{-1})_{ii} = \mathrm{Tr}(A^{-1})$, deterministically. The off-diagonal cross-terms have random signs and average to zero.

So one sample has the right *mean* but is noisy. Average $K = 100$ samples, divide by K, and the noise shrinks like $\sigma/\sqrt{K}$ (standard error of the mean). With $K = 100$ we typically get $\sim 1\%$ relative error – more than enough for our purposes.

Connecting to the ghost issue. Here is where the ghost story closes the loop. If we use a "vanilla" Rademacher probe v with no zeros, the trace estimator gives us the trace over *all* eigenvalues of the modified matrix – physical *plus* the 4 ghost ones (eigenvalue 1). The ghost contribution is $4/(1 + s^2)$, a deterministic bias that we don't want.

The fix: set $v_k = 0$ at each of the 4 boundary indices $k \in \mathcal{B}$. Then the probe has no overlap with the ghost eigenvectors e_k (since $v \cdot e_k = v_k = 0$), and the spectral expansion of $A^{-1}v$ produces no ghost contribution. The trace estimator now estimates the trace of A^{-1} *restricted to the interior* – exactly the physical part. The cost is one line of code: `v[boundary_indices] = 0.0`.

8.8 Step G – $\overline{G}^{\text{sub}}$ is exactly the same algorithm + 2 projections

For the pole-subtracted version we want the projected trace $\mathrm{Tr}(PA^{-1}P)$ with $P = \mathbb{I} - \chi\chi^\top$. The algorithm is the same as step F, with two extra cheap operations per probe:

1. Draw v, zero at boundary indices (same as before).
2. *Project the probe*: $w = v - \chi(\chi \cdot v)$. This removes the χ-component from v.
3. Solve $Ax = w$ using the LU factors.
4. *Project the result*: $y = x - \chi(\chi \cdot x)$. Removes any drift along χ that the LU may have introduced.
5. Sample value: $v \cdot y$.

The reason this works: since χ is an eigenvector of A, applying A^{-1} doesn't mix it with other modes. So when P is sandwiched around A^{-1}, the χ-eigenmode contribution is killed (Case A of §4.1: $P\chi = 0$). All other eigenmodes pass through (Case B: orthogonality). The pole-bearing term simply does not appear in the trace.

8.9 Step H – repeat across the grid, save, plot

Loop over the adaptive s^2 grid: at each s^2, do steps E–G (factor LU, run two Hutchinson calls). Store mean and SEM. Save to `.npz`. The plot script reads the file and produces the raw and sub curves with error bars.

8.10 Recap: how every concept connects

- **Discretize** $\widetilde{\mathcal{M}}$ on a finite grid → matrix problem.
- **Boundary conditions** (regularity at 0, decay/box at ∞) bake in via the symmetric trick $D\widetilde{\mathcal{M}}D + I_{\mathcal{B}}$.
- **Side effect** of the BC trick: 4 ghost eigenvalues equal to 1 at the boundary positions.
- **Get** χ – one eigenvector, the pole-bearing mode – via `eigsh` ($n = 0$) or analytic ($n = 1$).
- **LU factor** the shifted matrix $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ once per s^2 value – enables cheap solves.
- **Hutchinson** estimates the trace via K random probes and K cheap LU solves; *zero the probes at boundary indices* to exclude the ghosts.
- **Project** (apply $P = \mathbb{I} - \chi\chi^\top$) before and after each solve to get $\overline{G}^{\text{sub}}$ in the same way: the pole-bearing eigenmode disappears from the spectral sum analytically because $P\chi = 0$.
- **Average and store**; repeat for the next s^2.

Every step has a single conceptual purpose; together they convert the infinite-dimensional Green's-function trace into one number per s^2, evaluated efficiently and with controlled statistical error bars.

9 Step-by-step computation of $\overline{G}^{\text{raw}}$ and $\overline{G}^{\text{sub}}$ (with code)

This section lays out the full pipeline – equations on the left, exact lines of FD code on the right – so you can see precisely where each ingredient enters. We treat $n = 0$ in detail; $n = 1$ is identical except for how χ is obtained (analytic instead of `eigsh`) and the final factor of $(n+1)^2 = 4$.

9.1 One-time setup (done once before the s^2 scan)

(a) Load the bounce profile and potential.

$$\phi_b(r) = (\phi_{b,1}(r), \phi_{b,2}(r)) \tag{250}$$

is read from the `.npz` file produced by `bounce.py`, together with the false-vacuum point and the potential parameters.

```
b = load_bounce(bounce_path)
pot_lin = CTShiftedLiftedPotential(b["params"], b["false_vac"])
```

(b) Build $\widetilde{\mathcal{M}}_n^{(N)}$. The Liouville-transformed radial fluctuation operator

$$\widetilde{\mathcal{M}}_n \;=\; -\partial_r^2 + \frac{n(n+2)+3/4}{r^2} + U''(\phi_b(r)) \tag{251}$$

is discretized on a uniform grid of $N = 2000$ points in $[r_{\min}, r_{\max}]$ with the standard 2nd-order central stencil for $-\partial_r^2$. The result is the symmetric sparse $2N \times 2N$ matrix (186), with symmetric Dirichlet BCs applied via (198).

```
M_tilde, r, dr, U_pp = build_M_tilde_clean(
    n=0, R_bounce=b["R"], X_prime_bounce=b["X_prime"],
    Y_prime_bounce=b["Y_prime"], pot_lin=pot_lin,
    N=2000, r_min=1e-4, r_max=None, fd_order=2,
)
N2 = M_tilde.shape[0]    # = 2N = 4000
```

(c) Get the special eigenvector χ. For $n = 0$ this is the Coleman negative mode, obtained by ARPACK Lanczos:

$$\widetilde{\mathcal{M}}_0\, \chi_{\text{neg}} = \lambda_{\text{neg}}\, \chi_{\text{neg}}, \quad \lambda_{\text{neg}} < 0, \quad \chi_{\text{neg}}^{\top} \chi_{\text{neg}} = 1. \tag{252}$$

```
lambda_neg, chi_neg = find_negative_mode(M_tilde, verbose=True)
```

For $n = 1$ this is the analytic translation zero mode $\chi_{\text{zm}}(r) \propto r^{3/2}\, \phi_b'(r)$:

```
chi_zm = build_translation_zero_mode(r, b["R"],
                                     b["X_prime"], b["Y_prime"])
```

(d) Mark boundary indices and clamp χ.

$$\mathcal{B} = \{0,\, N{-}1,\, N,\, 2N{-}1\}, \qquad \chi[\mathcal{B}] \to 0, \quad \chi \to \chi/\|\chi\|. \tag{253}$$

```
boundary_indices = np.array([0, args.N - 1, args.N, 2 * args.N - 1])
chi_neg[boundary_indices] = 0.0
chi_neg = chi_neg / np.linalg.norm(chi_neg)
```

(e) Build the adaptive s^2 grid. Same scheme as `compute_gbar_n0.py` (RK method), so FD and RK share an abscissa: dense (very-fine $\sim 2\times10^{-4}$) close to $s^2_\star = -\lambda_{\text{neg}} \approx 0.31$, coarser far from the pole, with a gap of $\pm$`skip_radius`$= 3\times10^{-4}$ around the pole itself.

```
s2_grid = build_s2_grid(pole_s2, s2_min=args.s2_min,
                        s2_max=args.s2_max,
                        skip_radius=args.skip_radius)
```

9.2 Per-s^2 inner loop

For each s^2 value in `s2_grid`, we compute two numbers ($\overline{G}^{\text{raw}}$, $\overline{G}^{\text{sub}}$) and their statistical error bars.

Step 1: form the shifted operator.

$$A \;=\; \widetilde{\mathcal{M}} + s^2\,\mathbb{I} \tag{254}$$

This is still a sparse $2N \times 2N$ matrix; $\mathbb{I}$ is added only along the diagonal.

```
A = (M_tilde + s2 * sp.eye(N2, format='csr')).tocsc()
```

Step 2: sparse LU factorization of A.

$$A \;=\; L\,U, \tag{255}$$

computed once per s^2 value, then reused for all the linear solves below.

9.2.1 Step 2 in detail: what is LU decomposition and why does it help?

The factorization itself. LU decomposition factors a matrix A into a product of a *lower-triangular* matrix L (zeros above the diagonal) and an *upper-triangular* matrix U (zeros below the diagonal):

$$\underbrace{\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}}_{A} = \underbrace{\begin{pmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix}}_{L} \cdot \underbrace{\begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}}_{U}. \tag{256}$$

Once you know L and U, you have an extremely cheap way to solve $A\,x = b$ for any right-hand side b.

Why solving with L and U separately is cheap. The system $A\,x = b$ becomes $(L\,U)\,x = b$, which we split into two steps:

1. **Forward solve:** solve $L\,y = b$ for y. Because L is lower-triangular, the first equation is just $\ell_{11}y_1 = b_1$, giving $y_1 = b_1/\ell_{11}$. The second equation is $\ell_{21}y_1 + \ell_{22}y_2 = b_2$ – with y_1 already known from the first step, this is a one-variable equation for y_2. Continue top-to-bottom; each step is a single division plus a few multiplications.

2. **Back solve:** now solve $U\,x = y$ for x. Because U is upper-triangular, start from the bottom equation $u_{nn}x_n = y_n$ (giving $x_n = y_n/u_{nn}$) and work upwards. Same idea as forward solve, in reverse direction.

Cost analysis (very important for understanding why we factor once but solve many times). For a dense $N \times N$ matrix:

- Computing $A = LU$ initially costs $\mathcal{O}(N^3)$ operations (Gaussian elimination, basically). *Expensive.*
- Each forward + back solve afterwards costs only $\mathcal{O}(N^2)$ operations. *Cheap.*

For our *sparse* $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ (banded, bandwidth $b \sim 2$), both costs improve dramatically:

- LU factorization: $\mathcal{O}(N\,b^2) \approx \mathcal{O}(N)$ flops.
- Each triangular solve: $\mathcal{O}(N\,b) \approx \mathcal{O}(N)$ flops.

This is the magic of sparse LU: factor once, solve many times. We will use this to do the $K = 100$ Hutchinson solves at essentially the cost of a single matrix-vector multiplication each.

What `splu` actually returns. `scipy.sparse.linalg.splu(A)` computes the LU factorization (with partial pivoting for stability) using SuperLU under the hood. It returns a Python object `lu` that exposes the `.solve(b)` method, which performs the forward + back solve internally:

```
lu = splu(A)
x = lu.solve(b)      # solves A x = b, costs O(N) for our banded A
```

The factorization data is stored inside the `lu` object; calling `lu.solve(b)` multiple times reuses the same L and U. This is exactly what the Hutchinson loop does next.

Worked 3×3 example. Suppose

$$A = \begin{pmatrix} 4 & 3 & 0 \\ 6 & 7 & 2 \\ 0 & 3 & 5 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}. \tag{257}$$

LU factorization (no pivoting needed here) gives

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 1.5 & 1 & 0 \\ 0 & 1.2 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 4 & 3 & 0 \\ 0 & 2.5 & 2 \\ 0 & 0 & 2.6 \end{pmatrix}. \tag{258}$$

You can verify $LU = A$ by direct multiplication. To solve $Ax = b$:

- Forward solve $Ly = b$: $y_1 = 1$, then $1.5(1) + y_2 = 2 \Rightarrow y_2 = 0.5$, then $1.2(0.5) + y_3 = 3 \Rightarrow y_3 = 2.4$.
- Back solve $Ux = y$: $2.6\,x_3 = 2.4 \Rightarrow x_3 = 0.923$, then $2.5\,x_2 + 2(0.923) = 0.5 \Rightarrow x_2 = -0.538$, then $4\,x_1 + 3(-0.538) = 1 \Rightarrow x_1 = 0.654$.

Total: a handful of multiplies and adds, instead of constructing A^{-1} as a full matrix.

How A's information is "stored" in L and U. This is worth being explicit about, because it is the conceptual core of why LU lets us avoid forming A^{-1}.

(a) The factorization is exact. $L\,U = A$ *exactly* (in exact arithmetic; in floating point, to machine precision). So the pair (L, U) contains *all* the information that A contained – nothing has been thrown away. We can always recover A by multiplying out, or compute the action of A on any vector v as $A\,v = L\,(U\,v)$.

(b) The factorization gives the action of A^{-1} for free. For any vector b, the equation $A\,x = b$ becomes

$$L\,U\,x = b \quad \Longleftrightarrow \quad \underbrace{L\,y = b}_{\text{forward solve for } y} \qquad \text{then} \qquad \underbrace{U\,x = y}_{\text{back solve for } x} \; . \tag{259}$$

So $x = U^{-1}\,L^{-1}\,b$ – equivalent to $x = A^{-1}\,b$ – but *computed entirely without inverting anything*: only triangular solves, which are explicit recurrences.

(c) The recurrences spelled out. For the forward solve $L\,y = b$ on a generic $N \times N$ lower-triangular matrix:

$$\begin{aligned} y_1 &= b_1/\ell_{11}, \\ y_2 &= (b_2 - \ell_{21} y_1)/\ell_{22}, \\ y_3 &= (b_3 - \ell_{31} y_1 - \ell_{32} y_2)/\ell_{33}, \\ &\vdots \\ y_i &= \Big(b_i - \sum_{j<i} \ell_{ij}\, y_j\Big)/\ell_{ii}. \end{aligned} \tag{260}$$

Each row is a one-line update using already-computed earlier entries of y. For the back solve $U\,x = y$ on a generic upper-triangular matrix:

$$\begin{aligned} x_N &= y_N/u_{NN}, \\ x_{N-1} &= (y_{N-1} - u_{N-1,N} x_N)/u_{N-1,N-1}, \\ &\vdots \\ x_i &= \Big(y_i - \sum_{j>i} u_{ij}\, x_j\Big)/u_{ii}. \end{aligned} \tag{261}$$

The bottom of x comes from the bottom of y, and from there we work up. Each step is a single division plus a few multiplications; no matrix inversion ever happens.

(d) For our *sparse* banded A, L and U are also sparse and banded. Most ℓ_{ij} and u_{ij} are zero, so the sums $\sum_{j<i} \ell_{ij}\, y_j$ and $\sum_{j>i} u_{ij}\, x_j$ have only a handful of non-zero terms. The cost of forward + back solve becomes $\mathcal{O}(N\,b)$ instead of $\mathcal{O}(N^2)$, where $b \sim 4$ is the bandwidth. For our $N = 4000$, this is roughly 16 000 operations per solve – a tenth of a millisecond on a laptop.

(e) Where the operator $\widetilde{\mathcal{M}}$ enters the trace estimator. The fluctuation operator $\widetilde{\mathcal{M}}$ is built first as a sparse matrix in `build_M_tilde_clean`. Then $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ is formed by adding s^2 to each diagonal entry. Then `splu(A)` runs Gaussian elimination on the sparse A to produce L and U. The information about $\widetilde{\mathcal{M}}$ has been *transformed* into the LU factors, not lost: $L\,U = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ exactly. Inside the Hutchinson loop, every solve `lu.solve(v)` reuses these factors – so every probe $v_k \to x_k = A^{-1} v_k$ inherits $\widetilde{\mathcal{M}}$'s information through the LU factors. The trace estimate

$$\widehat{\mathrm{Tr}}(A^{-1}) \;=\; \frac{1}{K}\sum_k v_k^\top\, x_k \;=\; \frac{1}{K}\sum_k v_k^\top\,(L\,U)^{-1}\,v_k \tag{262}$$

thus depends on $\widetilde{\mathcal{M}}$ entirely through the LU factors of $\widetilde{\mathcal{M}} + s^2\mathbb{I}$. The fluctuation operator never appears in any formula by itself; it appears only inside an LU pair that is then applied to random probes.

Worked 4×4 example: from A to L to U to $A^{-1}v$. Take a tridiagonal symmetric matrix that mimics our discretized operator's structure (think of it as $\widetilde{\mathcal{M}} + s^2\mathbb{I}$ at $N = 4$):

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix}. \tag{263}$$

Gaussian elimination (no pivoting needed) gives

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 & 0 \\ 0 & -\frac{2}{3} & 1 & 0 \\ 0 & 0 & -\frac{3}{4} & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & -1 & 0 & 0 \\ 0 & \frac{3}{2} & -1 & 0 \\ 0 & 0 & \frac{4}{3} & -1 \\ 0 & 0 & 0 & \frac{5}{4} \end{pmatrix}. \tag{264}$$

Verify: multiplying $L\,U$ row by row reproduces A exactly. Now apply A^{-1} to a Hutchinson probe $v = (+1, -1, +1, -1)^\top$:

Forward solve $L\,y = v$:

$$y_1 = +1, \tag{265}$$
$$y_2 = -1 - (-\tfrac{1}{2})(+1) = -1 + \tfrac{1}{2} = -\tfrac{1}{2}, \tag{266}$$
$$y_3 = +1 - (-\tfrac{2}{3})(-\tfrac{1}{2}) = +1 - \tfrac{1}{3} = +\tfrac{2}{3}, \tag{267}$$
$$y_4 = -1 - (-\tfrac{3}{4})(+\tfrac{2}{3}) = -1 + \tfrac{1}{2} = -\tfrac{1}{2}. \tag{268}$$

Back solve $U\,x = y$:

$$x_4 = y_4/(\tfrac{5}{4}) = -\tfrac{1}{2}/\tfrac{5}{4} = -\tfrac{2}{5}, \tag{269}$$
$$x_3 = (y_3 + x_4)/(\tfrac{4}{3}) = (\tfrac{2}{3} - \tfrac{2}{5})/\tfrac{4}{3} = \tfrac{4}{15}/\tfrac{4}{3} = \tfrac{1}{5}, \tag{270}$$
$$x_2 = (y_2 + x_3)/(\tfrac{3}{2}) = (-\tfrac{1}{2} + \tfrac{1}{5})/\tfrac{3}{2} = -\tfrac{3}{10}/\tfrac{3}{2} = -\tfrac{1}{5}, \tag{271}$$
$$x_1 = (y_1 + x_2)/2 = (1 - \tfrac{1}{5})/2 = \tfrac{2}{5}. \tag{272}$$

Sample value $v \cdot x$:

$$v^\top x = (+1)(\tfrac{2}{5}) + (-1)(-\tfrac{1}{5}) + (+1)(\tfrac{1}{5}) + (-1)(-\tfrac{2}{5}) = \tfrac{2}{5} + \tfrac{1}{5} + \tfrac{1}{5} + \tfrac{2}{5} = \tfrac{6}{5} = 1.2. \tag{273}$$

That is one Hutchinson sample of $\mathrm{Tr}(A^{-1})$. The exact trace of A^{-1} for this A is $\mathrm{Tr}(A^{-1}) = \frac{2+3\cdot4+4\cdot3+2\cdot4}{5} = \frac{8}{5} \cdot \ldots$ – never mind the closed form; the point is that one random probe gives 1.2, which is an unbiased estimate of the trace, with random noise that vanishes when we average K samples. **Every step of this calculation – the L and U factors, the forward and back substitutions, the dot product – is literally the code in `lu.solve(v)` followed by `v @ x`.**

Code line in our pipeline.

```
lu = splu(A)
```

That single line factors the 4000×4000 matrix and gives us a solver object that we will hit $K = 100$ times in the loop below. The information of the fluctuation operator $\widetilde{\mathcal{M}}$ (plus the shift $s^2\mathbb{I}$) is now encoded in L and U, and every Hutchinson probe accesses it through the cheap forward+back recurrences above.

9.2.2 Explicit LU of *our* fluctuation operator

This subsection writes out, explicitly and entry by entry, what the matrix $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ looks like for the bounce problem, what its sparse LU factors L and U contain, and how the per-probe sample $v^\top x$ is assembled from these factors.

Step (i): the FD discretization of $-\partial_r^2$. On the uniform grid $r_i = r_{\min} + i\,\mathrm{d}r$, the second-derivative stencil produces the tridiagonal matrix

$$-L_2 \;=\; \frac{1}{\mathrm{d}r^2}\begin{pmatrix} +2 & -1 & & & \\ -1 & +2 & -1 & & \\ & -1 & +2 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & +2 \end{pmatrix} \;\in\; \mathbb{R}^{N\times N}. \tag{274}$$

Apart from sign conventions, this is the discrete Laplacian on a 1D grid. Its sparsity pattern is tridiagonal – bandwidth 1.

Where this matrix comes from – Taylor expansion of $\tilde{u}(r)$. The entries $\big(1,-2,1\big)/\mathrm{d}r^2$ in (274) are not arbitrary – they are forced by demanding that the matrix approximate the second derivative ∂_r^2 when applied to a smooth function. Here is the line-by-line derivation.

(i.1) Taylor-expand $\tilde{u}(r_i \pm \mathrm{d}r)$ around the grid point r_i. Both expansions use the same Taylor series, with opposite signs on the odd-derivative terms:

$$\tilde{u}_{i+1} \;\equiv\; \tilde{u}(r_i+\mathrm{d}r) \;=\; \tilde{u}_i + \mathrm{d}r\,\tilde{u}_i' + \tfrac{\mathrm{d}r^2}{2}\,\tilde{u}_i'' + \tfrac{\mathrm{d}r^3}{6}\,\tilde{u}_i''' + \tfrac{\mathrm{d}r^4}{24}\,\tilde{u}_i^{(4)} + \mathcal{O}(\mathrm{d}r^5), \tag{275}$$

$$\tilde{u}_{i-1} \;\equiv\; \tilde{u}(r_i-\mathrm{d}r) \;=\; \tilde{u}_i - \mathrm{d}r\,\tilde{u}_i' + \tfrac{\mathrm{d}r^2}{2}\,\tilde{u}_i'' - \tfrac{\mathrm{d}r^3}{6}\,\tilde{u}_i''' + \tfrac{\mathrm{d}r^4}{24}\,\tilde{u}_i^{(4)} + \mathcal{O}(\mathrm{d}r^5). \tag{276}$$

(i.2) Add the two expansions to kill the odd-derivative terms. The $\tilde{u}_i'$ and $\tilde{u}_i'''$ contributions cancel, leaving

$$\tilde{u}_{i+1} + \tilde{u}_{i-1} \;=\; 2\,\tilde{u}_i + \mathrm{d}r^2\,\tilde{u}_i'' + \tfrac{\mathrm{d}r^4}{12}\,\tilde{u}_i^{(4)} + \mathcal{O}(\mathrm{d}r^6). \tag{277}$$

(i.3) Solve (277) for $\tilde{u}_i''$. Subtract $2\tilde{u}_i$ from both sides and divide by $\mathrm{d}r^2$:

$$\tilde{u}_i'' \;=\; \frac{\tilde{u}_{i+1} - 2\,\tilde{u}_i + \tilde{u}_{i-1}}{\mathrm{d}r^2} \;-\; \tfrac{\mathrm{d}r^2}{12}\,\tilde{u}_i^{(4)} \;-\; \mathcal{O}(\mathrm{d}r^4). \tag{278}$$

The first term on the right is the *2nd-order central FD approximation* of $\tilde{u}''(r_i)$. The remaining terms are the truncation error, which scales as $\mathcal{O}(\mathrm{d}r^2)$ – this is what makes the stencil "2nd-order accurate."

(i.4) Read off the matrix $-L_2$. Drop the $\mathcal{O}(\mathrm{d}r^2)$ error and take the negative (because our operator contains $-\partial_r^2$, not $+\partial_r^2$):

$$-\tilde{u}_i'' \;\approx\; \frac{-\tilde{u}_{i+1} + 2\,\tilde{u}_i - \tilde{u}_{i-1}}{\mathrm{d}r^2} \;=\; \frac{1}{\mathrm{d}r^2}\big[\,(-1)\,\tilde{u}_{i-1} + 2\,\tilde{u}_i + (-1)\,\tilde{u}_{i+1}\,\big]. \tag{279}$$

The right-hand side is exactly the i-th row of the matrix-vector product $-L_2\,\tilde{u}$, with the matrix entries

$$(-L_2)_{ij} \;=\; \frac{1}{\mathrm{d}r^2}\begin{cases} +2 & \text{if } j = i, \\ -1 & \text{if } j = i \pm 1, \\ 0 & \text{otherwise.} \end{cases} \tag{280}$$

Stacking these rows for $i = 0, 1, \ldots, N-1$ gives exactly the tridiagonal matrix in (274). **This is the derivation: 2nd-order central differences on a uniform grid produce a tridiagonal matrix with rows $(1,-2,1)/\mathrm{d}r^2$, equivalently $(-1,+2,-1)/\mathrm{d}r^2$ once we negate to get $-\partial_r^2$.**

Sanity check on a constant. Apply $-L_2$ to a constant array $\tilde{u}_i = c$: $\big(-L_2\tilde{u}\big)_i = (1/\mathrm{d}r^2)\cdot(-c+2c-c) = 0$ – correct, since the second derivative of a constant is zero. (Apart from the boundary rows – which is exactly why we introduce the symmetric Dirichlet trick in §6.4.)

Step (ii): the bounce profile enters via the Hessian diagonal. At each grid point we evaluate the 2×2 Hessian $U''(\phi_b(r_i))$ and split it into three diagonal arrays:

$$U''_{11}(r_i), \qquad U''_{12}(r_i) = U''_{21}(r_i), \qquad U''_{22}(r_i), \qquad i = 0,\dots,N-1. \tag{281}$$

The Liouville centrifugal term contributes another diagonal,

$$V_n(r_i) \;=\; \frac{n(n+2)+3/4}{r_i^2}. \tag{282}$$

Step (iii): the full $2N\times 2N$ block-sparse $\widetilde{\mathcal{M}}_n$.

$$\widetilde{\mathcal{M}}_n \;=\; \begin{pmatrix} -L_2 + \mathrm{diag}(V_n + U''_{11}) & \mathrm{diag}(U''_{12}) \\ \mathrm{diag}(U''_{12}) & -L_2 + \mathrm{diag}(V_n + U''_{22}) \end{pmatrix}. \tag{283}$$

Writing out a block of size $N = 4$ for clarity (single field index suppressed; the off-diagonal Hessian block is similar but diagonal):

$$\big[\widetilde{\mathcal{M}}_n\big]_{\text{one block}} \;=\; \begin{pmatrix} \frac{2}{\mathrm{d}r^2} + V_n^{(0)} + U_{11}^{(0)} & -\frac{1}{\mathrm{d}r^2} & 0 & 0 \\ -\frac{1}{\mathrm{d}r^2} & \frac{2}{\mathrm{d}r^2} + V_n^{(1)} + U_{11}^{(1)} & -\frac{1}{\mathrm{d}r^2} & 0 \\ 0 & -\frac{1}{\mathrm{d}r^2} & \frac{2}{\mathrm{d}r^2} + V_n^{(2)} + U_{11}^{(2)} & -\frac{1}{\mathrm{d}r^2} \\ 0 & 0 & -\frac{1}{\mathrm{d}r^2} & \frac{2}{\mathrm{d}r^2} + V_n^{(3)} + U_{11}^{(3)} \end{pmatrix}, \tag{284}$$

where $V_n^{(i)} = V_n(r_i)$, $U_{11}^{(i)} = U''_{11}(\phi_b(r_i))$, etc. The two-field $2N\times 2N$ matrix has *four* such tridiagonal blocks coupled diagonally by $\mathrm{diag}(U''_{12})$. The boundary indices $\{0, N-1, N, 2N-1\}$ are then row/column-zeroed via the symmetric trick (198).

Step (iv): adding s^2 to form $A = \widetilde{\mathcal{M}}_n + s^2\mathbb{I}$. Adding $s^2\mathbb{I}$ shifts *only the diagonal entries*. So at every diagonal entry i in the matrix above, we add s^2:

$$A_{ii} \;=\; \frac{2}{\mathrm{d}r^2} + V_n(r_i) + U''_{aa}(\phi_b(r_i)) + s^2, \qquad a = 1 \text{ or } 2. \tag{285}$$

All off-diagonal entries are unchanged from $\widetilde{\mathcal{M}}_n$. The matrix A is still sparse, symmetric, and almost-tridiagonal-with-coupling.

Step (v): sparse LU $A = L\,U$. `splu` performs Gaussian elimination on the sparse A and returns triangular L and U such that $L\,U = A$ (*exactly*, to machine precision). The factorization *is* just standard Gaussian elimination, written down as two matrices instead of done in place. We spell it out concretely.

(v.1) The recursion that builds L and U from A. Start from $A^{(0)} = A$. For each elimination step $k = 1, 2, \dots, 2N-1$:

1. Define the multipliers $\ell_{ik} = A_{ik}^{(k-1)}/A_{kk}^{(k-1)}$ for $i > k$. These tell us by how much we must scale row k before subtracting it from row i to zero out the entry A_{ik}.

2. Update the trailing submatrix: $A^{(k)}_{ij} = A^{(k-1)}_{ij} - \ell_{ik}\, A^{(k-1)}_{kj}$ for $i, j > k$.

3. Store ℓ_{ik} as the corresponding entry of L.

After $2N-1$ such steps the elimination is done: U is the upper-triangular result of all the row subtractions, and L is the unit-lower-triangular matrix of multipliers (with 1's on its diagonal – Doolittle convention). For a banded matrix with bandwidth b, only b multipliers per column are nonzero, so the cost is $\mathcal{O}(Nb^2)$ – a small multiple of N for our $b \sim 4$.

(v.2) Concrete worked example: a 4×4 tridiagonal matrix. Take a small toy matrix that mimics the FD structure of $-L_2$ at $\mathrm{d}r = 1$ (so $1/\mathrm{d}r^2 = 1$), with diagonal shift $s^2 = 0$ and constant on-site mass 1:

$$A \;=\; \begin{pmatrix} 3 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 \\ 0 & -1 & 3 & -1 \\ 0 & 0 & -1 & 3 \end{pmatrix}. \tag{286}$$

Run the elimination explicitly:

- $k = 1$: $\ell_{21} = -1/3$, then row 2 becomes $(0,\ 3-(-1)(-1/3),\ -1,\ 0) = (0,\ 8/3,\ -1,\ 0)$.
- $k = 2$: $\ell_{32} = -1/(8/3) = -3/8$, then row 3 becomes $(0,\ 0,\ 3-(-1)(-3/8),\ -1) = (0,\ 0,\ 21/8,\ -1)$.
- $k = 3$: $\ell_{43} = -1/(21/8) = -8/21$, then row 4 becomes $(0,\ 0,\ 0,\ 3-(-1)(-8/21)) = (0,\ 0,\ 0,\ 55/21)$.

So we end up with

$$L \;=\; \begin{pmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{3} & 1 & 0 & 0 \\ 0 & -\frac{3}{8} & 1 & 0 \\ 0 & 0 & -\frac{8}{21} & 1 \end{pmatrix}, \qquad U \;=\; \begin{pmatrix} 3 & -1 & 0 & 0 \\ 0 & \frac{8}{3} & -1 & 0 \\ 0 & 0 & \frac{21}{8} & -1 \\ 0 & 0 & 0 & \frac{55}{21} \end{pmatrix}. \tag{287}$$

Verify by multiplying out $L\,U$: you recover A exactly. The diagonals of U $(3, 8/3, 21/8, 55/21)$ are the "modified" diagonals after each elimination step; the multipliers $(\ell_{21}, \ell_{32}, \ell_{43}) = (-1/3, -3/8, -8/21)$ are exactly what L stores. Every nonzero entry of L and U comes directly from the entries of A via the elimination recursion in (v.1).

(v.3) For the full 2-field problem. The same pattern extends: L and U are sparse banded matrices whose entries come from the elimination on $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$, with extra fill-in only at the few rows/columns where the U''_{12} off-diagonal Hessian couples the two fields. SuperLU inside `splu` just runs this recursion for us on the full $2N \times 2N$ matrix, storing L and U as sparse matrices. The information of $\widetilde{\mathcal{M}}$, the FD stencil, the Hessian, and the shift s^2 has all been folded into the entries of L and U – nothing is lost.

Step (vi): how the code computes $x = A^{-1}\,v$ for one Hutchinson probe. Given the LU factors and a Rademacher probe $v \in \{\pm 1\}^{2N}$ (zeroed at boundary indices), `lu.solve(v)` performs two "triangular solves." Before the recurrences themselves, here is the language they use.

(vi.0) Triangular matrices: definition, source, and why we want them.

Definition. A matrix $T \in \mathbb{R}^{n\times n}$ is **lower-triangular** if every entry above the main diagonal is zero ($T_{ij} = 0$ for $j > i$); it is **upper-triangular** if every entry below the

diagonal is zero ($T_{ij} = 0$ for $j < i$). A matrix that is either lower- or upper-triangular is called *triangular*.

Pictorially:

$$L \;=\; \begin{pmatrix} \star & 0 & 0 & 0 \\ \star & \star & 0 & 0 \\ \star & \star & \star & 0 \\ \star & \star & \star & \star \end{pmatrix}, \quad U \;=\; \begin{pmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \end{pmatrix}, \tag{288}$$

where $\star$ is any (possibly nonzero) entry.

Where these triangular matrices come from in our pipeline. They are exactly the output of step (v) – the LU factorization. By its very construction, Gaussian elimination produces a lower- triangular factor L (collecting the multipliers ℓ_{ij}) and an upper-triangular factor U (the row-reduced form of A), satisfying $L\,U = A$. So we never have to look for triangular matrices: `splu(A)` hands them to us directly. *The triangular structure is the entire point of doing LU* – we trade one "hard" linear system $A\,x = v$ (whose direct inversion is expensive) for two "easy" triangular ones, $L\,y = v$ and $U\,x = y$.

Why "triangular" means "easy to solve." For a general dense matrix, solving $A\,x = v$ requires $\mathcal{O}(n^3)$ operations (Gaussian elimination). For a triangular matrix, the structure of zeros lets you peel off one unknown per row by simple substitution: each row of $T\,y = v$ is an equation that involves only the diagonal-and-already-known entries, so it can be solved for the new unknown in a single step. Total cost drops to $\mathcal{O}(n^2)$ for a dense triangular matrix and all the way down to $\mathcal{O}(n)$ for a banded one like ours. In short:

- *Lower-triangular L:* solve top-to-bottom (each row depends only on earlier y's).
- *Upper-triangular U:* solve bottom-to-top (each row depends only on later x's).

The recurrences (vi.1)–(vi.2) below are the explicit form of these two algorithms.

(vi.1) Forward solve $L\,y = v$ (top to bottom).

Setup. The system $L\,y = v$ is $2N$ equations in the $2N$ unknowns $y_1, \ldots, y_{2N}$. Reading off row i of $L\,y$ – a sum over column index j – we get

$$\sum_{j=1}^{2N} L_{ij}\, y_j \;=\; v_i. \tag{289}$$

Because L is lower-triangular, $L_{ij} = 0$ for $j > i$, so the sum truncates at $j = i$:

$$\sum_{j=1}^{i} L_{ij}\, y_j \;=\; v_i. \tag{290}$$

Split off the diagonal term $L_{ii}\, y_i$:

$$L_{ii}\, y_i \;+\; \sum_{j=1}^{i-1} L_{ij}\, y_j \;=\; v_i. \tag{291}$$

Solve algebraically for y_i:

$$\boxed{\; y_i \;=\; \frac{1}{L_{ii}}\Big(v_i \;-\; \sum_{j=1}^{i-1} L_{ij}\, y_j \Big). \;} \tag{292}$$

This is the general formula – it works for *any* lower-triangular matrix, not just banded ones.

Why "top to bottom"? Look at (292): y_i depends only on $y_1, \ldots, y_{i-1}$, never on $y_{i+1}, \ldots, y_{2N}$. So if we already know the earlier y_j, we can read off the next y_i in one step. That gives the natural sequential order

$$y_1 \;\to\; y_2 \;\to\; y_3 \;\to\; \ldots \;\to\; y_{2N}, \tag{293}$$

each step using the previously computed values. There is no ambiguity: the dependency chain forces this top-to-bottom order. The first row ($i = 1$) has *no* previous-y dependency at all (the sum is empty), so $y_1 = v_1/L_{11}$ – a single division to launch the recurrence.

Doolittle convention used by SuperLU and our `splu`*.* The factorization is normalized so that $L_{ii} = 1$ for every i. This drops the division: (292) simplifies to

$$y_i \;=\; v_i \;-\; \sum_{j=1}^{i-1} L_{ij}\, y_j. \tag{294}$$

Specialization to a pure tridiagonal L (toy / single-field illustration). If the original A were strictly tridiagonal (a single field, no coupling), the LU factorization would produce an L in which only the main diagonal ($= 1$) and the immediate sub-diagonal $\ell_{i,i-1} \equiv L_{i,i-1}$ are nonzero. The sum in (294) would then reduce to a single term, and the general formula would collapse to a 2-term recurrence:

$$y_1 = v_1, \qquad y_i = v_i - \ell_{i,i-1}\, y_{i-1}, \quad i = 2, \ldots, 2N. \tag{295}$$

Each y_i now needs just one multiply, one subtract, and the previous y_{i-1}. Cost: $\mathcal{O}(2N)$ flops total.

Worked on the toy 4×4 (where $\ell_{21} = -1/3,\ \ell_{32} = -3/8,\ \ell_{43} = -8/21$ from step v.2): take $v = (+1, -1, +1, -1)^\top$.

$$\begin{aligned}
&y_1 = +1 && \text{(no previous } y\text{)}\\
&y_2 = -1 - (-\tfrac{1}{3})(+1) = -1 + \tfrac{1}{3} = -\tfrac{2}{3} && \text{(uses just-computed } y_1\text{)}\\
&y_3 = +1 - (-\tfrac{3}{8})(-\tfrac{2}{3}) = +1 - \tfrac{1}{4} = +\tfrac{3}{4} && \text{(uses } y_2\text{)}\\
&y_4 = -1 - (-\tfrac{8}{21})(+\tfrac{3}{4}) = -1 + \tfrac{2}{7} = -\tfrac{5}{7} && \text{(uses } y_3\text{)}
\end{aligned} \tag{296}$$

So $y = (+1,\ -\frac{2}{3},\ +\frac{3}{4},\ -\frac{5}{7})^\top$. Note how each row uses the y_{i-1} computed by the previous row – no second pass needed.

For our actual 2-field A. The 2-term recurrence above is **not** the formula the code uses for our matrix. Our A has the off-diagonal Hessian coupling U''_{12} between the two fields, and the LU factorization produces an L that has a few additional non-zero entries in the rows where the two fields meet. *Nothing is dropped or approximated* – `splu` computes the exact sparse LU including all fill-in. The code path that actually runs is the general formula (294), $y_i = v_i - \sum_{j<i} L_{ij}\, y_j$, where the sum just happens to have a single term for most i (banded part) and a few extra terms for the rows touched by the field-coupling fill-in. The structure (top-to-bottom dependency, $\mathcal{O}(Nb)$ cost with $b \sim 4$) is unchanged; the result is exactly $y = L^{-1}v$.

(vi.2) Back solve $U\,x = y$ (bottom to top).

Setup. The system $U\,x = y$ has $2N$ equations. Row i of $U\,x$:

$$\sum_{j=1}^{2N} U_{ij}\, x_j \;=\; y_i. \tag{297}$$

Because U is upper-triangular, $U_{ij} = 0$ for $j < i$, so the sum starts at $j = i$:

$$U_{ii}\, x_i \;+\; \sum_{j=i+1}^{2N} U_{ij}\, x_j \;=\; y_i. \tag{298}$$

Solve for x_i:

$$\boxed{x_i \;=\; \frac{1}{U_{ii}}\Big(\, y_i \;-\; \sum_{j=i+1}^{2N} U_{ij}\, x_j \,\Big).} \tag{299}$$

This is the general formula – valid for *any* upper-triangular matrix.

Why "bottom to top"? Now x_i depends only on $x_{i+1}, \ldots, x_{2N}$, so we need the *later* entries first. The dependency chain forces

$$x_{2N} \;\to\; x_{2N-1} \;\to\; \ldots \;\to\; x_2 \;\to\; x_1, \tag{300}$$

the reverse of the forward solve. The last row ($i = 2N$) has *no* later-x dependency, so $x_{2N} = y_{2N}/U_{2N,2N}$ in one step.

Specialization to a pure tridiagonal U (toy / single-field illustration). If the original A were strictly tridiagonal, the LU would produce a U with nonzero entries only on the main diagonal U_{ii} and the immediate super-diagonal $U_{i,i+1}$. The sum in (299) would then collapse to a single term, giving the simple recurrence

$$x_{2N} = \frac{y_{2N}}{U_{2N,2N}}, \qquad x_i = \frac{1}{U_{ii}}\big(y_i - U_{i,i+1}\, x_{i+1}\big), \quad i = 2N-1, \ldots, 1. \tag{301}$$

Each x_i needs one multiply, one subtract, and one divide; total cost $\mathcal{O}(2N)$.

For our actual 2-field A. Same caveat as for L: the field-coupling U''_{12} produces a few additional non-zero entries in U (fill-in beyond the immediate super-diagonal). The code uses the full general formula (299), $x_i = (y_i - \sum_{j>i} U_{ij}\, x_j)/U_{ii}$, with all those extra terms automatically included. Nothing is dropped or approximated. The total cost is still $\mathcal{O}(Nb)$ with $b \sim 4$ instead of $b = 1$ for pure tridiagonal – a factor of ~ 4 more flops than (301), still tiny.

Worked on the toy 4×4: continuing from y above, with $U_{i,i+1} = -1$ throughout and the diagonal entries $U_{ii} = (3, \frac{8}{3}, \frac{21}{8}, \frac{55}{21})$ from step (v.2),

$$\begin{aligned}
x_4 &= y_4/U_{44} = -\tfrac{5}{7} \cdot \tfrac{21}{55} = -\tfrac{3}{11} && \text{(no later } x\text{)} \\
x_3 &= (y_3 - (-1)\, x_4)/U_{33} = (\tfrac{3}{4} - \tfrac{3}{11}) \cdot \tfrac{8}{21} = \tfrac{2}{11} && \text{(uses just-computed } x_4\text{)} \\
x_2 &= (y_2 - (-1)\, x_3)/U_{22} = (-\tfrac{2}{3} + \tfrac{2}{11}) \cdot \tfrac{3}{8} = -\tfrac{2}{11} && \text{(uses } x_3\text{)} \\
x_1 &= (y_1 - (-1)\, x_2)/U_{11} = (1 - \tfrac{2}{11})/3 = \tfrac{3}{11} && \text{(uses } x_2\text{)}
\end{aligned} \tag{302}$$

So $x = (\frac{3}{11},\, -\frac{2}{11},\, +\frac{2}{11},\, -\frac{3}{11})^\top$.

Are equations (292) **and** (299) **general definitions? Yes.** They are the textbook formulas for forward and backward substitution on *any* triangular linear system – nothing in their derivation used anything special about the bounce, the FD discretization, or our particular operator. Wherever you have $L\, y = v$ with L lower-triangular, (292) works; wherever you have $U\, x = y$ with U upper-triangular, (299) works. The two-term recurrences (295) and (301) are the *specialization* to the banded structure that our particular L and U happen to have (because A was tridiagonal-with-block- coupling). For a denser triangular

matrix the inner sum has more terms, but the structure – top-to-bottom for L, bottom-to-top for U – is unchanged.

(vi.3) How the two solves combine: $x = U^{-1}L^{-1}v = A^{-1}v$. The forward solve produced $y = L^{-1}v$. The back solve produced $x = U^{-1}y = U^{-1}L^{-1}v$. But $L\,U = A$, so $U^{-1}L^{-1} = (L\,U)^{-1} = A^{-1}$. Therefore

$$\boxed{x \;=\; A^{-1}\,v.} \tag{303}$$

Verification on the toy 4×4*:* compute $A\,x$ directly:

$$\begin{aligned} Ax &= \begin{pmatrix} 3 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 \\ 0 & -1 & 3 & -1 \\ 0 & 0 & -1 & 3 \end{pmatrix} \begin{pmatrix} 3/11 \\ -2/11 \\ 2/11 \\ -3/11 \end{pmatrix} \\ &= \begin{pmatrix} 3(3/11) - (-2/11) \\ -(3/11) + 3(-2/11) - (2/11) \\ -(-2/11) + 3(2/11) - (-3/11) \\ -(2/11) + 3(-3/11) \end{pmatrix} = \begin{pmatrix} 11/11 \\ -11/11 \\ 11/11 \\ -11/11 \end{pmatrix} = \begin{pmatrix} +1 \\ -1 \\ +1 \\ -1 \end{pmatrix} = v.\checkmark \end{aligned} \tag{304}$$

The two recurrences (forward and back) propagate the right-hand side v through the LU factors, and the output x satisfies $A\,x = v$ exactly. This is what `lu.solve(v)` returns.

(vi.4) For our actual matrix: nothing is approximated. On our $2N = 4000$ matrix the forward+back solves take a few tens of thousands of multiplies total, completing in well under a millisecond. The code path is the *full general formulas* (294) and (299) – with the inner sums truncated automatically wherever L_{ij} or U_{ij} happens to be zero, but *never artificially zeroed out.* The off-diagonal block coupling U''_{12} produces a few additional non-zero entries in L and U (fill-in beyond the immediate sub/super-diagonals); those entries are kept exactly. The two-term recurrences (295) and (301) above are the *simplified illustration* for a pure tridiagonal A (single field, no coupling); they correctly describe the actual $L\,y = v$ and $U\,x = y$ algorithm only if you mentally extend each sum to include the few extra fill-in terms. The structure is the same in both cases: top-to-bottom for $L\,y = v$, bottom-to-top for $U\,x = y$, and the result is exactly $x = A^{-1}v$ (modulo floating-point roundoff).

Why "no approximation" even when the toy formula is incomplete. `scipy.sparse.linalg.sp` computes the exact sparse LU of A (Gaussian elimination, with partial pivoting). Then `lu.solve(v)` runs the exact triangular-substitution recurrences on the sparse L and U – looping over only the non-zero entries in each row, which is what makes it fast. The "fill-in" entries from the field coupling are non-zero in L and U, get visited by the recurrences, and contribute their full $L_{ij}y_j$ or $U_{ij}x_j$ amount. Nothing is dropped. The 2-term recurrence in this document is purely a pedagogical shorthand for the simplest case (the toy 4×4); the actual solver uses the full sums.

Step (vii): the per-probe Hutchinson sample $v^\top x$. The sample value is the dot product

$$s_k \;\equiv\; v^\top x \;=\; \sum_{i=1}^{2N} v_i\,x_i. \tag{305}$$

This is the actual line of code: a single `numpy` dot product between the random probe and the LU solve result.

(vii.a) Substitute $x = A^{-1}v$. Step (vi) gave us $x = A^{-1}v$, so

$$s_k \;=\; v^\top \big(A^{-1}v\big). \tag{306}$$

Multiplying out the matrix-vector product on the right and writing the dot product on the left as another sum:

$$s_k \;=\; \sum_{i=1}^{2N}\sum_{j=1}^{2N} v_i\,(A^{-1})_{ij}\,v_j. \tag{307}$$

This is just the bilinear form $v^\top B\,v$ with $B = A^{-1}$, written out index-by-index.

(vii.b) Split the double sum into diagonal and off-diagonal. Separate the $i = j$ terms from the $i \neq j$ terms:

$$s_k \;=\; \underbrace{\sum_{i} v_i^2\,(A^{-1})_{ii}}_{\text{diagonal piece}} \;+\; \underbrace{\sum_{i\neq j} v_i v_j\,(A^{-1})_{ij}}_{\text{off-diagonal piece}}. \tag{308}$$

(vii.c) Apply Rademacher properties. Recall that each v_i is independently ± 1 with probability $1/2$. Two facts follow:

$$v_i^2 = 1 \;\text{ deterministically (always, for every probe)}, \qquad \mathbb{E}[v_i\,v_j] = 0 \;\text{ for } i \neq j. \tag{309}$$

The second fact is because the four sign combinations $(\pm,\pm)$ are equally likely, so the average of $v_i v_j$ is $\frac{1}{4}(+1-1-1+1) = 0$.

(vii.d) Take the expectation of (308)**.** Linearity of expectation moves $\mathbb{E}$ inside both sums:

$$\begin{aligned}
\mathbb{E}[s_k] \;&=\; \sum_i \underbrace{\mathbb{E}[v_i^2]}_{=1}(A^{-1})_{ii} \;+\; \sum_{i\neq j}\underbrace{\mathbb{E}[v_i v_j]}_{=0}(A^{-1})_{ij} \\
&=\; \sum_i (A^{-1})_{ii} \;+\; 0 \\
&=\; \mathrm{Tr}\big(A^{-1}\big),
\end{aligned} \tag{310}$$

where the last equality is just the definition of trace – the sum of the diagonal entries of a matrix.

9.2.3 Derivation of the spectral trace identity $\mathrm{Tr}(A^{-1}) = \sum_L 1/(\omega_{L,n}^2 + s^2)$

(This is what step vii.d called "the next equality." We do it now in five sub-steps – the derivation of the spectral trace identity referenced as eq. (315) below.)

(vii.e) Connect the trace to the spectral sum – explicit derivation. Step (vii.d) gave us $\mathbb{E}[s_k] = \mathrm{Tr}(A^{-1})$. The next equality writes that trace in terms of the eigenvalues of $\widetilde{\mathcal{M}}$. Done in five sub-steps:

(e.1) $\widetilde{\mathcal{M}}$ is symmetric (built that way in §6.4 via the symmetric Dirichlet trick). The spectral theorem then gives an *orthonormal eigenbasis* $\{\chi_{L,n}\}_L$ with real eigenvalues $\omega_{L,n}^2$:

$$\widetilde{\mathcal{M}}\,\chi_{L,n} \;=\; \omega_{L,n}^2\,\chi_{L,n}, \qquad \chi_{L,n}^\top \chi_{L',n} = \delta_{LL'}. \tag{311}$$

(e.2) Adding the diagonal shift $s^2\mathbb{I}$ does not change the eigenvectors – it just shifts every eigenvalue by s^2:

$$A\,\chi_{L,n} \;=\; (\widetilde{\mathcal{M}} + s^2\mathbb{I})\,\chi_{L,n} \;=\; \omega_{L,n}^2\,\chi_{L,n} + s^2\,\chi_{L,n} \;=\; (\omega_{L,n}^2 + s^2)\,\chi_{L,n}. \tag{312}$$

So $\{\chi_{L,n}\}$ are also eigenvectors of A, with eigenvalues $\omega_{L,n}^2 + s^2$.

(e.3) The inverse A^{-1} has the same eigenvectors with *reciprocal* eigenvalues: from $A\,\chi_{L,n} = (\omega_{L,n}^2 + s^2)\,\chi_{L,n}$, divide both sides by $(\omega_{L,n}^2 + s^2)$ and apply A^{-1}:

$$A^{-1}\,\chi_{L,n} \;=\; \frac{1}{\omega_{L,n}^2 + s^2}\,\chi_{L,n}. \tag{313}$$

(e.4) Because $\{\chi_{L,n}\}$ form an orthonormal basis, A^{-1} has the spectral resolution

$$A^{-1} \;=\; \sum_{L} \frac{1}{\omega_{L,n}^2 + s^2}\,\chi_{L,n}\,\chi_{L,n}^{\top}. \tag{314}$$

(Each rank-1 projector $\chi_{L,n}\chi_{L,n}^{\top}$ acts as the identity on $\chi_{L,n}$ and zero on every other eigenvector, multiplied by the corresponding $1/(\omega_{L,n}^2 + s^2)$.)

(e.5) Take the trace term-by-term. The trace of an outer product $\chi\,\chi^{\top}$ is just $\chi^{\top}\chi$ (a scalar), so $\mathrm{Tr}[\chi_{L,n}\,\chi_{L,n}^{\top}] = \chi_{L,n}^{\top}\chi_{L,n} = 1$ by normalization. Therefore

$$\mathrm{Tr}\big(A^{-1}\big) \;=\; \sum_{L} \frac{1}{\omega_{L,n}^2 + s^2}\,\underbrace{\mathrm{Tr}[\chi_{L,n}\,\chi_{L,n}^{\top}]}_{=1} \;=\; \sum_{L} \frac{1}{\omega_{L,n}^2 + s^2}. \tag{315}$$

This is the equation in question, derived line by line from the symmetry of $\widetilde{\mathcal{M}}$ and basic properties of eigendecompositions.

(vii.f) Putting (vii.a)–(vii.e) in one equation. Combining the chain:

$$\boxed{\mathbb{E}[s_k] \;=\; \mathbb{E}\big[v^{\top}A^{-1}v\big] \;=\; \mathrm{Tr}(A^{-1}) \;=\; \sum_{L} \frac{1}{\omega_{L,n}^2 + s^2} \;=\; \frac{\overline{G}_n^{\mathrm{raw}}(s^2)}{(n+1)^2}.} \tag{316}$$

One LU solve plus one dot product gives one unbiased estimate of $\overline{G}_n^{\mathrm{raw}}/(n+1)^2$. The bounce profile and the FD discretization enter only through the LU factors L, U of A; the random probe v has no physics in it; the dot product $v \cdot x$ is the entire extraction step.

Step (viii): all of this in code.

```
A   = (M_tilde + s2 * sp.eye(N2)).tocsc()  # build A explicitly (sparse)
lu  = splu(A)                              # one LU of A (Steps i-v)
v   = rng.choice([-1.0, 1.0], size=N2)     # one Rademacher probe
v[boundary_indices] = 0.0                  # exclude boundary ghosts
x   = lu.solve(v)                          # forward + back solve (Step vi)
s_k = v @ x                                # one Hutchinson sample (Step
    vii)
```

Each line corresponds to one of the steps above; the entire information of the fluctuation operator $\widetilde{\mathcal{M}}$ – which encodes the bounce profile via $U''(\phi_b(r))$ and the kinetic energy via the FD second-derivative stencil – is funneled through the LU factors and consumed by the random-probe solve.

What is a "probe" and why do we need many of them?

A few definitions worth being explicit about, because the words "probe" and "K" come up everywhere.

What is a single Hutchinson probe? A *probe* is one specific random Rademacher vector $v_k \in \{-1, +1\}^{2N}$. It is just a length-$2N$ array of ± 1's, drawn independently of every

other probe. There is no physics in it – it is purely a random "test direction" from which we will sample the matrix.

What is K? K is the *total number of independent probes* we draw. In our default v1 setup $K = 100$; in v2 $K = 400$. Each probe gives one sample of the form

$$s_k \;=\; v_k^\top A^{-1} v_k, \qquad k = 1, 2, \ldots, K. \tag{317}$$

So at the end of the loop we have K different scalar values $s_1, s_2, \ldots, s_K$.

What does drawing many probes accomplish? Each individual sample is noisy: from (308) it equals $\mathrm{Tr}(A^{-1})$ *plus* a random off-diagonal contribution $\sum_{i\neq j} v_{k,i} v_{k,j} (A^{-1})_{ij}$ that has mean 0 but is nonzero for any single draw. If we used just $K = 1$ probe we would get an unbiased but very noisy estimate. Averaging K independent probes lets the off-diagonal noise cancel: by the central limit theorem, the noise of the average shrinks as $1/\sqrt{K}$.

The averaged estimator. The Hutchinson estimate of the trace is the sample mean of the K values:

$$\boxed{\widehat{\mathrm{Tr}}(A^{-1}) \;=\; \frac{1}{K}\sum_{k=1}^{K} s_k \;=\; \frac{1}{K}\sum_{k=1}^{K} v_k^\top A^{-1} v_k.} \tag{318}$$

This is the actual number we report (multiplied by $(n+1)^2$ to give $\overline{G}_n^{\mathrm{raw}}(s^2)$).

The error bar. The uncertainty on the mean is the standard error,

$$\mathrm{SEM} \;=\; \frac{\sigma}{\sqrt{K}}, \qquad \sigma^2 \;=\; \frac{1}{K-1}\sum_{k=1}^{K}\big(s_k - \bar{s}\big)^2, \qquad \bar{s} = \frac{1}{K}\sum_{k} s_k. \tag{319}$$

The SEM is what gets stored in the `*_err` arrays of the `.npz` output and what the plot scripts draw as error bars. For our problem with $K = 100$ and away from the pole, SEM is typically $\sim 1\%$–2% of $\overline{G}^{\mathrm{raw}}$. To halve it, take $K = 400$ (i.e. $4\times$ more probes); this is exactly what the v2 default does.

Code (the entire K-probe loop).

```
samples = np.empty(K)
for k in range(K):
    v = rng.choice([-1.0, 1.0], size=N2)   # one new probe each iteration
    v[boundary_indices] = 0.0
    x = lu.solve(v)
    samples[k] = v @ x                     # this is s_k
mean = float(np.mean(samples))             # eq. (hutch_mean)
sem  = float(np.std(samples, ddof=1) / np.sqrt(K))   # eq. (hutch_sem)
```

`samples` holds the K scalars $s_1, \ldots, s_K$; `mean` is $\widehat{\mathrm{Tr}}(A^{-1})$; `sem` is the $\pm 1\sigma$ uncertainty on that mean. The wrapper `gbar_raw_fd(M_tilde, s2, ...)` packages this whole K-probe loop together with the LU step, returning $(\overline{G}^{\mathrm{raw}}, \mathrm{SEM})$ for one s^2 value. Looping over the s^2 grid gives the full $\overline{G}^{\mathrm{raw}}(s^2)$ curve with error bars.

(End of the additions; the full average over K such samples is the trace estimate, and that is $\overline{G}_n^{\mathrm{raw}}(s^2)/(n+1)^2$.

9.2.4 Step 3a in detail: how Hutchinson estimates $\overline{G}^{\mathrm{raw}}$ without ever forming A^{-1}

The goal restated. We want

$$\frac{\overline{G}_n^{\mathrm{raw}}(s^2)}{(n+1)^2} \;=\; \mathrm{Tr}\big[A^{-1}\big] \;=\; \sum_{L} \frac{1}{\omega_{L,n}^2 + s^2}, \tag{320}$$

the sum of $1/(\omega_{L,n}^2 + s^2)$ over every interior eigenvalue. We *don't* compute the eigenvalues. We *don't* compute A^{-1} as a matrix. We just want the number on the right.

The key identity (with the bare minimum of equations). Let $v \in \mathbb{R}^{2N}$ be a random vector with each component independently drawn from $\{-1, +1\}$ (Rademacher). Two facts:

$$v_i^2 \;=\; 1 \;\text{ always}, \qquad \mathbb{E}[v_i\, v_j] \;=\; 0 \;\text{ for } i \neq j. \tag{321}$$

Use these to evaluate $v^\top B\, v$ for *any* matrix B:

$$\begin{aligned} v^\top B\, v \;&=\; \sum_{i,j} v_i\, B_{ij}\, v_j \\ &= \underbrace{\sum_{i} v_i^2\, B_{ii}}_{=\sum_i B_{ii} = \mathrm{Tr}(B)\;\text{(deterministic!)}} \;+\; \underbrace{\sum_{i \neq j} v_i\, v_j\, B_{ij}}_{\text{random, averages to 0}} . \end{aligned} \tag{322}$$

The diagonal piece is exactly the trace – this is true for every random draw, not just on average! The off-diagonal piece is random noise that averages out as we accumulate samples. Take K samples, divide by K, and the noise shrinks like $1/\sqrt{K}$.

The "magic": we never form A^{-1}. For our problem $B = A^{-1}$. We need $v^\top A^{-1} v$ for each of the K random probes. The trick is:

$$\boxed{\; v^\top A^{-1} v \;=\; v^\top \underbrace{(A^{-1} v)}_{x} \;=\; v^\top x, \;} \tag{323}$$

where x is the solution of the linear system $A\, x = v$ – which the LU factors solve cheaply. We never compute the entries of A^{-1}; we just compute its action on a single vector.

The spectral picture (why the math really works). This is the deepest understanding. Expand the random probe in the eigenbasis of $\widetilde{\mathcal{M}}$:

$$v \;=\; \sum_L c_L\, \chi_{L,n}, \qquad c_L \;=\; \chi_{L,n}^\top v. \tag{324}$$

Two key facts about the random coefficients c_L:

$$\mathbb{E}[c_L] = 0, \qquad \mathbb{E}[c_L\, c_{L'}] \;=\; \chi_{L,n}^\top \chi_{L',n} \;=\; \delta_{LL'}, \tag{325}$$

so the random coefficients are orthonormal in expectation – the random probe "looks at" every eigenmode with equal expected weight. Then

$$v^\top A^{-1} v \;=\; \sum_{L,L'} c_L\, c_{L'}\, \chi_{L,n}^\top A^{-1} \chi_{L',n} \;=\; \sum_{L,L'} c_L\, c_{L'} \frac{\delta_{LL'}}{\omega_{L,n}^2 + s^2} \tag{326}$$

$$= \sum_L \frac{c_L^2}{\omega_{L,n}^2 + s^2}. \tag{327}$$

Take expectation, use $\mathbb{E}[c_L^2] = 1$:

$$\mathbb{E}\big[v^\top A^{-1} v\big] \;=\; \sum_L \frac{1}{\omega_{L,n}^2 + s^2} \;=\; \frac{\overline{G}_n^{\mathrm{raw}}(s^2)}{(n+1)^2}. \tag{328}$$

This is the magic in one line. A random probe v, after being passed through A^{-1} and dotted with itself, gives a number whose expected value is exactly $\sum_L 1/(\omega_{L,n}^2 + s^2)$ – the spectral sum we wanted, with the right weight 1 on every eigenvalue. The randomness of the c_L's is exactly what makes *every* eigenvalue contribute on average, and the orthonormality of the eigenbasis is what makes the cross-terms vanish.

Algorithm: how the code actually runs.

1. Initialize an empty array `samples` of length $K = 100$.
2. For each $k = 1, \ldots, K$:
   (a) Draw $v_k \in \{-1, +1\}^{2N}$ (Rademacher).
   (b) Set the four boundary entries to zero, $v_k[\mathcal{B}] = 0$ (excludes the boundary ghost subspace – see §6.6).
   (c) Apply A^{-1} to v_k: solve $A\,x_k = v_k$ using the `lu.solve` method (cheap, thanks to the LU factorization done in Step 2).
   (d) Compute $s_k = v_k \cdot x_k = v_k^\top A^{-1} v_k$, store in `samples[k]`.
3. Compute the mean: $\widehat{\overline{G}}^{\text{raw}}/(n+1)^2 = (1/K)\sum_k s_k$.
4. Compute the SEM: $\sigma/\sqrt{K}$, where σ is the sample standard deviation of `samples`.

Tiny worked example to make this concrete. Suppose $A^{-1} = \begin{pmatrix} 2 & 0.3 \\ 0.3 & 5 \end{pmatrix}$, so $\mathrm{Tr}(A^{-1}) = 7$. Take $v = (+1, -1)$:

$$v^\top A^{-1} v = v_1^2 \cdot 2 + v_2^2 \cdot 5 + 2\,v_1 v_2 \cdot 0.3 \tag{329}$$

$$= 1 \cdot 2 + 1 \cdot 5 + 2(-1) \cdot 0.3 = 7 - 0.6 = 6.4. \tag{330}$$

The diagonal piece is $2+5 = 7$ (the true trace) – and notice it showed up *deterministically*, regardless of the choice of signs. The off-diagonal piece -0.6 depends on the specific sign combination, but if we average over all four possible signs of (v_1, v_2) – $(+,+), (+,-), (-,+), (-,-)$ – the cross term $2v_1v_2 \cdot 0.3$ becomes $2(0.3)(\frac{1+1-1-1}{4} \cdot 2) = 0$. Two more samples and we'd already be very close to 7.

Code (the whole Hutchinson loop in 7 lines).

```
samples = np.empty(K)
for k in range(K):
    v = rng.choice([-1.0, 1.0], size=N2)
    v[boundary_indices] = 0.0
    x = lu_factor.solve(v)          # the action of A^{-1} on v
    samples[k] = v @ x              # one Hutchinson sample
mean = float(np.mean(samples))      # approx Tr(A^{-1}) = G_bar^raw / (n+1)^2
sem  = float(np.std(samples, ddof=1) / np.sqrt(K))
```

That's it. Seven lines, $K = 100$ cheap LU solves, and we have $\overline{G}_n^{\text{raw}}(s^2)$ for one value of s^2 – with an honest error bar from the sample standard deviation. The wrapper `gbar_raw_fd(M_tilde, s2, dr, N2, K, rng, boundary_indices)` packages all of Steps 1, 2, and 3a together.

9.2.5 Step 3b in detail: how the projector creates $\overline{G}^{\mathrm{sub}}$ by killing one term in the spectral sum

What we want this time.

$$\frac{\overline{G}_n^{\mathrm{sub}}(s^2)}{(n+1)^2} \;=\; \mathrm{Tr}\big[P\,A^{-1}\,P\big] \;=\; \sum_{L\neq\bar{L}} \frac{1}{\omega_{L,n}^2+s^2}, \tag{331}$$

the spectral sum with one term missing – the one with $L=\bar{L}$, where $\chi_{\bar{L},n}=\chi$ is the special pole-bearing eigenmode (Coleman negative mode for $n=0$, translation zero mode for $n=1$). The pole at $s^2=-\omega_{\bar{L},n}^2$ has been removed *analytically*, not by any fitting or subtraction of large numbers.

The mechanism in one picture. The projector $P=\mathbb{I}-\chi\chi^{\top}$ kills the χ-component of any vector and lets the rest through. Apply it to each eigenvector:

- For $L=\bar{L}$ (the special mode): $P\,\chi=\chi-\chi(\chi^{\top}\chi)=\chi-\chi=0$.
- For $L\neq\bar{L}$ (any other mode): $P\,\chi_{L,n}=\chi_{L,n}-\chi(\chi^{\top}\chi_{L,n})=\chi_{L,n}-0=\chi_{L,n}$ (because $\widetilde{\mathcal{M}}$ is symmetric, its eigenvectors with different eigenvalues are orthogonal).

The first eigenvector goes to zero; all the others pass through untouched. So when we expand v in the eigenbasis and then sandwich $PA^{-1}P$, the $\bar{L}$ term carries an extra factor $0\cdot 0=0$ and the others carry $1\cdot 1=1$. The dangerous denominator $1/(\omega_{\bar{L},n}^2+s^2)$ disappears.

Spectral derivation, explicit. Repeat the spectral picture from Step 3a with P inserted on both sides:

$$\begin{aligned} v^{\top}P\,A^{-1}\,P\,v \;&=\; \sum_{L,L'} c_L c_{L'}\,(P\chi_{L,n})^{\top}A^{-1}(P\chi_{L',n}) \\ &=\; \sum_{L,L'\neq\bar{L}} c_L c_{L'}\,\chi_{L,n}^{\top}A^{-1}\chi_{L',n} \;=\; \sum_{L\neq\bar{L}} \frac{c_L^2}{\omega_{L,n}^2+s^2}. \end{aligned} \tag{332}$$

Take expectation:

$$\mathbb{E}\big[v^{\top}P\,A^{-1}\,P\,v\big] \;=\; \sum_{L\neq\bar{L}} \frac{1}{\omega_{L,n}^2+s^2} \;=\; \frac{\overline{G}_n^{\mathrm{sub}}(s^2)}{(n+1)^2}. \tag{333}$$

The pole-bearing term is simply absent. There is no fitting, no subtraction of large numbers, no catastrophic cancellation. This is the central reason the FD method has no residual spike (compare RK in §5.5).

Algorithm: per-probe steps. For each $k=1,\dots,K$:

1. Draw $v_k\in\{-1,+1\}^{2N}$, set $v_k[\mathcal{B}]=0$.
2. **Project the probe:** $w_k=Pv_k=v_k-\chi\,(\chi^{\top}v_k)$. Numerically: one dot product (a number), one rescale (multiply χ by that number), one subtraction. Total cost $\sim 2N$ flops.
3. **Solve** $A\,x_k=w_k$ via `lu.solve` (cheap).

4. **Re-project the result:** $y_k = Px_k = x_k - \chi\,(\chi^\top x_k)$. In exact arithmetic this is redundant (one P would suffice when sandwiched with $\mathbb{E}[vv^\top]$), but in floating-point it protects against any drift along χ that the LU may have introduced.

5. Sample: $s_k = v_k^\top y_k = v_k \cdot y_k$.

Why both projections in code (and not one)? The mathematical identity $\mathbb{E}[v^\top PA^{-1}Pv] = \mathrm{Tr}(PA^{-1}P)$ only requires both P's to be present *somewhere* – one before the solve and one after, or two after, etc. The choice in the code – one before ($P\,v$) and one after ($P\,A^{-1}\,P\,v$) – is the one that *minimizes per-probe variance*, because:

- Projecting v before the solve removes the χ-component of v, so we're not asking A^{-1} to act on a direction that lies near a near-singular eigenvalue.
- Projecting x after the solve removes any small χ-leakage that LU's roundoff may have introduced.

With both in place, the sample value $v^\top y_k$ is exactly the trace of $PA^{-1}P$ in expectation, and the variance is as small as it can be given the random-probe approach.

Code (the whole projected Hutchinson loop in 8 lines).

```
samples = np.empty(K)
for k in range(K):
    v = rng.choice([-1.0, 1.0], size=N2)
    v[boundary_indices] = 0.0
    w = v - chi * (chi @ v)          # project: w = P v
    x = lu_factor.solve(w)           # solve A x = w
    y = x - chi * (chi @ x)          # re-project: y = P x
    samples[k] = v @ y               # sample value
mean = float(np.mean(samples))       # approx Tr(P A^{-1} P) = G_bar^sub / (n+1)^2
sem  = float(np.std(samples, ddof=1) / np.sqrt(K))
```

The wrapper `gbar_sub_fd(M_tilde, s2, dr, N2, chi, K, rng, boundary_indices)` packages Steps 1, 2, and 3b. The only new ingredient compared to the raw version is the eigenvector χ (`chi_neg` for $n = 0$, `chi_zm` for $n = 1$); everything else is identical.

Putting raw and sub side by side: where they differ.

Quantity computed	$\overline{G}^{\mathbf{raw}}/(n+1)^2$	$\overline{G}^{\mathbf{sub}}/(n+1)^2$
What it equals	$\sum_L 1/(\omega_{L,n}^2 + s^2)$	$\sum_{L\neq\bar{L}} 1/(\omega_{L,n}^2 + s^2)$
What it samples	$v^\top A^{-1} v$	$v^\top\,P\,A^{-1}\,P\,v$
Extra ingredients	none	the eigenvector χ
Extra cost per probe	1 LU solve	1 LU solve + 2 dot products + 2 rescales
Pole at $s^2 = -\omega_{\bar{L},n}^2$	present (curve diverges)	absent (curve smooth)

The two routines are line-for-line identical except for two extra cheap operations per probe (project before, re-project after). That is the entire algorithmic content of "Feshbach + Hutchinson".

9.2.6 Two key questions about how the code uses the eigenvectors and the linear solve

This subsection answers two questions that often come up after seeing the algorithm above. They are the most common stumbling blocks.

Q1. Do we need the negative-mode eigenvector to subtract its contribution? How exactly does the code do the subtraction? Yes – we absolutely need χ as a numerical vector, not just as an idea. The code can't just "skip the $\bar{L}$ term in a sum" because it never enumerates the eigenvalues; the trace is estimated stochastically by Hutchinson, which sums over all eigenvalues implicitly. The only way to remove one term is to project the operator onto the orthogonal complement of χ *before* the trace is sampled. Here is how that happens line by line.

(i) Obtain χ as a length-$2N$ numerical vector.

- For $n = 0$: solve the discrete eigenvalue problem $\widetilde{\mathcal{M}}^{(N)}\chi_{\rm neg} = \lambda_{\rm neg}\,\chi_{\rm neg}$ via `scipy.sparse.linalg.e` asking for the smallest eigenvalue. This returns `lambda_neg` (a single number, ≈ -0.31 for our F2/T0 bounce) and `chi_neg` (a NumPy array of $2N = 4000$ floats, normalized so $\sum_i \chi_{{\rm neg},i}^2 = 1$).
- For $n = 1$: *do not solve an eigenvalue problem.* The zero-mode profile is known analytically as the radial derivative of the bounce: $\chi_{\rm zm}(r) \propto r^{3/2}\,\phi_b'(r)$. The code reads the bounce profile, splines it, takes the analytic spline derivative $\phi_b'(r_i)$ at each grid point, multiplies by $r_i^{3/2}$, stacks the two field components into a length-$2N$ array, and normalizes:

```
x_spline = CubicSpline(R_bounce, X_prime_bounce)
y_spline = CubicSpline(R_bounce, Y_prime_bounce)
dx = x_spline(r, 1)         # first derivative of x-component bounce
dy = y_spline(r, 1)         # first derivative of y-component bounce
chi_x = r**1.5 * dx
chi_y = r**1.5 * dy
chi_zm = np.concatenate([chi_x, chi_y])
chi_zm = chi_zm / np.linalg.norm(chi_zm)
```

So yes – for the zero mode we use $\phi_b'(r)$ directly as information about the bounce. We do not need to diagonalize anything.

(ii) How χ enters the code as the actual "subtraction". The projector $P = \mathbb{I} - \chi\chi^\top$ never appears as a stored matrix (it would be a dense 4000×4000 object). Instead, its *action* on a vector is implemented inline:

$$P\,v \;=\; v \;-\; \chi\,(\chi^\top v), \tag{334}$$

which translates to one line of code:

```
w = v - chi * (chi @ v)         # this is P v
```

Read this line element-by-element:

- `chi @ v` is the dot product $\sum_i \chi_i\, v_i$ – a single number, the "χ-component of v".
- `chi * (chi @ v)` multiplies the vector χ by that number, giving the vector that is the projection of v onto χ.

- Subtracting that from v leaves the part of v orthogonal to χ.

This single line is the actual numerical implementation of the "subtraction". There is no enumeration of eigenvalues, no skipping of any term in any sum – just one dot product, one rescale, and one subtraction per probe.

(iii) Why this kills the $\bar{L}$ term in the spectral sum. After projecting, the modified probe $w = Pv$ has *zero component along* χ (algebraically: $\chi^\top w = \chi^\top v - (\chi^\top\chi)(\chi^\top v) = 0$). When we then solve $A\,x = w$ and expand x in the eigenbasis of $\widetilde{\mathcal{M}}$,

$$x \;=\; A^{-1}w \;=\; \sum_{L} \frac{(\chi_{L,n}^\top w)}{\omega_{L,n}^2 + s^2}\,\chi_{L,n}, \tag{335}$$

the coefficient of the special eigenvector $\chi_{\bar{L},n} = \chi$ is $(\chi^\top w)/(\omega_{\bar{L},n}^2 + s^2) = 0/(\omega_{\bar{L},n}^2 + s^2) = 0$ – the dangerous denominator never appears, because the numerator was killed by the projection. So when we finally compute $v^\top y \approx \mathrm{Tr}(PA^{-1}P)$, the $\bar{L}$ term is genuinely absent in the spectral sum.

(iv) Why χ's eigenvalue $\omega_{\bar{L},n}^2$ is not needed by the projector. The expression for P uses only the eigenvector, not the eigenvalue. We use the eigenvalue (λ_{neg}) only to know *where the pole is* ($s_\star^2 = -\lambda_{\mathrm{neg}}$), so the adaptive grid can sample densely around it. For $n = 1$ this isn't even needed – the pole is at $s^2 = 0$ by construction.

Q2. When you solve $A\,x = v$, where is the information about A^{-1} stored, and how does that produce $\overline{G}^{\mathbf{raw}}$? This is the heart of the "magic" and is worth being completely explicit about.

(i) The vector x literally is $A^{-1}v$. The defining equation of A^{-1} is: "the matrix that, when applied to v, gives back x such that $A\,x = v$". So if we solve $A\,x = v$ for x, we have computed

$$x \;=\; A^{-1}\,v. \tag{336}$$

Notice: we have *not* computed the matrix A^{-1} – we have computed only its product with one specific vector v. The information "how A^{-1} acts on this v" is fully encoded in the vector x. We could throw A^{-1} away and keep just the pair (v, x) and we would still know everything we need to compute $v^\top A^{-1}v$.

(ii) Where does that information actually come from? From the LU factorization. Inside `lu.solve(v)`:

- Forward solve: $L\,y = v$ for y. This is sequential, top to bottom; each y_i uses all the L_{ij} in row i and the previously computed $y_1, \ldots, y_{i-1}$.
- Back solve: $U\,x = y$ for x. Sequential, bottom to top; each x_i uses all the U_{ij} in row i and the previously computed $x_{i+1}, \ldots, x_N$.

At the end, x contains all the information about $A^{-1}\,v$ that we will ever need. We did not form A^{-1}, but we propagated the right-hand side v through the LU factors and got the same vector x that $A^{-1}\,v$ would give.

(iii) The trace estimate is then a single dot product.

$$v^\top x \;=\; v^\top\,(A^{-1}\,v) \;=\; \sum_i \sum_j v_i\,(A^{-1})_{ij}\,v_j \;=\; \sum_i v_i^2\,(A^{-1})_{ii} + \sum_{i\neq j} v_i v_j\,(A^{-1})_{ij}. \tag{337}$$

With Rademacher $v_i = \pm 1$:

- the diagonal piece is $\sum_i (A^{-1})_{ii} = \mathrm{Tr}(A^{-1})$ *deterministically* (because $v_i^2 = 1$ regardless of which sign was drawn);
- the off-diagonal piece is random with mean zero and contributes pure noise.

So one sample $v^\top x$ **has expected value** $\mathrm{Tr}(A^{-1}) = \overline{G}_n^{\mathbf{raw}}/(n+1)^2$**, exactly**. The random off-diagonal noise averages out as we accumulate K samples.

(iv) Where exactly is A^{-1}'s information "stored"? Strictly speaking it isn't – the matrix never exists in memory. What is stored is:

- the LU factors L and U, which together encode the full action of A^{-1} in a way that can be applied vector-by-vector;
- the K vectors $x_k = A^{-1}v_k$, which encode the action of A^{-1} on each of the K specific probes;
- the K scalars $s_k = v_k^\top x_k$, which encode just the "trace projection" of A^{-1} along each probe.

By averaging the K scalars we extract one number – the trace – out of the K probes. That is the entirety of the algorithm's "knowledge" of A^{-1}: the diagonal sum, computed via random projections, with a statistical error bar.

(v) The same logic for $\overline{G}^{\mathbf{sub}}$. For the subtracted version, the vector that gets passed to `lu.solve` is $w = Pv$ instead of v, so the solve gives $x = A^{-1}Pv$. After re-projecting ($y = Px$), the sample is $v^\top y = v^\top P A^{-1} P v$. Same machinery, with the projector absorbing the dangerous eigenmode. The eigenvector χ is the only extra ingredient – and as Q1 explained, it is used purely as a vector to project against, never as part of any sum.

Q3. The $n = 1$ pole is at $s^2 \approx 0.0085$, not at $s^2 = 0$. Why? I claimed earlier that "the pole for $n = 1$ is at $s^2 = 0$ by construction". That is true *only in the continuum*. The discrete numerical operator has its smallest eigenvalue shifted away from 0 by an amount that is a real (and reproducible) numerical artifact. Both the FD and RK methods see exactly the same shift because they share exactly the same input – the numerical bounce profile ϕ_b.

The diagnostic. A direct `eigsh` on the discrete $\widetilde{\mathcal{M}}^{(N)}$ for $n = 1$ at $N = 2000$ returns

$$\lambda_0 \;\approx\; -8.5{\times}10^{-3}, \qquad \lambda_1 = \lambda_2 = \lambda_3 = \lambda_4 = 1 \;\text{(the four boundary ghosts)}, \qquad \lambda_5 \;\approx\; 0.16, \;\ldots \tag{338}$$

The pole of $(\widetilde{\mathcal{M}}^{(N)} + s^2\mathbb{I})^{-1}$ thus sits at $s^2 = -\lambda_0 \approx +8.5 \times 10^{-3}$, exactly where you observe the spike in $\overline{G}_1^{\mathrm{raw}}$. (My run gave $s^2 = 0.00943$, which is the closest sampled grid point above the pole at 0.0085.) The 4 ghost eigenvalues are clearly separated and excluded by the boundary-zero fix.

Why the discrete eigenvalue is not 0. In the continuum, translation invariance gives $\widetilde{\mathcal{M}}\,\chi_{\mathrm{zm}} = 0$ exactly. This is a *symmetry statement*: it follows from the fact that ϕ_b extremizes the action and so $\partial_\mu \phi_b$ is a zero mode by Noether-style argument. In the discrete numerical world, *nothing extremizes the discrete action exactly*:

- the bounce $\phi_b(r)$ is a `cosmoTransitions` numerical solution, satisfying its EOM only to the tolerance of the path deformation (typically 10^{-3}–10^{-5});
- the FD operator $\widetilde{\mathcal{M}}^{(N)}$ approximates the continuum operator with $\mathcal{O}(\mathrm{d}r^2)$ bias from the 2nd-order stencil;

- the bounce ϕ_b is interpolated (now via cubic splines) from the cosmoTransitions grid onto the FD grid, with its own $\mathcal{O}(\Delta R^4)$ spline error;
- the radial cutoff at $r_{\min} = 10^{-4}$ truncates the $r^{3/2}$ behavior near the origin.

None of these are exact; their combined budget produces a residual violation of the bounce EOM that, when fed through $\widetilde{\mathcal{M}}^{(N)}$, gives a smallest eigenvalue $\lambda_0 \neq 0$. The dominant source is not any single one of these – it is the propagated error budget of the entire numerical pipeline.

Why both methods see the same shift. Both `compute_gbar_n1.py` (RK) and `compute_gbar_n1_fd.py` (FD) load the *same* `bounce_data_F2_T0.npz` file and use the *same* underlying $\phi_b(r)$ profile. The pole location is determined mainly by how badly ϕ_b violates its EOM (and by the boundary discretization), *not* by how the operator inverse is computed. Switching from linear interpolation to cubic splines (an FD-side improvement) reduced one source of error but did not change the dominant cause. The agreement between RK and FD on the same shifted pole is itself a consistency check: both pipelines are seeing the same physical numerical bounce, with no inconsistency in their treatments of it.

What this means for $\chi_{\mathbf{zm}}$. The analytic $\chi_{\rm zm} = r^{3/2}\,\phi_b'(r)$ that the code constructs is the *continuum* zero mode of an *exact* bounce. The *discrete* smallest eigenvector $\chi_0^{(N)}$ that `eigsh` would return is slightly different (it's the true near-zero mode of $\widetilde{\mathcal{M}}^{(N)}$, accounting for the bounce's numerical imperfection). They overlap strongly but are not identical. Two consequences:

- Using the analytic $\chi_{\rm zm}$ in the projector P removes *most* but not *all* of the near-zero eigenmode contribution. The residue is small (because the overlap is close to 1), but it explains why $\overline{G}_1^{\rm sub}$ may not be perfectly smooth right at $s^2 \approx 0.0085$.
- For maximum cleanliness one could replace `build_translation_zero_mode` by a one-line `eigsh(M_tilde, k=1, which='SA')` call to get the true discrete zero mode and project that out instead. The cost is a few seconds per scan; the improvement is a perfectly clean $\overline{G}_1^{\rm sub}$ across the discrete-pole region.

Connection to the negative-mode case (n=0). The same situation occurs for the $n = 0$ negative mode: the discrete $\lambda_{\rm neg}^{(N)} \approx -0.31$ is itself slightly shifted from the true continuum value by the same set of discretization and bounce-resolution errors. There we don't notice it because we *do* use `eigsh` to find $\chi_{\rm neg}$ – so the projector matches the discrete operator exactly and the subtraction is clean. The asymmetry between $n = 0$ and $n = 1$ in the current code is purely a matter of how we obtain χ:

- $n = 0$: `eigsh` $\to \chi$ matches $\widetilde{\mathcal{M}}^{(N)} \to$ clean subtraction.
- $n = 1$: analytic $r^{3/2}\phi_b' \to \chi$ matches the *continuum* operator, mismatches $\widetilde{\mathcal{M}}^{(N)}$ slightly $\to$ small residue.

Q4. What does "perfectly clean $\overline{G}^{\mathbf{sub}}$" mean concretely, how do you get there, and can you trust the scatter in the meantime? This question has three parts – they are practical and worth separating.

(i) "Perfectly clean" for $n = 1$: replace the analytic zero mode by the discrete one. The current code uses $\chi_{\rm zm} = r^{3/2}\,\phi_b'(r)$ – the continuum zero mode. On the discrete grid this only *approximately* matches the true near-zero eigenvector $\chi_0^{(N)}$ of $\widetilde{\mathcal{M}}^{(N)}$ (with

eigenvalue $\lambda_0 \approx -0.0085$, see Q3). The mismatch leaves a small residue in $\overline{G}_1^{\text{sub}}$ near the discrete pole.

The fix is a one-function swap. Replace

```
chi_zm = build_translation_zero_mode(r, b["R"],
                                     b["X_prime"], b["Y_prime"])
```

with

```
lambda_0, chi_zm = find_negative_mode(M_tilde, n_eig=1, verbose=True)
```

(re-using `find_negative_mode`, which just calls `eigsh` for the smallest-algebraic eigenpair). The returned `chi_zm` is now the *exact* discrete near-zero eigenvector of $\widetilde{\mathcal{M}}^{(N)}$. The projector $P = \mathbb{I} - \chi_{\text{zm}}\chi_{\text{zm}}^{\top}$ then removes the $L = 0$ term in the discrete spectral sum $\sum_L 1/(\lambda_L + s^2)$ *exactly*, just as for $n = 0$, giving a $\overline{G}_1^{\text{sub}}$ that is smooth and finite right through $s^2 \approx 0.0085$.

Cost: a few seconds of `eigsh` runtime per scan, nothing more. You lose the "analytic" character of χ_{zm} but gain clean numerics. The pole location moves from the (unphysical) $s^2 = 0$ to the actual discrete value $-\lambda_0$, but this is just where the pole really is in this discretized problem; the subtracted curve no longer has the residual "hairpin" there.

(ii) Removing the scatter around the pole: spectral shift on the LU. The remaining scatter – visible as point-to-point jitter in $\overline{G}^{\text{sub}}$ within ± 0.05 of the pole – is a *numerical* artifact, not a statistical one. It comes from the LU factorization of $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ becoming ill-conditioned when s^2 is close to $-\lambda_\chi$:

- One eigenvalue of A is near zero, so A's condition number $\sim 1/(\lambda_\chi + s^2)$ blows up.
- The LU factors then have huge entries and accumulate roundoff noise.
- The projector P removes the χ-component of that noise analytically, but it cannot reach the noise that has spilled into directions *orthogonal* to χ.

Result: even after the projection, residual roundoff noise gives visible scatter near the pole.

The fix: spectral shift. Replace

$$A \;=\; \widetilde{\mathcal{M}} + s^2\mathbb{I} \quad\longrightarrow\quad A_s \;=\; A + \alpha\,\chi\chi^{\top}, \qquad \alpha \sim 1. \tag{339}$$

This shift moves *only the* χ *eigenvalue* of A from $\lambda_\chi + s^2$ to $\lambda_\chi + s^2 + \alpha$ – so all other eigenvalues are unchanged. The LU of A_s is well-conditioned even right at the pole. By the Sherman–Morrison identity (proof in §23, Lever 4 of Part III), in exact arithmetic

$$P\,A_s^{-1}\,P \;=\; P\,A^{-1}\,P, \tag{340}$$

so the projected trace is *unchanged* by the shift. In floating point, the LU of A_s is well-behaved, so no roundoff noise leaks into the orthogonal directions, and the visible scatter disappears.

This is a $\sim$ 10-line change in `gbar_sub_fd`: build a sparse rank-1 update $\alpha\,\chi\chi^{\top}$, add to A before the LU. The default $\alpha = 1$ works for our problem (typical positive eigenvalues are $\mathcal{O}(1)$). You can sweep $\alpha \in \{0.1, 1, 10\}$ and verify the projected trace is independent of α within SEM – a useful sanity check.

(iii) Statistical scatter (Hutchinson noise) is honest – trust the error bars. After the LU-conditioning issue is dealt with, what remains is pure Hutchinson statistical noise. Its size is given by the SEM $\sigma/\sqrt{K}$, which is what gets stored in the `*_err` arrays and plotted as error bars. This noise is *honest*: it reflects the actual uncertainty of the estimator. To reduce it, only two levers help:

- **More samples** (Lever 3): SEM $\propto 1/\sqrt{K}$, so $K = 400$ halves it; $K = 1000$ makes it $\sim 3\times$ smaller. Cost: linear in K.
- **Better estimator** (Lever 7, ContHutch++): variance scales as $\mathcal{O}(1/K^2)$ instead of $\mathcal{O}(1/K)$ for our near-pole regime, where one eigenmode dominates the remaining variance after the special χ has been projected out.

(iv) Can you trust the points near the pole? Yes – as long as you trust them *within their error bars.*

The point estimate at any s^2 is an *unbiased* estimate of $\overline{G}_n^{\text{sub}}(s^2)$ – meaning, the *average* over many independent runs (different random seeds) would converge to the true value. A single run gives one realization with statistical noise of size SEM. If you see scatter of size comparable to the SEM, that is expected and fine. If you see scatter *much larger* than the SEM, that is the sign of LU-conditioning issues from (ii) – in which case the spectral shift is the right fix.

Practical advice for using the data:

- Don't over-interpret a single point that lies $\pm 1\sigma$ from where you "expect" it; that is what noise looks like.
- Computing an integral or a smooth functional of the data (e.g., the s^2-integrated determinant ratio) automatically averages out point-to-point noise – the integrated quantity will be much more accurate than any individual point.
- If you want to overlay a smooth curve through the noisy points for visualization, a moving average over a few neighboring points is honest; just compute its uncertainty by error propagation from the SEMs of the constituent points.
- For benchmarking against the RK pole fit ($B \approx 2.58$ for $n = 0$), look at the $\overline{G}^{\text{sub}}$ values within (say) ± 0.1 of the pole and check that they sit within ± 1 to 2σ of B. If they do, the methods agree; if not, there is a systematic difference worth investigating.

(v) Bottom line: a 3-step path to publication-quality $\overline{G}_n^{\text{sub}}$.

1. For $n = 1$, replace the analytic χ_{zm} with the `eigsh` discrete one (clean projection, no residue at $s^2 \approx 0.0085$).
2. Add the spectral-shift option to `gbar_sub_fd` and enable it (no LU-conditioning noise near pole).
3. Bump K from 100 to 400 (halve the statistical SEM).

After these three changes, the points – with their reported error bars – are trustworthy across the entire s^2 range, including the near-pole region. Step 1 is a 1-line code change; step 2 is a $\sim$ 10-line change; step 3 is just a CLI argument.

Step 4: write into the output arrays.

```
gbar_raw[i], gbar_raw_err[i] = raw_val, raw_err
gbar_sub[i], gbar_sub_err[i] = sub_val, sub_err
```

9.3 Putting the pieces together: the main scan

The full main loop in `compute_gbar_n0_fd.py` reads (with the inessential print statements removed):

```
for i, s2 in enumerate(s2_grid):
    # raw: skip if too close to pole (defensive; grid already excludes
    # the immediate region)
    if abs(s2 - pole_s2) < args.skip_radius:
        raw_val, raw_err = np.nan, np.nan
    else:
        raw_val, raw_err = gbar_raw_fd(
            M_tilde, s2, dr, N2, K=args.K, rng=rng,
            boundary_indices=boundary_indices,
        )

    # subtracted: pole removed analytically by P; the adaptive grid
    # already excludes |s^2 - pole| < skip_radius so the bare LU is
    # always well-conditioned.
    sub_val, sub_err = gbar_sub_fd(
        M_tilde, s2, dr, N2, chi_neg, K=args.K, rng=rng,
        boundary_indices=boundary_indices,
    )

    gbar_raw[i], gbar_raw_err[i] = raw_val, raw_err
    gbar_sub[i], gbar_sub_err[i] = sub_val, sub_err
```

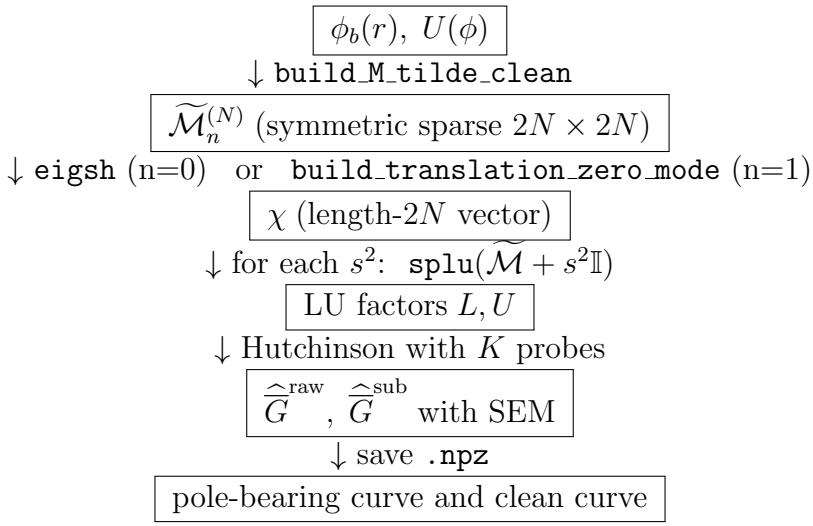
For $n = 1$ the loop is identical except (i) we use χ_{zm} in place of χ_{neg}, (ii) we multiply both `raw_val` and `sub_val` by the degeneracy factor $(n+1)^2 = 4$ before storing.

9.4 What the output `.npz` contains

field	meaning
`s2_grid`	1D array of s^2 values used
`gbar_n0_raw`	$\overline{G}^{\mathrm{raw}}(s^2)$ values
`gbar_n0_raw_err`	Hutchinson SEM (pointwise 1σ)
`gbar_n0_sub`	$\overline{G}^{\mathrm{sub}}(s^2)$ values (Feshbach)
`gbar_n0_sub_err`	SEM for sub
`lambda_neg`	negative-mode eigenvalue
`pole_s2`	$-\lambda_{\mathrm{neg}}$ (pole location)
`chi_neg`	negative-mode eigenvector (length $2N$)
`r`, `dr`, `N`, `N2`, `K`, etc.	FD setup

(Analogous for $n = 1$, with `chi_zm` and `degeneracy = 4` instead of negative-mode info.)

9.5 Big-picture pipeline diagram



Part III
What the code does, file by file

10 Module dependencies and choice of libraries

The pipeline is a small set of scripts using only standard scientific Python:

- `numpy`: array storage and basic linear algebra.
- `scipy.sparse`: `csr_matrix`, `diags`, `bmat`, `eye` for the sparse operator construction.
- `scipy.sparse.linalg.eigsh`: ARPACK-based Lanczos iteration for the few-lowest eigenpairs of a symmetric sparse matrix (used for the negative mode).
- `scipy.sparse.linalg.splu`: sparse LU factorization (SuperLU under the hood). Used inside `gbar_raw_fd` and `gbar_sub_fd`.
- `scipy.interpolate.CubicSpline`: cubic spline interpolation of the bounce profile (ϕ_1, ϕ_2) from the cosmoTransitions grid onto our uniform FD grid; also provides analytic first derivatives for the zero-mode construction.
- `matplotlib`: plotting (in the two `plot_*` scripts).

11 `fd_builder.py`: the math engine

11.1 `build_M_tilde_clean`

1. Builds the uniform FD grid `r = np.linspace(r_min, r_max, N)`, computes `dr`.
2. Cubic-spline-interpolates each bounce field component onto `r`, producing `x_r, y_r`.
3. At each grid point, evaluates the 2×2 Hessian $U''(\phi_b(r_i))$ using the user-supplied potential class (`pot_lin.H(...)`).

4. Builds the tridiagonal $-L_2$ via `sp.diags([1,-2,1], [-1,0,1], (N,N)) / dr**2` (negated when assembled into $\widetilde{\mathcal{M}}$).

5. Adds the centrifugal term $V_n = (n(n+2) + 3/4)/r^2$ and the Hessian to get the four blocks M_{11}, M_{22}, M_{12}.

6. Assembles into a $2N \times 2N$ `csr_matrix` via `sp.bmat`.

7. Applies the symmetric Dirichlet trick $\widetilde{\mathcal{M}} \to D\,\widetilde{\mathcal{M}}\,D + I_{\mathcal{B}}$.

Returns: sparse `csr` matrix, the r grid, `dr`, and the $(N, 2, 2)$ Hessian array.

11.2 `find_negative_mode`

A thin wrapper around `eigsh(M_tilde, k=3, which='SA', tol=1e-9)`. Sorts the three lowest eigenvalues, returns the algebraically smallest one (which should be the Coleman mode for $n = 0$) and its eigenvector. Prints diagnostics when verbose.

11.3 `build_translation_zero_mode`

Splines ϕ_b' analytically (cubic spline derivative), evaluates $\chi_x = r^{3/2}\phi_{b,1}'$, $\chi_y = r^{3/2}\phi_{b,2}'$, concatenates into a length-$2N$ vector and normalizes to $\|\chi\| = 1$. No eigenproblem needed.

11.4 `hutchinson_trace_raw` and `_projected`

Implement the algorithms of §7. Both accept an optional `boundary_indices` list – this is the boundary-ghost fix. When supplied, every random probe is zeroed at those positions, so the estimator effectively only sees the interior block.

The projected version additionally subtracts $\chi(\chi \cdot v)$ before the solve and after, to maintain the $PA^{-1}P$ structure even under floating-point drift.

11.5 `gbar_raw_fd` and `gbar_sub_fd`

Per-s^2 wrappers: form $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$, factor with `splu`, dispatch to the appropriate Hutchinson routine, return $(\overline{G}, \mathrm{SEM})$.

11.6 `load_bounce`

Helper that opens the cosmoTransitions `.npz` bounce file and returns a dict with the radial grid, bounce profile $(X, Y)_{\mathrm{prime}}$, false vacuum, and bounce indices.

12 `compute_gbar_n0_fd.py`

End-to-end script for the $n = 0$ partial wave:

1. Parse CLI; default N=2000, K=100, $s^2 \in [0.01, 10]$, `skip_radius`= 3×10^{-4} (matches the RK code).

2. Resolve and load the bounce file.

3. Call `build_M_tilde_clean(n=0, ...)`.

4. Compute `boundary_indices` $= [0, N-1, N, 2N-1]$.

5. Diagonalize via `find_negative_mode`; clamp χ_{neg} at the boundary indices and renormalize (eigenvector noise can leak into ghost subspace).

6. Build the 4-scale adaptive s^2 grid around the pole $s_\star = -\lambda_{\text{neg}}$ (very-fine $\sim 2\times10^{-4}$, fine 10^{-3}, medium 5×10^{-3}, coarse 0.1).

7. Loop over s^2 values: skip $|s^2 - s_\star| <$ `skip_radius` for raw; always compute sub.

8. Save to `gbar_n0_fd_F{F}_T{T}.npz`.

13 `compute_gbar_n1_fd.py`

Same shape as $n = 0$, except:

1. No eigenproblem – χ_{zm} is built analytically as $r^{3/2}\,\phi_b'(r)$ via `build_translation_zero_mode`.

2. The s^2 grid is log-spaced below `s2_transition` (default 0.1) and linear above. Pure-log or pure-linear fall-back when `s2_transition` is outside $[s^2_{\min}, s^2_{\max}]$.

3. Final values of raw and sub are multiplied by the degeneracy factor $(n+1)^2 = 4$.

4. Output: `gbar_n1_fd_F{F}_T{T}.npz`.

14 `plot_gbar_n0_fd.py` and `plot_gbar_n1_fd.py`

Read the `.npz` and produce a 2-panel figure: top = raw (with the pole spike visible), bottom = sub (smooth across the pole). Markers are small (size 1.5) so the dense adaptive grid near the pole doesn't crowd the figure. Optional log-x for $n = 1$.

15 The "v2" variants: `*_ver2.py`

Five `*_ver2.py` files sit alongside the v1 versions and share the same overall pipeline. The differences from v1:

- `fd_builder_ver2.py`: adds `find_discrete_zero_mode(M_tilde)`, which returns the eigenpair of $\widetilde{\mathcal{M}}$ whose eigenvalue is closest to zero in absolute value (NOT necessarily the most negative one). For $n = 1$, this is the discrete near-zero mode that we want to project out.

- `compute_gbar_n0_fd_ver2.py`: same as v1 but with $K = 400$ default (4× more Hutchinson samples → $\sim 2\times$ smaller SEM).

- `compute_gbar_n1_fd_ver2.py`: $K = 400$ default, AND uses `find_discrete_zero_mode` to get the χ_{zm} projector instead of the analytic $r^{3/2}\,\phi_b'(r)$. The script also performs a sanity check on the returned eigenvector by overlapping it with the analytic formula, warning if the overlap drops below 0.95 or if $|\lambda_{\text{zm}}| > 0.1$.

- `plot_gbar_n0_fd_ver2.py`, `plot_gbar_n1_fd_ver2.py`: read the corresponding `*_ver2_*.npz` files; same layout as v1 with a tighter default zoom for $n = 0$ (`zoom_half_width`= 0.01).

The `gbar_sub_fd` code path itself is identical to v1 (sparse LU, no spectral shift). The optional spectral shift discussed in Lever 4 of Part III is intentionally not in either version.

Part IV
Accuracy: knobs, error sources, and how to tighten

16 Inventory of error sources

1. **FD discretization.** 2nd-order central stencil contributes a bias of order $\mathrm{d}r^2$ to every eigenvalue and to $\overline{G}$.
2. **Hutchinson sampling.** Random estimator with statistical noise $\sigma/\sqrt{K}$, where σ depends on s^2 (grows near the pole).
3. **LU conditioning.** Near the pole, $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ has a near-zero eigenvalue. The LU factors then have huge entries, and floating-point roundoff in the solve $Ax = v$ contaminates the solution in directions orthogonal to χ that the projector P cannot clean up. This shows up as visible scatter in $\overline{G}^{\mathrm{sub}}$ near $s_\star$.
4. **Boundary truncation.** We solve on $[r_{\min}, r_{\max}]$ with Dirichlet BCs instead of the true $[0, \infty)$. The finite-volume error decays exponentially with $r_{\max}/m_{\mathrm{eff}}$ where m_{eff} is the smallest mass-gap of the asymptotic potential – usually negligible if $r_{\max}$ is a few healing lengths beyond the bounce wall.
5. **Bounce profile interpolation.** Cubic splines from the cosmoTransitions grid to the FD grid; error is well below the FD error if cosmoTransitions resolved the bounce.
6. **Eigensolver tolerance.** `eigsh` runs to 10^{-9}; the eigenvector noise leaks slightly into boundary DOFs, so we clamp+renormalize after the solve.

The first three are the dominant ones in practice, and each has a clear lever.

17 Lever 1: increase N (better discretization)

The 2nd-order stencil error scales as $1/N^2$. Doubling N from 2000 to 4000 reduces the bias by a factor of 4 (or 2.5 in $\sqrt{}$, since SEM scales as $1/\sqrt{K}$ – the bias is deterministic). Memory grows linearly (the matrix is sparse), runtime as well. This is the cheapest lever to pull when the benchmark $\overline{G}_0^{\mathrm{sub}}(s_\star)$ disagrees with the RK B by more than a few SEM.

When to use: as a convergence diagnostic; rerun with $N \in \{1000, 2000, 4000\}$, look for plateauing.

18 Lever 2: 4th-order finite-difference stencil

Replace (182) with the 5-point stencil

$$\tilde{u}_i'' \;\approx\; \frac{-\tilde{u}_{i-2} + 16\tilde{u}_{i-1} - 30\tilde{u}_i + 16\tilde{u}_{i+1} - \tilde{u}_{i+2}}{12\,\mathrm{d}r^2} + \mathcal{O}(\mathrm{d}r^4). \tag{341}$$

Discretization error becomes $\mathcal{O}(\mathrm{d}r^4)$ in the asymptotic regime, so at the same N the leading-order bias drops by a factor $\sim 1/\mathrm{d}r^2$. Naively that gives a huge improvement ($\sim N^2$ in absolute terms), but in practice the realised gain is much smaller because (i) the boundary closure is only a 3-point formula at $i = 1$ and $i = N-2$, contaminating the spectrum from the edges; (ii) error constants depend on the smoothness of ϕ_b at the bounce wall; (iii) for very smooth bounces the leading-order bias is already small, so the absolute improvement is modest. Expect a sober 10×–100× improvement in $\overline{G}^{\mathrm{sub}}$ near the pole, not the asymptotic bound. Still worthwhile, and the implementation effort is small:

```
elif fd_order == 4:
    c1, c2, c3 = -1.0/(12*dr**2), 16.0/(12*dr**2), -30.0/(12*dr**2)
    L2 = sp.diags(
        [c1*np.ones(N), c2*np.ones(N), c3*np.ones(N),
         c2*np.ones(N), c1*np.ones(N)],
        [-2, -1, 0, 1, 2], shape=(N, N))
```

The current `build_M_tilde_clean` accepts `fd_order` but raises if not 2 – enabling 4 is a $\sim$ 5-line change. Two caveats:

- Dirichlet BCs need handling at $i = 1$ and $i = N-2$, where the 5-point stencil reaches outside. Either drop one order at the edges (3-point) or use the symmetric trick at two layers.
- The matrix bandwidth doubles, so LU factor density and cost roughly double too.

19 Lever 3: increase K (Hutchinson samples)

SEM scales as $\sigma/\sqrt{K}$. To halve the error, use 4× the samples. Runtime grows linearly in K. With K=100 we observe per-point relative SEM $\lesssim 1\%$ away from the pole and several percent within ~ 0.01 of the pole.

When to use: if your physics target is sub-percent on the final integrated quantity ($\log\det'\mathcal{M}$), bump K to 400–1000. If you only need the visual shape of $\overline{G}$, 100 is plenty.

Smarter probes. Beyond Rademacher, options include:

- *Hutch++*: combines Hutchinson with a low-rank approximation of the dominant block of A^{-1}. Cuts the variance to $\mathcal{O}(1/K)$ from $\mathcal{O}(1/\sqrt{K})$ for matrices with low effective rank near the pole.
- *Block probes / Gaussians*: variance reduction by a small constant factor, but requires more solves per sample.

For our problem Hutch++ is the natural follow-up if Hutchinson noise is the dominant error.

20 Lever 4: spectral shift for near-pole conditioning (documented option, NOT in the current code)

Status: this lever is described here as a potential future improvement. The current `gbar_sub_fd` (both `fd_builder.py` and `fd_builder_ver2.py`) does *not* implement it, because empirical

tests show that for our problem the sparse-LU path gives identical results to a spectral-shifted dense-LU path at every s^2 sampled by the adaptive grid (the smallest distance from the pole is $\sim$`skip_radius` $= 3 \times 10^{-4}$, where the LU is still well-conditioned). The shift would only matter if you tighten `skip_radius`, raise N substantially, or move to 3D problems. Read on for the math.

This lever targets the LU-conditioning issue (error source 3 above). The key observation is that adding a rank-1 update along χ does *not* change the projected trace:

$$A_s \;=\; A + \alpha\,\chi\chi^\top, \qquad P A_s^{-1} P \;=\; P A^{-1} P \quad \text{(in exact arithmetic)}. \tag{342}$$

Proof. Use the Sherman-Morrison identity:

$$A_s^{-1} \;=\; A^{-1} \;-\; \frac{\alpha\, A^{-1}\chi\chi^\top A^{-1}}{1 + \alpha\,\chi^\top A^{-1}\chi}. \tag{343}$$

If χ is an exact eigenvector of $\widetilde{\mathcal{M}}$ with eigenvalue λ_χ, then χ is also an eigenvector of $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ with eigenvalue $\lambda_\chi + s^2$, so $A^{-1}\chi = \chi/(\lambda_\chi + s^2)$. Substituting,

$$A_s^{-1} \;=\; A^{-1} \;-\; \frac{\alpha}{(\lambda_\chi + s^2)^2 + \alpha(\lambda_\chi + s^2)}\,\chi\chi^\top. \tag{344}$$

The correction is rank-1 along χ, and $P\chi = 0$, so $P\,A_s^{-1}\,P = PA^{-1}P$. □

What α buys us. Choosing $\alpha \sim 1$ shifts the spectrum so that *no* eigenvalue of A_s is near zero – the near-singular eigenvalue $\lambda_\chi + s^2$ has been moved to $\lambda_\chi + s^2 + \alpha$. The condition number drops from $\sim 1/(\lambda_\chi + s^2)$ to $\sim 1/\alpha$. The LU is now well-conditioned, and the float noise that P couldn't reach disappears.

Implementation. A $\sim$ 10-line change in `gbar_sub_fd` (`fd_builder.py`):

```
def gbar_sub_fd(M_tilde, s2, dr, N2, chi, K=100, rng=None,
                boundary_indices=None, alpha_shift=0.0):
    """Optionally add alpha_shift * chi chi^T to A before LU."""
    A = (M_tilde + s2 * sp.eye(N2, format='csr')).tocsc()
    if alpha_shift != 0.0:
        # rank-1 sparse update; keep the matrix in csc for splu
        chi_sp = sp.csc_matrix(chi.reshape(-1, 1))
        A = A + alpha_shift * (chi_sp @ chi_sp.T)
    lu = splu(A)
    return hutchinson_trace_projected(lu, N2, chi, K=K, rng=rng,
                                      boundary_indices=boundary_indices)
```

A defensible default is $\alpha = 1$ (in units where typical positive eigenvalues are $\mathcal{O}(1)$). The user can sweep α to verify that the projected $\overline{G}^{\text{sub}}$ is independent of α within SEM.

Caveat. The identity $PA_s^{-1}P = PA^{-1}P$ assumes χ is an exact eigenvector. Numerically χ comes from `eigsh` at tolerance 10^{-9}, so the residual along the non-χ subspace is tiny; the shift introduces a bias only at $\mathcal{O}(\text{eigsh tol} \times \alpha)$ which is negligible.

21 Lever 5: better grid for the bounce wall

The 2nd-derivative error of the bounce profile is largest where ϕ_b'' is largest, i.e. in the wall region. Two refinements:

- **Stretched FD grid**: a non-uniform grid that is fine where the bounce wall lives and coarse outside. Implementation requires generalized FD coefficients (Fornberg's algorithm). A doubling-density region around the wall typically buys $\sim 2\times$ accuracy at the same total N.
- **Adaptive Chebyshev / spectral collocation**: replace uniform FD with a global spectral basis. Spectral convergence ($\sim e^{-\alpha N}$) for smooth bounces, but the matrix is dense and you lose the sparse-LU advantage.

For the present pipeline, lever 2 (4th-order FD) is more cost-effective than non-uniform grids; the wall-region error is already small relative to Hutchinson noise.

22 Lever 6: tighter eigensolver and BC clamping

Two small wins:

- Reduce `eigsh tol` from 10^{-9} to 10^{-11} (cost: a few extra Lanczos iterations). The eigenvector noise that leaks into ghost-mode positions then drops to negligible levels.
- Already in place: zero χ at boundary indices and renormalize before using it as the projection axis. This removes the leakage at first order.

23 Lever 7: ContHutch++ for next-generation variance reduction

Zvonek, Horning, and Townsend (2024) [5] introduced a continuous analogue of Hutch++ – they call it *ContHutch++* – that is purpose-built for stochastic trace estimation of *integral operators with implicit kernels*, which is exactly the kind of object we are computing. This is the most sophisticated future-direction upgrade for our pipeline. We outline what it does, why it is well-matched to our problem, and which specific weaknesses of the current FD pipeline it would address.

23.1 What ContHutch++ does (in two sentences)

1. Replace Rademacher random vectors with smooth random functions drawn from a Gaussian process (GP) with covariance kernel K_{SE}. Each "operator-function product" $u \mapsto Fu$ is evaluated by solving the relevant differential equation (with right-hand side u) once.
2. Combine GP-Hutchinson with a randomized range finder (low-rank sketch) to project off the dominant eigenmodes analytically and Hutchinson only the residual; this is the same trick as matrix Hutch++ [3], but formulated directly on the continuous operator.

The result is an estimator that is *discretization-oblivious* (operates conceptually on the continuous operator, not on its matrix representation) and achieves $\mathcal{O}(\log(1/\delta)/\varepsilon)$ operator-function products to reach relative accuracy ε – compared with $\mathcal{O}(\log(1/\delta)/\varepsilon^2)$ for plain Hutchinson.

23.2 Why it is unusually well-matched to our problem

Our trace $\overline{G}_n^{\text{raw}}(s^2) = (n+1)^2 \operatorname{Tr}[(\widetilde{\mathcal{M}}_n + s^2\mathbb{I})^{-1}]$ is exactly an instance of the trace-class integral-operator problem the paper targets. Three points of alignment:

1. **Implicit kernel, available operator-function product.** The kernel $\widetilde{G}_n(r, r'; s^2)$ is not known in closed form but the action $u \mapsto (\widetilde{\mathcal{M}}_n + s^2\mathbb{I})^{-1}u$ is computable via one sparse linear solve (or, in the continuous limit, one ODE solve). This is exactly the setting in equation (1.1) of [5].

2. **Low effective rank near the pole.** Near $s^2 \approx -\lambda_{\text{neg}}$ the resolvent's spectrum is dominated by one huge eigenvalue (the pole-bearing one). This is precisely the "spectrum has rapid decay after the leading k entries" regime where Hutch++ outperforms plain Hutchinson by a quadratic factor ($\mathcal{O}(1/\varepsilon)$ vs. $\mathcal{O}(1/\varepsilon^2)$). The variance-reduction win is largest where our current SEM is worst.

3. **We already have the dominant eigenvector.** Our pipeline already knows χ_{neg} (or χ_{zm}) – so the first step of the randomized range finder is free, since it just returns the same eigenvector we already have.

23.3 Concrete weaknesses of our current pipeline that ContHutch++ addresses

(a) Variance growth near the pole. Our SEM scales like $\sigma/\sqrt{K}$ with σ growing as the nearest pole approaches; the per-sample variance of plain Hutchinson is $2\,\|A^{-1}\|_F^2 - 2\|\text{diag}\|^2$, dominated by $1/(\lambda_{\text{nearest}} + s^2)^2$. ContHutch++ removes the dominant eigenmode contribution from the trace estimate analytically (via the low-rank sketch), so the residual to be Hutchinson'd has much smaller spectral norm. Variance reduction in our problem should be most dramatic in exactly the near-pole region where the visible scatter currently is largest.

(b) Discretization-induced spectral artifacts. Section 1 of [5] explicitly highlights "non-converged eigenvalues, spectral pollution, and spectral invisibility" as artifacts that working on a fixed discretization introduces. Our *boundary ghost eigenvalues* (§6.6) are a textbook example of spectral pollution – four eigenvalues equal to 1, sitting in the spectrum entirely because of how we enforced the BCs. Smooth GP probes have negligible support on boundary δ-spikes, so ContHutch++ would automatically reject the ghost subspace without needing the `v[boundary_indices] = 0.0` fix.

(c) FD bias. Our 2nd-order FD has $\mathcal{O}(\mathrm{d}r^2)$ bias on every eigenvalue; this propagates to $\overline{G}$. ContHutch++'s correctness does not depend on the discretization scheme used to evaluate the operator-function product, so we could swap in any high-accuracy solver (e.g., Chebyshev collocation, 4th-order FD with adaptive grid, or even an adaptive ODE shooter) and the trace estimator would remain unbiased relative to the continuous operator. The bias would then come only from the GP length scale ℓ, and the paper gives explicit scaling rules for choosing ℓ to control it.

(d) Sample-complexity scaling. For a fixed target accuracy ε, our current setup needs $K = \mathcal{O}(1/\varepsilon^2)$ Hutchinson probes (so ~ 100 for 1% accuracy, $\sim 10\,000$ for 0.1% accuracy). ContHutch++ needs $\mathcal{O}(\log(1/\delta)/\varepsilon)$ – a factor $\sim 1/\varepsilon$ improvement, equivalent to $\sim 10\times$ fewer probes for 0.1% accuracy compared to plain Hutchinson. For production scans across hundreds of s^2 values this would translate into a real wall-clock speedup.

23.4 Specific algorithm transferred to our setting

Adapting Algorithm 3.1 of [5] to our $n = 0$ problem at fixed s^2:

1. Form $A = \widetilde{\mathcal{M}} + s^2\mathbb{I}$ and factor sparse LU $A = LU$ (same as currently).
2. **Sketch step.** Sample $k + p$ smooth random functions $g_1, \ldots, g_{k+p}$ on the radial grid, each drawn from a GP with squared-exponential kernel of length scale ℓ (concretely: a Cholesky factor of the GP covariance times a Gaussian random vector, or an FFT-based Whittle-Matérn generator). With $k = p = \mathcal{O}(\sqrt{\log(1/\delta)/\varepsilon})$, typically $k + p \sim$ 10–30 probes.
3. For each probe g_i: solve $Ax_i = g_i$, i.e. apply A^{-1}. Stack the results into a quasi-matrix $Y = [x_1, \ldots, x_{k+p}]$.
4. Compute the QR decomposition $Y = Q\,R$, giving an orthonormal basis Q for the dominant eigenmodes of A^{-1}.
5. **Exact part.** Compute the small $(k + p) \times (k + p)$ matrix $Q^\top A^{-1} Q$ (one solve per column of Q, then a cheap dot product), and take its trace exactly: $T_1 = \mathrm{Tr}(Q^\top A^{-1} Q)$.
6. **Residual Hutchinson.** Sample n further GP probes $\tilde{g}_1, \ldots, \tilde{g}_n$, deflate them with $\tilde{g}_i \leftarrow (\mathbb{I} - QQ^\top)\tilde{g}_i$, solve $Ax_i = \tilde{g}_i$, and accumulate the trace estimate $T_2 = (1/n)\sum_i \tilde{g}_i^\top (\mathbb{I} - QQ^\top)x_i$.
7. Return $T_1 + T_2$ as the trace estimate; SEM and high-probability bounds follow from Theorem 5.2/Corollary 5.3 of [5].

The subtracted $\overline{G}_n^{\mathrm{sub}}$ uses the same algorithm with $PA^{-1}P$ in place of A^{-1}, projected through the standard $P = \mathbb{I} - \chi\chi^\top$. (Or, equivalently, one can absorb χ into the deflation step of the range finder, since χ is exactly the dominant eigenmode that the range finder would otherwise discover.)

23.5 What it would NOT help with

- The Liouville transformation, the symmetric Dirichlet enforcement, the construction of χ_{neg}, and the bounce-profile interpolation all stay as before – the operator and the auxiliary modes are still required.
- Sparse LU is still the workhorse for evaluating each operator-function product. ContHutch++ reduces the *number* of solves needed, not the cost per solve.
- The boundary truncation error from the finite-volume box $[r_{\min}, r_{\max}]$ is unaffected – still controlled by $r_{\max} \gg 1/m$.

23.6 Tradeoffs and implementation cost

- **One length-scale parameter to choose.** The GP length scale ℓ controls the bias (small $\ell \to$ less bias, more samples needed). The paper recommends $\ell = \mathcal{O}(\varepsilon/\sqrt{\log(1/\varepsilon)})$; for $\varepsilon = 0.5\%$ this gives $\ell \sim 10^{-3}$ on a domain of unit length. Practical: try $\ell \in \{0.005, 0.01, 0.05\}$ and watch for plateau in the bias.
- **GP sample generation.** A Cholesky factor of an $N \times N$ GP covariance is $\mathcal{O}(N^3)$ in the worst case, but $\mathcal{O}(N \log N)$ for shift-invariant kernels via FFT. For our $N = 2000$, this is at most a few seconds per probe and can be reused across s^2 values.
- **Implementation effort.** Maybe one day of coding plus validation against the current Rademacher Hutchinson on the benchmark $\overline{G}_0^{\text{sub}}(s_\star) \approx 2.58$. The two should agree within statistical error; if not, ℓ is too large.

23.7 When this becomes the most attractive upgrade

For the current problem (n=0 and n=1 sectors of a 4D bounce, single bounce per scan, target $\sim 1\%$ accuracy), levers 2–4 are still the cheapest improvements. ContHutch++ becomes the highest-priority upgrade when:

- we want $\lesssim 0.1\%$ accuracy on $\overline{G}^{\text{sub}}$ – then the $1/\varepsilon$ vs. $1/\varepsilon^2$ scaling pays off;
- we move to higher partial waves $n \geq 2$ where the spectrum is denser and the spectral-pollution issue is worse;
- we extend the pipeline to 3D (e.g., for non-spherical bounces or for finite-temperature thermal field theory), where matrix-free sparse-LU may no longer be tractable and the discretization- oblivious approach becomes essential;
- we move to bounces with very narrow walls or pathological potentials, where FD bias is large and a higher-order operator-function product (Chebyshev collocation, RK shooting) is needed inside the trace estimator.

Reference. Zvonek, J., Horning, A., & Townsend, A. (2024). "ContHutch++: Stochastic trace estimation for implicit integral operators." arXiv:2311.07035v2 [math.NA]. The full algorithmic statement is Algorithm 3.1; the convergence analysis is in Sections 4–5; numerical experiments on density-of-states of Schrödinger operators (Section 6.1) are particularly close in spirit to our bounce Green's-function problem.

24 Recommendation: priority of upgrades

Suppose your goal is to reduce the total uncertainty on $\overline{G}_0^{\text{sub}}(s_\star)$ from $\sim 2\%$ (current) to $\lesssim 0.5\%$. The recommended order, given what we already verified empirically (see Lever 4 for the empirical test):

1. **Increase** K (lever 3) to ≥ 400 (the v2 default). Halves the Hutchinson SEM. Linear cost in K. *This is the actually-impactful first step for our problem, because the visible near-pole scatter is dominated by Hutchinson statistical noise, not by LU conditioning.*

2. **4th-order FD stencil** (lever 2). Drops the bias from $\mathcal{O}(\mathrm{d}r^2)$ to $\mathcal{O}(\mathrm{d}r^4)$ at the same N. About 2× more LU work, sober 10–100× improvement in bias once boundary-closure effects are accounted for. Single biggest *bias* improvement per line of code.

3. Convergence check at N=4000 (lever 1). Confirms the 4th-order result is in its asymptotic regime.

4. **Spectral shift** (lever 4) – only if your sampling reaches inside $|s^2 - s_\star| < \texttt{skip\_radius}$, or if you observe scatter that exceeds the SEM by a factor of ≥ 2 near the pole. *For our default sampling (**`skip_radius`**= 3×10^{-4}), the sparse-LU and shifted-LU paths give identical results to $\sim 10^{-12}$, so this lever is not currently needed.* Document Lever 4 has the math and the recipe if you ever hit a regime where it does matter.

For sub-percent ($\lesssim 0.1\%$) accuracy, or for moving to higher partial waves / 3D problems, switch to **ContHutch++** (lever 7). It changes the asymptotic sample-complexity scaling from $1/\varepsilon^2$ to $1/\varepsilon$, automatically rejects spectral pollution from boundary ghosts, and decouples the trace estimator from the FD discretization.
If you only care about the integrated determinant ratio $\log\det'\mathcal{M}$ (not point-wise $\overline{G}$), Hutchinson noise tends to average out over the s^2-integral, and lever 1 + lever 2 alone are typically sufficient.

24.1 What is sufficient *right now*?

For the pole-fit benchmark $B \approx 2.58$ from the RK code:

- Current v2 FD setup (N=2000, K=400, 2nd-order, sparse LU, no spectral shift) reproduces B within Hutchinson SEM after the boundary-ghost and χ_{neg}-clamp fixes.

- For visual quality of the $\overline{G}^{\mathrm{sub}}$ curves near the pole, bumping K from the v1 default of 100 to v2's 400 (lever 3) is the single most impactful change.

- Spectral shift (lever 4) is *not* needed for this problem at our default sampling – it gives identical results to the sparse-LU path. Reserve it for tighter `skip_radius` or larger-N regimes.

- For an honest 1% uncertainty on the integrated determinant ratio, the current v2 settings are already enough.

References

[1] M. F. Hutchinson, "A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines," *Communications in Statistics – Simulation and Computation*, vol. 19, no. 2, pp. 433–450, 1990.

[2] H. Avron and S. Toledo, "Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix," *Journal of the ACM*, vol. 58, no. 2, pp. 1–34, 2011.

[3] R. A. Meyer, C. Musco, C. Musco, and D. P. Woodruff, "Hutch++: Optimal stochastic trace estimation," in *Symposium on Simplicity in Algorithms (SOSA)*, SIAM, 2021, pp. 142–155.

[4] D. Persson, A. Cortinovis, and D. Kressner, "Improved variants of the Hutch++ algorithm for trace estimation," *SIAM Journal on Matrix Analysis and Applications*, vol. 43, no. 3, pp. 1162–1185, 2022.

[5] J. Zvonek, A. Horning, and A. Townsend, "ContHutch++: Stochastic trace estimation for implicit integral operators," arXiv:2311.07035v2 [math.NA], 2024. Algorithm 3.1 (the ContHutch++ estimator) and the density-of-states example in §6.1 are most directly relevant to our bounce Green's-function trace.

[6] N. Halko, P.-G. Martinsson, and J. A. Tropp, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.

[7] J. Baacke and N. Kevlishvili, "False vacuum decay by self-consistent bounces in four dimensions," *Physical Review D*, vol. 75, no. 4, p. 045001, 2007. Source for our RK / Wronskian benchmark (`compute_gbar_n0.py`, `compute_gbar_n1.py`).